File Reference: template-preview.js

A file-specific explanation covering purpose, owner layer, metadata, source details, implementation signals, source excerpts, and safe-change rules.

Item	Explanation
File	template-preview.js
Category	JavaScript source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	454,437 bytes
Extension	.js
MIME type	application/javascript
Text lines	10,826

Why this file exists

- Builder frontend brain: state handling, rich editors, project builder, previews, parser triggers, publishing UI, compile workspace, and guide windows.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

API endpoints mentioned or called

Item	Explanation
/api/save-draft	Line 1637. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/upload	Line 5430. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/upload	Line 8413. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/upload	Line 8473. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/apply-catalog	Line 8613. Loads project/catalog data for the builder.
/api/apply-catalog	Line 8637. Loads project/catalog data for the builder.
/api/apply-catalog	Line 8647. Loads project/catalog data for the builder.
/api/apply-catalog	Line 8662. Loads project/catalog data for the builder.
/api/apply-catalog	Line 8674. Loads project/catalog data for the builder.
/api/save-draft	Line 8685. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/save-draft	Line 10770. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/apply-catalog	Line 10772. Loads project/catalog data for the builder.
/api/save-draft	Line 10796. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.

Important implementation signals

Item	Explanation
Browser DOM at line 1	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideOpen = document.querySelector("#builder-guide-open");
Browser DOM at line 2	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideDialog = document.querySelector("#builder-guide-dialog");
Browser DOM at line 3	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideClose = document.querySelector("#builder-guide-close");
Browser DOM at line 4	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideSearch = document.querySelector("#builder-guide-search");
Browser DOM at line 5	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideResults = document.querySelector("#builder-guide-results");
Browser DOM at line 6	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideSections = document.querySelector("#builder-guide-sections");
Browser DOM at line 7	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideTopicDialog = document.querySelector("#builder-guide-topic-dialog");
Browser DOM at line 8	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideTopicTitle = document.querySelector("#builder-guide-topic-title");
Browser DOM at line 9	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideTopicBody = document.querySelector("#builder-guide-topic-body");
Browser DOM at line 10	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideTopicClose = document.querySelector("#builder-guide-topic-close");
Browser DOM at line 11	Builder or website interface behavior in the browser/Electron renderer. Evidence: const templateSelect = document.querySelector("#template-select");
Browser DOM at line 12	Builder or website interface behavior in the browser/Electron renderer. Evidence: const newCategorySelect = document.querySelector("#new-category-select");
Browser DOM at line 13	Builder or website interface behavior in the browser/Electron renderer. Evidence: const placementSelect = document.querySelector("#placement-select");
Browser DOM at line 14	Builder or website interface behavior in the browser/Electron renderer. Evidence: const newProjectTitle = document.querySelector("#new-project-title");
Browser DOM at line 15	Builder or website interface behavior in the browser/Electron renderer. Evidence: const addProjectButton = document.querySelector("#add-project");
Browser DOM at line 16	Builder or website interface behavior in the browser/Electron renderer. Evidence: const addCategoryButton = document.querySelector("#add-category");
Browser DOM at line 17	Builder or website interface behavior in the browser/Electron renderer. Evidence: const addSiteSectionButton = document.querySelector("#add-site-section");
Browser DOM at line 18	Builder or website interface behavior in the browser/Electron renderer. Evidence: const funFactsInput = document.querySelector("#fun-facts-input");
Browser DOM at line 19	Builder or website interface behavior in the browser/Electron renderer. Evidence: const funFactsCount = document.querySelector("#fun-facts-count");
Browser DOM at line 20	Builder or website interface behavior in the browser/Electron renderer. Evidence: const saveFunFactsButton = document.querySelector("#save-fun-facts");
Browser DOM at line 21	Builder or website interface behavior in the browser/Electron renderer. Evidence: const clearFunFactsButton = document.querySelector("#clear-fun-facts");
Browser DOM at line 22	Builder or website interface behavior in the browser/Electron renderer. Evidence: const siteHeroEyebrowInput = document.querySelector("#site-hero-eyebrow");
Browser DOM at line 23	Builder or website interface behavior in the browser/Electron renderer. Evidence: const siteHeroTitleInput = document.querySelector("#site-hero-title");
Browser DOM at line 24	Builder or website interface behavior in the browser/Electron renderer. Evidence: const siteHeroCopyInput = document.querySelector("#site-hero-copy");
Browser DOM at line 25	Builder or website interface behavior in the browser/Electron renderer. Evidence: const saveSiteContentButton = document.querySelector("#save-site-content");
Browser DOM at line 26	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileDisplayNameInput = document.querySelector("#profile-display-name");
Browser DOM at line 27	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profilePortfolioLabelInput = document.querySelector("#profile-portfolio-label");

Item	Explanation
Browser DOM at line 28	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileContactIntroInput = document.querySelector("#profile-contact-intro");
Browser DOM at line 29	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileEmailInput = document.querySelector("#profile-email");
Browser DOM at line 30	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profilePhoneInput = document.querySelector("#profile-phone");
Browser DOM at line 31	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileGithubInput = document.querySelector("#profile-github");
Browser DOM at line 32	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileLinkedinInput = document.querySelector("#profile-linkedin");
Browser DOM at line 33	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileWebsiteInput = document.querySelector("#profile-website");
Browser DOM at line 34	Builder or website interface behavior in the browser/Electron renderer. Evidence: const saveProfileContentButton = document.querySelector("#save-profile-content");
Browser DOM at line 35	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileImageUploadButton = document.querySelector("#profile-image-upload");
Browser DOM at line 36	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileHeroUploadButton = document.querySelector("#profile-hero-upload");
Browser DOM at line 37	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileResumeUploadButton = document.querySelector("#profile-resume-upload");
Browser DOM at line 38	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileBrandUploadButton = document.querySelector("#profile-brand-upload");
Browser DOM at line 39	Builder or website interface behavior in the browser/Electron renderer. Evidence: const profileAssetStatus = document.querySelector("#profile-asset-status");
Browser DOM at line 40	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryDropdown = document.querySelector("#category-dropdown");
Browser DOM at line 41	Builder or website interface behavior in the browser/Electron renderer. Evidence: const siteSectionList = document.querySelector("#site-section-list");
Browser DOM at line 42	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectCreateDialog = document.querySelector("#project-create-dialog");
Browser DOM at line 43	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectCreateForm = document.querySelector("#project-create-form");
Browser DOM at line 44	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectCreateCategory = document.querySelector("#project-create-category");
Browser DOM at line 45	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectCreateCancel = document.querySelector("#project-create-cancel");
Browser DOM at line 46	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryDialog = document.querySelector("#category-dialog");
Browser DOM at line 47	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryForm = document.querySelector("#category-form");
Browser DOM at line 48	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryTitle = document.querySelector("#category-title");
Browser DOM at line 49	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryDescription = document.querySelector("#category-description");
Browser DOM at line 50	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryAccent = document.querySelector("#category-accent");
Browser DOM at line 51	Builder or website interface behavior in the browser/Electron renderer. Evidence: const categoryCancel = document.querySelector("#category-cancel");
Browser DOM at line 52	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectTree = document.querySelector("#project-tree");
Browser DOM at line 53	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectSearchInput = document.querySelector("#project-search");
Browser DOM at line 54	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectSearchClear = document.querySelector("#project-search-clear");

Item	Explanation
Browser DOM at line 55	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectSearchStatus = document.querySelector("#project-search-status");
Browser DOM at line 56	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectDialog = document.querySelector("#project-dialog");
Browser DOM at line 57	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectWindowTitle = document.querySelector("#project-window-title");
Browser DOM at line 58	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectWindowClose = document.querySelector("#project-window-close");
Browser DOM at line 59	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectWindowDelete = document.querySelector("#project-window-delete");
Browser DOM at line 60	Builder or website interface behavior in the browser/Electron renderer. Evidence: const viewProjectPreviewButton = document.querySelector("#view-project-preview");
Browser DOM at line 61	Builder or website interface behavior in the browser/Electron renderer. Evidence: const saveProjectButton = document.querySelector("#save-project");
Browser DOM at line 62	Builder or website interface behavior in the browser/Electron renderer. Evidence: const saveProjectCloseButton = document.querySelector("#save-project-close");
Browser DOM at line 63	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectTitleMenu = document.querySelector("#project-title-menu");
Browser DOM at line 64	Builder or website interface behavior in the browser/Electron renderer. Evidence: const titleRenameActions = document.querySelector("#title-rename-actions");
Browser DOM at line 65	Builder or website interface behavior in the browser/Electron renderer. Evidence: const titleRenameSave = document.querySelector("#title-rename-save");
Browser DOM at line 66	Builder or website interface behavior in the browser/Electron renderer. Evidence: const titleRenameCancel = document.querySelector("#title-rename-cancel");
Browser DOM at line 67	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectMetaDialog = document.querySelector("#project-meta-dialog");
Browser DOM at line 68	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectMetaTitle = document.querySelector("#project-meta-title");
Browser DOM at line 69	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectMetaGrid = document.querySelector("#project-meta-grid");
Browser DOM at line 70	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectMetaClose = document.querySelector("#project-meta-close");
Browser DOM at line 71	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectFields = document.querySelector("#project-fields");
Browser DOM at line 72	Builder or website interface behavior in the browser/Electron renderer. Evidence: const sectionTabs = document.querySelector("#section-tabs");
Browser DOM at line 73	Builder or website interface behavior in the browser/Electron renderer. Evidence: const sectionContent = document.querySelector("#section-content");
Browser DOM at line 74	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectWindowContent = document.querySelector(".project-window-content");
Browser DOM at line 75	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectPreviewDialog = document.querySelector("#project-preview-dialog");
Browser DOM at line 76	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectPreviewTitle = document.querySelector("#project-preview-title");
Browser DOM at line 77	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectPreviewContent = document.querySelector("#project-preview-content");
Browser DOM at line 78	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectPreviewClose = document.querySelector("#project-preview-close");
Browser DOM at line 79	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectPreviewBack = document.querySelector("#project-preview-back");
Browser DOM at line 80	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectPreviewForward = document.querySelector("#project-preview-forward");

Important constants and variables

Item	Explanation
builderGuideOpen	Line 1: const builderGuideOpen = document.querySelector("#builder-guide-open");
builderGuideDialog	Line 2: const builderGuideDialog = document.querySelector("#builder-guide-dialog");
builderGuideClose	Line 3: const builderGuideClose = document.querySelector("#builder-guide-close");
builderGuideSearch	Line 4: const builderGuideSearch = document.querySelector("#builder-guide-search");
builderGuideResults	Line 5: const builderGuideResults = document.querySelector("#builder-guide-results");
builderGuideSections	Line 6: const builderGuideSections = document.querySelector("#builder-guide-sections");
builderGuideTopicDialog	Line 7: const builderGuideTopicDialog = document.querySelector("#builder-guide-topic-dialog");
builderGuideTopicTitle	Line 8: const builderGuideTopicTitle = document.querySelector("#builder-guide-topic-title");
builderGuideTopicBody	Line 9: const builderGuideTopicBody = document.querySelector("#builder-guide-topic-body");
builderGuideTopicClose	Line 10: const builderGuideTopicClose = document.querySelector("#builder-guide-topic-close");
templateSelect	Line 11: const templateSelect = document.querySelector("#template-select");
newCategorySelect	Line 12: const newCategorySelect = document.querySelector("#new-category-select");
placementSelect	Line 13: const placementSelect = document.querySelector("#placement-select");
newProjectTitle	Line 14: const newProjectTitle = document.querySelector("#new-project-title");
addProjectButton	Line 15: const addProjectButton = document.querySelector("#add-project");
addCategoryButton	Line 16: const addCategoryButton = document.querySelector("#add-category");
addSiteSectionButton	Line 17: const addSiteSectionButton = document.querySelector("#add-site-section");
funFactsInput	Line 18: const funFactsInput = document.querySelector("#fun-facts-input");
funFactsCount	Line 19: const funFactsCount = document.querySelector("#fun-facts-count");
saveFunFactsButton	Line 20: const saveFunFactsButton = document.querySelector("#save-fun-facts");
clearFunFactsButton	Line 21: const clearFunFactsButton = document.querySelector("#clear-fun-facts");
siteHeroEyebrowInput	Line 22: const siteHeroEyebrowInput = document.querySelector("#site-hero-eyebrow");
siteHeroTitleInput	Line 23: const siteHeroTitleInput = document.querySelector("#site-hero-title");
siteHeroCopyInput	Line 24: const siteHeroCopyInput = document.querySelector("#site-hero-copy");
saveSiteContentButton	Line 25: const saveSiteContentButton = document.querySelector("#save-site-content");
profileDisplayNameInput	Line 26: const profileDisplayNameInput = document.querySelector("#profile-display-name");
profilePortfolioLabelInput	Line 27: const profilePortfolioLabelInput = document.querySelector("#profile-portfolio-label");
profileContactIntroInput	Line 28: const profileContactIntroInput = document.querySelector("#profile-contact-intro");
profileEmailInput	Line 29: const profileEmailInput = document.querySelector("#profile-email");
profilePhoneInput	Line 30: const profilePhoneInput = document.querySelector("#profile-phone");
profileGithubInput	Line 31: const profileGithubInput = document.querySelector("#profile-github");
profileLinkedinInput	Line 32: const profileLinkedinInput = document.querySelector("#profile-linkedin");
profileWebsiteInput	Line 33: const profileWebsiteInput = document.querySelector("#profile-website");
saveProfileContentButton	Line 34: const saveProfileContentButton = document.querySelector("#save-profile-content");
profileImageUploadButton	Line 35: const profileImageUploadButton = document.querySelector("#profile-image-upload");
profileHeroUploadButton	Line 36: const profileHeroUploadButton = document.querySelector("#profile-hero-upload");
profileResumeUploadButton	Line 37: const profileResumeUploadButton = document.querySelector("#profile-resume-upload");
profileBrandUploadButton	Line 38: const profileBrandUploadButton = document.querySelector("#profile-brand-upload");
profileAssetStatus	Line 39: const profileAssetStatus = document.querySelector("#profile-asset-status");
categoryDropdown	Line 40: const categoryDropdown = document.querySelector("#category-dropdown");
siteSectionList	Line 41: const siteSectionList = document.querySelector("#site-section-list");

Item	Explanation
projectCreateDialog	Line 42: const projectCreateDialog = document.querySelector("#project-create-dialog");
projectCreateForm	Line 43: const projectCreateForm = document.querySelector("#project-create-form");
projectCreateCategory	Line 44: const projectCreateCategory = document.querySelector("#project-create-category");
projectCreateCancel	Line 45: const projectCreateCancel = document.querySelector("#project-create-cancel");
categoryDialog	Line 46: const categoryDialog = document.querySelector("#category-dialog");
categoryForm	Line 47: const categoryForm = document.querySelector("#category-form");
categoryTitle	Line 48: const categoryTitle = document.querySelector("#category-title");
categoryDescription	Line 49: const categoryDescription = document.querySelector("#category-description");
categoryAccent	Line 50: const categoryAccent = document.querySelector("#category-accent");
categoryCancel	Line 51: const categoryCancel = document.querySelector("#category-cancel");
projectTree	Line 52: const projectTree = document.querySelector("#project-tree");
projectSearchInput	Line 53: const projectSearchInput = document.querySelector("#project-search");
projectSearchClear	Line 54: const projectSearchClear = document.querySelector("#project-search-clear");
projectSearchStatus	Line 55: const projectSearchStatus = document.querySelector("#project-search-status");
projectDialog	Line 56: const projectDialog = document.querySelector("#project-dialog");
projectWindowTitle	Line 57: const projectWindowTitle = document.querySelector("#project-window-title");
projectWindowClose	Line 58: const projectWindowClose = document.querySelector("#project-window-close");
projectWindowDelete	Line 59: const projectWindowDelete = document.querySelector("#project-window-delete");
viewProjectPreviewButton	Line 60: const viewProjectPreviewButton = document.querySelector("#view-project-preview");
saveProjectButton	Line 61: const saveProjectButton = document.querySelector("#save-project");
saveProjectCloseButton	Line 62: const saveProjectCloseButton = document.querySelector("#save-project-close");
projectTitleMenu	Line 63: const projectTitleMenu = document.querySelector("#project-title-menu");
titleRenameActions	Line 64: const titleRenameActions = document.querySelector("#title-rename-actions");
titleRenameSave	Line 65: const titleRenameSave = document.querySelector("#title-rename-save");
titleRenameCancel	Line 66: const titleRenameCancel = document.querySelector("#title-rename-cancel");
projectMetaDialog	Line 67: const projectMetaDialog = document.querySelector("#project-meta-dialog");
projectMetaTitle	Line 68: const projectMetaTitle = document.querySelector("#project-meta-title");
projectMetaGrid	Line 69: const projectMetaGrid = document.querySelector("#project-meta-grid");
projectMetaClose	Line 70: const projectMetaClose = document.querySelector("#project-meta-close");
projectFields	Line 71: const projectFields = document.querySelector("#project-fields");
sectionTabs	Line 72: const sectionTabs = document.querySelector("#section-tabs");
sectionContent	Line 73: const sectionContent = document.querySelector("#section-content");
projectWindowContent	Line 74: const projectWindowContent = document.querySelector(".project-window-content");
projectPreviewDialog	Line 75: const projectPreviewDialog = document.querySelector("#project-preview-dialog");
projectPreviewTitle	Line 76: const projectPreviewTitle = document.querySelector("#project-preview-title");
projectPreviewContent	Line 77: const projectPreviewContent = document.querySelector("#project-preview-content");
projectPreviewClose	Line 78: const projectPreviewClose = document.querySelector("#project-preview-close");
projectPreviewBack	Line 79: const projectPreviewBack = document.querySelector("#project-preview-back");
projectPreviewForward	Line 80: const projectPreviewForward = document.querySelector("#project-preview-forward");

JSON/YAML-style keys detected

Item	Explanation
skin-light-blue	Line 433: "skin-light-blue": "appearance-light-blue-red-click",
skin-clean-white	Line 434: "skin-clean-white": "appearance-white-blue-click",
skin-deep-navy	Line 435: "skin-deep-navy": "appearance-deep-navy-cyan-click",
skin-red-warm	Line 436: "skin-red-warm": "appearance-warm-red-pale-click",
skin-emerald-instrument	Line 437: "skin-emerald-instrument": "appearance-emerald-mint-click",
skin-amber-power	Line 438: "skin-amber-power": "appearance-amber-cream-click",
skin-violet-mixed	Line 439: "skin-violet-mixed": "appearance-violet-lilac-click",
skin-graphite-asic	Line 440: "skin-graphite-asic": "appearance-graphite-white-click",
analog-opamp-topology	Line 441: "analog-opamp-topology": "appearance-light-blue-red-click",
analog-power-charger	Line 442: "analog-power-charger": "appearance-amber-cream-click",
analog-filter-front-end	Line 443: "analog-filter-front-end": "appearance-light-blue-red-click",
analog-sensor-interface	Line 444: "analog-sensor-interface": "appearance-emerald-mint-click",
analog-mixed-signal-timing	Line 445: "analog-mixed-signal-timing": "appearance-violet-lilac-click",
digital-fpga-pipeline	Line 446: "digital-fpga-pipeline": "appearance-deep-navy-cyan-click",
digital-asic-block	Line 447: "digital-asic-block": "appearance-graphite-white-click",
digital-verification-suite	Line 448: "digital-verification-suite": "appearance-graphite-white-click",
digital-interface-controller	Line 449: "digital-interface-controller": "appearance-deep-navy-cyan-click",
digital-signal-processing	Line 450: "digital-signal-processing": "appearance-violet-lilac-click",
embedded-stm32-sensor	Line 451: "embedded-stm32-sensor": "appearance-emerald-mint-click",
embedded-rtos-control	Line 452: "embedded-rtos-control": "appearance-graphite-white-click",
embedded-power-monitor	Line 453: "embedded-power-monitor": "appearance-amber-cream-click",
embedded-wireless-node	Line 454: "embedded-wireless-node": "appearance-deep-navy-cyan-click",
embedded-motor-control	Line 455: "embedded-motor-control": "appearance-violet-lilac-click"
engineering-file	Line 2512: "engineering-file": "Engineering file link",
google-drive	Line 2515: "google-drive": "Google Drive link",
Brief	Line 2656: "Brief": "Overview",
Project Brief	Line 2657: "Project Brief": "Overview",
Design Brief	Line 2658: "Design Brief": "Design Overview",
Simulation Brief	Line 2659: "Simulation Brief": "Simulation Overview"
--responsive-section-card-min	Line 2901: "--responsive-section-card-min": profile.sectionCardMin,
--responsive-content-max	Line 2902: "--responsive-content-max": profile.contentMax,
--responsive-media-max	Line 2903: "--responsive-media-max": profile.mediaMax,
--responsive-touch-target	Line 2904: "--responsive-touch-target": profile.touchTarget
--accent	Line 2919: "--accent": accent,
--template-bg	Line 2920: "--template-bg": visual.background,
--template-panel	Line 2921: "--template-panel": visual.panel,
--template-accent	Line 2922: "--template-accent": visual.accent,
--template-hover	Line 2923: "--template-hover": visual.hover,
--template-click	Line 2924: "--template-click": visual.click,
--template-click-text	Line 2925: "--template-click-text": visual.clickText,
--template-line	Line 2926: "--template-line": visual.line,

Item	Explanation
--template-text	Line 2927: "--template-text": visual.text,
--accent	Line 2952: "--accent": accent,
--template-bg	Line 2953: "--template-bg": visual.background,
--template-panel	Line 2954: "--template-panel": visual.panel,
--template-accent	Line 2955: "--template-accent": visual.accent,
--template-hover	Line 2956: "--template-hover": visual.hover,
--template-click	Line 2957: "--template-click": visual.click,
--template-click-text	Line 2958: "--template-click-text": visual.clickText,
--template-line	Line 2959: "--template-line": visual.line,
--template-text	Line 2960: "--template-text": visual.text,
compile-code	Line 7449: "compile-code": () => renderCompileCodeSection(project),

Event handlers and user interactions

Item	Explanation
Line 1048	document.addEventListener("pointerdown", beginDialogResize);
Line 1049	document.addEventListener("pointerdown", beginDialogDrag);
Line 1050	document.addEventListener("pointermove", moveDialogResize);
Line 1051	document.addEventListener("pointermove", moveDialogDrag);
Line 1052	document.addEventListener("pointerup", endDialogResize);
Line 1053	document.addEventListener("pointerup", endDialogDrag);
Line 1054	document.addEventListener("pointercancel", endDialogResize);
Line 1055	document.addEventListener("pointercancel", endDialogDrag);
Line 1056	document.addEventListener("click", (event) => {
Line 1061	document.addEventListener("click", (event) => {
Line 1067	dialog.addEventListener("close", () => {
Line 1076	window.addEventListener("resize", () => {
Line 1319	document.addEventListener("visibilitychange", () => {
Line 6166	siteLinkDialog.addEventListener("close", onClose);
Line 8325	dialog.addEventListener("close", onClose);
Line 8747	templateSelect.addEventListener("change", () => {
Line 8752	templatePreviewClose.addEventListener("click", () => {
Line 8880	builderGuideOpen.addEventListener("click", () => {
Line 8887	builderGuideClose.addEventListener("click", () => {
Line 8891	builderGuideTopicClose?.addEventListener("click", () => {
Line 8895	builderGuideTopicDialog?.addEventListener("close", () => {
Line 8899	builderGuideSearch?.addEventListener("input", filterBuilderGuide);
Line 8900	builderGuideDialog?.addEventListener("click", (event) => {
Line 8915	builderGuideDialog?.addEventListener("keydown", (event) => {
Line 8923	projectWindowTitle.addEventListener("dblclick", (event) => {
Line 8929	projectWindowTitle.addEventListener("keydown", (event) => {
Line 8946	projectWindowTitle.addEventListener("contextmenu", (event) => {

Item	Explanation
Line 8954	projectTitleMenu.addEventListener("click", async (event) => {
Line 8991	titleRenameSave.addEventListener("click", () => endTitleRename(true));
Line 8992	titleRenameCancel.addEventListener("click", () => endTitleRename(false));
Line 8993	projectMetaClose.addEventListener("click", () => closeDialogElement(projectMetaDialog));
Line 8995	viewProjectPreviewButton.addEventListener("click", openProjectPortfolioPreview);
Line 8996	projectPreviewClose.addEventListener("click", closeProjectPreviewWindow);
Line 8997	projectPreviewDialog.addEventListener("close", () => {
Line 9001	projectPreviewBack.addEventListener("click", projectPreviewBackStep);
Line 9002	projectPreviewForward.addEventListener("click", projectPreviewForwardStep);
Line 9004	projectPreviewContent.addEventListener("click", (event) => {
Line 9012	saveProjectButton.addEventListener("click", saveSelectedProjectToPortfolio);
Line 9013	saveProjectCloseButton.addEventListener("click", saveSelectedProjectAndClose);
Line 9014	openPortfolioPreviewButton.addEventListener("click", openPortfolioPreview);
Line 9015	saveAllSectionsButton.addEventListener("click", saveAllSections);
Line 9016	portfolioPreviewClose.addEventListener("click", () => {
Line 9019	portfolioPreviewDialog.addEventListener("close", () => {
Line 9022	publishTargetOpen?.addEventListener("click", openPublishTargetDialog);
Line 9023	publishTargetClose?.addEventListener("click", () => {
Line 9026	publishTargetCancel?.addEventListener("click", () => {
Line 9029	publishTargetForm?.addEventListener("submit", savePublishTarget);
Line 9030	publishTargetSync?.addEventListener("click", syncFromPublishTarget);
Line 9031	publishTargetCheck?.addEventListener("click", loadSystemReadiness);
Line 9032	publishTargetInstallGit?.addEventListener("click", installGitForPublishing);
Line 9033	publishTargetRepository?.addEventListener("input", () => {
Line 9036	publishTargetDomain?.addEventListener("input", () => {
Line 9040	publishTargetUsername?.addEventListener("input", markPublishTargetNeedsAuthentication);
Line 9041	publishTargetPassword?.addEventListener("input", markPublishTargetNeedsAuthentication);
Line 9042	publishResultClose.addEventListener("click", () => {
Line 9046	appUpdateClose?.addEventListener("click", () => {
Line 9050	appUpdateLater?.addEventListener("click", () => {
Line 9058	appUpdateSkip?.addEventListener("click", () => {
Line 9065	appUpdateApply?.addEventListener("click", () => {
Line 9069	appUpdateCheckButton?.addEventListener("click", () => {
Line 9073	securityReportOpen?.addEventListener("click", openSecurityReport);
Line 9074	securityReportRefresh?.addEventListener("click", openSecurityReport);
Line 9075	securityReportClose?.addEventListener("click", () => {
Line 9078	securityReportOk?.addEventListener("click", () => {
Line 9082	addProjectButton.addEventListener("click", () => {
Line 9086	projectSearchInput?.addEventListener("input", () => {
Line 9091	projectSearchClear?.addEventListener("click", () => {
Line 9100	quickOpenSelected?.addEventListener("click", () => {

Item	Explanation
Line 9106	quickPreviewSelected?.addEventListener("click", () => {
Line 9112	addCategoryButton?.addEventListener("click", openCategoryDialog);
Line 9113	categoryCancel?.addEventListener("click", () => closeDialogElement(categoryDialog, "cancel"));
Line 9114	categoryForm?.addEventListener("submit", (event) => {
Line 9120	addSiteSectionButton.addEventListener("click", addSiteSection);
Line 9122	funFactsInput?.addEventListener("input", () => {
Line 9128	saveFunFactsButton?.addEventListener("click", () => {
Line 9135	clearFunFactsButton?.addEventListener("click", () => {
Line 9146	input.addEventListener("input", () => {
Line 9154	saveSiteContentButton?.addEventListener("click", () => {
Line 9171	input.addEventListener("input", () => {
Line 9179	saveProfileContentButton?.addEventListener("click", () => {

Function-by-function detail

isHdlLanguage (template-preview.js, lines 240-242)

isHdlLanguage part of the private builder frontend and editing experience. In this file, it appears around lines 240-242 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes, and normalizeCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 241: return ["verilog", "systemverilog"].includes(normalizeCodeLanguage(language));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
240: function isHdlLanguage(language = "") {
241:   return ["verilog", "systemverilog"].includes(normalizeCodeLanguage(language));
242: }
```

inferCompileFileRole (template-preview.js, lines 244-251)

inferCompileFileRole part of the compile code language/tool/build/run flow. In this file, it appears around lines 244-251 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isHdlLanguage, toLowerCase, b, and tb. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 245: if (!isHdlLanguage(language)) return "source";; line 248: return "testbench";; line 250: return "design";.

Important local values include haystack at line 246. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
244: function inferCompileFileRole(fileName = "", code = "", language = "") {
245:   if (!isHdlLanguage(language)) return "source";
246:   const haystack = `${fileName}\n${code}`.toLowerCase();
247:   if (/\b(tb|testbench)\b|(^|[_-])tb([_-]|\.)|test[_-]?bench|\$dumpfile|\$dumpvars|module\s+tb[_a-z0-
9]*/i.test(haystack)) {
248:     return "testbench";
249:   }
250:   return "design";
251: }
```

normalizeCompileFileRole (template-preview.js, lines 253-257)

normalizeCompileFileRole part of the compile code language/tool/build/run flow. In this file, it appears around lines 253-257 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isHdlLanguage, trim, toLowerCase, replace, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 254: if (!isHdlLanguage(language)) return "source";; line 256: return ["tb", "testbench", "bench"].includes(clean) ? "testbench" : "design";.

Important local values include clean at line 255. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
253: function normalizeCompileFileRole(value = "", language = "") {
254:   if (!isHdlLanguage(language)) return "source";
255:   const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");
256:   return ["tb", "testbench", "bench"].includes(clean) ? "testbench" : "design";
257: }
```

compileFileRoleLabel (template-preview.js, lines 259-263)

compileFileRoleLabel part of the compile code language/tool/build/run flow. In this file, it appears around lines 259-263 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 260: if (role === "testbench") return "Testbench";; line 261: if (role === "design") return "Design";; line 262: return "Program source";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
259: function compileFileRoleLabel(role = "source") {
260:   if (role === "testbench") return "Testbench";
261:   if (role === "design") return "Design";
262:   return "Program source";
263: }
```

setStatus (template-preview.js, lines 509-511)

setStatus part of the private builder frontend and editing experience. In this file, it appears around lines 509-511 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
509: function setStatus(message) {
510:   builderStatus.textContent = message;
511: }
```

setSaveState (template-preview.js, lines 513-517)

setSaveState part of the private builder frontend and editing experience. In this file, it appears around lines 513-517 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 514: if (!builderSaveState) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
513: function setSaveState(state, message) {
514:   if (!builderSaveState) return;
515:   builderSaveState.dataset.state = state || "loaded";
516:   builderSaveState.textContent = message || "Loaded";
517: }
```

projectSearchText (template-preview.js, lines 519-521)

projectSearchText part of the private builder frontend and editing experience. In this file, it appears around lines 519-521 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, replace, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 520: return String(value || "").toLowerCase().replace(/\s+/g, " ").trim();.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
519: function projectSearchText(value = "") {
520:   return String(value || "").toLowerCase().replace(/\s+/g, " ").trim();
521: }
```

regexEscape (template-preview.js, lines 523-525)

regexEscape part of the private builder frontend and editing experience. In this file, it appears around lines 523-525 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 524: return String(value).replace(/[.*+?^${}()|[\]\\]/g, "\\$&");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
523: function regexEscape(value = "") {
524:   return String(value).replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
525: }
```

highlightSearchText (template-preview.js, lines 527-533)

highlightSearchText part of the private builder frontend and editing experience. In this file, it appears around lines 527-533 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectSearchText, escapeHtml, regexEscape, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 530: if (!cleanQuery) return escapeHtml(text);; line 532: return escapeHtml(text).replace(matcher, "<mark>$1</mark>");.

Important local values include text at line 528; cleanQuery at line 529; matcher at line 531. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
527: function highlightSearchText(value = "", query = "") {
```

```
528:   const text = String(value || "");
529:   const cleanQuery = projectSearchText(query);
530:   if (!cleanQuery) return escapeHtml(text);
531:   const matcher = new RegExp(`(${regexEscape(cleanQuery)})`, "ig");
532:   return escapeHtml(text).replace(matcher, "<mark>$1</mark>");
533: }
```

projectSearchHaystack (template-preview.js, lines 535-555)

projectSearchHaystack part of the private builder frontend and editing experience. In this file, it appears around lines 535-555 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectSearchText, stringify, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 536: return projectSearchText([.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
535: function projectSearchHaystack(project = {}, category = {}) {
536:   return projectSearchText([
537:     category.label,
538:     category.description,
539:     project.title,
540:     project.summary,
541:     project.status,
542:     project.category,
543:     project.templateLabel,
544:     ...(project.focus || []),
545:     ...(project.highlights || []),
546:     JSON.stringify(project.sections || []),
547:     JSON.stringify(project.design || {}),
548:     JSON.stringify(project.documents || []),
549:     JSON.stringify(project.tests || []),
550:     JSON.stringify(project.tools || []),
551:     JSON.stringify(project.links || []),
552:     JSON.stringify(project.media || []),
553:     JSON.stringify(project.compileCode || {})
554:   ].filter(Boolean).join(" "));
555: }
```

projectMatchesSearch (template-preview.js, lines 557-561)

projectMatchesSearch part of the private builder frontend and editing experience. In this file, it appears around lines 557-561 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectSearchText, projectSearchHaystack, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 559: if (!cleanQuery) return true;; line 560: return projectSearchHaystack(project, category).includes(cleanQuery);.

Important local values include cleanQuery at line 558. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
557: function projectMatchesSearch(project, category, query) {
558:   const cleanQuery = projectSearchText(query);
559:   if (!cleanQuery) return true;
560:   return projectSearchHaystack(project, category).includes(cleanQuery);
561: }
```

updateProjectSearchStatus (template-preview.js, lines 563-575)

updateProjectSearchStatus updates ui, state, cache, or stored data. In this file, it appears around lines 563-575 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectSearchText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 569: if (!projectSearchStatus) return;; line 572: return;.

Important local values include cleanQuery at line 564. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  563: function updateProjectSearchStatus(matchCount = catalog.projects.length) {
  564:   const cleanQuery = projectSearchText(projectSearchQuery);
  565:   if (projectSearchInput && document.activeElement !== projectSearchInput) {
  566:     projectSearchInput.value = projectSearchQuery;
  567:   }
  568:   if (projectSearchClear) projectSearchClear.hidden = !cleanQuery;
  569:   if (!projectSearchStatus) return;
  570:   if (!cleanQuery) {
  571:     projectSearchStatus.textContent = "Showing all projects.";
  572:     return;
  573:   }
  574:   projectSearchStatus.textContent = `${matchCount} matching project${matchCount === 1 ? "" : "s"} for
"${projectSearchQuery}".`;
  575: }
```

updateBuilderWorkflow (template-preview.js, lines 577-597)

updateBuilderWorkflow updates ui, state, cache, or stored data. In this file, it appears around lines 577-597 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, Set, map, filter, has, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include project at line 578; savedIds at line 579; savedCount at line 580; visibleSections at line 581. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  577: function updateBuilderWorkflow() {
  578:   const project = selectedProject();
  579:   const savedIds = new Set((savedPortfolioCatalog.projects || []).map((item) => item.id));
  580:   const savedCount = (catalog.projects || []).filter((projectItem) =>
savedIds.has(projectItem.id)).length;
  581:   const visibleSections = (catalog.siteSections || []).filter(siteSectionRenderable).length;
  582:   if (workflowSelectedProject) {
  583:     workflowSelectedProject.textContent = project ? project.title || "Untitled project" : "No project
selected";
  584:   }
  585:   if (workflowTotalProjects) {
  586:     workflowTotalProjects.textContent = `${catalog.projects.length} project${catalog.projects.length ===
1 ? "" : "s"}`;
  587:   }
  588:   if (workflowSavedProjects) {
  589:     workflowSavedProjects.textContent = `${savedCount} parsed`;
  590:   }
  591:   if (workflowVisibleSections) {
  592:     workflowVisibleSections.textContent = `${visibleSections} live section${visibleSections === 1 ? "" :
"s"}`;
  593:   }
  594:   [quickOpenSelected, quickPreviewSelected].forEach((button) => {
  595:     if (button) button.disabled = !project;
  596:   });
  597: }
```

savedCount (template-preview.js, lines 580-584)

savedCount persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 580-584 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, and has. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include savedCount at line 580; visibleSections at line 581. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  580:   const savedCount = (catalog.projects || []).filter((projectItem) =>
savedIds.has(projectItem.id)).length;
  581:   const visibleSections = (catalog.siteSections || []).filter(siteSectionRenderable).length;
  582:   if (workflowSelectedProject) {
  583:     workflowSelectedProject.textContent = project ? project.title || "Untitled project" : "No project
selected";
  584:   }
```

dialogDragHandles (template-preview.js, lines 599-604)

dialogDragHandles part of the private builder frontend and editing experience. In this file, it appears around lines 599-604 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 600: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
  599: function dialogDragHandles(dialog) {
  600:   return [
  601:     ...dialog.querySelectorAll(".project-dialog-heading, .project-preview-heading, .template-dialog-
heading"),
  602:     ...dialog.querySelectorAll(".asset-dialog form > div:first-child")
  603:   ];
  604: }
```

dialogWindowActionTarget (template-preview.js, lines 606-608)

dialogWindowActionTarget part of the private builder frontend and editing experience. In this file, it appears around lines 606-608 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 607: return handle.querySelector(".project-window-actions, .section-view-actions") || handle;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
  606: function dialogWindowActionTarget(handle) {
  607:   return handle.querySelector(".project-window-actions, .section-view-actions") || handle;
  608: }
```

updateDialogWindowButtons (template-preview.js, lines 610-635)

updateDialogWindowButtons updates ui, state, cache, or stored data. In this file, it appears around lines 610-635 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, querySelectorAll, forEach, setAttribute, dialogCanStepBack, and dialogCanStepForward. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include isMinimized at line 611; isHidden at line 612; isMaximized at line 613. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
610: function updateDialogWindowButtons(dialog) {
611:   const isMinimized = dialog.classList.contains("is-minimized-dialog");
612:   const isHidden = dialog.classList.contains("is-hidden-dialog");
613:   const isMaximized = dialog.classList.contains("is-maximized-dialog");
614:   dialog.querySelectorAll("[data-dialog-minimize='true']").forEach((button) => {
615:     button.textContent = isMinimized ? "+" : "\u2212";
616:     button.title = isMinimized ? "Restore window" : "Minimize window";
617:     button.setAttribute("aria-label", isMinimized ? "Restore window" : "Minimize window");
618:   });
619:   dialog.querySelectorAll("[data-dialog-hide='true']").forEach((button) => {
620:     button.textContent = isHidden ? "\u25b4" : "\u25be";
621:     button.title = isHidden ? "Show window" : "Hide window";
622:     button.setAttribute("aria-label", isHidden ? "Show window" : "Hide window");
623:   });
624:   dialog.querySelectorAll("[data-dialog-maximize='true']").forEach((button) => {
625:     button.textContent = isMaximized ? "\u2750" : "\u25a1";
626:     button.title = isMaximized ? "Restore size" : "Maximize window";
627:     button.setAttribute("aria-label", isMaximized ? "Restore size" : "Maximize window");
628:   });
629:   dialog.querySelectorAll("[data-dialog-back='true']").forEach((button) => {
630:     button.disabled = !dialogCanStepBack(dialog);
631:   });
632:   dialog.querySelectorAll("[data-dialog-forward='true']").forEach((button) => {
633:     button.disabled = !dialogCanStepForward(dialog);
634:   });
635: }
```

dialogTitleForDock (template-preview.js, lines 637-639)

dialogTitleForDock part of the private builder frontend and editing experience. In this file, it appears around lines 637-639 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 638: return dialog.querySelector("h2")?.textContent?.trim() || "Window";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
637: function dialogTitleForDock(dialog) {
638:   return dialog.querySelector("h2")?.textContent?.trim() || "Window";
639: }
```

makeDialogControl (template-preview.js, lines 641-650)

makeDialogControl part of the private builder frontend and editing experience. In this file, it appears around lines 641-650 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, and setAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 649: return button;.

Important local values include button at line 642. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
641: function makeDialogControl(action, label, title) {
642:   const button = document.createElement("button");
643:   button.type = "button";
644:   button.className = `dialog-window-control dialog-window-${action}-control`;
645:   button.dataset.dialogAction = action;
646:   button.textContent = label;
647:   button.title = title;
648:   button.setAttribute("aria-label", title);
649:   return button;
650: }
```

ensureDialogLeftControls (template-preview.js, lines 652-674)

ensureDialogLeftControls part of the private builder frontend and editing experience. In this file, it appears around lines 652-674 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, createElement, makeDialogControl, append, prepend, forEach, toUpperCase, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 653: if (handle.querySelector(".dialog-window-left-controls")) return;.

Important local values include titlebar at line 654; controls at line 655; refresh at line 659. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  652: function ensureDialogLeftControls(dialog, handle) {
  653:   if (handle.querySelector(".dialog-window-left-controls")) return;
  654:   const titlebar = handle.querySelector(".section-view-titlebar");
  655:   const controls = document.createElement("div");
  656:   controls.className = "dialog-window-left-controls";
  657:
  658:   if (titlebar?.querySelector(".section-view-back, .section-view-forward")) {
  659:     const refresh = makeDialogControl("refresh", "\u21bb", "Refresh window");
  660:     refresh.dataset.dialogRefresh = "true";
  661:     controls.append(refresh);
  662:     titlebar.prepend(controls);
  663:   } else {
  664:     [
  665:       makeDialogControl("back", "\u2190", "Back"),
  666:       makeDialogControl("forward", "\u2192", "Forward"),
  667:       makeDialogControl("refresh", "\u21bb", "Refresh window")
  668:     ].forEach((button) => {
  669:
button.dataset[`dialog${button.dataset.dialogAction[0].toUpperCase()}${button.dataset.dialogAction.slice(1)}`] =
 "true";
  670:       controls.append(button);
  671:     });
  672:     handle.prepend(controls);
  673:   }
  674: }
```

ensureDialogWindowControls (template-preview.js, lines 676-720)

ensureDialogWindowControls part of the private builder frontend and editing experience. In this file, it appears around lines 676-720 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, remove, ensureDialogLeftControls, dialogWindowActionTarget, querySelector, createElement, append, makeDialogControl, find, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include target at line 679; cluster at line 680; hide at line 688; minimize at line 693; maximize at line 698; closeButton at line 703; cloneClose at line 714. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  676: function ensureDialogWindowControls(dialog, handle) {
  677:   dialog.querySelectorAll(".dialog-minimize-button").forEach((button) => button.remove());
  678:   ensureDialogLeftControls(dialog, handle);
  679:   const target = dialogWindowActionTarget(handle);
  680:   let cluster = target.querySelector(".dialog-window-control-cluster");
  681:   if (!cluster) {
  682:     cluster = document.createElement("div");
  683:     cluster.className = "dialog-window-control-cluster";
  684:     target.append(cluster);
  685:   }
  686:
  687:   if (!cluster.querySelector("[data-dialog-hide='true']")) {
  688:     const hide = makeDialogControl("hide", "\u25be", "Hide window");
  689:     hide.dataset.dialogHide = "true";
  690:     cluster.append(hide);
  691:   }
  692:   if (!cluster.querySelector("[data-dialog-minimize='true']")) {
  693:     const minimize = makeDialogControl("minimize", "\u2212", "Minimize window");
  694:     minimize.dataset.dialogMinimize = "true";
  695:     cluster.append(minimize);
```

```
  696:   }
  697:   if (!cluster.querySelector("[data-dialog-maximize='true']")) {
  698:     const maximize = makeDialogControl("maximize", "\u25a1", "Maximize window");
  699:     maximize.dataset.dialogMaximize = "true";
  700:     cluster.append(maximize);
  701:   }
  702:
  703:   let closeButton = cluster.querySelector(".project-window-close, .close-control, [data-dialog-
close='true']");
  704:   if (!closeButton) {
  705:     closeButton = [...dialog.querySelectorAll(".project-window-close, .close-control")]
  706:       .find((button) => button.closest(".project-dialog-heading, .project-preview-heading, .template-
dialog-heading, .asset-dialog form > div:first-child") === handle);
  707:   }
  708:   if (closeButton) {
  709:     closeButton.classList.add("dialog-window-control", "dialog-close-button");
  710:     closeButton.dataset.dialogAction = "close";
  711:     closeButton.dataset.dialogClose = "true";
  712:     cluster.append(closeButton);
  713:   } else {
  714:     const cloneClose = makeDialogControl("close", "\u00d7", "Close window");
  715:     cloneClose.classList.add("project-window-close", "dialog-close-button");
  716:     cloneClose.dataset.dialogClose = "true";
  717:     cluster.append(cloneClose);
  718:   }
  719:   updateDialogWindowButtons(dialog);
  720: }
```

ensureDialogResizeHandles (template-preview.js, lines 722-731)

ensureDialogResizeHandles part of the private builder frontend and editing experience. In this file, it appears around lines 722-731 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, forEach, createElement, setAttribute, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 723: if (dialog.querySelector("[data-dialog-resize-handle]")) return;.

Important local values include handle at line 725. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  722: function ensureDialogResizeHandles(dialog) {
  723:   if (dialog.querySelector("[data-dialog-resize-handle]")) return;
  724:   ["n", "e", "s", "w", "ne", "nw", "se", "sw"].forEach((direction) => {
  725:     const handle = document.createElement("div");
  726:     handle.className = `dialog-resize-handle resize-${direction}`;
  727:     handle.dataset.dialogResizeHandle = direction;
  728:     handle.setAttribute("aria-hidden", "true");
  729:     dialog.append(handle);
  730:   });
  731: }
```

markDialogDragHandles (template-preview.js, lines 733-740)

markDialogDragHandles part of the private builder frontend and editing experience. In this file, it appears around lines 733-740 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references dialogDragHandles, forEach, ensureDialogWindowControls, and ensureDialogResizeHandles. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
  733: function markDialogDragHandles(dialog) {
  734:   dialogDragHandles(dialog).forEach((handle) => {
  735:     handle.dataset.dialogDragHandle = "true";
  736:     handle.title = handle.title || "Drag to move this window";
  737:     ensureDialogWindowControls(dialog, handle);
  738:   });
  739:   ensureDialogResizeHandles(dialog);
  740: }
```

draggableDialogFromEvent (template-preview.js, lines 742-751)

draggableDialogFromEvent part of the private builder frontend and editing experience. In this file, it appears around lines 742-751 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, and contains. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 743: if (event.target.closest("[data-dialog-resize-handle]")) return null;; line 745: if (!handle) return null;; line 747: if (!dialog?.open) return null;; line 748: if (dialog.classList.contains("is-hidden-dialog")) return null;. There are 2 additional return statements in the function body.

Important local values include handle at line 744; dialog at line 746. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  742: function draggableDialogFromEvent(event) {
  743:   if (event.target.closest("[data-dialog-resize-handle]")) return null;
  744:   const handle = event.target.closest("[data-dialog-drag-handle='true']");
  745:   if (!handle) return null;
  746:   const dialog = handle.closest("dialog");
  747:   if (!dialog?.open) return null;
  748:   if (dialog.classList.contains("is-hidden-dialog")) return null;
  749:   if (event.target.closest("button, a, input, textarea, select, label, summary,
[contenteditable='true']")) return null;
  750:   return dialog;
  751: }
```

clampDialogPosition (template-preview.js, lines 753-762)

clampDialogPosition part of the private builder frontend and editing experience. In this file, it appears around lines 753-762 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getBoundingClientRect, max, and min. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 758: return {.

Important local values include rect at line 754; margin at line 755; maxLeft at line 756; maxTop at line 757. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  753: function clampDialogPosition(left, top, dialog) {
  754:   const rect = dialog.getBoundingClientRect();
  755:   const margin = 12;
  756:   const maxLeft = Math.max(margin, window.innerWidth - rect.width - margin);
  757:   const maxTop = Math.max(margin, window.innerHeight - rect.height - margin);
  758:   return {
  759:     left: Math.min(Math.max(left, margin), maxLeft),
  760:     top: Math.min(Math.max(top, margin), maxTop)
  761:   };
  762: }
```

anchorDialogForDrag (template-preview.js, lines 764-773)

anchorDialogForDrag part of the private builder frontend and editing experience. In this file, it appears around lines 764-773 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getBoundingClientRect, clampDialogPosition, and add. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include rect at line 765; position at line 766. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
764: function anchorDialogForDrag(dialog) {
765:   const rect = dialog.getBoundingClientRect();
766:   const position = clampDialogPosition(rect.left, rect.top, dialog);
767:   dialog.classList.add("is-draggable-dialog");
768:   dialog.style.left = `${position.left}px`;
769:   dialog.style.top = `${position.top}px`;
770:   dialog.style.right = "auto";
771:   dialog.style.bottom = "auto";
772:   dialog.style.margin = "0";
773: }
```

saveDialogRestoreState (template-preview.js, lines 775-788)

saveDialogRestoreState persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 775-788 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getBoundingClientRect, and stringify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 776: if (dialog.dataset.restoreWindowState) return;.

Important local values include rect at line 777. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
775: function saveDialogRestoreState(dialog) {
776:   if (dialog.dataset.restoreWindowState) return;
777:   const rect = dialog.getBoundingClientRect();
778:   dialog.dataset.restoreWindowState = JSON.stringify({
779:     bottom: dialog.style.bottom || "",
780:     height: dialog.style.height || `${rect.height}px`,
781:     left: dialog.style.left || `${rect.left}px`,
782:     margin: dialog.style.margin || "",
783:     maxHeight: dialog.style.maxHeight || "",
784:     right: dialog.style.right || "",
785:     top: dialog.style.top || `${rect.top}px`,
786:     width: dialog.style.width || `${rect.width}px`
787:   });
788: }
```

restoreDialogWindowState (template-preview.js, lines 790-805)

restoreDialogWindowState part of the private builder frontend and editing experience. In this file, it appears around lines 790-805 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, removeAttribute, entries, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 800: return;.

Important local values include state at line 791. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
790: function restoreDialogWindowState(dialog) {
791:   let state = null;
792:   try {
793:     state = JSON.parse(dialog.dataset.restoreWindowState || "null");
794:   } catch {
795:     state = null;
796:   }
797:   delete dialog.dataset.restoreWindowState;
798:   if (!state) {
799:     dialog.removeAttribute("style");
800:     return;
801:   }
802:   Object.entries(state).forEach(([key, value]) => {
803:     dialog.style[key] = value;
804:   });
805: }
```

anchorDialogForResize (template-preview.js, lines 807-814)

anchorDialogForResize part of the private builder frontend and editing experience. In this file, it appears around lines 807-814 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references anchorDialogForDrag, getBoundingClientRect, and add. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include rect at line 809. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
807: function anchorDialogForResize(dialog) {
808:   anchorDialogForDrag(dialog);
809:   const rect = dialog.getBoundingClientRect();
810:   dialog.classList.add("is-resized-dialog");
811:   dialog.style.width = `${rect.width}px`;
812:   dialog.style.height = `${rect.height}px`;
813:   dialog.style.maxHeight = "none";
814: }
```

resizeDialogBounds (template-preview.js, lines 816-843)

resizeDialogBounds part of the private builder frontend and editing experience. In this file, it appears around lines 816-843 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references min, max, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 842: return { height, left, top, width };.

Important local values include margin at line 817; direction at line 818; minWidth at line 819; minHeight at line 820; dx at line 822; dy at line 823; right at line 832; bottom at line 837. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
816: function resizeDialogBounds(state, event) {
817:   const margin = 12;
818:   const direction = state.direction;
819:   const minWidth = Math.min(320, Math.max(180, window.innerWidth - margin * 2));
820:   const minHeight = Math.min(170, Math.max(120, window.innerHeight - margin * 2));
821:   let { left, top, width, height } = state.rect;
822:   const dx = event.clientX - state.startX;
823:   const dy = event.clientY - state.startY;
824:
825:   if (direction.includes("e")) {
826:     width = Math.min(Math.max(state.rect.width + dx, minWidth), window.innerWidth - left - margin);
827:   }
828:   if (direction.includes("s")) {
829:     height = Math.min(Math.max(state.rect.height + dy, minHeight), window.innerHeight - top - margin);
830:   }
831:   if (direction.includes("w")) {
832:     const right = state.rect.left + state.rect.width;
833:     left = Math.min(Math.max(state.rect.left + dx, margin), right - minWidth);
834:     width = right - left;
835:   }
836:   if (direction.includes("n")) {
837:     const bottom = state.rect.top + state.rect.height;
838:     top = Math.min(Math.max(state.rect.top + dy, margin), bottom - minHeight);
839:     height = bottom - top;
840:   }
841:
842:   return { height, left, top, width };
843: }
```

beginDialogResize (template-preview.js, lines 845-863)

beginDialogResize part of the private builder frontend and editing experience. In this file, it appears around lines 845-863 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, contains, anchorDialogForResize, getBoundingClientRect, add, and preventDefault. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 846: if (event.button !== 0) return;; line 849: if (!handle || !dialog?.open || dialog.classList.contains("is-minimized-dialog") || dialog.classList.contains("is-hidden-dialog")) return;.

Important local values include handle at line 847; dialog at line 848. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  845: function beginDialogResize(event) {
  846:   if (event.button !== 0) return;
  847:   const handle = event.target.closest("[data-dialog-resize-handle]");
  848:   const dialog = handle?.closest("dialog");
  849:   if (!handle || !dialog?.open || dialog.classList.contains("is-minimized-dialog") ||
dialog.classList.contains("is-hidden-dialog")) return;
  850:   anchorDialogForResize(dialog);
  851:   activeDialogResize = {
  852:     dialog,
  853:     direction: handle.dataset.dialogResizeHandle,
  854:     pointerId: event.pointerId,
  855:     pointerTarget: event.target,
  856:     rect: dialog.getBoundingClientRect(),
  857:     startX: event.clientX,
  858:     startY: event.clientY
  859:   };
  860:   event.target.setPointerCapture?.(event.pointerId);
  861:   dialog.classList.add("is-resizing-dialog");
  862:   event.preventDefault();
  863: }
```

moveDialogResize (template-preview.js, lines 865-872)

moveDialogResize part of the private builder frontend and editing experience. In this file, it appears around lines 865-872 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references resizeDialogBounds. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 866: if (!activeDialogResize) return;.

Important local values include next at line 867. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  865: function moveDialogResize(event) {
  866:   if (!activeDialogResize) return;
  867:   const next = resizeDialogBounds(activeDialogResize, event);
  868:   activeDialogResize.dialog.style.left = `${next.left}px`;
  869:   activeDialogResize.dialog.style.top = `${next.top}px`;
  870:   activeDialogResize.dialog.style.width = `${next.width}px`;
  871:   activeDialogResize.dialog.style.height = `${next.height}px`;
  872: }
```

endDialogResize (template-preview.js, lines 874-879)

endDialogResize part of the private builder frontend and editing experience. In this file, it appears around lines 874-879 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 875: if (!activeDialogResize) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
  874: function endDialogResize() {
  875:   if (!activeDialogResize) return;
  876:   activeDialogResize.pointerTarget?.releasePointerCapture?.(activeDialogResize.pointerId);
  877:   activeDialogResize.dialog.classList.remove("is-resizing-dialog");
  878:   activeDialogResize = null;
```

```
879: }
```

beginDialogDrag (template-preview.js, lines 881-897)

beginDialogDrag part of the private builder frontend and editing experience. In this file, it appears around lines 881-897 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references draggableDialogFromEvent, anchorDialogForDrag, getBoundingClientRect, add, and preventDefault. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 882: if (event.button !== 0) return;; line 884: if (!dialog) return;.

Important local values include dialog at line 883; rect at line 886. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
881: function beginDialogDrag(event) {
882:   if (event.button !== 0) return;
883:   const dialog = draggableDialogFromEvent(event);
884:   if (!dialog) return;
885:   anchorDialogForDrag(dialog);
886:   const rect = dialog.getBoundingClientRect();
887:   activeDialogDrag = {
888:     dialog,
889:     offsetX: event.clientX - rect.left,
890:     offsetY: event.clientY - rect.top,
891:     pointerId: event.pointerId,
892:     pointerTarget: event.target
893:   };
894:   event.target.setPointerCapture?.(event.pointerId);
895:   dialog.classList.add("is-dragging-dialog");
896:   event.preventDefault();
897: }
```

moveDialogDrag (template-preview.js, lines 899-908)

moveDialogDrag part of the private builder frontend and editing experience. In this file, it appears around lines 899-908 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clampDialogPosition. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 900: if (!activeDialogDrag) return;.

Important local values include next at line 901. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
899: function moveDialogDrag(event) {
900:   if (!activeDialogDrag) return;
901:   const next = clampDialogPosition(
902:     event.clientX - activeDialogDrag.offsetX,
903:     event.clientY - activeDialogDrag.offsetY,
904:     activeDialogDrag.dialog
905:   );
906:   activeDialogDrag.dialog.style.left = `${next.left}px`;
907:   activeDialogDrag.dialog.style.top = `${next.top}px`;
908: }
```

endDialogDrag (template-preview.js, lines 910-915)

endDialogDrag part of the private builder frontend and editing experience. In this file, it appears around lines 910-915 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 911: if (!activeDialogDrag) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
910: function endDialogDrag() {
911:   if (!activeDialogDrag) return;
912:   activeDialogDrag.pointerTarget?.releasePointerCapture?.(activeDialogDrag.pointerId);
913:   activeDialogDrag.dialog.classList.remove("is-dragging-dialog");
914:   activeDialogDrag = null;
915: }
```

toggleDialogMinimized (template-preview.js, lines 917-927)

toggleDialogMinimized part of the private builder frontend and editing experience. In this file, it appears around lines 917-927 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, toggleDialogHidden, anchorDialogForDrag, remove, toggle, and updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 918: if (!dialog) return;; line 921: return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
917: function toggleDialogMinimized(dialog) {
918:   if (!dialog) return;
919:   if (dialog.classList.contains("is-hidden-dialog")) {
920:     toggleDialogHidden(dialog);
921:     return;
922:   }
923:   anchorDialogForDrag(dialog);
924:   dialog.classList.remove("is-maximized-dialog");
925:   dialog.classList.toggle("is-minimized-dialog");
926:   updateDialogWindowButtons(dialog);
927: }
```

toggleDialogMaximized (template-preview.js, lines 929-949)

toggleDialogMaximized part of the private builder frontend and editing experience. In this file, it appears around lines 929-949 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, remove, restoreDialogWindowState, saveDialogRestoreState, add, and updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 930: if (!dialog) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
929: function toggleDialogMaximized(dialog) {
930:   if (!dialog) return;
931:   if (dialog.classList.contains("is-hidden-dialog")) dialog.classList.remove("is-hidden-dialog");
932:   if (dialog.classList.contains("is-maximized-dialog")) {
933:     dialog.classList.remove("is-maximized-dialog");
934:     restoreDialogWindowState(dialog);
935:   } else {
936:     saveDialogRestoreState(dialog);
937:     dialog.classList.remove("is-minimized-dialog");
938:     dialog.classList.add("is-draggable-dialog", "is-maximized-dialog");
939:     dialog.style.left = "0";
940:     dialog.style.top = "0";
941:     dialog.style.right = "auto";
942:     dialog.style.bottom = "auto";
943:     dialog.style.margin = "0";
944:     dialog.style.width = "100vw";
945:     dialog.style.height = "100dvh";
946:     dialog.style.maxHeight = "none";
947:   }
948:   updateDialogWindowButtons(dialog);
949: }
```

toggleDialogHidden (template-preview.js, lines 951-970)

toggleDialogHidden part of the private builder frontend and editing experience. In this file, it appears around lines 951-970 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, remove, restoreDialogWindowState, saveDialogRestoreState, add, min, calc, and updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 952: if (!dialog) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
951: function toggleDialogHidden(dialog) {
952:   if (!dialog) return;
953:   if (dialog.classList.contains("is-hidden-dialog")) {
954:     dialog.classList.remove("is-hidden-dialog");
955:     restoreDialogWindowState(dialog);
956:   } else {
957:     saveDialogRestoreState(dialog);
958:     dialog.classList.remove("is-minimized-dialog", "is-maximized-dialog");
959:     dialog.classList.add("is-draggable-dialog", "is-hidden-dialog");
960:     dialog.style.left = "auto";
961:     dialog.style.top = "auto";
962:     dialog.style.right = "12px";
963:     dialog.style.bottom = "10px";
964:     dialog.style.margin = "0";
965:     dialog.style.width = "min(300px, calc(100vw - 24px))";
966:     dialog.style.height = "32px";
967:     dialog.style.maxHeight = "32px";
968:   }
969:   updateDialogWindowButtons(dialog);
970: }
```

dialogCanStepBack (template-preview.js, lines 972-976)

dialogCanStepBack part of the private builder frontend and editing experience. In this file, it appears around lines 972-976 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 973: if (dialog === projectDialog) return projectWindowBackStack.length > 0;; line 974: if (dialog === projectPreviewDialog) return activeProjectPreviewSectionIndex >= 0;; line 975: return false;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
972: function dialogCanStepBack(dialog) {
973:   if (dialog === projectDialog) return projectWindowBackStack.length > 0;
974:   if (dialog === projectPreviewDialog) return activeProjectPreviewSectionIndex >= 0;
975:   return false;
976: }
```

dialogCanStepForward (template-preview.js, lines 978-982)

dialogCanStepForward part of the private builder frontend and editing experience. In this file, it appears around lines 978-982 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 979: if (dialog === projectDialog) return projectWindowForwardStack.length > 0;; line 980: if (dialog === projectPreviewDialog) return activeProjectPreviewForwardStack.length > 0;; line 981: return false;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
978: function dialogCanStepForward(dialog) {
979:   if (dialog === projectDialog) return projectWindowForwardStack.length > 0;
980:   if (dialog === projectPreviewDialog) return activeProjectPreviewForwardStack.length > 0;
981:   return false;
982: }
```

renderProjectWindowSectionFromHistory (template-preview.js, lines 984-990)

renderProjectWindowSectionFromHistory renders ui, markup, or display output. In this file, it appears around lines 984-990 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderAll, and updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
984: function renderProjectWindowSectionFromHistory(sectionId) {
985:   suppressProjectWindowHistory = true;
986:   activeSectionId = sectionId || "brief";
987:   renderAll();
988:   suppressProjectWindowHistory = false;
989:   updateDialogWindowButtons(projectDialog);
990: }
```

stepProjectWindowBack (template-preview.js, lines 992-996)

stepProjectWindowBack part of the private builder frontend and editing experience. In this file, it appears around lines 992-996 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references push, renderProjectWindowSectionFromHistory, and pop. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 993: if (!projectWindowBackStack.length) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
992: function stepProjectWindowBack() {
993:   if (!projectWindowBackStack.length) return;
994:   projectWindowForwardStack.push(activeSectionId);
995:   renderProjectWindowSectionFromHistory(projectWindowBackStack.pop());
996: }
```

stepProjectWindowForward (template-preview.js, lines 998-1002)

stepProjectWindowForward part of the private builder frontend and editing experience. In this file, it appears around lines 998-1002 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references push, renderProjectWindowSectionFromHistory, and pop. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 999: if (!projectWindowForwardStack.length) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 998: function stepProjectWindowForward() {
 999:   if (!projectWindowForwardStack.length) return;
1000:   projectWindowBackStack.push(activeSectionId);
1001:   renderProjectWindowSectionFromHistory(projectWindowForwardStack.pop());
1002: }
```

refreshDialogWindow (template-preview.js, lines 1004-1015)

refreshDialogWindow part of the private builder frontend and editing experience. In this file, it appears around lines 1004-1015 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references saveVisibleRichEditors, refreshOpenPreviews, renderPreview, renderAll, setStatus, and updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1004: function refreshDialogWindow(dialog) {
1005:   saveVisibleRichEditors();
1006:   if (dialog === projectPreviewDialog) {
1007:     refreshOpenPreviews();
1008:   } else if (dialog === portfolioPreviewDialog) {
1009:     renderPreview();
1010:   } else if (dialog === projectDialog) {
1011:     renderAll();
1012:   }
1013:   setStatus("Window refreshed.");
1014:   updateDialogWindowButtons(dialog);
1015: }
```

closeDialogFromControl (template-preview.js, lines 1017-1025)

closeDialogFromControl controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 1017-1025 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, find, contains, click, and closeDialogElement. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1022: return;.

Important local values include originalClose at line 1018. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1017: function closeDialogFromControl(dialog, button) {
1018:   const originalClose = [...dialog.querySelectorAll(".project-window-close, .close-control")]
1019:     .find((item) => item !== button && !item.dataset.dialogClose);
1020:   if (originalClose && !button.classList.contains("close-control") && !button.id) {
1021:     originalClose.click();
1022:     return;
1023:   }
1024:   closeDialogElement(dialog, "close");
1025: }
```

handleDialogWindowAction (template-preview.js, lines 1027-1044)

handleDialogWindowAction handles an event, request, command, or user action. In this file, it appears around lines 1027-1044 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, toggleDialogHidden, toggleDialogMinimized, toggleDialogMaximized, refreshDialogWindow, stepProjectWindowBack, projectPreviewBackStep, stepProjectWindowForward, projectPreviewForwardStep, and closeDialogFromControl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1029: if (!dialog?.open) return;.

Important local values include dialog at line 1028; action at line 1030. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function handleDialogWindowAction(button) {
  const dialog = button.closest("dialog");
  if (!dialog?.open) return;
  const action = button.dataset.dialogAction;
  if (action === "hide") toggleDialogHidden(dialog);
  if (action === "minimize") toggleDialogMinimized(dialog);
  if (action === "maximize") toggleDialogMaximized(dialog);
  if (action === "refresh") refreshDialogWindow(dialog);
  if (action === "back") {
    if (dialog === projectDialog) stepProjectWindowBack();
    else if (dialog === projectPreviewDialog) projectPreviewBackStep();
  }
  if (action === "forward") {
    if (dialog === projectDialog) stepProjectWindowForward();
    else if (dialog === projectPreviewDialog) projectPreviewForwardStep();
  }
  if (action === "close") closeDialogFromControl(dialog, button);
}
```

enableDraggableDialogs (template-preview.js, lines 1046-1084)

enableDraggableDialogs part of the private builder frontend and editing experience. In this file, it appears around lines 1046-1084 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, addEventListener, closest, handleDialogWindowAction, toggleDialogHidden, remove, updateDialogWindowButtons, getBoundingClientRect, and clampDialogPosition. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1058: if (!button) return;; line 1063: if (!hiddenDialog || event.target.closest("button")) return;.

Important local values include button at line 1057; hiddenDialog at line 1062; rect at line 1078; next at line 1079. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function enableDraggableDialogs() {
  document.querySelectorAll("dialog").forEach(markDialogDragHandles);
  document.addEventListener("pointerdown", beginDialogResize);
  document.addEventListener("pointerdown", beginDialogDrag);
  document.addEventListener("pointermove", moveDialogResize);
  document.addEventListener("pointermove", moveDialogDrag);
  document.addEventListener("pointerup", endDialogResize);
  document.addEventListener("pointerup", endDialogDrag);
  document.addEventListener("pointercancel", endDialogResize);
  document.addEventListener("pointercancel", endDialogDrag);
  document.addEventListener("click", (event) => {
    const button = event.target.closest("[data-dialog-action]");
    if (!button) return;
    handleDialogWindowAction(button);
  });
  document.addEventListener("click", (event) => {
    const hiddenDialog = event.target.closest("dialog.is-hidden-dialog");
    if (!hiddenDialog || event.target.closest("button")) return;
    toggleDialogHidden(hiddenDialog);
  });
  document.querySelectorAll("dialog").forEach((dialog) => {
    dialog.addEventListener("close", () => {
      dialog.classList.remove("is-minimized-dialog");
      dialog.classList.remove("is-hidden-dialog");
      dialog.classList.remove("is-maximized-dialog");
      dialog.classList.remove("is-resizing-dialog");
      delete dialog.dataset.restoreWindowState;
      updateDialogWindowButtons(dialog);
    });
  });
  window.addEventListener("resize", () => {
    document.querySelectorAll("dialog.is-draggable-dialog[open]").forEach((dialog) => {
      const rect = dialog.getBoundingClientRect();
      const next = clampDialogPosition(rect.left, rect.top, dialog);
      dialog.style.left = `${next.left}px`;
      dialog.style.top = `${next.top}px`;
    });
  });
}
```

markDraftNeedsSave (template-preview.js, lines 1086-1089)

markDraftNeedsSave part of the private builder frontend and editing experience. In this file, it appears around lines 1086-1089 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setSaveState. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1086: function markDraftNeedsSave() {
1087:   draftSavedSinceChanges = false;
1088:   setSaveState("dirty", "Save draft needed");
1089: }
```

showBuilderError (template-preview.js, lines 1091-1097)

showBuilderError controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 1091-1097 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1091: function showBuilderError(title, message, details = "") {
1092:   publishResultEyebrow.textContent = "Builder check";
1093:   publishResultTitle.textContent = title;
1094:   publishResultMessage.textContent = message;
1095:   publishResultOutput.textContent = details;
1096:   publishResultDialog.showModal();
1097: }
```

appUpdateWasSnoozed (template-preview.js, lines 1105-1110)

appUpdateWasSnoozed part of the private builder frontend and editing experience. In this file, it appears around lines 1105-1110 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getItem, and now. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are omb-snoozed-update-at, and omb-snoozed-update-version.

It has 2 return statement(s). The most important visible returns are: line 1106: if (!version) return false;; line 1109: return snoozedVersion === version && Date.now() - snoozedAt < APP_UPDATE_SNOOZE_MS;.

Important local values include snoozedVersion at line 1107; snoozedAt at line 1108. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1105: function appUpdateWasSnoozed(version = "") {
1106:   if (!version) return false;
1107:   const snoozedVersion = localStorage.getItem("omb-snoozed-update-version") || "";
1108:   const snoozedAt = Number(localStorage.getItem("omb-snoozed-update-at") || 0);
1109:   return snoozedVersion === version && Date.now() - snoozedAt < APP_UPDATE_SNOOZE_MS;
1110: }
```

shouldShowUpdateDialog (template-preview.js, lines 1112-1117)

shouldShowUpdateDialog part of the private builder frontend and editing experience. In this file, it appears around lines 1112-1117 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getItem, and appUpdateWasSnoozed. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are omb-dismissed-update-version, and omb-skipped-update-version.

It has 3 return statement(s). The most important visible returns are: line 1113: if (!update.updateAvailable || !update.latestVersion) return false;; line 1115: if (skippedVersion === update.latestVersion) return false;; line 1116: return !appUpdateWasSnoozed(update.latestVersion);.

Important local values include skippedVersion at line 1114. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 1112: function shouldShowUpdateDialog(update = {}) {
 1113:   if (!update.updateAvailable || !update.latestVersion) return false;
 1114:   const skippedVersion = localStorage.getItem("omb-skipped-update-version") || localStorage.getItem("omb-
dismissed-update-version") || "";
 1115:   if (skippedVersion === update.latestVersion) return false;
 1116:   return !appUpdateWasSnoozed(update.latestVersion);
 1117: }
```

showUpdateDialog (template-preview.js, lines 1119-1158)

showUpdateDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 1119-1158 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references shouldShowUpdateDialog, escapeHtml, filter, join, showModal, and show. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1121: if (!appUpdateDialog || (!force && !shouldShowUpdateDialog(update))) return;; line 1152: if (appUpdateDialog.open) return;.

Important local values include force at line 1120; hasInstaller at line 1123; updateBlocked at line 1124; hasUpdate at line 1125. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 1119: function showUpdateDialog(update = {}, options = {}) {
 1120:   const force = Boolean(options.force);
 1121:   if (!appUpdateDialog || (!force && !shouldShowUpdateDialog(update))) return;
 1122:   pendingAppUpdate = update;
 1123:   const hasInstaller = Boolean(update.installerUrl);
 1124:   const updateBlocked = Boolean(update.updateBlocked);
 1125:   const hasUpdate = Boolean(update.updateAvailable && update.latestVersion);
 1126:   appUpdateTitle.textContent = updateBlocked ? "Update paused" : hasUpdate ? "Update available" :
"Builder is up to date";
 1127:   appUpdateMessage.textContent = updateBlocked
 1128:     ? `OMB Portfolio Builder ${update.latestVersion || "this release"} is available, but this version is
paused for automatic updates.`
 1129:     : hasUpdate
 1130:     ? `OMB Portfolio Builder ${update.latestVersion} is available. You are running
${update.currentVersion || "an older version"}.`
 1131:     : `You are running OMB Portfolio Builder ${update.currentVersion || "the current installed
version"}.`;
 1132:   appUpdateDetails.textContent = updateBlocked
 1133:     ? (update.blockedReason || "This release is temporarily blocked from automatic updates. Check again
after the next release is available.")
 1134:     : hasUpdate
 1135:     ? "Choose Update to download the latest installer, close this app, and install the new version
automatically. Remind me later postpones the prompt; Skip this version ignores this release."
 1136:     : "No newer released builder was found on GitHub Releases.";
 1137:   appUpdateReleaseMeta.innerHTML = (hasUpdate || updateBlocked)
 1138:     ? [
 1139:       `<span>Current version: ${escapeHtml(update.currentVersion || "unknown")}</span>`,
 1140:       `<span>Available version: ${escapeHtml(update.latestVersion || "unknown")}</span>`,
 1141:       update.releaseUrl ? `<span>Release source: GitHub Releases</span>` : ""
 1142:     ].filter(Boolean).join("")
 1143:     : [
 1144:       `<span>Current version: ${escapeHtml(update.currentVersion || "unknown")}</span>`,
 1145:       update.checkedAt ? `<span>Checked: ${escapeHtml(update.checkedAt)}</span>` : ""
 1146:     ].filter(Boolean).join("");
 1147:   appUpdateLater.textContent = hasUpdate ? "Remind me later" : "Close";
 1148:   appUpdateSkip.hidden = !hasUpdate;
 1149:   appUpdateApply.hidden = !hasUpdate;
 1150:   appUpdateApply.disabled = hasUpdate && !hasInstaller;
 1151:   appUpdateApply.textContent = hasInstaller ? "Update" : "Open release page";
 1152:   if (appUpdateDialog.open) return;
 1153:   try {
 1154:     appUpdateDialog.showModal();
 1155:   } catch {
 1156:     appUpdateDialog.show();
 1157:   }
 1158: }
```

startAppUpdateInstall (template-preview.js, lines 1160-1193)

startAppUpdateInstall part of the private builder frontend and editing experience. In this file, it appears around lines 1160-1193 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, stringify, json, Error, and showBuilderError. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include update at line 1161; response at line 1168; result at line 1174; fallback at line 1184. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1160: async function startAppUpdateInstall() {
1161:   const update = pendingAppUpdate || {};
1162:   appUpdateApply.disabled = true;
1163:   appUpdateLater.disabled = true;
1164:   appUpdateSkip.disabled = true;
1165:   appUpdateApply.textContent = "Updating...";
1166:   appUpdateDetails.textContent = "Downloading the installer. The builder will close automatically, then
the installer will update the app.";
1167:   try {
1168:     const response = await fetch(`/api/app-update/install?t=${Date.now()}`, {
1169:       method: "POST",
1170:       cache: "no-store",
1171:       headers: { "Content-Type": "application/json" },
1172:       body: JSON.stringify({ latestVersion: update.latestVersion || "" })
1173:     });
1174:     const result = await response.json();
1175:     if (!response.ok || !result.ok) throw new Error(result.error || "The update could not be started.");
1176:     appUpdateMessage.textContent = "Update started";
1177:     appUpdateDetails.textContent = "The installer is ready. This window will close so the new version can
be installed.";
1178:   } catch (error) {
1179:     appUpdateApply.disabled = false;
1180:     appUpdateLater.disabled = false;
1181:     appUpdateSkip.disabled = false;
1182:     appUpdateApply.textContent = "Update";
1183:     appUpdateDetails.textContent = error.message || "The update could not be started.";
1184:     const fallback = update.releaseUrl || update.installerUrl || update.portableUrl || "";
1185:     if (fallback) {
1186:       showBuilderError(
1187:         "Automatic update could not start",
1188:         "The app could not run the installer automatically. The release page can still be opened
manually.",
1189:         fallback
1190:       );
1191:     }
1192:   }
1193: }
```

checkForAppUpdates (template-preview.js, lines 1195-1235)

checkForAppUpdates part of the private builder frontend and editing experience. In this file, it appears around lines 1195-1235 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references now, fetch, json, showUpdateDialog, and toLocaleString. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1198: if (!force && Date.now() - lastAppUpdateCheckAt < 5 * 60 * 1000) return;.

Important local values include force at line 1196; manual at line 1197; response at line 1207; update at line 1208. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1195: async function checkForAppUpdates(options = {}) {
1196:   const force = Boolean(options.force);
1197:   const manual = Boolean(options.manual);
1198:   if (!force && Date.now() - lastAppUpdateCheckAt < 5 * 60 * 1000) return;
1199:   lastAppUpdateCheckAt = Date.now();
1200:   if (manual) {
1201:     if (appUpdateCheckButton) {
1202:       appUpdateCheckButton.disabled = true;
1203:       appUpdateCheckButton.textContent = "Checking...";
1204:     }
1205:   }
```

```
1206:   try {
1207:     const response = await fetch(`/api/app-update?t=${Date.now()}`, { cache: "no-store" });
1208:     const update = await response.json();
1209:     if (response.ok && update.ok !== false) {
1210:       showUpdateDialog({ ...update, checkedAt: new Date().toLocaleString() }, { force: manual });
1211:     } else if (manual) {
1212:       showUpdateDialog({
1213:         currentVersion: update.currentVersion || "unknown",
1214:         updateAvailable: false,
1215:         checkedAt: new Date().toLocaleString()
1216:       }, { force: true });
1217:     }
1218:   } catch {
1219:     if (manual) {
1220:       showUpdateDialog({
1221:         currentVersion: "unknown",
1222:         updateAvailable: false,
1223:         checkedAt: new Date().toLocaleString()
1224:       }, { force: true });
1225:     }
1226:     // Update checks should never interrupt builder work.
1227:   } finally {
1228:     if (manual) {
1229:       if (appUpdateCheckButton) {
1230:         appUpdateCheckButton.disabled = false;
1231:         appUpdateCheckButton.textContent = "Check updates";
1232:       }
1233:     }
1234:   }
1235: }
```

formatReportDate (template-preview.js, lines 1237-1240)

formatReportDate part of the private builder frontend and editing experience. In this file, it appears around lines 1237-1240 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isNaN, getTime, and toLocaleString. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1239: return Number.isNaN(date.getTime()) ? "" : date.toLocaleString();.

Important local values include date at line 1238. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1237: function formatReportDate(value = "") {
1238:   const date = new Date(value);
1239:   return Number.isNaN(date.getTime()) ? "" : date.toLocaleString();
1240: }
```

renderSecurityReport (template-preview.js, lines 1242-1294)

renderSecurityReport renders ui, markup, or display output. In this file, it appears around lines 1242-1294 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, map, escapeHtml, toLocaleString, join, and formatReportDate. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1243: if (!securityReportBody) return;.

Important local values include downloads at line 1244; assets at line 1245; auth at line 1246; access at line 1247; controls at line 1248; assetRows at line 1249. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1242: function renderSecurityReport(report = {}) {
1243:   if (!securityReportBody) return;
1244:   const downloads = report.appDownloads || {};
1245:   const assets = Array.isArray(downloads.assets) ? downloads.assets : [];
1246:   const auth = report.publishingAuth || {};
1247:   const access = report.websiteAccess || {};
1248:   const controls = Array.isArray(report.securityControls) ? report.securityControls : [];
1249:   const assetRows = assets.length
1250:     ? assets.map((asset) => `
1251:         <li>
```

```
          <span>${escapeHtml(asset.name || "Release asset")}</span>
          <span>${Number(asset.downloadCount || 0).toLocaleString()} downloads</span>
        </li>
      `).join("")
    : `<li><span>No release assets found</span><span>0 downloads</span></li>`;
  securityReportBody.innerHTML = `
    <div class="security-report-grid">
      <section class="security-report-card">
        <h3>Builder downloads</h3>
        <strong>${downloads.ok ? Number(downloads.totalDownloads || 0).toLocaleString() :
"Unavailable"}</strong>
        <p>${downloads.ok
          ? `Latest release: ${escapeHtml(downloads.releaseTag || downloads.releaseName || "latest")}`
          : escapeHtml(downloads.error || "GitHub release counts are not available.")}</p>
        <ul class="security-report-assets">${assetRows}</ul>
      </section>
      <section class="security-report-card">
        <h3>Publishing authorization</h3>
        <strong>${auth.authenticated ? "Active" : "Not active"}</strong>
        <p>${auth.authenticated
          ? `This builder has a remembered GitHub publishing session until
${escapeHtml(formatReportDate(auth.expiresAt) || auth.expiresAt || "the cache expires")}.`
          : "No fresh GitHub publishing session is cached for the current target."}</p>
        <ul>
          <li>Target: ${escapeHtml(auth.targetRepository || "No publishing target saved")}</li>
          <li>Branch: ${escapeHtml(auth.branch || "Not set")}</li>
          <li>Successful authorizations this week: ${Number(auth.successCountLastWeek ||
0).toLocaleString()}</li>
        </ul>
      </section>
      <section class="security-report-card">
        <h3>Website access reporting</h3>
        <strong>${access.available ? "Configured" : "Needs Cloudflare"}</strong>
        <p>${escapeHtml(access.summary || "Visitor and IP reporting needs a server-side analytics
source.")}</p>
        <ul>${(access.recommendedSetup || []).map((item) =>
`<li>${escapeHtml(item)}</li>`).join("")}</ul>
      </section>
      <section class="security-report-card">
        <h3>Security controls</h3>
        <strong>${controls.length.toLocaleString()}</strong>
        <p>Controls currently enabled or prepared for publishing.</p>
        <ul>${controls.map((item) => `<li>${escapeHtml(item)}</li>`).join("")}</ul>
      </section>
    </div>
    <p>Generated ${escapeHtml(formatReportDate(report.generatedAt) || "just now")}.</p>
  `;
}
```

openSecurityReport (template-preview.js, lines 1296-1314)

openSecurityReport controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 1296-1314 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references showModal, updateDialogWindowButtons, fetch, now, json, Error, renderSecurityReport, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1297: if (!securityReportDialog || !securityReportBody) return;.

Important local values include response at line 1302; report at line 1303. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
async function openSecurityReport() {
  if (!securityReportDialog || !securityReportBody) return;
  securityReportBody.innerHTML = "<p>Loading security report...</p>";
  if (!securityReportDialog.open) securityReportDialog.showModal();
  updateDialogWindowButtons(securityReportDialog);
  try {
    const response = await fetch(`/api/security-report?t=${Date.now()}`, { cache: "no-store" });
    const report = await response.json();
    if (!response.ok || report.ok === false) throw new Error(report.error || "Security report could not
be loaded.");
    renderSecurityReport(report);
  } catch (error) {
    securityReportBody.innerHTML = `
      <div class="security-report-card">
        <h3>Security report unavailable</h3>
        <p>${escapeHtml(error.message || "The report could not be loaded.")}</p>
      </div>
    `;
  }
```

```
1314: }
```

scheduleAppUpdateChecks (template-preview.js, lines 1316-1325)

scheduleAppUpdateChecks part of the private builder frontend and editing experience. In this file, it appears around lines 1316-1325 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references checkForAppUpdates, addEventListener, and now. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1320: if (document.visibilityState !== "visible") return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1316: function scheduleAppUpdateChecks() {
1317:   checkForAppUpdates({ force: true });
1318:   window.setInterval(() => checkForAppUpdates({ force: true }), APP_UPDATE_CHECK_INTERVAL_MS);
1319:   document.addEventListener("visibilitychange", () => {
1320:     if (document.visibilityState !== "visible") return;
1321:     if (Date.now() - lastAppUpdateCheckAt >= APP_UPDATE_FOCUS_RECHECK_MS) {
1322:       checkForAppUpdates({ force: true });
1323:     }
1324:   });
1325: }
```

renderPublishTargetInfo (template-preview.js, lines 1327-1353)

renderPublishTargetInfo renders ui, markup, or display output. In this file, it appears around lines 1327-1353 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLocaleString, map, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1328: if (!publishTargetCurrent) return;.

Important local values include checkedText at line 1330; expiresText at line 1331; trustText at line 1332; authorizationStatus at line 1333; rows at line 1336. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1327: function renderPublishTargetInfo(target = {}) {
1328:   if (!publishTargetCurrent) return;
1329:   currentPublishTarget = target;
1330:   const checkedText = target.checkedAt ? ` at ${new Date(target.checkedAt).toLocaleString()}` : "";
1331:   const expiresText = target.expiresAt ? `; remembered until ${new
Date(target.expiresAt).toLocaleString()}` : "";
1332:   const trustText = target.extendedTrust ? " (30-day trusted target)" : target.authorizationCached ? "
(daily cache)" : "";
1333:   const authorizationStatus = target.authorizationChecked
1334:     ? `Verified${checkedText}${trustText}${expiresText}`
1335:     : "Not verified for this repository and branch";
1336:   const rows = [
1337:     ["Builder workspace", target.workspace || "Not available"],
1338:     ["Portfolio workspace", target.portfolioWorkspace || target.workspace || "Not available"],
1339:     ["Repository", target.repository || target.remote || "No repository connected"],
1340:     ["Branch", target.branch || "No branch selected"],
1341:     ["GitHub authorization", authorizationStatus],
1342:     ["Compatible site", target.compatible ? "Yes" : "No"],
1343:     ["Git-backed", target.gitBacked ? "Yes" : "No"]
1344:   ];
1345:   publishTargetCurrent.innerHTML = rows
1346:     .map(([label, value]) =>
`<div><strong>${escapeHtml(label)}</strong><span>${escapeHtml(value)}</span></div>`)
1347:     .join("");
1348:   if (publishTargetRepository) {
1349:     publishTargetRepository.value = target.remote || "";
1350:     publishTargetRepository.dataset.loadedValue = target.remote || "";
1351:   }
1352:   if (publishTargetSync) publishTargetSync.disabled = !target.authorizationChecked;
1353: }
```

renderSystemReadiness (template-preview.js, lines 1355-1394)

renderSystemReadiness renders ui, markup, or display output. In this file, it appears around lines 1355-1394 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLocaleString, map, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1356: if (!publishTargetReadinessList) return;.

Important local values include rows at line 1357. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1355: function renderSystemReadiness(system = {}, target = {}) {
1356:   if (!publishTargetReadinessList) return;
1357:   const rows = [
1358:     {
1359:       ok: true,
1360:       text: `Electron/Node runtime: bundled with the app${system.nodeRuntime?.version ? `
(${system.nodeRuntime.version})` : ""}.`
1361:     },
1362:     {
1363:       ok: system.pnpm?.required === false,
1364:       text: "pnpm: not required for installed users."
1365:     },
1366:     {
1367:       ok: Boolean(system.git?.ok),
1368:       text: system.git?.ok
1369:         ? `Git for Windows: ${system.git.version || "installed"}.`
1370:         : "Git for Windows: missing. Install it before loading or publishing a target."
1371:     },
1372:     {
1373:       ok: Boolean(system.credentialManager?.ok),
1374:       text: system.credentialManager?.ok
1375:         ? `Git Credential Manager: ${system.credentialManager.version || "installed"}.`
1376:         : "Git Credential Manager: missing. Install Git for Windows with Git Credential Manager enabled."
1377:     },
1378:     {
1379:       ok: Boolean(target?.remote),
1380:       text: target?.remote
1381:         ? `Publishing target: ${target.repository || target.remote}.`
1382:         : "Publishing target: enter a repository URL and click Authenticate target."
1383:     },
1384:     {
1385:       ok: Boolean(target?.authorizationChecked),
1386:       text: target?.authorizationChecked
1387:         ? `GitHub authorization: verified for ${target.branch || "the selected branch"}${target.expiresAt
? ` until ${new Date(target.expiresAt).toLocaleString()}` : ""}.`
1388:         : "GitHub authorization: required before Load from target or Apply to site."
1389:     }
1390:   ];
1391:   publishTargetReadinessList.innerHTML = rows
1392:     .map((row) => `<li class="${row.ok ? "ready" : "blocked"}">${escapeHtml(row.ok ? `OK - ${row.text}` :
`Needs action - ${row.text}`)}</li>`)
1393:     .join("");
1394: }
```

markPublishTargetNeedsAuthentication (template-preview.js, lines 1396-1405)

markPublishTargetNeedsAuthentication part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1396-1405 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1396: function markPublishTargetNeedsAuthentication() {
1397:   if (publishTargetSync) publishTargetSync.disabled = true;
1398:   if (currentPublishTarget) {
1399:     currentPublishTarget = {
1400:       ...currentPublishTarget,
1401:       authorizationChecked: false,
```

```
1402:       authorizationCached: false
1403:     };
1404:   }
1405: }
```

loadSystemReadiness (template-preview.js, lines 1407-1418)

loadSystemReadiness loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1407-1418 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, json, Error, renderSystemReadiness, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1408: if (!publishTargetReadinessList) return;.

Important local values include response at line 1411; result at line 1412. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1407: async function loadSystemReadiness() {
1408:   if (!publishTargetReadinessList) return;
1409:   publishTargetReadinessList.innerHTML = "<li>Checking local publishing tools...</li>";
1410:   try {
1411:     const response = await fetch(`/api/system-check?t=${Date.now()}`, { cache: "no-store" });
1412:     const result = await response.json();
1413:     if (!response.ok || !result.ok) throw new Error(result.error || "System check failed.");
1414:     renderSystemReadiness(result.system || {}, result.target || currentPublishTarget || {});
1415:   } catch (error) {
1416:     publishTargetReadinessList.innerHTML = `<li class="blocked">${escapeHtml(error.message || "System
check failed.")}</li>`;
1417:   }
1418: }
```

setPublishTargetProgress (template-preview.js, lines 1420-1426)

setPublishTargetProgress part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1420-1426 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, toLocaleTimeString, map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1421: if (!publishTargetCurrent) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1420: function setPublishTargetProgress(title, steps = []) {
1421:   if (!publishTargetCurrent) return;
1422:   publishTargetCurrent.innerHTML = `
1423:     <div><strong>${escapeHtml(title)}</strong><span>${escapeHtml(new
Date().toLocaleTimeString())}</span></div>
1424:     ${steps.map((step) =>
`<div><strong>${escapeHtml(step.label)}</strong><span>${escapeHtml(step.value)}</span></div>`).join("")}
1425:   `;
1426: }
```

loadPublishTargetInfo (template-preview.js, lines 1428-1439)

loadPublishTargetInfo loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1428-1439 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, json, Error, and renderPublishTargetInfo. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1429: if (!publishTargetDialog) return;.

Important local values include response at line 1432; result at line 1433. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1428: async function loadPublishTargetInfo() {
1429:   if (!publishTargetDialog) return;
1430:   publishTargetCurrent.textContent = "Current target is loading.";
1431:   try {
1432:     const response = await fetch(`/api/publish-target?t=${Date.now()}`, { cache: "no-store" });
1433:     const result = await response.json();
1434:     if (!response.ok || !result.ok) throw new Error(result.error || "Publishing target could not be
loaded.");
1435:     renderPublishTargetInfo(result.target || {});
1436:   } catch (error) {
1437:     publishTargetCurrent.textContent = error.message || "Publishing target could not be loaded.";
1438:   }
1439: }
```

openPublishTargetDialog (template-preview.js, lines 1441-1445)

openPublishTargetDialog part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1441-1445 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references loadPublishTargetInfo, loadSystemReadiness, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1441: async function openPublishTargetDialog() {
1442:   await loadPublishTargetInfo();
1443:   await loadSystemReadiness();
1444:   publishTargetDialog.showModal();
1445: }
```

savePublishTarget (template-preview.js, lines 1447-1450)

savePublishTarget persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 1447-1450 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references preventDefault, and authenticatePublishTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1447: async function savePublishTarget(event) {
1448:   event.preventDefault();
1449:   await authenticatePublishTarget({ fromSave: true });
1450: }
```

currentPublishTargetPayload (template-preview.js, lines 1452-1456)

currentPublishTargetPayload part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1452-1456 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1453: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1452: function currentPublishTargetPayload() {
```

```
  return {
    repositoryUrl: publishTargetRepository?.value || ""
  };
}
```

installGitForPublishing (template-preview.js, lines 1458-1481)

installGitForPublishing part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1458-1481 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, json, Error, open, loadSystemReadiness, and showBuilderError. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include response at line 1461; result at line 1467. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
async function installGitForPublishing() {
  publishTargetCurrent.textContent = "Starting Git for Windows setup...";
  try {
    const response = await fetch(`/api/install-git?t=${Date.now()}`, {
      method: "POST",
      cache: "no-store",
      headers: { "Content-Type": "application/json" },
      body: "{}"
    });
    const result = await response.json();
    if (!response.ok || !result.ok) throw new Error(result.error || "Git installer could not be
started.");
    if (!result.install?.launched && result.install?.downloadUrl) {
      window.open(result.install.downloadUrl, "_blank", "noreferrer");
    }
    await loadSystemReadiness();
    showBuilderError(
      result.install?.launched ? "Git setup started" : "Open Git download",
      result.install?.message || "Install Git for Windows with Git Credential Manager enabled, then
reopen the builder.",
      result.install?.downloadUrl || ""
    );
  } catch (error) {
    publishTargetCurrent.textContent = error.message || "Git installer could not be started.";
  }
}
```

authenticatePublishTarget (template-preview.js, lines 1483-1536)

authenticatePublishTarget part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1483-1536 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setPublishTargetProgress, fetch, now, stringify, currentPublishTargetPayload, json, Error, filter, join, renderPublishTargetInfo, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include fromSave at line 1484; response at line 1496; result at line 1502; message at line 1504; usedCachedAuth at line 1510. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
async function authenticatePublishTarget(options = {}) {
  const fromSave = Boolean(options.fromSave);
  if (publishTargetSync) publishTargetSync.disabled = true;
  setPublishTargetProgress(
    fromSave ? "Authenticating publishing target" : "Checking GitHub authorization",
    [
      { label: "Step 1", value: "Validating the repository URL and local Git tools." },
      { label: "Step 2", value: "Using the saved daily authorization if it is still valid." },
      { label: "Step 3", value: "Finding the target repository default branch." },
      { label: "Step 4", value: "Opening GitHub sign-in only when a fresh authorization is required." }
    ]
  );
  try {
    const response = await fetch(`/api/github-authenticate?t=${Date.now()}`, {
```

```
      method: "POST",
      cache: "no-store",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(currentPublishTargetPayload())
    });
    const result = await response.json();
    if (!response.ok || !result.ok) {
      const message = result.error || "GitHub authentication did not complete.";
      throw new Error([message, result.details].filter(Boolean).join("\n\n"));
    }
    renderPublishTargetInfo(result.target || {});
    renderSystemReadiness(result.auth?.system || {}, result.target || {});
    if (publishTargetPassword) publishTargetPassword.value = "";
    const usedCachedAuth = Boolean(result.auth?.authorizationCached ||
result.target?.authorizationCached);
    setPublishTargetProgress(
      usedCachedAuth ? "Target authenticated from saved authorization" : "Target authenticated",
      [
        { label: "Repository", value: result.target?.repository || result.auth?.repository || "Selected
target" },
        { label: "Branch", value: result.target?.branch || result.auth?.branch || "Detected target
branch" },
        { label: "Authorization", value: usedCachedAuth ? "A valid saved authorization was reused; no new
 GitHub sign-in was needed." : "GitHub write access was verified now." },
        { label: "Next step", value: "Click Load from target when you want to import the latest
compatible website files into this builder." }
      ]
    );
    if (publishTargetSync) publishTargetSync.disabled = false;
  } catch (error) {
    if (publishTargetSync) publishTargetSync.disabled = true;
    setPublishTargetProgress(
      "Target was not saved",
      [
        { label: "Reason", value: error.message || "GitHub authentication did not complete." },
        { label: "Status", value: "The previous publishing target was kept. Fix the repository or sign-
in, then try Authenticate target again." }
      ]
    );
    showBuilderError(
      "Target authentication failed",
      "The builder did not save this publishing target because GitHub write access was not verified.",
      error.message || "GitHub authentication did not complete."
    );
  }
}
```

syncFromPublishTarget (template-preview.js, lines 1538-1584)

syncFromPublishTarget part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1538-1584 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, json, Error, filter, join, renderPublishTargetInfo, closeDialogElement, loadData, and showBuilderError. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1539: if (!publishTargetDialog) return;.

Important local values include afterAuth at line 1540; response at line 1545; result at line 1551. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
async function syncFromPublishTarget(options = {}) {
  if (!publishTargetDialog) return;
  const afterAuth = Boolean(options.afterAuth);
  publishTargetCurrent.textContent = afterAuth
    ? "GitHub authentication was accepted. Importing the target portfolio now..."
    : "Checking GitHub authorization and loading compatible portfolio files from the target
repository...";
  try {
    const response = await fetch(`/api/sync-from-publish-target?t=${Date.now()}`, {
      method: "POST",
      cache: "no-store",
      headers: { "Content-Type": "application/json" },
      body: "{}"
    });
    const result = await response.json();
    if (!response.ok || !result.ok) {
      throw new Error([
        result.error || "Publishing target could not be imported.",
        result.details || "Click Authenticate target, complete the browser sign-in if prompted, then try
Load from target again."
      ].filter(Boolean).join("\n\n"));
```

```
    }
    renderPublishTargetInfo(result.target || {});
    closeDialogElement(publishTargetDialog);
    await loadData();
    draftSavedSinceChanges = true;
    showBuilderError(
      afterAuth ? "GitHub connected and target loaded" : "Target content loaded",
      afterAuth
        ? "Authentication succeeded, and compatible portfolio files were loaded into this local builder workspace. The local draft now matches the imported target catalog."
        : "Compatible portfolio files were loaded from the authenticated publishing target into this local builder workspace. The local draft now matches the imported target catalog.",
      [
        `Imported: ${(result.sync?.imported || []).join(", ")}`,
        result.sync?.backup ? `Backup: ${result.sync.backup}` : ""
      ].filter(Boolean).join("\n")
    );
  } catch (error) {
    publishTargetCurrent.textContent = afterAuth
      ? `Authentication completed, but target loading failed. ${error.message || "Publishing target could not be imported."}`
      : error.message || "Publishing target could not be imported.";
    if (afterAuth) {
      showBuilderError(
        "Target saved, loading failed",
        "GitHub authorization succeeded, but the builder could not import compatible portfolio files from that target.",
        error.message || "Publishing target could not be imported."
      );
    }
  }
}
```

showPublishResult (template-preview.js, lines 1586-1619)

showPublishResult part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1586-1619 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, join, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include publish at line 1587; pushed at line 1588; authorizationRequired at line 1589; output at line 1602. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function showPublishResult(result) {
  const publish = result.publish || {};
  const pushed = Boolean(publish.pushed);
  const authorizationRequired = Boolean(publish.authorizationRequired);
  publishResultEyebrow.textContent = "GitHub publish";
  publishResultTitle.textContent = pushed
    ? "Successfully pushed to GitHub"
    : authorizationRequired
      ? "Publishing access required"
      : "Site applied, push failed";
  publishResultMessage.textContent = pushed
    ? `Your portfolio changes were committed and pushed to ${publish.branch || "GitHub"}.`
    : authorizationRequired
      ? "No live website files were applied. Open Publishing target, click Authenticate with GitHub, complete the browser sign-in, then try Apply to site again."
      : "The site was applied locally, but Git push did not complete.";

  const output = [
    pushed ? "PUSH STATUS: SUCCESS" : "PUSH STATUS: FAILED",
    `FILE: ${result.file || "projects.json"}`,
    authorizationRequired ? "AUTHORIZATION: REQUIRED BEFORE WEBSITE CHANGES" : "",
    publish.repository ? `REPOSITORY: ${publish.repository}` : "",
    publish.remote ? `REMOTE: ${publish.remote}` : "",
    publish.branch ? `BRANCH: ${publish.branch}` : "",
    publish.committed === false ? "COMMIT: No file changes to commit." : "",
    publish.commitOutput ? `COMMIT OUTPUT:\n${publish.commitOutput}` : "",
    publish.pushOutput ? `PUSH OUTPUT:\n${publish.pushOutput}` : "",
    publish.error ? `ERROR:\n${publish.error}` : "",
    publish.details ? `DETAILS:\n${publish.details}` : "",
    publish.help ? `NEXT STEP:\n${publish.help}` : ""
  ].filter(Boolean).join("\n\n");

  publishResultOutput.textContent = output || "No Git output was returned.";
  publishResultDialog.showModal();
}
```

scheduleAutosave (template-preview.js, lines 1621-1627)

scheduleAutosave part of the private builder frontend and editing experience. In this file, it appears around lines 1621-1627 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references markDraftNeedsSave, and autosaveDraft. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1621: function scheduleAutosave(delay = 900) {
1622:   markDraftNeedsSave();
1623:   clearTimeout(autosaveTimer);
1624:   autosaveTimer = setTimeout(() => {
1625:     autosaveDraft();
1626:   }, delay);
1627: }
```

autosaveDraft (template-preview.js, lines 1629-1659)

autosaveDraft part of the private builder frontend and editing experience. In this file, it appears around lines 1629-1659 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setSaveState, fetch, stringify, json, setStatus, and scheduleAutosave. Endpoint references found inside it are /api/save-draft. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1632: return;.

Important local values include response at line 1637; result at line 1642. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1629: async function autosaveDraft() {
1630:   if (autosaveInFlight) {
1631:     autosaveQueued = true;
1632:     return;
1633:   }
1634:   autosaveInFlight = true;
1635:   setSaveState("saving", "Autosaving...");
1636:   try {
1637:     const response = await fetch("/api/save-draft", {
1638:       method: "POST",
1639:       headers: { "Content-Type": "application/json" },
1640:       body: JSON.stringify({ catalog })
1641:     });
1642:     const result = await response.json();
1643:     if (!response.ok) {
1644:       setStatus(result.error || "Autosave failed.");
1645:       setSaveState("error", "Autosave failed");
1646:     } else if (!draftSavedSinceChanges) {
1647:       setSaveState("dirty", "Autosaved locally");
1648:     }
1649:   } catch {
1650:     setStatus("Autosave failed. Make sure the local server is running.");
1651:     setSaveState("error", "Autosave failed");
1652:   } finally {
1653:     autosaveInFlight = false;
1654:     if (autosaveQueued) {
1655:       autosaveQueued = false;
1656:       scheduleAutosave(300);
1657:     }
1658:   }
1659: }
```

schedulePreviewRender (template-preview.js, lines 1661-1667)

schedulePreviewRender part of the private builder frontend and editing experience. In this file, it appears around lines 1661-1667 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderPreview. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1662: if (!portfolioPreviewDialog?.open) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1661: function schedulePreviewRender() {
1662:   if (!portfolioPreviewDialog?.open) return;
1663:   clearTimeout(previewRenderTimer);
1664:   previewRenderTimer = setTimeout(() => {
1665:     renderPreview();
1666:   }, 220);
1667: }
```

scheduleChromeRender (template-preview.js, lines 1669-1676)

scheduleChromeRender part of the private builder frontend and editing experience. In this file, it appears around lines 1669-1676 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderTree, renderSectionTabs, selectedProject, and renderPreview. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1669: function scheduleChromeRender() {
1670:   clearTimeout(chromeRenderTimer);
1671:   chromeRenderTimer = setTimeout(() => {
1672:     renderTree();
1673:     renderSectionTabs(selectedProject());
1674:     if (portfolioPreviewDialog?.open) renderPreview();
1675:   }, 260);
1676: }
```

updateAssetDialogVisibility (template-preview.js, lines 1678-1691)

updateAssetDialogVisibility updates ui, state, cache, or stored data. In this file, it appears around lines 1678-1691 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, trim, labelForUrlAssetKind, and inferUrlAssetKind. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include isLocal at line 1679; isCaptionFile at line 1680; url at line 1685. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1678: function updateAssetDialogVisibility() {
1679:   const isLocal = assetSource.value === "local";
1680:   const isCaptionFile = captionSource.value === "file";
1681:   document.querySelector(".asset-local-field").hidden = !isLocal;
1682:   document.querySelector(".asset-url-field").hidden = isLocal;
1683:   document.querySelector(".caption-text-field").hidden = isCaptionFile;
1684:   document.querySelector(".caption-file-field").hidden = !isCaptionFile;
1685:   const url = assetUrl?.value?.trim() || "";
1686:   if (assetDetectedNote) {
1687:     assetDetectedNote.textContent = isLocal || !url
1688:       ? ""
1689:       : `Detected: ${labelForUrlAssetKind(inferUrlAssetKind(url, assetSource.value))}.`;
1690:   }
1691: }
```

slugify (template-preview.js, lines 1693-1699)

slugify part of the private builder frontend and editing experience. In this file, it appears around lines 1693-1699 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, trim, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1694: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1693: function slugify(value) {
1694:   return String(value || "")
1695:     .toLowerCase()
1696:     .trim()
1697:     .replace(/[^a-z0-9]+/g, "-")
1698:     .replace(/^-+|-+$/g, "");
1699: }
```

wordCount (template-preview.js, lines 1701-1703)

wordCount part of the private builder frontend and editing experience. In this file, it appears around lines 1701-1703 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, split, and filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1702: return String(value || "").trim().split(/\s+/).filter(Boolean).length;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1701: function wordCount(value) {
1702:   return String(value || "").trim().split(/\s+/).filter(Boolean).length;
1703: }
```

limitWords (template-preview.js, lines 1705-1708)

limitWords part of the private builder frontend and editing experience. In this file, it appears around lines 1705-1708 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, split, filter, slice, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1707: return words.length > maxWords ? words.slice(0, maxWords).join(" ") : value;.

Important local values include words at line 1706. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1705: function limitWords(value, maxWords = 1000) {
1706:   const words = String(value || "").trim().split(/\s+/).filter(Boolean);
1707:   return words.length > maxWords ? words.slice(0, maxWords).join(" ") : value;
1708: }
```

normalizeFunFacts (template-preview.js, lines 1710-1716)

normalizeFunFacts validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 1710-1716 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, split, map, replace, trim, filter, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1712: return items.

Important local values include items at line 1711. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1710: function normalizeFunFacts(value) {
1711:   const items = Array.isArray(value) ? value : String(value || "").split(/\r?\n/);
1712:   return items
1713:     .map((item) => String(item || "").replace(/\s+/g, " ").trim())
1714:     .filter(Boolean)
1715:     .slice(0, 3);
1716: }
```

syncFunFactsFromInput (template-preview.js, lines 1718-1726)

syncFunFactsFromInput keeps two places aligned, such as local data and target files. In this file, it appears around lines 1718-1726 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references extractRichSummary, normalizeFunFacts, and plainTextFromRich. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1719: if (!funFactsInput) return [];; line 1725: return facts;.

Important local values include rich at line 1720; facts at line 1721. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1718: function syncFunFactsFromInput() {
1719:   if (!funFactsInput) return [];
1720:   const rich = extractRichSummary(funFactsInput);
1721:   const facts = normalizeFunFacts(plainTextFromRich(rich));
1722:   catalog.funFacts = facts;
1723:   catalog.funFactsRich = rich;
1724:   if (funFactsCount) funFactsCount.textContent = `${facts.length}/3`;
1725:   return facts;
1726: }
```

renderFunFactsEditor (template-preview.js, lines 1728-1734)

renderFunFactsEditor renders ui, markup, or display output. In this file, it appears around lines 1728-1734 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeFunFacts, populateStandaloneRichEditor, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1729: if (!funFactsInput) return;; line 1732: if (document.activeElement === funFactsInput) return;.

Important local values include facts at line 1730. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1728: function renderFunFactsEditor() {
1729:   if (!funFactsInput) return;
1730:   const facts = normalizeFunFacts(catalog.funFacts || []);
1731:   if (funFactsCount) funFactsCount.textContent = `${facts.length}/3`;
1732:   if (document.activeElement === funFactsInput) return;
1733:   populateStandaloneRichEditor(funFactsInput, catalog.funFactsRich, facts.join("\n"));
1734: }
```

populateTextOnlyRichField (template-preview.js, lines 1736-1740)

populateTextOnlyRichField part of the private builder frontend and editing experience. In this file, it appears around lines 1736-1740 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references populateStandaloneRichEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1737: if (!field) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1736: function populateTextOnlyRichField(field, rich, fallbackText = "") {
1737:   if (!field) return;
1738:   populateStandaloneRichEditor(field, rich, fallbackText);
1739:   field.dataset.richTextOnly = "true";
1740: }
```

richFieldValue (template-preview.js, lines 1742-1754)

richFieldValue part of the private builder frontend and editing experience. In this file, it appears around lines 1742-1754 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, textBlocksFromPlainText, extractRichSummary, and plainTextFromRich. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1744: return {; line 1750: return {.

Important local values include rich at line 1749. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1742: function richFieldValue(field, fallbackText = "") {
1743:   if (!field) {
1744:     return {
1745:       plain: String(fallbackText || "").trim(),
1746:       rich: { blocks: textBlocksFromPlainText(fallbackText) }
1747:     };
1748:   }
1749:   const rich = extractRichSummary(field);
1750:   return {
1751:     plain: plainTextFromRich(rich, "\n").trim(),
1752:     rich
1753:   };
1754: }
```

mergeRichFieldMaps (template-preview.js, lines 1756-1761)

mergeRichFieldMaps part of the private builder frontend and editing experience. In this file, it appears around lines 1756-1761 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clone. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1757: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1756: function mergeRichFieldMaps(current = {}, fallback = {}) {
1757:   return {
1758:     ...clone(fallback || {}),
1759:     ...clone(current || {})
1760:   };
1761: }
```

parsedSiteContentRich (template-preview.js, lines 1763-1765)

parsedSiteContentRich part of the private builder frontend and editing experience. In this file, it appears around lines 1763-1765 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references mergeRichFieldMaps. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1764: return mergeRichFieldMaps(catalog.siteContentRich, savedPortfolioCatalog.siteContentRich);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1763: function parsedSiteContentRich() {
1764:   return mergeRichFieldMaps(catalog.siteContentRich, savedPortfolioCatalog.siteContentRich);
1765: }
```

parsedProfileRich (template-preview.js, lines 1767-1769)

parsedProfileRich part of the private builder frontend and editing experience. In this file, it appears around lines 1767-1769 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references mergeRichFieldMaps. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1768: return mergeRichFieldMaps(catalog.profileRich, savedPortfolioCatalog.profileRich);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1767: function parsedProfileRich() {
1768:   return mergeRichFieldMaps(catalog.profileRich, savedPortfolioCatalog.profileRich);
1769: }
```

parsedFunFactsRich (template-preview.js, lines 1771-1775)

parsedFunFactsRich part of the private builder frontend and editing experience. In this file, it appears around lines 1771-1775 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references call, and clone. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1772: return Object.prototype.hasOwnProperty.call(catalog, "funFactsRich").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1771: function parsedFunFactsRich() {
1772:   return Object.prototype.hasOwnProperty.call(catalog, "funFactsRich")
1773:     ? clone(catalog.funFactsRich || null)
1774:     : clone(savedPortfolioCatalog.funFactsRich || null);
1775: }
```

syncPortfolioTextInputsForParsing (template-preview.js, lines 1777-1781)

syncPortfolioTextInputsForParsing keeps two places aligned, such as local data and target files. In this file, it appears around lines 1777-1781 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references syncFunFactsFromInput, syncSiteContentFromInputs, and syncProfileFromInputs. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1777: function syncPortfolioTextInputsForParsing() {
1778:   if (funFactsInput) syncFunFactsFromInput();
1779:   if (siteHeroTitleInput || siteHeroCopyInput || siteHeroEyebrowInput) syncSiteContentFromInputs();
1780:   if (profileDisplayNameInput || profileEmailInput || profilePhoneInput) syncProfileFromInputs();
1781: }
```

parsedPortfolioGlobals (template-preview.js, lines 1783-1796)

parsedPortfolioGlobals part of the private builder frontend and editing experience. In this file, it appears around lines 1783-1796 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references syncPortfolioTextInputsForParsing, clone, normalizeFieldStyles, normalizeFunFacts, parsedFunFactsRich, normalizeProfile, parsedProfileRich, normalizeSiteContent, parsedSiteContentRich, filter, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1785: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1783: function parsedPortfolioGlobals() {
1784:   syncPortfolioTextInputsForParsing();
1785:   return {
1786:     categories: clone(catalog.categories || savedPortfolioCatalog.categories || []),
1787:     fieldStyles: clone(normalizeFieldStyles(catalog.fieldStyles || savedPortfolioCatalog.fieldStyles ||
{})),
1788:     funFacts: normalizeFunFacts(catalog.funFacts || savedPortfolioCatalog.funFacts || []),
1789:     funFactsRich: parsedFunFactsRich(),
1790:     profile: clone(normalizeProfile(catalog.profile || savedPortfolioCatalog.profile || {})),
1791:     profileRich: parsedProfileRich(),
1792:     siteContent: clone(normalizeSiteContent(catalog.siteContent || savedPortfolioCatalog.siteContent ||
{})),
1793:     siteContentRich: parsedSiteContentRich(),
1794:     siteSections: (catalog.siteSections || savedPortfolioCatalog.siteSections ||
[]).filter(siteSectionRenderable).map(clone)
1795:   };
1796: }
```

syncParsedPortfolioGlobalsIntoSaved (template-preview.js, lines 1798-1800)

syncParsedPortfolioGlobalsIntoSaved keeps two places aligned, such as local data and target files. In this file, it appears around lines 1798-1800 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assign, and parsedPortfolioGlobals. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1798: function syncParsedPortfolioGlobalsIntoSaved() {
1799:   Object.assign(savedPortfolioCatalog, parsedPortfolioGlobals());
1800: }
```

siteContentRichValue (template-preview.js, lines 1802-1804)

siteContentRichValue part of the private builder frontend and editing experience. In this file, it appears around lines 1802-1804 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1803: return catalog.siteContentRich?.[key] || savedPortfolioCatalog.siteContentRich?.[key] || null;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1802: function siteContentRichValue(key) {
1803:   return catalog.siteContentRich?.[key] || savedPortfolioCatalog.siteContentRich?.[key] || null;
1804: }
```

profileRichValue (template-preview.js, lines 1806-1808)

profileRichValue part of the private builder frontend and editing experience. In this file, it appears around lines 1806-1808 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1807: return catalog.profileRich?.[key] || savedPortfolioCatalog.profileRich?.[key] || null;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1806: function profileRichValue(key) {
1807:   return catalog.profileRich?.[key] || savedPortfolioCatalog.profileRich?.[key] || null;
1808: }
```

syncSiteRichField (template-preview.js, lines 1810-1823)

syncSiteRichField keeps two places aligned, such as local data and target files. In this file, it appears around lines 1810-1823 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richFieldValue, normalizeSiteContent, markDraftNeedsSave, scheduleAutosave, and schedulePreviewRender. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1812: if (!key) return;.

Important local values include key at line 1811. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1810: function syncSiteRichField(editor) {
1811:   const key = editor?.dataset.siteField;
1812:   if (!key) return;
1813:   catalog.siteContentRich = catalog.siteContentRich || {};
1814:   const { plain, rich } = richFieldValue(editor, catalog.siteContent?.[key] || "");
1815:   catalog.siteContent = normalizeSiteContent({
1816:     ...(catalog.siteContent || {}),
1817:     [key]: plain
1818:   });
1819:   catalog.siteContentRich[key] = rich;
1820:   markDraftNeedsSave();
1821:   scheduleAutosave();
1822:   schedulePreviewRender();
1823: }
```

syncProfileRichField (template-preview.js, lines 1825-1838)

syncProfileRichField keeps two places aligned, such as local data and target files. In this file, it appears around lines 1825-1838 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richFieldValue, normalizeProfile, markDraftNeedsSave, scheduleAutosave, and schedulePreviewRender. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1827: if (!key) return;.

Important local values include key at line 1826. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1825: function syncProfileRichField(editor) {
1826:   const key = editor?.dataset.profileField;
1827:   if (!key) return;
1828:   catalog.profileRich = catalog.profileRich || {};
1829:   const { plain, rich } = richFieldValue(editor, catalog.profile?.[key] || "");
1830:   catalog.profile = normalizeProfile({
1831:     ...(catalog.profile || {}),
1832:     [key]: plain
1833:   });
1834:   catalog.profileRich[key] = rich;
1835:   markDraftNeedsSave();
1836:   scheduleAutosave();
1837:   schedulePreviewRender();
1838: }
```

renderSiteContentEditor (template-preview.js, lines 1840-1851)

renderSiteContentEditor renders ui, markup, or display output. In this file, it appears around lines 1840-1851 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeSiteContent, populateTextOnlyRichField, and siteContentRichValue. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include content at line 1841. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1840: function renderSiteContentEditor() {
1841:   const content = normalizeSiteContent(catalog.siteContent || {});
1842:   if (siteHeroEyebrowInput && document.activeElement !== siteHeroEyebrowInput) {
1843:     populateTextOnlyRichField(siteHeroEyebrowInput, siteContentRichValue("heroEyebrow"),
content.heroEyebrow);
1844:   }
1845:   if (siteHeroTitleInput && document.activeElement !== siteHeroTitleInput) {
1846:     populateTextOnlyRichField(siteHeroTitleInput, siteContentRichValue("heroTitle"), content.heroTitle);
1847:   }
1848:   if (siteHeroCopyInput && document.activeElement !== siteHeroCopyInput) {
1849:     populateTextOnlyRichField(siteHeroCopyInput, siteContentRichValue("heroCopy"), content.heroCopy);
1850:   }
1851: }
```

renderProfileEditor (template-preview.js, lines 1853-1881)

renderProfileEditor renders ui, markup, or display output. In this file, it appears around lines 1853-1881 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeProfile, forEach, populateTextOnlyRichField, profileRichValue, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include profile at line 1854; fields at line 1855; assetLabels at line 1871. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1853: function renderProfileEditor() {
1854:   const profile = normalizeProfile(catalog.profile || {});
1855:   const fields = [
1856:     [profileDisplayNameInput, profile.displayName],
1857:     [profilePortfolioLabelInput, profile.portfolioLabel],
1858:     [profileContactIntroInput, profile.contactIntro],
1859:     [profileEmailInput, profile.email],
1860:     [profilePhoneInput, profile.phone],
1861:     [profileGithubInput, profile.githubUrl],
1862:     [profileLinkedinInput, profile.linkedinUrl],
```

```
    [profileWebsiteInput, profile.websiteUrl]
  ];
  fields.forEach(([field, value]) => {
    if (field && document.activeElement !== field) {
      populateTextOnlyRichField(field, profileRichValue(field.dataset.profileField), value);
    }
  });
  if (profileAssetStatus) {
    const assetLabels = [
      profile.profileImage ? "profile photo" : "",
      profile.heroImage ? "main background" : "",
      profile.resumeUrl ? "resume" : "",
      profile.brandImage ? "brand icon" : ""
    ].filter(Boolean);
    profileAssetStatus.textContent = assetLabels.length
      ? `Added: ${assetLabels.join(", ")}.`
      : "No profile photo, resume, brand icon, or main background added yet.";
  }
}
```

syncSiteContentFromInputs (template-preview.js, lines 1883-1897)

syncSiteContentFromInputs keeps two places aligned, such as local data and target files. In this file, it appears around lines 1883-1897 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richFieldValue, and normalizeSiteContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1896: return catalog.siteContent;.

Important local values include eyebrow at line 1885; title at line 1886; copy at line 1887. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function syncSiteContentFromInputs() {
  catalog.siteContentRich = catalog.siteContentRich || {};
  const eyebrow = richFieldValue(siteHeroEyebrowInput, catalog.siteContent?.heroEyebrow || "");
  const title = richFieldValue(siteHeroTitleInput, catalog.siteContent?.heroTitle || "");
  const copy = richFieldValue(siteHeroCopyInput, catalog.siteContent?.heroCopy || "");
  catalog.siteContent = normalizeSiteContent({
    heroCopy: copy.plain,
    heroEyebrow: eyebrow.plain,
    heroTitle: title.plain
  });
  catalog.siteContentRich.heroCopy = copy.rich;
  catalog.siteContentRich.heroEyebrow = eyebrow.rich;
  catalog.siteContentRich.heroTitle = title.rich;
  return catalog.siteContent;
}
```

syncProfileFromInputs (template-preview.js, lines 1899-1926)

syncProfileFromInputs keeps two places aligned, such as local data and target files. In this file, it appears around lines 1899-1926 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richFieldValue, normalizeProfile, entries, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1925: return catalog.profile;.

Important local values include fields at line 1901. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function syncProfileFromInputs() {
  catalog.profileRich = catalog.profileRich || {};
  const fields = {
    contactIntro: richFieldValue(profileContactIntroInput, catalog.profile?.contactIntro || ""),
    displayName: richFieldValue(profileDisplayNameInput, catalog.profile?.displayName || ""),
    email: richFieldValue(profileEmailInput, catalog.profile?.email || ""),
    githubUrl: richFieldValue(profileGithubInput, catalog.profile?.githubUrl || ""),
    linkedinUrl: richFieldValue(profileLinkedinInput, catalog.profile?.linkedinUrl || ""),
    phone: richFieldValue(profilePhoneInput, catalog.profile?.phone || ""),
    portfolioLabel: richFieldValue(profilePortfolioLabelInput, catalog.profile?.portfolioLabel || ""),
    websiteUrl: richFieldValue(profileWebsiteInput, catalog.profile?.websiteUrl || "")
```

```
1910:   };
1911:   catalog.profile = normalizeProfile({
1912:     ...(catalog.profile || {}),
1913:     contactIntro: fields.contactIntro.plain,
1914:     displayName: fields.displayName.plain,
1915:     email: fields.email.plain,
1916:     githubUrl: fields.githubUrl.plain,
1917:     linkedinUrl: fields.linkedinUrl.plain,
1918:     phone: fields.phone.plain,
1919:     portfolioLabel: fields.portfolioLabel.plain,
1920:     websiteUrl: fields.websiteUrl.plain
1921:   });
1922:   Object.entries(fields).forEach(([key, value]) => {
1923:     catalog.profileRich[key] = value.rich;
1924:   });
1925:   return catalog.profile;
1926: }
```

clone (template-preview.js, lines 1928-1930)

clone part of the private builder frontend and editing experience. In this file, it appears around lines 1928-1930 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, and stringify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1929: return JSON.parse(JSON.stringify(value));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1928: function clone(value) {
1929:   return JSON.parse(JSON.stringify(value));
1930: }
```

closeDialogElement (template-preview.js, lines 1932-1948)

closeDialogElement controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 1932-1948 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references hasAttribute, close, removeAttribute, dispatchEvent, and Event. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1933: if (!dialog) return;; line 1942: if (!dialog.open && !dialog.hasAttribute("open")) return;.

Important local values include closeValue at line 1934; wasOpen at line 1935. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1932: function closeDialogElement(dialog, returnValue = "close") {
1933:   if (!dialog) return;
1934:   const closeValue = String(returnValue || "close");
1935:   const wasOpen = Boolean(dialog.open || dialog.hasAttribute("open"));
1936:   if (typeof dialog.close === "function") {
1937:     try {
1938:       if (dialog.open) dialog.close(closeValue);
1939:     } catch {
1940:       // Fall through to manual cleanup if the browser rejects the native close call.
1941:     }
1942:     if (!dialog.open && !dialog.hasAttribute("open")) return;
1943:   }
1944:   dialog.returnValue = closeValue;
1945:   dialog.open = false;
1946:   dialog.removeAttribute("open");
1947:   if (wasOpen) dialog.dispatchEvent(new Event("close"));
1948: }
```

getByPath (template-preview.js, lines 1950-1952)

getByPath part of the private builder frontend and editing experience. In this file, it appears around lines 1950-1952 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, filter, and reduce. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1951: return String(pathValue || "").split(".").filter(Boolean).reduce((current, key) => current?.[key], target);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 1950: function getByPath(target, pathValue) {
 1951:   return String(pathValue || "").split(".").filter(Boolean).reduce((current, key) => current?.[key],
target);
 1952: }
```

setByPath (template-preview.js, lines 1954-1963)

setByPath part of the private builder frontend and editing experience. In this file, it appears around lines 1954-1963 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, filter, pop, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include keys at line 1955; last at line 1956; current at line 1957. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1954: function setByPath(target, pathValue, value) {
1955:   const keys = String(pathValue || "").split(".").filter(Boolean);
1956:   const last = keys.pop();
1957:   let current = target;
1958:   keys.forEach((key) => {
1959:     current[key] = current[key] || {};
1960:     current = current[key];
1961:   });
1962:   if (last) current[last] = value;
1963: }
```

defaultDesignModel (template-preview.js, lines 1965-1971)

defaultDesignModel part of the private builder frontend and editing experience. In this file, it appears around lines 1965-1971 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1966: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1965: function defaultDesignModel() {
1966:     return {
1967:         brief: { summary: "", summaryRich: null, files: [] },
1968:         documentation: { summary: "", summaryRich: null, files: [], references: [], mathAnalysis: [] },
1969:         simulation: { summary: "", summaryRich: null, files: [], results: [] }
1970:     };
1971: }
```

isAnalogProject (template-preview.js, lines 1973-1975)

isAnalogProject part of the private builder frontend and editing experience. In this file, it appears around lines 1973-1975 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1974: return project?.category === "analog-mixed-signal";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1973: function isAnalogProject(project) {
1974:   return project?.category === "analog-mixed-signal";
1975: }
```

ensureDesignModel (template-preview.js, lines 1977-1992)

ensureDesignModel part of the private builder frontend and editing experience. In this file, it appears around lines 1977-1992 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references defaultDesignModel. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1978: if (!project) return null;; line 1991: return project.design;.

Important local values include defaults at line 1979. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1977: function ensureDesignModel(project) {
1978:   if (!project) return null;
1979:   const defaults = defaultDesignModel();
1980:   project.design = project.design || {};
1981:   project.design.brief = { ...defaults.brief, ...(project.design.brief || {}) };
1982:   project.design.documentation = { ...defaults.documentation, ...(project.design.documentation || {}) };
1983:   project.design.documentation.summaryRich = project.design.documentation.summaryRich || null;
1984:   project.design.simulation = { ...defaults.simulation, ...(project.design.simulation || {}) };
1985:   project.design.brief.files = project.design.brief.files || [];
1986:   project.design.documentation.files = project.design.documentation.files || [];
1987:   project.design.documentation.references = project.design.documentation.references || [];
1988:   project.design.documentation.mathAnalysis = project.design.documentation.mathAnalysis || [];
1989:   project.design.simulation.files = project.design.simulation.files || [];
1990:   project.design.simulation.results = project.design.simulation.results || [];
1991:   return project.design;
1992: }
```

ensureCompileCode (template-preview.js, lines 1994-2034)

ensureCompileCode part of the compile code language/tool/build/run flow. In this file, it appears around lines 1994-2034 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, map, normalizeCodeLanguage, detectCodeLanguage, safeClientCodeFileName, defaultCodeFileName, normalizeCompileFileRole, inferCompileFileRole, slugify, now, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1995: if (!project) return null;; line 2002: return {; line 2033: return project.compileCode;.

Important local values include language at line 1999; fileName at line 2000; role at line 2001; appendDestinations at line 2021. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1994: function ensureCompileCode(project) {
1995:   if (!project) return null;
1996:   project.compileCode = project.compileCode || {};
1997:   project.compileCode.files = Array.isArray(project.compileCode.files) ? project.compileCode.files : [];
1998:   project.compileCode.files = project.compileCode.files.map((file, index) => {
1999:     const language = normalizeCodeLanguage(file.language || detectCodeLanguage(file.code || "",
file.fileName || ""));
2000:     const fileName = safeClientCodeFileName(file.fileName || defaultCodeFileName(language), language);
2001:     const role = normalizeCompileFileRole(file.role || inferCompileFileRole(fileName, file.code || "",
```

```
language), language);
 2002:     return {
 2003:       id: file.id || slugify(`${fileName}-${index}-${Date.now()}`),
 2004:       title: file.title || fileName,
 2005:       fileName,
 2006:       language,
 2007:       role,
 2008:       code: String(file.code || ""),
 2009:       stdin: String(file.stdin || ""),
 2010:       savedPath: file.savedPath || file.workspacePath || "",
 2011:       savedAt: file.savedAt || "",
 2012:       waveform: file.waveform || file.lastResult?.waveform || null,
 2013:       lastResult: file.lastResult || null,
 2014:       dirty: Boolean(file.dirty)
 2015:     };
 2016:   });
 2017:   if (!project.compileCode.activeFileId || !project.compileCode.files.some((file) => file.id ===
project.compileCode.activeFileId)) {
 2018:     project.compileCode.activeFileId = project.compileCode.files[0]?.id || "";
 2019:   }
 2020:   project.compileCode.terminal = String(project.compileCode.terminal || "");
 2021:   const appendDestinations = compileAppendDestinations(project);
 2022:   if (!project.compileCode.appendTargetId || !appendDestinations.some((destination) => destination.id ===
 project.compileCode.appendTargetId)) {
 2023:     project.compileCode.appendTargetId = appendDestinations[0]?.id || "";
 2024:   }
 2025:   project.compileCode.messages = Array.isArray(project.compileCode.messages)
 2026:     ? project.compileCode.messages.map((message) => ({
 2027:       id: message.id || slugify(`message-${Date.now()}-${Math.random().toString(16).slice(2)}`),
 2028:       at: message.at || new Date().toISOString(),
 2029:       level: ["info", "success", "warning", "error"].includes(message.level) ? message.level : "info",
 2030:       text: String(message.text || "")
 2031:     })).filter((message) => message.text)
 2032:     : [];
 2033:   return project.compileCode;
 2034: }
```

escapeHtml (template-preview.js, lines 2036-2043)

escapeHtml part of the private builder frontend and editing experience. In this file, it appears around lines 2036-2043 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replaceAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2037: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2036: function escapeHtml(value) {
 2037:   return String(value || "")
 2038:     .replaceAll("&", "&amp;")
 2039:     .replaceAll("<", "&lt;")
 2040:     .replaceAll(">", "&gt;")
 2041:     .replaceAll('"', "&quot;")
 2042:     .replaceAll("'", "&#039;");
 2043: }
```

escapeCodeHtml (template-preview.js, lines 2045-2051)

escapeCodeHtml part of the private builder frontend and editing experience. In this file, it appears around lines 2045-2051 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replaceAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2046: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2045: function escapeCodeHtml(value) {
 2046:   return String(value || "")
 2047:     .replaceAll("&", "&amp;")
 2048:     .replaceAll("<", "&lt;")
```

```
2049:     .replaceAll(">", "&gt;")
2050:     .replaceAll('"', "&quot;");
2051: }
```

normalizeCodeLanguage (template-preview.js, lines 2053-2059)

normalizeCodeLanguage validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2053-2059 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, replace, find, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2058: return match?.id || "javascript";.

Important local values include clean at line 2054; match at line 2055. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2053: function normalizeCodeLanguage(value = "") {
2054:   const clean = String(value || "").trim().toLowerCase().replace(/[_-]+/g, " ");
2055:   const match = supportedCodeLanguages.find((language) =>
2056:     language.id === clean || language.aliases.includes(clean)
2057:   );
2058:   return match?.id || "javascript";
2059: }
```

codeLanguageProfile (template-preview.js, lines 2061-2063)

codeLanguageProfile part of the private builder frontend and editing experience. In this file, it appears around lines 2061-2063 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and normalizeCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2062: return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value)) || supportedCodeLanguages.find((language) => language.id === "javascript");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2061: function codeLanguageProfile(value = "") {
2062:   return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value)) ||
supportedCodeLanguages.find((language) => language.id === "javascript");
2063: }
```

codeLanguageLabel (template-preview.js, lines 2065-2067)

codeLanguageLabel part of the private builder frontend and editing experience. In this file, it appears around lines 2065-2067 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and normalizeCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2066: return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value))?.label || "Code";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2065: function codeLanguageLabel(value = "") {
2066:   return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value))?.label
|| "Code";
2067: }
```

sourceLooksCpp (template-preview.js, lines 2069-2073)

sourceLooksCpp part of the private builder frontend and editing experience. In this file, it appears around lines 2069-2073 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2071: return /#include\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text) ||.

Important local values include text at line 2070. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 2069: function sourceLooksCpp(source = "") {
 2070:   const text = String(source || "");
 2071:   return
/#include\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text)
||
 2072:     /\b(std::|cout|cin|cerr|namespace|template\s*<|class\s+\w+|new\s+\w|delete\s+|constexpr|nullptr|using
\s+namespace)\b/.test(text);
 2073: }
```

sourceLooksC (template-preview.js, lines 2075-2079)

sourceLooksC part of the private builder frontend and editing experience. In this file, it appears around lines 2075-2079 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2077: return /#include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||.

Important local values include text at line 2076. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2075: function sourceLooksC(source = "") {
2076:   const text = String(source || "");
2077:   return /#include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||
2078:     /\b(printf|scanf|malloc|calloc|realloc|free|struct\s+\w+|typedef\s+struct)\b/.test(text);
2079: }
```

languageFromFileName (template-preview.js, lines 2081-2090)

languageFromFileName part of the private builder frontend and editing experience. In this file, it appears around lines 2081-2090 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, includes, split, pop, sourceLooksCpp, sourceLooksC, and find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 2085: if (sourceLooksCpp(source)) return "cpp";; line 2086: if (sourceLooksC(source)) return "c";; line 2087: return "c";; line 2089: return supportedCodeLanguages.find((language) => language.extensions?.includes(extension))?.id || "";.

Important local values include clean at line 2082; extension at line 2083. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2081: function languageFromFileName(fileName = "", source = "") {
2082:   const clean = String(fileName || "").toLowerCase();
2083:   const extension = clean.includes(".") ? `.${clean.split(".").pop()}` : "";
2084:   if (extension === ".h") {
2085:     if (sourceLooksCpp(source)) return "cpp";
```

```
2086:     if (sourceLooksC(source)) return "c";
2087:     return "c";
2088:   }
2089:   return supportedCodeLanguages.find((language) => language.extensions?.includes(extension))?.id || "";
2090: }
```

defaultCodeFileName (template-preview.js, lines 2092-2094)

defaultCodeFileName part of the private builder frontend and editing experience. In this file, it appears around lines 2092-2094 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references codeLanguageProfile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2093: return codeLanguageProfile(language)?.defaultFile || "main.js";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2092: function defaultCodeFileName(language = "javascript") {
2093:   return codeLanguageProfile(language)?.defaultFile || "main.js";
2094: }
```

safeClientCodeFileName (template-preview.js, lines 2096-2104)

safeClientCodeFileName validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2096-2104 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references codeLanguageProfile, trim, replace, match, toLowerCase, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2103: return `${namePart}${profile.extensions?.includes(ext) ? ext : fallback.match(/(\.[^.]+)$/)?.[1] || ".js"}`;.

Important local values include profile at line 2097; fallback at line 2098; clean at line 2099; namePart at line 2100; extMatch at line 2101; ext at line 2102. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2096: function safeClientCodeFileName(fileName = "", language = "javascript") {
2097:   const profile = codeLanguageProfile(language);
2098:   const fallback = profile.defaultFile || "main.js";
2099:   const clean = String(fileName || fallback).trim().replace(/[\\/:*?"<>|]+/g, "-");
2100:   const namePart = clean.replace(/\.[^.]+$/, "").replace(/[^a-zA-Z0-9._-]+/g, "-").replace(/^-+|-+$/g,
"") || fallback.replace(/\.[^.]+$/, "");
2101:   const extMatch = clean.match(/(\.[a-zA-Z0-9_+-]+)$/);
2102:   const ext = (extMatch?.[1] || fallback.match(/(\.[^.]+)$/)?.[1] || ".js").toLowerCase();
2103:   return `${namePart}${profile.extensions?.includes(ext) ? ext : fallback.match(/(\.[^.]+)$/)?.[1] ||
".js"}`;
2104: }
```

normalizeCodePasteMode (template-preview.js, lines 2106-2108)

normalizeCodePasteMode validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2106-2108 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2107: return value === "source" ? "source" : "basic";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2106: function normalizeCodePasteMode(value = "") {
2107:   return value === "source" ? "source" : "basic";
2108: }
```

normalizeCodeText (template-preview.js, lines 2110-2119)

normalizeCodeText validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2110-2119 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, normalizeCodePasteMode, split, map, trimEnd, join, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2112: if (normalizeCodePasteMode(pasteMode) === "source") return normalized.replace(/\t/g, " ").replace(/\s+$/g, "");; line 2113: return normalized.

Important local values include normalized at line 2111. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2110: function normalizeCodeText(value = "", pasteMode = "source") {
2111:   const normalized = String(value || "").replace(/\r\n?/g, "\n").replace(/\u00a0/g, " ");
2112:   if (normalizeCodePasteMode(pasteMode) === "source") return normalized.replace(/\t/g, "
").replace(/\s+$/g, "");
2113:   return normalized
2114:     .split("\n")
2115:     .map((line) => line.trimEnd())
2116:     .join("\n")
2117:     .replace(/\n{4,}/g, "\n\n\n")
2118:     .trim();
2119: }
```

detectCodeLanguage (template-preview.js, lines 2121-2136)

detectCodeLanguage part of the private builder frontend and editing experience. In this file, it appears around lines 2121-2136 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references languageFromFileName, b, sourceLooksCpp, and sourceLooksC. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 10 return statement(s). The most important visible returns are: line 2123: if (byFile) return byFile;; line 2125: if (/<\/?[a-z][\s\S]*?>/i.test(code) || /<!doctype\s+html/i.test(code)) return "html";; line 2126: if (/\b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|class\s+\w+)\b/.test(code)) return "systemverilog";; line 2127: if (/\b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/.test(code)) return "verilog";. There are 6 additional return statements in the function body.

Important local values include byFile at line 2122; code at line 2124. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2121: function detectCodeLanguage(value = "", fileName = "") {
2122:   const byFile = languageFromFileName(fileName, value);
2123:   if (byFile) return byFile;
2124:   const code = String(value || "");
2125:   if (/<\/?[a-z][\s\S]*?>/i.test(code) || /<!doctype\s+html/i.test(code)) return "html";
2126:   if (/\b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|
class\s+\w+)\b/.test(code)) return "systemverilog";
2127:   if (/\b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/.test(code)) return
"verilog";
2128:   if (/^\s*\.?(tran|ac|dc|op|model|subckt|ends|param)\b/im.test(code) ||
/\bV\w+\s+\w+\s+\w+\s+(?:DC|SIN|PULSE)?/i.test(code)) return "ltspice";
2129:   if (/\b(def|elif|import\s+\w+|from\s+\w+\s+import|self|None|True|False)\b/.test(code)) return "python";
2130:   if (/\b(public\s+class|private\s+|protected\s+|static\s+void\s+main|System\.out)\b/.test(code)) return
"java";
2131:   if (sourceLooksCpp(code) || sourceLooksC(code) ||
/\b(#include|main\s*\(|puts\s*\(|fprintf|sizeof\s*\()\b/.test(code)) {
2132:     return sourceLooksCpp(code) ? "cpp" : "c";
2133:   }
2134:   if (/\b(const|let|var|function|=>|console\.log|document\.|window\.)\b/.test(code)) return "javascript";
2135:   return "javascript";
2136: }
```

tokenizedCodeHtml (template-preview.js, lines 2138-2192)

tokenizedCodeHtml part of the private builder frontend and editing experience. In this file, it appears around lines 2138-2192 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeCodeHtml, fromCharCode, floor, replace, tokenName, push, protect, includes, normalizeCodeLanguage, b, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 9 return statement(s). The most important visible returns are: line 2149: return `\uE000${output}\uE001`;; line 2155: return token;; line 2176: c: "_Alignas|_Alignof|_Atomic|_Bool|_Complex|_Generic|_Imaginary|_Noreturn|_Static_assert|_Thread_local|auto|break|case|char|const|continue|default|do|double|else|enum|extern|float|for|goto|if|inline|int|long|register|re; line 2177: cpp: "alignas|alignof|and|and_eq|asm|auto|bitand|bitor|bool|break|case|catch|char|char8_t|char16_t|char32_t|class|compl|concept|const|consteval|constexpr|constinit|const_cast|continue|co_await|co_return|co_yield|decltype. There are 5 additional return statements in the function body.

Important local values include html at line 2139; tokens at line 2140; tokenName at line 2141; value at line 2142; output at line 2143; protect at line 2151; token at line 2153; commentPattern at line 2163; plus 2 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2138: function tokenizedCodeHtml(code = "", language = "javascript") {
2139:   let html = escapeCodeHtml(code);
2140:   const tokens = [];
2141:   const tokenName = (index) => {
2142:     let value = Number(index) + 1;
2143:     let output = "";
2144:     while (value > 0) {
2145:       value -= 1;
2146:       output = String.fromCharCode(65 + (value % 26)) + output;
2147:       value = Math.floor(value / 26);
2148:     }
2149:     return `\uE000${output}\uE001`;
2150:   };
2151:   const protect = (pattern, className) => {
2152:     html = html.replace(pattern, (match) => {
2153:       const token = tokenName(tokens.length);
2154:       tokens.push({ token, html: `<span class="${className}">${match}</span>` });
2155:       return token;
2156:     });
2157:   };
2158:
2159:   if (language === "html") {
2160:     protect(/&lt;!--[\s\S]*?--&gt;/g, "code-token-comment");
2161:     protect(/(&lt;\/?)([a-zA-Z0-9:-]+)([\s\S]*?)(\/?&gt;)/g, "code-token-tag");
2162:   } else {
2163:     const commentPattern = ["python", "ltspice"].includes(normalizeCodeLanguage(language))
2164:       ? /\/\*[\s\S]*?\*\/|\/\/[^\n]*|#[^\n]*/g
2165:       : /\/\*[\s\S]*?\*\/|\/\/[^\n]*/g;
2166:     protect(commentPattern, "code-token-comment");
2167:     protect(/"(?:\\.|[^"\\])*"|'(?:\\.|[^'\\])*'|`(?:\\.|[^`\\])*`/g, "code-token-string");
2168:     if (["c", "cpp"].includes(normalizeCodeLanguage(language))) {
2169:       protect(/^#\s*\w+[^\n]*/gm, "code-token-preprocessor");
2170:     }
2171:   }
2172:
2173:   protect(/\b(?:0x[\da-f]+|\d+(?:\.\d+)?(?:e[+-]?\d+)?)\b/gi, "code-token-number");
2174:
2175:   const keywordMap = {
2176:     c: "_Alignas|_Alignof|_Atomic|_Bool|_Complex|_Generic|_Imaginary|_Noreturn|_Static_assert|_Thread_loc
al|auto|break|case|char|const|continue|default|do|double|else|enum|extern|float|for|goto|if|inline|int|long|regi
ster|restrict|return|short|signed|sizeof|static|struct|switch|typedef|union|unsigned|void|volatile|while",
2177:     cpp: "alignas|alignof|and|and_eq|asm|auto|bitand|bitor|bool|break|case|catch|char|char8_t|char16_t|ch
ar32_t|class|compl|concept|const|consteval|constexpr|constinit|const_cast|continue|co_await|co_return|co_yield|d
ecltype|default|delete|do|double|dynamic_cast|else|enum|explicit|export|extern|false|float|for|friend|if|inline|
int|long|mutable|namespace|new|noexcept|not|not_eq|nullptr|operator|or|or_eq|private|protected|public|register|r
einterpret_cast|requires|return|short|signed|sizeof|static|static_assert|static_cast|struct|switch|template|this
|thread_local|throw|true|try|typedef|typeid|typename|union|unsigned|using|virtual|void|volatile|wchar_t|while|xo
r|xor_eq",
2178:     verilog: "always|and|assign|begin|buf|case|casex|casez|deassign|default|defparam|disable|edge|else|en
d|endcase|endfunction|endmodule|endprimitive|endspecify|endtable|endtask|event|for|force|forever|fork|function|g
enerate|genvar|if|initial|inout|input|integer|join|module|nand|negedge|nor|not|or|output|parameter|posedge|primi
tive|reg|release|repeat|signed|specify|supply0|supply1|table|task|tri|wand|while|wire|wor|xnor|xor",
2179:     systemverilog: "always|always_comb|always_ff|always_latch|assign|automatic|begin|bit|case|class|clock
ing|covergroup|default|disable|do|else|end|endcase|endclass|endclocking|endfunction|endmodule|endpackage|endtask
|enum|for|forever|function|generate|genvar|if|initial|input|int|interface|logic|module|negedge|output|package|pa
rameter|posedge|reg|return|signed|task|typedef|wire",
2180:     ltspice: "ac|dc|end|ends|four|func|global|ic|include|lib|meas|model|nodeset|op|options|param|plot|pro
be|save|step|subckt|temp|tran",
2181:     java: "abstract|assert|boolean|break|byte|case|catch|char|class|const|continue|default|do|double|else
|enum|extends|final|finally|float|for|if|implements|import|instanceof|int|interface|long|native|new|null|package
|private|protected|public|return|short|static|strictfp|super|switch|synchronized|this|throw|throws|transient|try
```

```
|void|volatile|while",
 2182:     javascript: "async|await|break|case|catch|class|const|continue|debugger|default|delete|do|else|export
|extends|false|finally|for|from|function|if|import|in|instanceof|let|new|null|of|return|static|super|switch|this
|throw|true|try|typeof|undefined|var|void|while|yield",
 2183:     python: "and|as|assert|async|await|break|class|continue|def|del|elif|else|except|False|finally|for|fr
om|global|if|import|in|is|lambda|None|nonlocal|not|or|pass|raise|return|True|try|while|with|yield",
 2184:     html: "html|head|body|main|section|article|div|span|script|style|link|meta|title|header|footer|nav|bu
tton|input|form|label|img|a|p|pre|code"
 2185:   };
 2186:   const keywords = keywordMap[normalizeCodeLanguage(language)] || keywordMap.javascript;
 2187:   html = html.replace(new RegExp(`\\b(${keywords})\\b`, "g"), '<span class="code-token-
keyword">$1</span>');
 2188:   tokens.forEach((token) => {
 2189:     html = html.replaceAll(token.token, token.html);
 2190:   });
 2191:   return html;
 2192: }
```

tokenName (template-preview.js, lines 2141-2150)

tokenName part of the private builder frontend and editing experience. In this file, it appears around lines 2141-2150 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fromCharCode, and floor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2149: return `\uE000${output}\uE001`;.

Important local values include tokenName at line 2141; value at line 2142; output at line 2143. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2141:   const tokenName = (index) => {
2142:     let value = Number(index) + 1;
2143:     let output = "";
2144:     while (value > 0) {
2145:       value -= 1;
2146:       output = String.fromCharCode(65 + (value % 26)) + output;
2147:       value = Math.floor(value / 26);
2148:     }
2149:     return `\uE000${output}\uE001`;
2150:   };
```

protect (template-preview.js, lines 2151-2157)

protect part of the private builder frontend and editing experience. In this file, it appears around lines 2151-2157 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, tokenName, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2155: return token;.

Important local values include protect at line 2151; token at line 2153. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2151:   const protect = (pattern, className) => {
2152:     html = html.replace(pattern, (match) => {
2153:       const token = tokenName(tokens.length);
2154:       tokens.push({ token, html: `<span class="${className}">${match}</span>` });
2155:       return token;
2156:     });
2157:   };
```

renderRichCodeBlock (template-preview.js, lines 2194-2204)

renderRichCodeBlock renders ui, markup, or display output. In this file, it appears around lines 2194-2204 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCodeLanguage, detectCodeLanguage, normalizeCodeText, codeLanguageLabel, escapeHtml, and tokenizedCodeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2198: return `.

Important local values include language at line 2195; code at line 2196; label at line 2197. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2194: function renderRichCodeBlock(block = {}) {
2195:   const language = normalizeCodeLanguage(block.language || detectCodeLanguage(block.code || ""));
2196:   const code = normalizeCodeText(block.code || "", block.pasteMode || "source");
2197:   const label = codeLanguageLabel(language);
2198:   return `
2199:     <figure class="rich-code-block rich-code-language-${language}">
2200:       <figcaption><span>&lt;/&gt;</span> ${escapeHtml(label)}</figcaption>
2201:       <pre><code>${tokenizedCodeHtml(code, language)}</code></pre>
2202:     </figure>
2203:   `;
2204: }
```

renderPlainMultiline (template-preview.js, lines 2206-2208)

renderPlainMultiline renders ui, markup, or display output. In this file, it appears around lines 2206-2208 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2207: return escapeHtml(value).replace(/\r\n?|\n/g, "
");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2206: function renderPlainMultiline(value = "") {
2207:   return escapeHtml(value).replace(/\r\n?|\n/g, "<br>");
2208: }
```

normalizeSiteContent (template-preview.js, lines 2210-2216)

normalizeSiteContent validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2210-2216 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2211: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2210: function normalizeSiteContent(content = {}) {
2211:   return {
2212:     heroCopy: String(content.heroCopy || "").trim() || defaultSiteContent.heroCopy,
2213:     heroEyebrow: String(content.heroEyebrow || "").trim() || defaultSiteContent.heroEyebrow,
2214:     heroTitle: String(content.heroTitle || "").trim() || defaultSiteContent.heroTitle
2215:   };
2216: }
```

normalizeProfile (template-preview.js, lines 2218-2234)

normalizeProfile validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2218-2234 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2219: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2218: function normalizeProfile(profile = {}) {
2219:   return {
2220:     brandImage: String(profile.brandImage || "").trim(),
2221:     brandText: String(profile.brandText || "").trim() || String(profile.displayName || "").trim() ||
defaultProfile.brandText,
2222:     contactIntro: String(profile.contactIntro || "").trim(),
2223:     displayName: String(profile.displayName || "").trim(),
2224:     email: String(profile.email || "").trim(),
2225:     githubUrl: String(profile.githubUrl || "").trim(),
2226:     heroImage: String(profile.heroImage || "").trim(),
2227:     linkedinUrl: String(profile.linkedinUrl || "").trim(),
2228:     phone: String(profile.phone || "").trim(),
2229:     portfolioLabel: String(profile.portfolioLabel || "").trim() || defaultProfile.portfolioLabel,
2230:     profileImage: String(profile.profileImage || "").trim(),
2231:     resumeUrl: String(profile.resumeUrl || "").trim(),
2232:     websiteUrl: String(profile.websiteUrl || "").trim()
2233:   };
2234: }
```

normalizePlainFieldStyle (template-preview.js, lines 2236-2250)

normalizePlainFieldStyle validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2236-2250 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cleanFontFamily, normalizeFontPx, normalizeTextColor, and rgbColorToHex. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2249: return normalized;.

Important local values include fontFamily at line 2237; fontPx at line 2238; color at line 2239; normalized at line 2240. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2236: function normalizePlainFieldStyle(style = {}) {
2237:   const fontFamily = cleanFontFamily(style.fontFamily || "");
2238:   const fontPx = normalizeFontPx(style.fontPx || style.fontSize || "");
2239:   const color = normalizeTextColor(style.color || "");
2240:   const normalized = {};
2241:
2242:   if (fontFamily) normalized.fontFamily = fontFamily;
2243:   if (fontPx) normalized.fontPx = fontPx;
2244:   if (color) normalized.color = rgbColorToHex(color);
2245:   if (style.bold === true || style.bold === "true") normalized.bold = true;
2246:   if (style.italic === true || style.italic === "true") normalized.italic = true;
2247:   if (style.underline === true || style.underline === "true") normalized.underline = true;
2248:
2249:   return normalized;
2250: }
```

normalizeFieldStyles (template-preview.js, lines 2252-2259)

normalizeFieldStyles validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2252-2259 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fromEntries, entries, filter, has, map, normalizePlainFieldStyle, and keys. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2253: return Object.fromEntries(.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2252: function normalizeFieldStyles(styles = {}) {
2253:   return Object.fromEntries(
2254:     Object.entries(styles || {})
2255:       .filter(([fieldId]) => persistentPlainStyleFieldIds.has(fieldId))
2256:       .map(([fieldId, style]) => [fieldId, normalizePlainFieldStyle(style)])
2257:       .filter(([, style]) => Object.keys(style).length)
2258:   );
2259: }
```

loadBuilderPreferences (template-preview.js, lines 2261-2273)

loadBuilderPreferences loads data from disk, network, browser storage, or a service. In this file, it appears around lines 2261-2273 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, getItem, includes, and applyBuilderPreferences. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include stored at line 2263. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2261: function loadBuilderPreferences() {
2262:   try {
2263:     const stored = JSON.parse(localStorage.getItem(preferenceStorageKey) || "{}");
2264:     builderPreferences = {
2265:       ...defaultBuilderPreferences,
2266:       ...stored,
2267:       theme: ["light", "dark"].includes(stored.theme) ? stored.theme : defaultBuilderPreferences.theme
2268:     };
2269:   } catch {
2270:     builderPreferences = { ...defaultBuilderPreferences };
2271:   }
2272:   applyBuilderPreferences();
2273: }
```

applyBuilderPreferences (template-preview.js, lines 2275-2280)

applyBuilderPreferences part of the private builder frontend and editing experience. In this file, it appears around lines 2275-2280 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include theme at line 2276. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2275: function applyBuilderPreferences() {
2276:   const theme = ["light", "dark"].includes(builderPreferences.theme) ? builderPreferences.theme :
"light";
2277:   document.documentElement.dataset.builderTheme = theme;
2278:   document.body.dataset.builderTheme = theme;
2279:   if (preferenceTheme) preferenceTheme.value = theme;
2280: }
```

setBuilderTheme (template-preview.js, lines 2282-2287)

setBuilderTheme part of the private builder frontend and editing experience. In this file, it appears around lines 2282-2287 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes, setItem, stringify, applyBuilderPreferences, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2282: function setBuilderTheme(theme = "light") {
2283:   builderPreferences.theme = ["light", "dark"].includes(theme) ? theme : "light";
2284:   localStorage.setItem(preferenceStorageKey, JSON.stringify(builderPreferences));
2285:   applyBuilderPreferences();
2286:   setStatus(`Builder ${builderPreferences.theme} mode enabled.`);
2287: }
```

openPreferencesDialog (template-preview.js, lines 2289-2292)

openPreferencesDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 2289-2292 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references applyBuilderPreferences, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2289: function openPreferencesDialog() {
2290:   applyBuilderPreferences();
2291:   if (!preferencesDialog?.open) preferencesDialog.showModal();
2292: }
```

saveBuilderPreferencesFromDialog (template-preview.js, lines 2294-2297)

saveBuilderPreferencesFromDialog persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2294-2297 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setBuilderTheme, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2294: function saveBuilderPreferencesFromDialog() {
2295:   setBuilderTheme(preferenceTheme?.value || "light");
2296:   setStatus("Preferences saved.");
2297: }
```

plainFieldStyleToCss (template-preview.js, lines 2299-2309)

plainFieldStyleToCss part of the private builder frontend and editing experience. In this file, it appears around lines 2299-2309 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizePlainFieldStyle, push, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2308: return rules.join("; ");.

Important local values include normalized at line 2300; rules at line 2301. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2299: function plainFieldStyleToCss(style = {}) {
2300:   const normalized = normalizePlainFieldStyle(style);
2301:   const rules = [];
2302:   if (normalized.fontFamily) rules.push(`font-family: ${normalized.fontFamily}`);
2303:   if (normalized.fontPx) rules.push(`font-size: ${normalized.fontPx}px`);
```

```
2304:   if (normalized.color) rules.push(`color: ${normalized.color}`);
2305:   if (normalized.bold) rules.push("font-weight: 700");
2306:   if (normalized.italic) rules.push("font-style: italic");
2307:   if (normalized.underline) rules.push("text-decoration: underline");
2308:   return rules.join("; ");
2309: }
```

plainFieldStyleAttribute (template-preview.js, lines 2311-2314)

plainFieldStyleAttribute part of the private builder frontend and editing experience. In this file, it appears around lines 2311-2314 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references plainFieldStyleToCss, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2313: return css ? ` style="${escapeHtml(css)}"` : "";.

Important local values include css at line 2312. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2311: function plainFieldStyleAttribute(styles = {}, fieldId = "") {
2312:   const css = plainFieldStyleToCss(styles[fieldId]);
2313:   return css ? ` style="${escapeHtml(css)}"` : "";
2314: }
```

plainStyleFromControl (template-preview.js, lines 2316-2326)

plainStyleFromControl part of the private builder frontend and editing experience. In this file, it appears around lines 2316-2326 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizePlainFieldStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2317: if (!control) return {};; line 2318: return normalizePlainFieldStyle({.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2316: function plainStyleFromControl(control) {
2317:   if (!control) return {};
2318:   return normalizePlainFieldStyle({
2319:     bold: control.dataset.builderBold,
2320:     color: control.dataset.builderColor,
2321:     fontFamily: control.dataset.builderFontFamily,
2322:     fontPx: control.dataset.builderFontPx,
2323:     italic: control.dataset.builderItalic,
2324:     underline: control.dataset.builderUnderline
2325:   });
2326: }
```

applyPlainFieldStyleToControl (template-preview.js, lines 2328-2343)

applyPlainFieldStyleToControl part of the private builder frontend and editing experience. In this file, it appears around lines 2328-2343 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizePlainFieldStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2329: if (!control) return;.

Important local values include normalized at line 2330. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2328: function applyPlainFieldStyleToControl(control, style = {}) {
```

```
2329:   if (!control) return;
2330:   const normalized = normalizePlainFieldStyle(style);
2331:   control.dataset.builderFontFamily = normalized.fontFamily || "";
2332:   control.dataset.builderFontPx = normalized.fontPx || "";
2333:   control.dataset.builderColor = normalized.color || "";
2334:   control.dataset.builderBold = normalized.bold ? "true" : "false";
2335:   control.dataset.builderItalic = normalized.italic ? "true" : "false";
2336:   control.dataset.builderUnderline = normalized.underline ? "true" : "false";
2337:   control.style.fontFamily = normalized.fontFamily || "";
2338:   control.style.fontSize = normalized.fontPx ? `${normalized.fontPx}px` : "";
2339:   control.style.color = normalized.color || "";
2340:   control.style.fontWeight = normalized.bold ? "700" : "";
2341:   control.style.fontStyle = normalized.italic ? "italic" : "";
2342:   control.style.textDecoration = normalized.underline ? "underline" : "";
2343: }
```

applyStoredPlainFieldStyle (template-preview.js, lines 2345-2349)

applyStoredPlainFieldStyle part of the private builder frontend and editing experience. In this file, it appears around lines 2345-2349 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references has, normalizeFieldStyles, and applyPlainFieldStyleToControl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2346: if (!control?.id || !persistentPlainStyleFieldIds.has(control.id)) return;.

Important local values include styles at line 2347. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2345: function applyStoredPlainFieldStyle(control) {
2346:   if (!control?.id || !persistentPlainStyleFieldIds.has(control.id)) return;
2347:   const styles = normalizeFieldStyles(catalog.fieldStyles || {});
2348:   applyPlainFieldStyleToControl(control, styles[control.id] || {});
2349: }
```

storePlainControlStyle (template-preview.js, lines 2351-2365)

storePlainControlStyle persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2351-2365 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references has, normalizeFieldStyles, plainStyleFromControl, keys, clone, markDraftNeedsSave, scheduleAutosave, and schedulePreviewRender. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2352: if (!control?.id || !persistentPlainStyleFieldIds.has(control.id)) return;.

Important local values include nextStyles at line 2353; style at line 2354. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2351: function storePlainControlStyle(control) {
2352:   if (!control?.id || !persistentPlainStyleFieldIds.has(control.id)) return;
2353:   const nextStyles = normalizeFieldStyles(catalog.fieldStyles || {});
2354:   const style = plainStyleFromControl(control);
2355:   if (Object.keys(style).length) {
2356:     nextStyles[control.id] = style;
2357:   } else {
2358:     delete nextStyles[control.id];
2359:   }
2360:   catalog.fieldStyles = nextStyles;
2361:   savedPortfolioCatalog.fieldStyles = clone(nextStyles);
2362:   markDraftNeedsSave();
2363:   scheduleAutosave();
2364:   schedulePreviewRender();
2365: }
```

mailComposeLink (template-preview.js, lines 2367-2370)

mailComposeLink part of the private builder frontend and editing experience. In this file, it appears around lines 2367-2370 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2369: return clean ? `https://mail.google.com/mail/?view=cm&fs=1&to=${encodeURIComponent(clean)}` : "";.

Important local values include clean at line 2368. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2367: function mailComposeLink(email = "") {
2368:   const clean = String(email || "").trim();
2369:   return clean ? `https://mail.google.com/mail/?view=cm&fs=1&to=${encodeURIComponent(clean)}` : "";
2370: }
```

phoneLink (template-preview.js, lines 2372-2375)

phoneLink part of the private builder frontend and editing experience. In this file, it appears around lines 2372-2375 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2374: return clean ? `tel:${clean.replace(/[^\d+]/g, "")}` : "";.

Important local values include clean at line 2373. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2372: function phoneLink(phone = "") {
2373:   const clean = String(phone || "").trim();
2374:   return clean ? `tel:${clean.replace(/[^\d+]/g, "")}` : "";
2375: }
```

textBlocksFromPlainText (template-preview.js, lines 2377-2385)

textBlocksFromPlainText part of the private builder frontend and editing experience. In this file, it appears around lines 2377-2385 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2378: return [{.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2377: function textBlocksFromPlainText(text) {
2378:   return [{
2379:     align: "left",
2380:     fontFamily: "Arial",
2381:     fontSize: "normal",
2382:     text: String(text || ""),
2383:     type: "paragraph"
2384:   }];
2385: }
```

richBlocksForProject (template-preview.js, lines 2387-2389)

richBlocksForProject part of the private builder frontend and editing experience. In this file, it appears around lines 2387-2389 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clone, and textBlocksFromPlainText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2388: return project?.summaryRich?.blocks?.length ? clone(project.summaryRich.blocks) : textBlocksFromPlainText(project?.summary || "");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2387: function richBlocksForProject(project) {
 2388:   return project?.summaryRich?.blocks?.length ? clone(project.summaryRich.blocks) :
textBlocksFromPlainText(project?.summary || "");
 2389: }
```

plainTextFromRich (template-preview.js, lines 2391-2402)

plainTextFromRich part of the private builder frontend and editing experience. In this file, it appears around lines 2391-2402 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 2392: return (rich?.blocks || []); line 2394: if (block.type === "paragraph") return block.text || "";; line 2395: if (block.type === "formula") return block.formula || "";; line 2396: if (block.type === "code") return block.code || "";. There are 2 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2391: function plainTextFromRich(rich, separator = "\n\n") {
2392:   return (rich?.blocks || [])
2393:     .map((block) => {
2394:       if (block.type === "paragraph") return block.text || "";
2395:       if (block.type === "formula") return block.formula || "";
2396:       if (block.type === "code") return block.code || "";
2397:       if (block.type === "image") return [block.title, block.caption].filter(Boolean).join(" ");
2398:       return "";
2399:     })
2400:     .filter(Boolean)
2401:     .join(separator);
2402: }
```

unwrapFormula (template-preview.js, lines 2404-2417)

unwrapFormula part of the private builder frontend and editing experience. In this file, it appears around lines 2404-2417 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and match. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2414: if (match) return match[group].trim();; line 2416: return text;.

Important local values include text at line 2405; wrappers at line 2406; match at line 2413. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2404: function unwrapFormula(value) {
2405:   const text = String(value || "").trim();
2406:   const wrappers = [
2407:     [/^\$\$([\s\S]+)\$\$/, 1],
2408:     [/^\\\[([\s\S]+)\\\]$/, 1],
2409:     [/^\\\(([\s\S]+)\\\)$/, 1],
2410:     [/^\$([^$]+)\$$/, 1]
2411:   ];
2412:   for (const [pattern, group] of wrappers) {
2413:     const match = text.match(pattern);
2414:     if (match) return match[group].trim();
2415:   }
```

```
2416:   return text;
2417: }
```

isFormulaOnly (template-preview.js, lines 2419-2426)

isFormulaOnly part of the private builder frontend and editing experience. In this file, it appears around lines 2419-2426 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and looksLikeWebOrContactText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 2421: if (!text) return false;; line 2422: if (looksLikeWebOrContactText(text)) return false;; line 2423: if (/^\$\$[\s\S]+\$\$$/.test(text) || /^\\\[[\s\S]+\\\]$/.test(text) || /^\\\([\s\S]+\\\)$/.test(text)) return true;; line 2425: return text.length <= 180 && mathSignals.test(text) && !/[.!?]\s*$/.test(text);.

Important local values include text at line 2420; mathSignals at line 2424. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2419: function isFormulaOnly(value) {
2420:   const text = String(value || "").trim();
2421:   if (!text) return false;
2422:   if (looksLikeWebOrContactText(text)) return false;
2423:   if (/^\$\$[\s\S]+\$\$$/.test(text) || /^\\\[[\s\S]+\\\]$/.test(text) || /^\\\([\s\S]+\\\)$/.test(text))
return true;
2424:   const mathSignals = /\\frac|\\sqrt|\\sum|\\int|\\Delta|\\omega|\\pi|[=^_√∫ΣπΩμ]/;
2425:   return text.length <= 180 && mathSignals.test(text) && !/[.!?]\s*$/.test(text);
2426: }
```

renderInlineMath (template-preview.js, lines 2428-2437)

renderInlineMath renders ui, markup, or display output. In this file, it appears around lines 2428-2437 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, replace, b, match, slice, and normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2431: return withMath.replace(/\b(https?:\/\/[^\s<]+|www\.[^\s<]+|(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\/[^\s<]+)/gi, (match) => {; line 2435: return `<a href="${href}" target="_blank" rel="noreferrer">${clean}</a>${trailing}`;.

Important local values include escaped at line 2429; withMath at line 2430; trailing at line 2432; clean at line 2433; href at line 2434. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2428: function renderInlineMath(text) {
2429:   const escaped = escapeHtml(text);
2430:   const withMath = escaped.replace(/\$([^$]+)\$/g, '<span class="rich-inline-formula">$1</span>');
2431:   return withMath.replace(/\b(https?:\/\/[^\s<]+|www\.[^\s<]+|(?:github|linkedin|gitlab|bitbucket|drive|d
ocs)\.com\/[^\s<]+)/gi, (match) => {
2432:     const trailing = match.match(/[),.;:!?]+$/)?.[0] || "";
2433:     const clean = trailing ? match.slice(0, -trailing.length) : match;
2434:     const href = normalizeLinkTarget(clean, { assumeWeb: true });
2435:     return `<a href="${href}" target="_blank" rel="noreferrer">${clean}</a>${trailing}`;
2436:   });
2437: }
```

renderMultilineInlineText (template-preview.js, lines 2439-2441)

renderMultilineInlineText renders ui, markup, or display output. In this file, it appears around lines 2439-2441 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderInlineMath, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2440: return renderInlineMath(text).replace(/\r\n?|\n/g, "
");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2439: function renderMultilineInlineText(text = "") {
2440:   return renderInlineMath(text).replace(/\r\n?|\n/g, "<br>");
2441: }
```

looksLikeBareWebAddress (template-preview.js, lines 2443-2450)

looksLikeBareWebAddress part of the private builder frontend and editing experience. In this file, it appears around lines 2443-2450 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, split, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 2445: if (!clean || /\s/.test(clean) || /^(\.?\.?\/|#|mailto:|tel:)/i.test(clean)) return false;; line 2446: if (/^[a-z][a-z0-9+.-]*:/i.test(clean)) return false;; line 2448: if (!/^[a-z0-9-]+(\.[a-z0-9-]+)+$/.test(host)) return false;; line 2449: return !/\.(pdf|docx?|xlsx?|pptx?|zip|7z|rar|png|jpe?g|gif|svg|webp|txt|md|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhdl?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb)$/i.test(host);.

Important local values include clean at line 2444; host at line 2447. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2443: function looksLikeBareWebAddress(value = "") {
2444:   const clean = String(value || "").trim();
2445:   if (!clean || /\s/.test(clean) || /^(\.?\.?\/|#|mailto:|tel:)/i.test(clean)) return false;
2446:   if (/^[a-z][a-z0-9+.-]*:/i.test(clean)) return false;
2447:   const host = clean.split(/[/?#]/)[0].toLowerCase();
2448:   if (!/^[a-z0-9-]+(\.[a-z0-9-]+)+$/.test(host)) return false;
2449:   return !/\.(pdf|docx?|xlsx?|pptx?|zip|7z|rar|png|jpe?g|gif|svg|webp|txt|md|csv|json|xml|log|c|h|cpp|hpp
|py|js|mjs|ts|v|sv|vhdl?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb)$/i.test(host);
2450: }
```

looksLikeWebOrContactText (template-preview.js, lines 2452-2460)

looksLikeWebOrContactText part of the private builder frontend and editing experience. In this file, it appears around lines 2452-2460 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, b, looksLikeBareWebAddress, split, some, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 2454: if (!clean) return false;; line 2455: if (/\b(?:https?:\/\/|www\.)[^\s<>"']+/i.test(clean)) return true;; line 2456: if (/\b[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}\b/i.test(clean)) return true;; line 2457: if (/\b(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\/[^\s<>"']+/i.test(clean)) return true;. There are 2 additional return statements in the function body.

Important local values include clean at line 2453. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2452: function looksLikeWebOrContactText(value = "") {
2453:   const clean = String(value || "").trim();
2454:   if (!clean) return false;
2455:   if (/\b(?:https?:\/\/|www\.)[^\s<>"']+/i.test(clean)) return true;
2456:   if (/\b[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}\b/i.test(clean)) return true;
2457:   if (/\b(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\/[^\s<>"']+/i.test(clean)) return true;
2458:   if (looksLikeBareWebAddress(clean)) return true;
2459:   return clean.split(/\s+/).some((word) => looksLikeBareWebAddress(word.replace(/^[("']+|[)"'.,;:!?]+$/g,
""")));
2460: }
```

normalizeLinkTarget (template-preview.js, lines 2462-2469)

normalizeLinkTarget validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2462-2469 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and looksLikeBareWebAddress. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 2464: if (!clean) return "";; line 2465: if (/^\/\//.test(clean)) return `https:${clean}`;; line 2466: if (/^www\./i.test(clean)) return `https://${clean}`;; line 2467: if (options.assumeWeb && looksLikeBareWebAddress(clean)) return `https://${clean}`;. There are 1 additional return statements in the function body.

Important local values include clean at line 2463. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2462: function normalizeLinkTarget(target = "", options = {}) {
2463:   const clean = String(target || "").trim();
2464:   if (!clean) return "";
2465:   if (/^\/\//.test(clean)) return `https:${clean}`;
2466:   if (/^www\./i.test(clean)) return `https://${clean}`;
2467:   if (options.assumeWeb && looksLikeBareWebAddress(clean)) return `https://${clean}`;
2468:   return clean;
2469: }
```

isWebsiteLinkItem (template-preview.js, lines 2471-2479)

isWebsiteLinkItem part of the private builder frontend and editing experience. In this file, it appears around lines 2471-2479 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, filter, join, toLowerCase, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2473: if (/^(https?:)?\/\//i.test(clean) || /^www\./i.test(clean)) return true;; line 2478: return /\b(link|website|web link|url|external|github|linkedin|drive|social|profile)\b/.test(typeText);.

Important local values include clean at line 2472; typeText at line 2474. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2471: function isWebsiteLinkItem(item = {}, target = "") {
2472:   const clean = String(target || "").trim();
2473:   if (/^(https?:)?\/\//i.test(clean) || /^www\./i.test(clean)) return true;
2474:   const typeText = [item.kind, item.type, item.status, item.meta, item.label, item.title]
2475:     .filter(Boolean)
2476:     .join(" ")
2477:     .toLowerCase();
2478:   return /\b(link|website|web link|url|external|github|linkedin|drive|social|profile)\b/.test(typeText);
2479: }
```

extensionFromUrl (template-preview.js, lines 2481-2485)

extensionFromUrl part of the private builder frontend and editing experience. In this file, it appears around lines 2481-2485 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, match, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2484: return match ? match[1].toLowerCase() : "";.

Important local values include clean at line 2482; match at line 2483. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2481: function extensionFromUrl(value = "") {
2482:   const clean = String(value || "").split(/[?#]/)[0];
2483:   const match = clean.match(/(\.[a-z0-9_]+)$/i);
2484:   return match ? match[1].toLowerCase() : "";
2485: }
```

urlLooksLikeDirectImage (template-preview.js, lines 2487-2491)

urlLooksLikeDirectImage part of the private builder frontend and editing experience. In this file, it appears around lines 2487-2491 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2489: return /^(data:image\/|blob:)/i.test(clean) ||.

Important local values include clean at line 2488. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2487: function urlLooksLikeDirectImage(value = "") {
2488:   const clean = normalizeLinkTarget(value, { assumeWeb: true });
2489:   return /^(data:image\/|blob:)/i.test(clean) ||
2490:     /\.(png|jpe?g|webp|gif|svg|bmp|avif)([?#].*)?$/i.test(clean);
2491: }
```

inferUrlAssetKind (template-preview.js, lines 2493-2506)

inferUrlAssetKind part of the private builder frontend and editing experience. In this file, it appears around lines 2493-2506 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, extensionFromUrl, urlLooksLikeDirectImage, includes, and looksLikeBareWebAddress. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 10 return statement(s). The most important visible returns are: line 2496: if (source === "drive" || /(^https?:\/\/)?(drive|docs)\.google\.com\//i.test(clean)) return "google-drive";; line 2497: if (/github\.com|raw\.githubusercontent\.com/i.test(clean)) return "github";; line 2498: if (/linkedin\.com/i.test(clean)) return "linkedin";; line 2499: if (urlLooksLikeDirectImage(clean)) return "image";. There are 6 additional return statements in the function body.

Important local values include clean at line 2494; extension at line 2495. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2493: function inferUrlAssetKind(value = "", source = "web") {
2494:   const clean = normalizeLinkTarget(value, { assumeWeb: true });
2495:   const extension = extensionFromUrl(clean);
2496:   if (source === "drive" || /(^https?:\/\/)?(drive|docs)\.google\.com\//i.test(clean)) return "google-
drive";
2497:   if (/github\.com|raw\.githubusercontent\.com/i.test(clean)) return "github";
2498:   if (/linkedin\.com/i.test(clean)) return "linkedin";
2499:   if (urlLooksLikeDirectImage(clean)) return "image";
2500:   if ([".pdf"].includes(extension)) return "pdf";
2501:   if ([".doc", ".docx", ".ppt", ".pptx", ".xls", ".xlsx", ".csv", ".txt", ".md"].includes(extension))
return "document";
2502:   if ([".zip", ".7z", ".rar"].includes(extension)) return "archive";
2503:   if ([".c", ".h", ".cpp", ".hpp", ".py", ".js", ".mjs", ".ts", ".v", ".sv", ".vhd", ".vhdl", ".spice",
".cir", ".net", ".asc", ".sch", ".kicad_sch", ".kicad_pcb"].includes(extension)) return "engineering-file";
2504:   if (/^https?:\/\//i.test(clean) || /^www\./i.test(value) || looksLikeBareWebAddress(value)) return
"webpage";
2505:   return extension ? "file" : "link";
2506: }
```

labelForUrlAssetKind (template-preview.js, lines 2508-2522)

labelForUrlAssetKind part of the private builder frontend and editing experience. In this file, it appears around lines 2508-2522 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2509: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2508: function labelForUrlAssetKind(kind = "link") {
2509:   return {
2510:     archive: "Archive link",
2511:     document: "Document link",
2512:     "engineering-file": "Engineering file link",
2513:     file: "File link",
2514:     github: "GitHub link",
2515:     "google-drive": "Google Drive link",
2516:     image: "Image URL",
2517:     linkedin: "LinkedIn link",
2518:     link: "Link",
2519:     pdf: "PDF link",
2520:     webpage: "Website link"
2521:   }[kind] || "Link";
2522: }
```

displayNameFromUrl (template-preview.js, lines 2524-2533)

displayNameFromUrl part of the private builder frontend and editing experience. In this file, it appears around lines 2524-2533 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, URL, split, filter, pop, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2529: return decodeURIComponent(lastPath || parsed.hostname.replace(/^www\./, "")) || fallback;; line 2531: return String(value || "").split("/").filter(Boolean).pop() || fallback;.

Important local values include clean at line 2525; parsed at line 2527; lastPath at line 2528. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2524: function displayNameFromUrl(value = "", fallback = "Linked asset") {
2525:   const clean = normalizeLinkTarget(value, { assumeWeb: true });
2526:   try {
2527:     const parsed = new URL(clean);
2528:     const lastPath = parsed.pathname.split("/").filter(Boolean).pop();
2529:     return decodeURIComponent(lastPath || parsed.hostname.replace(/^www\./, "")) || fallback;
2530:   } catch {
2531:     return String(value || "").split("/").filter(Boolean).pop() || fallback;
2532:   }
2533: }
```

linkifyRichTextNodes (template-preview.js, lines 2535-2570)

linkifyRichTextNodes part of the private builder frontend and editing experience. In this file, it appears around lines 2535-2570 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createTreeWalker, acceptNode, closest, nextNode, push, forEach, createDocumentFragment, replace, b, append, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2538: if (!node.textContent || !/(https?:\/\/|www\.|(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\/)/i.test(node.textContent)) return NodeFilter.FILTER_REJECT;; line 2539: return node.parentElement?.closest("a") ? NodeFilter.FILTER_REJECT : NodeFilter.FILTER_ACCEPT;; line 2565: return match;.

Important local values include walker at line 2536; nodes at line 2542; node at line 2543; text at line 2550; fragment at line 2551; cursor at line 2552; trailing at line 2555; clean at line 2556; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2535: function linkifyRichTextNodes(root) {
2536:   const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {
2537:     acceptNode(node) {
2538:       if (!node.textContent ||
!/(https?:\/\/|www\.|(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\/)/i.test(node.textContent)) return
NodeFilter.FILTER_REJECT;
2539:       return node.parentElement?.closest("a") ? NodeFilter.FILTER_REJECT : NodeFilter.FILTER_ACCEPT;
2540:     }
2541:   });
2542:   const nodes = [];
2543:   let node = walker.nextNode();
```

```
  while (node) {
    nodes.push(node);
    node = walker.nextNode();
  }

  nodes.forEach((textNode) => {
    const text = textNode.textContent || "";
    const fragment = document.createDocumentFragment();
    let cursor = 0;
    text.replace(/\b(https?:\/\/[^\s<]+|www\.[^\s<]+|(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com
\/[^\s<]+)/gi, (match, _url, offset) => {
      if (offset > cursor) fragment.append(document.createTextNode(text.slice(cursor, offset)));
      const trailing = match.match(/[),.;:!?]+$/)?.[0] || "";
      const clean = trailing ? match.slice(0, -trailing.length) : match;
      const link = document.createElement("a");
      link.href = normalizeLinkTarget(clean, { assumeWeb: true });
      link.target = "_blank";
      link.rel = "noreferrer";
      link.textContent = clean;
      fragment.append(link);
      if (trailing) fragment.append(document.createTextNode(trailing));
      cursor = offset + match.length;
      return match;
    });
    if (cursor < text.length) fragment.append(document.createTextNode(text.slice(cursor)));
    textNode.replaceWith(fragment);
  });
}
```

inlineStyleSignature (template-preview.js, lines 2572-2583)

inlineStyleSignature part of the private builder frontend and editing experience. In this file, it appears around lines 2572-2583 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2574: if (!style) return "";; line 2575: return [.

Important local values include style at line 2573. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function inlineStyleSignature(element) {
  const style = element?.style;
  if (!style) return "";
  return [
    style.fontFamily || "",
    style.fontSize || "",
    style.color || "",
    style.fontWeight || "",
    style.fontStyle || "",
    style.textDecoration || ""
  ].join("|");
}
```

normalizeInlineStyleSpans (template-preview.js, lines 2585-2605)

normalizeInlineStyleSpans validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2585-2605 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, remove, getAttribute, replaceWith, normalize, matches, inlineStyleSignature, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2586: if (!root) return;; line 2591: return;.

Important local values include next at line 2597; remove at line 2600. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function normalizeInlineStyleSpans(root) {
  if (!root) return;
```

```
2587:   root.querySelectorAll("span.rich-inline-style, span[style]").forEach((span) => {
2588:     span.style.display = "inline";
2589:     if (!span.textContent) {
2590:       span.remove();
2591:       return;
2592:     }
2593:     if (!span.getAttribute("style")) span.replaceWith(...span.childNodes);
2594:   });
2595:   root.normalize();
2596:   root.querySelectorAll("span.rich-inline-style, span[style]").forEach((span) => {
2597:     let next = span.nextSibling;
2598:     while (next?.nodeType === Node.ELEMENT_NODE && next.matches("span.rich-inline-style, span[style]") &&
inlineStyleSignature(next) === inlineStyleSignature(span)) {
2599:       span.append(...next.childNodes);
2600:       const remove = next;
2601:       next = next.nextSibling;
2602:       remove.remove();
2603:     }
2604:   });
2605: }
```

sanitizeRichInlineHtml (template-preview.js, lines 2607-2651)

sanitizeRichInlineHtml validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2607-2651 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, Set, querySelectorAll, forEach, has, replaceWith, getAttribute, trim, normalizeLinkTarget, cleanFontFamily, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2615: return;; line 2650: return template.innerHTML;.

Important local values include template at line 2608; allowedTags at line 2610; href at line 2618; normalizedHref at line 2619; safeHref at line 2620; fontFamily at line 2621; fontPx at line 2622; color at line 2623; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2607: function sanitizeRichInlineHtml(value = "") {
2608:   const template = document.createElement("template");
2609:   template.innerHTML = String(value || "");
2610:   const allowedTags = new Set(["A", "B", "BR", "EM", "I", "SPAN", "STRONG", "U"]);
2611:
2612:   [...template.content.querySelectorAll("*")].forEach((node) => {
2613:     if (!allowedTags.has(node.tagName)) {
2614:       node.replaceWith(...node.childNodes);
2615:       return;
2616:     }
2617:
2618:     const href = node.tagName === "A" ? String(node.getAttribute("href") || "").trim() : "";
2619:     const normalizedHref = normalizeLinkTarget(href, { assumeWeb: true });
2620:     const safeHref = /^(https?:|mailto:|tel:|#|\/)/i.test(normalizedHref) ? normalizedHref : "";
2621:     const fontFamily = cleanFontFamily(node.style.fontFamily || "");
2622:     const fontPx = normalizeFontPx(parseFloat(node.style.fontSize || ""));
2623:     const color = normalizeTextColor(node.style.color || "");
2624:     const rawFontWeight = node.style.fontWeight || "";
2625:     const fontWeight = /^(bold|[6-9]00)$/i.test(rawFontWeight)
2626:       ? "700"
2627:       : /^(normal|[1-5]00)$/i.test(rawFontWeight)
2628:         ? "400"
2629:         : "";
2630:     const fontStyle = node.style.fontStyle === "italic" ? "italic" : "";
2631:     const textDecoration = node.style.textDecoration.includes("underline") ? "underline" : "";
2632:
2633:     [...node.attributes].forEach((attribute) => node.removeAttribute(attribute.name));
2634:
2635:     if (node.tagName === "A" && safeHref) {
2636:       node.setAttribute("href", safeHref);
2637:       node.setAttribute("target", "_blank");
2638:       node.setAttribute("rel", "noreferrer");
2639:     }
2640:     if (fontFamily) node.style.fontFamily = fontFamily;
2641:     if (fontPx) node.style.fontSize = `${fontPx}px`;
2642:     if (color) node.style.color = color;
2643:     if (fontWeight) node.style.fontWeight = fontWeight;
2644:     if (fontStyle) node.style.fontStyle = fontStyle;
2645:     if (textDecoration) node.style.textDecoration = textDecoration;
2646:   });
2647:
2648:   linkifyRichTextNodes(template.content);
2649:   normalizeInlineStyleSpans(template.content);
2650:   return template.innerHTML;
```

```
2651: }
```

displayTitle (template-preview.js, lines 2653-2662)

displayTitle part of the private builder frontend and editing experience. In this file, it appears around lines 2653-2662 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2661: return replacements[title] || title;.

Important local values include title at line 2654; replacements at line 2655. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2653: function displayTitle(value, fallback = "Untitled item") {
2654:   const title = String(value || fallback).trim();
2655:   const replacements = {
2656:     "Brief": "Overview",
2657:     "Project Brief": "Overview",
2658:     "Design Brief": "Design Overview",
2659:     "Simulation Brief": "Simulation Overview"
2660:   };
2661:   return replacements[title] || title;
2662: }
```

cleanFontFamily (template-preview.js, lines 2664-2671)

cleanFontFamily part of the private builder frontend and editing experience. In this file, it appears around lines 2664-2671 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, replace, trim, filter, slice, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2665: return String(value).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2664: function cleanFontFamily(value = "") {
2665:   return String(value)
2666:     .split(",")
2667:     .map((item) => item.replace(/[^\w\s-]/g, "").trim())
2668:     .filter(Boolean)
2669:     .slice(0, 3)
2670:     .join(", ");
2671: }
```

normalizeFontPx (template-preview.js, lines 2673-2676)

normalizeFontPx validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2673-2676 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2675: return Number.isFinite(size) && size >= 8 && size <= 24 ? String(size) : "";.

Important local values include size at line 2674. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2673: function normalizeFontPx(value) {
2674:   const size = Number(value);
2675:   return Number.isFinite(size) && size >= 8 && size <= 24 ? String(size) : "";
```

```
2676: }
```

normalizeTextColor (template-preview.js, lines 2677-2684)

normalizeTextColor validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2677-2684 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2679: return /^#([0-9a-f]{3}|[0-9a-f]{6})$/i.test(color) ||.

Important local values include color at line 2678. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2677: function normalizeTextColor(value = "") {
2678:     const color = String(value || "").trim();
2679:     return /^#([0-9a-f]{3}|[0-9a-f]{6})$/i.test(color) ||
2680:         /^rgba?\(/i.test(color) ||
2681:         /^hsla?\(/i.test(color)
2682:         ? color
2683:         : "";
2684: }
```

summaryRichPathFromTextPath (template-preview.js, lines 2686-2688)

summaryRichPathFromTextPath part of the private builder frontend and editing experience. In this file, it appears around lines 2686-2688 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2687: return String(pathValue || "").replace(/\.summary$/, ".summaryRich");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2686: function summaryRichPathFromTextPath(pathValue = "") {
2687:     return String(pathValue || "").replace(/\.summary$/, ".summaryRich");
2688: }
```

overviewFolderFromPath (template-preview.js, lines 2690-2695)

overviewFolderFromPath part of the private builder frontend and editing experience. In this file, it appears around lines 2690-2695 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 2691: if (pathValue.includes("design.brief")) return "design-overview";; line 2692: if (pathValue.includes("design.documentation")) return "documentation-overview";; line 2693: if (pathValue.includes("design.simulation")) return "simulation-overview";; line 2694: return "overview";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2690: function overviewFolderFromPath(pathValue = "") {
2691:     if (pathValue.includes("design.brief")) return "design-overview";
2692:     if (pathValue.includes("design.documentation")) return "documentation-overview";
2693:     if (pathValue.includes("design.simulation")) return "simulation-overview";
2694:     return "overview";
2695: }
```

richBlocksFromValue (template-preview.js, lines 2697-2699)

richBlocksFromValue part of the private builder frontend and editing experience. In this file, it appears around lines 2697-2699 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clone, and textBlocksFromPlainText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2698: return rich?.blocks?.length ? clone(rich.blocks) : textBlocksFromPlainText(fallbackText);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2697: function richBlocksFromValue(rich, fallbackText = "") {
2698:     return rich?.blocks?.length ? clone(rich.blocks) : textBlocksFromPlainText(fallbackText);
2699: }
```

richTextStyle (template-preview.js, lines 2700-2714)

richTextStyle part of the private builder frontend and editing experience. In this file, it appears around lines 2700-2714 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cleanFontFamily, normalizeFontPx, normalizeTextColor, push, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2713: return styles.length ? ` style="${escapeHtml(styles.join("; "))}"` : "";.

Important local values include styles at line 2701; fontFamily at line 2702; fontPx at line 2703; color at line 2704. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2700: function richTextStyle(block = {}) {
2701:     const styles = [];
2702:     const fontFamily = cleanFontFamily(block.fontFamily || "Arial") || "Arial";
2703:     const fontPx = normalizeFontPx(block.fontPx);
2704:     const color = normalizeTextColor(block.color || "");
2705:
2706:     if (fontFamily) styles.push(`font-family: ${fontFamily}`);
2707:     if (fontPx) styles.push(`font-size: ${fontPx}px`);
2708:     if (color) styles.push(`color: ${color}`);
2709:     if (block.bold) styles.push("font-weight: 700");
2710:     if (block.italic) styles.push("font-style: italic");
2711:     if (block.underline) styles.push("text-decoration: underline");
2712:
2713:     return styles.length ? ` style="${escapeHtml(styles.join("; "))}"` : "";
2714: }
```

normalizeCropAspect (template-preview.js, lines 2716-2718)

normalizeCropAspect validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2716-2718 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2717: return ["1 / 1", "4 / 3", "16 / 9", "3 / 4"].includes(value) ? value : "original";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2716: function normalizeCropAspect(value = "") {
2717:   return ["1 / 1", "4 / 3", "16 / 9", "3 / 4"].includes(value) ? value : "original";
2718: }
```

normalizeCropZoom (template-preview.js, lines 2720-2723)

normalizeCropZoom validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2720-2723 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2722: return Number.isFinite(zoom) ? Math.min(3, Math.max(1, zoom)) : 1;.

Important local values include zoom at line 2721. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2720: function normalizeCropZoom(value = 1) {
2721:   const zoom = Number(value);
2722:   return Number.isFinite(zoom) ? Math.min(3, Math.max(1, zoom)) : 1;
2723: }
```

normalizeCropPosition (template-preview.js, lines 2725-2728)

normalizeCropPosition validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2725-2728 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2727: return Number.isFinite(position) ? Math.min(100, Math.max(0, position)) : 50;.

Important local values include position at line 2726. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2725: function normalizeCropPosition(value = 50) {
2726:   const position = Number(value);
2727:   return Number.isFinite(position) ? Math.min(100, Math.max(0, position)) : 50;
2728: }
```

richImageCropStyle (template-preview.js, lines 2730-2738)

richImageCropStyle part of the private builder frontend and editing experience. In this file, it appears around lines 2730-2738 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCropAspect, normalizeCropZoom, normalizeCropPosition, push, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2737: return ` style="${escapeHtml(styles.join("; "))}"`;.

Important local values include aspect at line 2731; zoom at line 2732; x at line 2733; y at line 2734; styles at line 2735. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2730: function richImageCropStyle(block = {}) {
2731:   const aspect = normalizeCropAspect(block.cropAspect);
2732:   const zoom = normalizeCropZoom(block.cropZoom);
2733:   const x = normalizeCropPosition(block.cropX);
2734:   const y = normalizeCropPosition(block.cropY);
2735:   const styles = [`--crop-zoom: ${zoom}`, `--crop-x: ${x}%`, `--crop-y: ${y}%`];
2736:   if (aspect !== "original") styles.push(`--crop-aspect: ${aspect}`);
2737:   return ` style="${escapeHtml(styles.join("; "))}"`;
2738: }
```

richImageDownloadLink (template-preview.js, lines 2740-2745)

richImageDownloadLink part of the private builder frontend and editing experience. In this file, it appears around lines 2740-2745 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, cleanRichImageTitle, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, and downloadAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2744: return `<p class="rich-download-only"><a class="resource-link" href="${escapeHtml(target)}"${linkAttributes(target, block)}${downloadAttribute(target, block)}>${label}</a></p>`;.

Important local values include label at line 2741; url at line 2742; target at line 2743. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2740: function richImageDownloadLink(block = {}) {
2741:   const label = escapeHtml(cleanRichImageTitle(block) || block.caption || "Image file");
2742:   const url = block.url || "#";
2743:   const target = normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(block, url) });
2744:   return `<p class="rich-download-only"><a class="resource-link"
href="${escapeHtml(target)}"${linkAttributes(target, block)}${downloadAttribute(target,
block)}>${label}</a></p>`;
2745: }
```

cleanRichImageTitle (template-preview.js, lines 2747-2750)

cleanRichImageTitle part of the private builder frontend and editing experience. In this file, it appears around lines 2747-2750 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2749: return /^pasted image\b/i.test(title) ? "" : title;.

Important local values include title at line 2748. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2747: function cleanRichImageTitle(block = {}) {
2748:   const title = String(block.title || "").trim();
2749:   return /^pasted image\b/i.test(title) ? "" : title;
2750: }
```

renderRichContent (template-preview.js, lines 2752-2787)

renderRichContent renders ui, markup, or display output. In this file, it appears around lines 2752-2787 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textBlocksFromPlainText, map, includes, richImageDownloadLink, cleanRichImageTitle, normalizeLinkTarget, normalizeCropAspect, richImageCropStyle, escapeHtml, unwrapFormula, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 2754: return `; line 2759: if (block.display === "download") return richImageDownloadLink(block);; line 2762: return `; line 2774: return `<p class="rich-paragraph rich-text-normal text-${align}">${renderInlineMath(formula)}</p>`;. There are 3 additional return statements in the function body.

Important local values include blocks at line 2753; align at line 2757; title at line 2760; imageSrc at line 2761; formula at line 2772; size at line 2779; content at line 2780. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2752: function renderRichContent(rich, fallbackText = "") {
2753:   const blocks = rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText);
2754:   return `
2755:     <div class="rich-content">
```

```
      ${blocks.map((block) => {
        const align = ["left", "center", "right"].includes(block.align) ? block.align : "left";
        if (block.type === "image") {
          if (block.display === "download") return richImageDownloadLink(block);
          const title = cleanRichImageTitle(block);
          const imageSrc = normalizeLinkTarget(block.url, { assumeWeb: true });
          return `
            <figure class="rich-image justify-${align}">
              <span class="rich-image-viewport crop-${normalizeCropAspect(block.cropAspect) ===
"original" ? "original" : "active"}"${richImageCropStyle(block)}>
                <img src="${escapeHtml(imageSrc)}" alt="${escapeHtml(title || "Overview image")}">
              </span>
              ${(title || block.caption) ? `<figcaption>${title ? `<strong>${escapeHtml(title)}</strong>`
: ""}${block.caption ? `<span>${escapeHtml(block.caption)}</span>` : ""}</figcaption>` : ""}
            </figure>
          `;
        }
        if (block.type === "formula") {
          const formula = unwrapFormula(block.formula);
          if (looksLikeWebOrContactText(formula)) {
            return `<p class="rich-paragraph rich-text-normal
text-${align}">${renderInlineMath(formula)}</p>`;
          }
          return `<div class="rich-formula justify-${align}">${escapeHtml(formula)}</div>`;
        }
        if (block.type === "code") return renderRichCodeBlock(block);
        const size = ["small", "normal", "large"].includes(block.fontSize) ? block.fontSize : "normal";
        const content = block.html
          ? sanitizeRichInlineHtml(block.html)
          : renderInlineMath(block.text || "");
        return `<p class="rich-paragraph rich-text-${size}
text-${align}"${richTextStyle(block)}>${content}</p>`;
      }).join("")}
    </div>
  `;
}
```

renderRichFieldContent (template-preview.js, lines 2789-2799)

renderRichFieldContent renders ui, markup, or display output. In this file, it appears around lines 2789-2799 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textBlocksFromPlainText, map, includes, sanitizeRichInlineHtml, renderInlineMath, richTextStyle, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2791: return blocks.map((block) => {; line 2792: if (block.type !== "paragraph") return "";; line 2797: return `<span class="rich-field-line text-${align}"${richTextStyle(block)}>${content}</span>`;.

Important local values include blocks at line 2790; align at line 2793; content at line 2794. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function renderRichFieldContent(rich, fallbackText = "") {
  const blocks = rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText);
  return blocks.map((block) => {
    if (block.type !== "paragraph") return "";
    const align = ["left", "center", "right"].includes(block.align) ? block.align : "left";
    const content = block.html
      ? sanitizeRichInlineHtml(block.html)
      : renderInlineMath(block.text || "");
    return `<span class="rich-field-line text-${align}"${richTextStyle(block)}>${content}</span>`;
  }).filter(Boolean).join("");
}
```

saveSelectedProjectToPortfolio (template-preview.js, lines 2801-2817)

saveSelectedProjectToPortfolio persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2801-2817 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, parseProjectForPortfolio, syncParsedPortfolioGlobalsIntoSaved, filter, Set, map, has, findIndex, max, splice, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2803: if (!project) return false;; line 2816: return true;.

Important local values include project at line 2802; parsedProject at line 2804; nextProjects at line 2806; workingIds at line 2807; workingIndex at line 2809; insertAt at line 2810. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2801: function saveSelectedProjectToPortfolio() {
2802:   const project = selectedProject();
2803:   if (!project) return false;
2804:   const parsedProject = parseProjectForPortfolio(project);
2805:   syncParsedPortfolioGlobalsIntoSaved();
2806:   const nextProjects = (savedPortfolioCatalog.projects || []).filter((item) => item.id !== project.id);
2807:   const workingIds = new Set((catalog.projects || []).map((item) => item.id));
2808:   savedPortfolioCatalog.projects = nextProjects.filter((item) => workingIds.has(item.id));
2809:   const workingIndex = catalog.projects.findIndex((item) => item.id === project.id);
2810:   const insertAt = Math.max(0, workingIndex);
2811:   savedPortfolioCatalog.projects.splice(insertAt, 0, parsedProject);
2812:   markDraftNeedsSave();
2813:   refreshOpenPreviews();
2814:   setStatus(`${project.title} saved into the portfolio preview.`);
2815:   updateBuilderWorkflow();
2816:   return true;
2817: }
```

nextProjects (template-preview.js, lines 2806-2814)

nextProjects part of the private builder frontend and editing experience. In this file, it appears around lines 2806-2814 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, Set, map, has, findIndex, max, splice, markDraftNeedsSave, refreshOpenPreviews, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include nextProjects at line 2806; workingIds at line 2807; workingIndex at line 2809; insertAt at line 2810. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2806:   const nextProjects = (savedPortfolioCatalog.projects || []).filter((item) => item.id !== project.id);
2807:   const workingIds = new Set((catalog.projects || []).map((item) => item.id));
2808:   savedPortfolioCatalog.projects = nextProjects.filter((item) => workingIds.has(item.id));
2809:   const workingIndex = catalog.projects.findIndex((item) => item.id === project.id);
2810:   const insertAt = Math.max(0, workingIndex);
2811:   savedPortfolioCatalog.projects.splice(insertAt, 0, parsedProject);
2812:   markDraftNeedsSave();
2813:   refreshOpenPreviews();
2814:   setStatus(`${project.title} saved into the portfolio preview.`);
```

removeProjectFromPortfolio (template-preview.js, lines 2819-2821)

removeProjectFromPortfolio part of the private builder frontend and editing experience. In this file, it appears around lines 2819-2821 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2819: function removeProjectFromPortfolio(projectId) {
2820:   savedPortfolioCatalog.projects = (savedPortfolioCatalog.projects || []).filter((item) => item.id !==
projectId);
2821: }
```

deleteProjectById (template-preview.js, lines 2823-2836)

deleteProjectById part of the private builder frontend and editing experience. In this file, it appears around lines 2823-2836 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, filter, removeProjectFromPortfolio, closeDialogElement, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2825: if (!project) return;.

Important local values include project at line 2824. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2823: function deleteProjectById(projectId) {
2824:   const project = catalog.projects.find((item) => item.id === projectId);
2825:   if (!project) return;
2826:   catalog.projects = catalog.projects.filter((item) => item.id !== project.id);
2827:   removeProjectFromPortfolio(project.id);
2828:   selectedProjectId = catalog.projects[0]?.id || "";
2829:   activeSectionId = "brief";
2830:   if (projectDialog?.open && project.id === projectWindowTitle.dataset.projectId) {
2831:     closeDialogElement(projectDialog);
2832:   }
2833:   setStatus("Project deleted locally.");
2834:   scheduleAutosave();
2835:   renderAll();
2836: }
```

openDeleteConfirm (template-preview.js, lines 2838-2843)

openDeleteConfirm controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 2838-2843 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2838: function openDeleteConfirm({ title = "Delete project", message, onConfirm }) {
2839:   pendingDeleteAction = onConfirm;
2840:   deleteConfirmTitle.textContent = title;
2841:   deleteConfirmMessage.textContent = message;
2842:   deleteConfirmDialog.showModal();
2843: }
```

requestProjectDelete (template-preview.js, lines 2845-2853)

requestProjectDelete part of the private builder frontend and editing experience. In this file, it appears around lines 2845-2853 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, openDeleteConfirm, and deleteProjectById. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2847: if (!project) return;.

Important local values include project at line 2846. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2845: function requestProjectDelete(projectId) {
2846:   const project = catalog.projects.find((item) => item.id === projectId);
2847:   if (!project) return;
2848:   openDeleteConfirm({
2849:     title: "Delete project",
2850:     message: "Are you sure you want to delete this project?",
```

```
2851:     onConfirm: () => deleteProjectById(project.id)
2852:   });
2853: }
```

parserItem (template-preview.js, lines 2855-2866)

parserItem part of the private builder frontend and editing experience. In this file, it appears around lines 2855-2866 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, and clone. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2865: return item;.

Important local values include item at line 2856. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2855: function parserItem(title, description = "", url = "", meta = "", kind = "text", rich = null, children =
[]) {
2856:   const item = {
2857:     children: children.filter((child) => child && (child.title || child.description || child.url ||
child.children?.length)),
2858:     description: description || "",
2859:     kind,
2860:     meta: meta || "",
2861:     title: title || "Untitled item",
2862:     url: url || ""
2863:   };
2864:   if (rich?.blocks?.length) item.rich = clone(rich);
2865:   return item;
2866: }
```

canonicalTemplateId (template-preview.js, lines 2868-2870)

canonicalTemplateId part of the private builder frontend and editing experience. In this file, it appears around lines 2868-2870 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2869: return legacyTemplateSkins[id] || id || "";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2868: function canonicalTemplateId(id) {
2869:   return legacyTemplateSkins[id] || id || "";
2870: }
```

projectTemplateId (template-preview.js, lines 2872-2874)

projectTemplateId part of the private builder frontend and editing experience. In this file, it appears around lines 2872-2874 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references canonicalTemplateId. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2873: return canonicalTemplateId(project?.portfolioView?.template?.id || project?.templateId || "");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2872: function projectTemplateId(project) {
2873:   return canonicalTemplateId(project?.portfolioView?.template?.id || project?.templateId || "");
2874: }
```

projectTemplateFor (template-preview.js, lines 2876-2879)

projectTemplateFor part of the private builder frontend and editing experience. In this file, it appears around lines 2876-2879 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateId, and find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2878: return templates.find((template) => template.id === templateId) || {};.

Important local values include templateId at line 2877. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2876: function projectTemplateFor(project) {
2877:   const templateId = projectTemplateId(project);
2878:   return templates.find((template) => template.id === templateId) || {};
2879: }
```

projectTemplateClass (template-preview.js, lines 2881-2884)

projectTemplateClass part of the private builder frontend and editing experience. In this file, it appears around lines 2881-2884 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateId. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2883: return templateId ? `project-template project-template-${templateId}` : "project-template-white";.

Important local values include templateId at line 2882. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2881: function projectTemplateClass(project) {
2882:   const templateId = projectTemplateId(project);
2883:   return templateId ? `project-template project-template-${templateId}` : "project-template-white";
2884: }
```

responsiveClassName (template-preview.js, lines 2886-2891)

responsiveClassName part of the private builder frontend and editing experience. In this file, it appears around lines 2886-2891 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2887: return String(value || fallback || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2886: function responsiveClassName(value, fallback) {
2887:   return String(value || fallback || "")
2888:     .toLowerCase()
2889:     .replace(/[^a-z0-9]+/g, "-")
2890:     .replace(/^-|-$/g, "") || fallback;
2891: }
```

projectResponsiveClass (template-preview.js, lines 2893-2896)

projectResponsiveClass part of the private builder frontend and editing experience. In this file, it appears around lines 2893-2896 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectResponsiveProfile, and responsiveClassName. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2895: return `responsive-project responsive-density-${responsiveClassName(profile.density, "balanced")} responsive-card-${responsiveClassName(profile.cardLayout, "single")}`;.

Important local values include profile at line 2894. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2893: function projectResponsiveClass(project) {
2894:   const profile = projectResponsiveProfile(project);
2895:   return `responsive-project responsive-density-${responsiveClassName(profile.density, "balanced")}
responsive-card-${responsiveClassName(profile.cardLayout, "single")}`;
2896: }
```

responsiveStyleValues (template-preview.js, lines 2898-2906)

responsiveStyleValues part of the private builder frontend and editing experience. In this file, it appears around lines 2898-2906 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectResponsiveProfile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2900: return {.

Important local values include profile at line 2899. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2898: function responsiveStyleValues(project) {
2899:   const profile = projectResponsiveProfile(project);
2900:   return {
2901:     "--responsive-section-card-min": profile.sectionCardMin,
2902:     "--responsive-content-max": profile.contentMax,
2903:     "--responsive-media-max": profile.mediaMax,
2904:     "--responsive-touch-target": profile.touchTarget
2905:   };
2906: }
```

hasPublicTemplate (template-preview.js, lines 2908-2910)

hasPublicTemplate part of the private builder frontend and editing experience. In this file, it appears around lines 2908-2910 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateId. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2909: return Boolean(projectTemplateId(project));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2908: function hasPublicTemplate(project) {
2909:   return Boolean(projectTemplateId(project));
2910: }
```

projectTemplateVisual (template-preview.js, lines 2912-2914)

projectTemplateVisual part of the private builder frontend and editing experience. In this file, it appears around lines 2912-2914 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateFor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2913: return project?.portfolioView?.template?.visual || project?.templateVisual || projectTemplateFor(project).visual || null;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 2912: function projectTemplateVisual(project) {
 2913:   return project?.portfolioView?.template?.visual || project?.templateVisual ||
projectTemplateFor(project).visual || null;
 2914: }
```

projectTemplateStyle (template-preview.js, lines 2916-2934)

projectTemplateStyle part of the private builder frontend and editing experience. In this file, it appears around lines 2916-2934 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateVisual, responsiveStyleValues, entries, filter, map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2930: return Object.entries(values).

Important local values include visual at line 2917; values at line 2918. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2916: function projectTemplateStyle(project, accent) {
2917:   const visual = projectTemplateVisual(project) || {};
2918:   const values = {
2919:     "--accent": accent,
2920:     "--template-bg": visual.background,
2921:     "--template-panel": visual.panel,
2922:     "--template-accent": visual.accent,
2923:     "--template-hover": visual.hover,
2924:     "--template-click": visual.click,
2925:     "--template-click-text": visual.clickText,
2926:     "--template-line": visual.line,
2927:     "--template-text": visual.text,
2928:     ...responsiveStyleValues(project)
2929:   };
2930:   return Object.entries(values)
2931:     .filter(([, value]) => value)
2932:     .map(([key, value]) => `${key}: ${value}`)
2933:     .join("; ");
2934: }
```

applyProjectTemplateToElement (template-preview.js, lines 2936-2970)

applyProjectTemplateToElement part of the private builder frontend and editing experience. In this file, it appears around lines 2936-2970 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, startsWith, forEach, remove, projectTemplateClass, split, add, projectResponsiveClass, projectTemplateVisual, responsiveStyleValues, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2937: if (!element) return;.

Important local values include visual at line 2950; values at line 2951. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2936: function applyProjectTemplateToElement(element, project, accent) {
2937:   if (!element) return;
2938:   [...element.classList]
2939:     .filter((className) =>
2940:       className === "project-template" ||
2941:       className === "project-template-white" ||
2942:       className === "responsive-project" ||
2943:       className.startsWith("project-template-") ||
2944:       className.startsWith("responsive-density-") ||
```

```
      className.startsWith("responsive-card-")
    )
    .forEach((className) => element.classList.remove(className));
  projectTemplateClass(project).split(" ").forEach((className) => element.classList.add(className));
  projectResponsiveClass(project).split(" ").forEach((className) => element.classList.add(className));
  const visual = projectTemplateVisual(project) || {};
  const values = {
    "--accent": accent,
    "--template-bg": visual.background,
    "--template-panel": visual.panel,
    "--template-accent": visual.accent,
    "--template-hover": visual.hover,
    "--template-click": visual.click,
    "--template-click-text": visual.clickText,
    "--template-line": visual.line,
    "--template-text": visual.text,
    ...responsiveStyleValues(project)
  };
  Object.entries(values).forEach(([key, value]) => {
    if (value) {
      element.style.setProperty(key, value);
    } else {
      element.style.removeProperty(key);
    }
  });
}
```

resourcesFrom (template-preview.js, lines 2972-2988)

resourcesFrom part of the private builder frontend and editing experience. In this file, it appears around lines 2972-2988 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and parserItem. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 2973: return (project[key] || []).map((item) => {; line 2975: return parserItem(item.name, item.result || item.method, item.artifact, item.method || item.status || "Test", "test", item.richDescription);; line 2978: return parserItem(item.name, item.revision || item.status, item.artifact, item.status || "PCB", "pcb", item.richDescription);; line 2981: return parserItem(item.title, item.caption, item.url, item.status || "Image", "image", item.richDescription);. There are 2 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
function resourcesFrom(project, key) {
  return (project[key] || []).map((item) => {
    if (key === "tests") {
      return parserItem(item.name, item.result || item.method, item.artifact, item.method || item.status
|| "Test", "test", item.richDescription);
    }
    if (key === "pcbs") {
      return parserItem(item.name, item.revision || item.status, item.artifact, item.status || "PCB",
"pcb", item.richDescription);
    }
    if (key === "media") {
      return parserItem(item.title, item.caption, item.url, item.status || "Image", "image",
item.richDescription);
    }
    if (key === "links") {
      return parserItem(item.label, item.description || "", item.url, "Link", "link",
item.richDescription);
    }
    return parserItem(item.title || item.name || item.label, item.description || item.caption, item.url
|| item.artifact, item.type || item.status || "Document", "document", item.richDescription);
  });
}
```

resourcesFromItems (template-preview.js, lines 2990-3003)

resourcesFromItems part of the private builder frontend and editing experience. In this file, it appears around lines 2990-3003 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and parserItem. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2991: return (items || []).map((item) => {; line 2994: return parserItem(.

Important local values include url at line 2992; kind at line 2993. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2990: function resourcesFromItems(items = [], fallbackKind = "document") {
2991:   return (items || []).map((item) => {
2992:     const url = item.url || item.artifact || "";
2993:     const kind = item.kind || fallbackKind;
2994:     return parserItem(
2995:       item.title || item.name || item.label || "Untitled item",
2996:       item.description || item.caption || item.result || item.method || "",
2997:       url,
2998:       item.type || item.status || item.method || "",
2999:       kind,
3000:       item.richDescription
3001:     );
3002:   });
3003: }
```

parsedSubsection (template-preview.js, lines 3005-3007)

parsedSubsection part of the private builder frontend and editing experience. In this file, it appears around lines 3005-3007 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parserItem. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3006: return parserItem(title, description, "", "Subsection", "subsection", rich, items);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3005: function parsedSubsection(title, description = "", items = [], rich = null) {
3006:   return parserItem(title, description, "", "Subsection", "subsection", rich, items);
3007: }
```

richHasContent (template-preview.js, lines 3009-3018)

richHasContent part of the private builder frontend and editing experience. In this file, it appears around lines 3009-3018 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3010: return Boolean(rich?.blocks?.some((block) =>.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3009: function richHasContent(rich) {
3010:   return Boolean(rich?.blocks?.some((block) =>
3011:     block.type === "image" && block.url ||
3012:     block.type === "formula" && block.formula ||
3013:     block.type === "code" && block.code ||
3014:     block.text ||
3015:     block.title ||
3016:     block.caption
3017:   ));
3018: }
```

parsedItemHasContent (template-preview.js, lines 3020-3031)

parsedItemHasContent part of the private builder frontend and editing experience. In this file, it appears around lines 3020-3031 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richHasContent, some, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 3021: if (!item) return false;; line 3022: if (item.kind === "summary") return Boolean(item.description || richHasContent(item.rich));; line 3025: return Boolean(item.description || item.url || richHasContent(item.rich) || children.some(parsedItemHasContent));; line 3028: return Boolean(item.title || item.description || item.url || richHasContent(item.rich) || children.some(parsedItemHasContent));. There are 1 additional return statements in the function body.

Important local values include children at line 3023. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 3020: function parsedItemHasContent(item) {
 3021:   if (!item) return false;
 3022:   if (item.kind === "summary") return Boolean(item.description || richHasContent(item.rich));
 3023:   const children = item.children || item.items || [];
 3024:   if (item.kind === "subsection") {
 3025:     return Boolean(item.description || item.url || richHasContent(item.rich) ||
children.some(parsedItemHasContent));
 3026:   }
 3027:   if (["tool", "language"].includes(item.kind)) {
 3028:     return Boolean(item.title || item.description || item.url || richHasContent(item.rich) ||
children.some(parsedItemHasContent));
 3029:   }
 3030:   return Boolean(item.description || item.url || richHasContent(item.rich) ||
children.some(parsedItemHasContent));
 3031: }
```

parsedSectionHasContent (template-preview.js, lines 3033-3035)

parsedSectionHasContent part of the private builder frontend and editing experience. In this file, it appears around lines 3033-3035 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richHasContent, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3034: return Boolean(section?.description || richHasContent(section?.rich) || (section?.items || []).some(parsedItemHasContent));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3033: function parsedSectionHasContent(section) {
 3034:   return Boolean(section?.description || richHasContent(section?.rich) || (section?.items ||
[]).some(parsedItemHasContent));
 3035: }
```

responsiveChildren (template-preview.js, lines 3037-3039)

responsiveChildren part of the private builder frontend and editing experience. In this file, it appears around lines 3037-3039 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3038: return node?.items || node?.children || [];.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3037: function responsiveChildren(node) {
 3038:   return node?.items || node?.children || [];
 3039: }
```

richContainsMedia (template-preview.js, lines 3041-3043)

richContainsMedia part of the private builder frontend and editing experience. In this file, it appears around lines 3041-3043 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3042: return Boolean(rich?.blocks?.some((block) => block.type === "image" && block.url));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3041: function richContainsMedia(rich) {
3042:   return Boolean(rich?.blocks?.some((block) => block.type === "image" && block.url));
3043: }
```

responsiveNodeHasMedia (template-preview.js, lines 3045-3051)

responsiveNodeHasMedia part of the private builder frontend and editing experience. In this file, it appears around lines 3045-3051 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richContainsMedia, responsiveChildren, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 3046: if (!node) return false;; line 3047: if (node.kind === "image" || richContainsMedia(node.rich)) return true;; line 3049: if (/\.(apng|avif|gif|jpe?g|png|svg|webp)(\?|#|$)/i.test(url)) return true;; line 3050: return responsiveChildren(node).some(responsiveNodeHasMedia);.

Important local values include url at line 3048. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3045: function responsiveNodeHasMedia(node) {
3046:   if (!node) return false;
3047:   if (node.kind === "image" || richContainsMedia(node.rich)) return true;
3048:   const url = String(node.url || "");
3049:   if (/\.(apng|avif|gif|jpe?g|png|svg|webp)(\?|#|$)/i.test(url)) return true;
3050:   return responsiveChildren(node).some(responsiveNodeHasMedia);
3051: }
```

responsiveNodeCount (template-preview.js, lines 3053-3056)

responsiveNodeCount part of the private builder frontend and editing experience. In this file, it appears around lines 3053-3056 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parsedItemHasContent, responsiveChildren, and reduce. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3054: if (!node || !parsedItemHasContent(node)) return 0;; line 3055: return 1 + responsiveChildren(node).reduce((count, child) => count + responsiveNodeCount(child), 0);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3053: function responsiveNodeCount(node) {
3054:   if (!node || !parsedItemHasContent(node)) return 0;
3055:   return 1 + responsiveChildren(node).reduce((count, child) => count + responsiveNodeCount(child), 0);
3056: }
```

responsiveNodeDepth (template-preview.js, lines 3058-3063)

responsiveNodeDepth part of the private builder frontend and editing experience. In this file, it appears around lines 3058-3063 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parsedItemHasContent, responsiveChildren, filter, max, and map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3059: if (!node || !parsedItemHasContent(node)) return 0;; line 3061: if (!children.length) return 1;; line 3062: return 1 + Math.max(...children.map(responsiveNodeDepth));.

Important local values include children at line 3060. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3058: function responsiveNodeDepth(node) {
3059:   if (!node || !parsedItemHasContent(node)) return 0;
3060:   const children = responsiveChildren(node).filter(parsedItemHasContent);
3061:   if (!children.length) return 1;
3062:   return 1 + Math.max(...children.map(responsiveNodeDepth));
3063: }
```

buildResponsiveProfile (template-preview.js, lines 3065-3096)

buildResponsiveProfile creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 3065-3096 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, parsedSectionHasContent, reduce, responsiveNodeCount, max, map, some, responsiveNodeHasMedia, and hasPublicTemplate. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3081: return {.

Important local values include visibleSections at line 3066; itemCount at line 3067; maxDepth at line 3071; hasMedia at line 3075; density at line 3076. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3065: function buildResponsiveProfile(project, parsedSections = []) {
3066:   const visibleSections = (parsedSections || []).filter((section) => section?.id !== "brief" &&
parsedSectionHasContent(section));
3067:   const itemCount = visibleSections.reduce(
3068:     (count, section) => count + (section.items || []).reduce((sum, item) => sum +
responsiveNodeCount(item), 0),
3069:     0
3070:   );
3071:   const maxDepth = visibleSections.reduce(
3072:     (depth, section) => Math.max(depth, ...(section.items || []).map(responsiveNodeDepth), 1),
3073:     1
3074:   );
3075:   const hasMedia = visibleSections.some((section) => responsiveNodeHasMedia(section));
3076:   const density = itemCount >= 18 || visibleSections.length >= 6 || maxDepth >= 4
3077:     ? "dense"
3078:     : itemCount <= 6 && visibleSections.length <= 2
3079:       ? "simple"
3080:       : "balanced";
3081:   return {
3082:     version: 1,
3083:     breakpoints: {
3084:       mobile: 640,
3085:       tablet: 900,
3086:       desktop: 1180
3087:     },
3088:     cardLayout: hasPublicTemplate(project) ? "single" : "split",
3089:     contentMax: hasMedia || maxDepth >= 3 ? "1180px" : "980px",
3090:     density,
3091:     mediaMax: hasMedia ? "860px" : "720px",
3092:     sectionCardMin: density === "dense" ? "210px" : density === "simple" ? "280px" : "240px",
3093:     touchTarget: "44px",
3094:     windowMode: "full-screen"
3095:   };
3096: }
```

visibleSections (template-preview.js, lines 3066-3095)

visibleSections part of the private builder frontend and editing experience. In this file, it appears around lines 3066-3095 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, parsedSectionHasContent, reduce, responsiveNodeCount, max, map, some, responsiveNodeHasMedia, and hasPublicTemplate. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3081: return {.

Important local values include visibleSections at line 3066; itemCount at line 3067; maxDepth at line 3071; hasMedia at line 3075; density at line 3076. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 3066:   const visibleSections = (parsedSections || []).filter((section) => section?.id !== "brief" &&
parsedSectionHasContent(section));
 3067:   const itemCount = visibleSections.reduce(
 3068:     (count, section) => count + (section.items || []).reduce((sum, item) => sum +
responsiveNodeCount(item), 0),
 3069:     0
 3070:   );
 3071:   const maxDepth = visibleSections.reduce(
 3072:     (depth, section) => Math.max(depth, ...(section.items || []).map(responsiveNodeDepth), 1),
 3073:     1
 3074:   );
 3075:   const hasMedia = visibleSections.some((section) => responsiveNodeHasMedia(section));
 3076:   const density = itemCount >= 18 || visibleSections.length >= 6 || maxDepth >= 4
 3077:     ? "dense"
 3078:     : itemCount <= 6 && visibleSections.length <= 2
 3079:       ? "simple"
 3080:       : "balanced";
 3081:   return {
 3082:     version: 1,
 3083:     breakpoints: {
 3084:       mobile: 640,
 3085:       tablet: 900,
 3086:       desktop: 1180
 3087:     },
 3088:     cardLayout: hasPublicTemplate(project) ? "single" : "split",
 3089:     contentMax: hasMedia || maxDepth >= 3 ? "1180px" : "980px",
 3090:     density,
 3091:     mediaMax: hasMedia ? "860px" : "720px",
 3092:     sectionCardMin: density === "dense" ? "210px" : density === "simple" ? "280px" : "240px",
 3093:     touchTarget: "44px",
 3094:     windowMode: "full-screen"
 3095:   };
```

projectResponsiveProfile (template-preview.js, lines 3098-3107)

projectResponsiveProfile part of the private builder frontend and editing experience. In this file, it appears around lines 3098-3107 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references buildResponsiveProfile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3101: return {; line 3106: return buildResponsiveProfile(project, project?.portfolioView?.sections || []);.

Important local values include savedProfile at line 3099. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3098: function projectResponsiveProfile(project) {
3099:   const savedProfile = project?.portfolioView?.responsive;
3100:   if (savedProfile?.version) {
3101:     return {
3102:       ...buildResponsiveProfile(project, project?.portfolioView?.sections || []),
3103:       ...savedProfile
3104:     };
3105:   }
3106:   return buildResponsiveProfile(project, project?.portfolioView?.sections || []);
3107: }
```

parsedSection (template-preview.js, lines 3109-3118)

parsedSection part of the private builder frontend and editing experience. In this file, it appears around lines 3109-3118 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, richHasContent, and clone. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3117: return section;.

Important local values include section at line 3110. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3109: function parsedSection(id, title, items = [], description = "", rich = null) {
3110:   const section = {
3111:     description,
3112:     id,
3113:     items: items.filter(parsedItemHasContent),
3114:     title
3115:   };
3116:   if (richHasContent(rich)) section.rich = clone(rich);
3117:   return section;
3118: }
```

parsedDesignSection (template-preview.js, lines 3120-3146)

parsedDesignSection part of the private builder frontend and editing experience. In this file, it appears around lines 3120-3146 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isAnalogProject, ensureDesignModel, resourcesFrom, resourcesFromItems, parsedSubsection, and parsedSection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3133: return parsedSection("design", "Design", [; line 3141: return parsedSection("design", "Design", [.

Important local values include design at line 3122; documentationItems at line 3123; boardMediaItems at line 3129. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3120: function parsedDesignSection(project) {
3121:   if (isAnalogProject(project)) {
3122:     const design = ensureDesignModel(project);
3123:     const documentationItems = [
3124:       ...resourcesFrom(project, "documents"),
3125:       ...resourcesFromItems(design.documentation.files, "document"),
3126:       parsedSubsection("References", "", resourcesFromItems(design.documentation.references,
"reference")),
3127:       parsedSubsection("Math / Analysis", "", resourcesFromItems(design.documentation.mathAnalysis,
"analysis"))
3128:     ];
3129:     const boardMediaItems = [
3130:       ...resourcesFrom(project, "pcbs"),
3131:       ...resourcesFrom(project, "media")
3132:     ];
3133:     return parsedSection("design", "Design", [
3134:       parsedSubsection("Design Overview", design.brief.summary || "", [
3135:         ...resourcesFromItems(design.brief.files, "document")
3136:       ], design.brief.summaryRich),
3137:       parsedSubsection("Documentation", design.documentation.summary || "", documentationItems,
design.documentation.summaryRich),
3138:       parsedSubsection("PCB and Visual Evidence", "", boardMediaItems)
3139:     ]);
3140:   }
3141:   return parsedSection("design", "Design", [
3142:     ...resourcesFrom(project, "documents"),
3143:     ...resourcesFrom(project, "pcbs"),
3144:     ...resourcesFrom(project, "media")
3145:   ]);
3146: }
```

parsedSimulationSection (template-preview.js, lines 3148-3156)

parsedSimulationSection part of the private builder frontend and editing experience. In this file, it appears around lines 3148-3156 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isAnalogProject, ensureDesignModel, parsedSection, parsedSubsection, and resourcesFromItems. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3149: if (!isAnalogProject(project)) return null;; line 3151: return parsedSection("simulation", "Simulation", [.

Important local values include design at line 3150. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 3148: function parsedSimulationSection(project) {
 3149:   if (!isAnalogProject(project)) return null;
 3150:   const design = ensureDesignModel(project);
 3151:   return parsedSection("simulation", "Simulation", [
 3152:     parsedSubsection("Simulation Overview", design.simulation.summary || "", [],
design.simulation.summaryRich),
 3153:     parsedSubsection("Simulation Files", "", resourcesFromItems(design.simulation.files, "simulation-
file")),
 3154:     parsedSubsection("Simulation Results", "", resourcesFromItems(design.simulation.results, "simulation-
result"))
 3155:   ]);
 3156: }
```

parsedToolsSection (template-preview.js, lines 3158-3169)

parsedToolsSection part of the private builder frontend and editing experience. In this file, it appears around lines 3158-3169 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, parserItem, toolName, toolDescription, itemTitle, parsedSection, and resourcesFrom. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3162: return parserItem(label, "", "", "Language", "language");; line 3164: return parsedSection("tools", "Tools", [.

Important local values include toolItems at line 3159; languageItems at line 3160; label at line 3161. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3158: function parsedToolsSection(project) {
3159:   const toolItems = (project.tools || []).map((tool) => parserItem(toolName(tool), toolDescription(tool),
"", "Tool", "tool", tool?.richDescription));
3160:   const languageItems = (project.languages || []).map((language) => {
3161:     const label = typeof language === "string" ? language : itemTitle(language);
3162:     return parserItem(label, "", "", "Language", "language");
3163:   });
3164:   return parsedSection("tools", "Tools", [
3165:     ...toolItems,
3166:     ...languageItems,
3167:     ...resourcesFrom(project, "links")
3168:   ]);
3169: }
```

toolItems (template-preview.js, lines 3159-3163)

toolItems part of the private builder frontend and editing experience. In this file, it appears around lines 3159-3163 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, parserItem, toolName, toolDescription, and itemTitle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3162: return parserItem(label, "", "", "Language", "language");.

Important local values include toolItems at line 3159; languageItems at line 3160; label at line 3161. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3159:   const toolItems = (project.tools || []).map((tool) => parserItem(toolName(tool), toolDescription(tool),
"", "Tool", "tool", tool?.richDescription));
3160:   const languageItems = (project.languages || []).map((language) => {
3161:     const label = typeof language === "string" ? language : itemTitle(language);
3162:     return parserItem(label, "", "", "Language", "language");
3163:   });
```

languageItems (template-preview.js, lines 3160-3163)

languageItems part of the private builder frontend and editing experience. In this file, it appears around lines 3160-3163 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, itemTitle, and parserItem. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3162: return parserItem(label, "", "", "Language", "language");.

Important local values include languageItems at line 3160; label at line 3161. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3160:   const languageItems = (project.languages || []).map((language) => {
3161:     const label = typeof language === "string" ? language : itemTitle(language);
3162:     return parserItem(label, "", "", "Language", "language");
3163:   });
```

parseProjectForPortfolio (template-preview.js, lines 3171-3219)

This function converts editable project state into the clean public project object used by previews and the website. It filters empty sections, normalizes overview content, applies appearance templates, gathers resources, and builds responsive profile metadata.

The builder data is messy because it contains drafts, controls, and editor state. The public website needs a clean display model. This parser is that boundary.

It calls or references projectTemplateFor, sectionOptions, flatMap, parsedSection, parserItem, parsedDesignSection, parsedSimulationSection, resourcesFrom, parsedToolsSection, startsWith, and 11 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 8 return statement(s). The most important visible returns are: line 3175: return parsedSection("brief", "Overview", [; line 3179: if (section.id === "design") return parsedDesignSection(project);; line 3180: if (section.id === "simulation") return parsedSimulationSection(project);; line 3181: if (section.id === "tests") return parsedSection("tests", "Tests", resourcesFrom(project, "tests"));. There are 4 additional return statements in the function body.

Important local values include template at line 3172; parsedSections at line 3173; sectionId at line 3184; custom at line 3185. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3171: function parseProjectForPortfolio(project) {
3172:   const template = projectTemplateFor(project);
3173:   const parsedSections = sectionOptions(project).flatMap((section) => {
3174:     if (section.id === "brief") {
3175:       return parsedSection("brief", "Overview", [
3176:         parserItem("Overview", project.summary || "", "", "Overview", "summary", project.summaryRich)
3177:       ]);
3178:     }
3179:     if (section.id === "design") return parsedDesignSection(project);
3180:     if (section.id === "simulation") return parsedSimulationSection(project);
3181:     if (section.id === "tests") return parsedSection("tests", "Tests", resourcesFrom(project, "tests"));
3182:     if (section.id === "tools") return parsedToolsSection(project);
3183:     if (section.id.startsWith("custom:")) {
3184:       const sectionId = section.id.replace("custom:", "");
3185:       const custom = (project.sections || []).find((item) => item.id === sectionId);
3186:       return parsedSection(
3187:         `custom:${sectionId}`,
3188:         custom?.title || section.label,
3189:         (custom?.items || []).map((item) => item.url
3190:           ? parserItem(item.title, item.description, item.url, item.type || item.status || "File",
"file", item.richDescription)
3191:           : parsedSubsection(
3192:               item.title,
3193:               item.description,
3194:               resourcesFromItems(item.children || item.items || item.files || [], "file"),
3195:               item.richDescription
3196:             )),
3197:         custom?.description || "",
3198:         custom?.richDescription
```

```
3199:       );
3200:     }
3201:     return null;
3202:   }).filter(parsedSectionHasContent);
3203:
3204:   return {
3205:     ...clone(project),
3206:     portfolioView: {
3207:       builtAt: new Date().toISOString(),
3208:       template: {
3209:         group: "Appearance",
3210:         id: projectTemplateId(project),
3211:         label: template.label || project.templateLabel || "",
3212:         visual: template.visual ? clone(template.visual) : projectTemplateVisual(project)
3213:       },
3214:       responsive: buildResponsiveProfile(project, parsedSections),
3215:       title: project.title,
3216:       sections: parsedSections
3217:     }
3218:   };
3219: }
```

custom (template-preview.js, lines 3185-3187)

custom part of the private builder frontend and editing experience. In this file, it appears around lines 3185-3187 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and parsedSection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3186: return parsedSection(.

Important local values include custom at line 3185. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3185:       const custom = (project.sections || []).find((item) => item.id === sectionId);
3186:       return parsedSection(
3187:         `custom:${sectionId}`,
```

refreshOpenPreviews (template-preview.js, lines 3221-3230)

refreshOpenPreviews part of the private builder frontend and editing experience. In this file, it appears around lines 3221-3230 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, find, renderProjectWebsitePreview, and renderPreview. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include project at line 3223; savedProject at line 3224. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3221: function refreshOpenPreviews() {
3222:   if (projectPreviewDialog?.open) {
3223:     const project = selectedProject();
3224:     const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project?.id);
3225:     renderProjectWebsitePreview(savedProject || project);
3226:   }
3227:   if (portfolioPreviewDialog?.open) {
3228:     renderPreview();
3229:   }
3230: }
```

savedProject (template-preview.js, lines 3224-3227)

savedProject persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 3224-3227 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and renderProjectWebsitePreview. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include savedProject at line 3224. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3224:     const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project?.id);
3225:     renderProjectWebsitePreview(savedProject || project);
3226:   }
3227:   if (portfolioPreviewDialog?.open) {
```

saveSelectedProjectAndClose (template-preview.js, lines 3232-3236)

saveSelectedProjectAndClose persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 3232-3236 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references saveSelectedProjectToPortfolio, and closeDialogElement. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3232: function saveSelectedProjectAndClose() {
3233:   if (saveSelectedProjectToPortfolio()) {
3234:     closeDialogElement(projectDialog);
3235:   }
3236: }
```

commitActiveProjectEdits (template-preview.js, lines 3238-3246)

commitActiveProjectEdits part of the private builder frontend and editing experience. In this file, it appears around lines 3238-3246 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references saveVisibleRichEditors, and saveRichSummary. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3242: return saveRichSummary();; line 3245: return true;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3238: async function commitActiveProjectEdits() {
3239:   saveVisibleRichEditors();
3240:
3241:   if (summaryEditorProjectId) {
3242:     return saveRichSummary();
3243:   }
3244:
3245:   return true;
3246: }
```

saveAllSections (template-preview.js, lines 3248-3259)

saveAllSections persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 3248-3259 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references commitActiveProjectEdits, syncParsedPortfolioGlobalsIntoSaved, map, parseProjectForPortfolio, markDraftNeedsSave, closeDialogElement, refreshOpenPreviews, setStatus, and updateBuilderWorkflow. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3250: if (!(await commitActiveProjectEdits())) return false;; line 3258: return true;.

Important local values include settings at line 3249. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 3248: async function saveAllSections(options = {}) {
 3249:   const settings = options && options.currentTarget ? {} : options;
 3250:   if (!(await commitActiveProjectEdits())) return false;
 3251:   syncParsedPortfolioGlobalsIntoSaved();
 3252:   savedPortfolioCatalog.projects = (catalog.projects || []).map((project) =>
parseProjectForPortfolio(project));
 3253:   markDraftNeedsSave();
 3254:   if (projectDialog?.open) closeDialogElement(projectDialog);
 3255:   if (settings.refreshOpenPreviews !== false) refreshOpenPreviews();
 3256:   setStatus(`Saved ${savedPortfolioCatalog.projects.length}
project${savedPortfolioCatalog.projects.length === 1 ? "" : "s"} into the portfolio preview.`);
 3257:   updateBuilderWorkflow();
 3258:   return true;
 3259: }
```

selectedProject (template-preview.js, lines 3261-3263)

selectedProject part of the private builder frontend and editing experience. In this file, it appears around lines 3261-3263 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3262: return catalog.projects.find((project) => project.id === selectedProjectId) || catalog.projects[0];.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3261: function selectedProject() {
 3262:   return catalog.projects.find((project) => project.id === selectedProjectId) || catalog.projects[0];
 3263: }
```

sectionWindowId (template-preview.js, lines 3265-3269)

sectionWindowId part of the private builder frontend and editing experience. In this file, it appears around lines 3265-3269 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3266: if (["documents", "pcbs", "media"].includes(sectionId)) return "design";; line 3267: if (["links", "highlights", "tools", "languages"].includes(sectionId)) return "tools";; line 3268: return sectionId || "brief";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3265: function sectionWindowId(sectionId) {
 3266:   if (["documents", "pcbs", "media"].includes(sectionId)) return "design";
 3267:   if (["links", "highlights", "tools", "languages"].includes(sectionId)) return "tools";
 3268:   return sectionId || "brief";
 3269: }
```

resetProjectWindowScroll (template-preview.js, lines 3271-3279)

resetProjectWindowScroll part of the private builder frontend and editing experience. In this file, it appears around lines 3271-3279 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references requestAnimationFrame. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3271: function resetProjectWindowScroll() {
3272:   projectWindowContent.scrollTop = 0;
3273:   requestAnimationFrame(() => {
3274:     projectWindowContent.scrollTop = 0;
3275:     requestAnimationFrame(() => {
3276:       projectWindowContent.scrollTop = 0;
3277:     });
3278:   });
3279: }
```

openProjectWindow (template-preview.js, lines 3281-3302)

openProjectWindow controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3281-3302 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sectionWindowId, removeAttribute, remove, renderAll, add, showModal, resetProjectWindowScroll, and updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3281: function openProjectWindow(projectId, sectionId = "brief") {
3282:   selectedProjectId = projectId;
3283:   projectWindowTitle.dataset.projectId = projectId;
3284:   activeSectionId = sectionWindowId(sectionId);
3285:   projectWindowBackStack = [];
3286:   projectWindowForwardStack = [];
3287:   projectTitleMenu.hidden = true;
3288:   projectDialog.removeAttribute("style");
3289:   projectDialog.classList.remove(
3290:     "is-draggable-dialog",
3291:     "is-hidden-dialog",
3292:     "is-maximized-dialog",
3293:     "is-minimized-dialog",
3294:     "is-resized-dialog"
3295:   );
3296:   delete projectDialog.dataset.restoreWindowState;
3297:   renderAll();
3298:   document.body.classList.add("project-window-open");
3299:   if (!projectDialog.open) projectDialog.showModal();
3300:   resetProjectWindowScroll();
3301:   updateDialogWindowButtons(projectDialog);
3302: }
```

categoryById (template-preview.js, lines 3304-3306)

categoryById part of the private builder frontend and editing experience. In this file, it appears around lines 3304-3306 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3305: return catalog.categories.find((category) => category.id === id) || {};.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3304: function categoryById(id) {
3305:   return catalog.categories.find((category) => category.id === id) || {};
3306: }
```

normalizeCategory (template-preview.js, lines 3308-3318)

normalizeCategory validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 3308-3318 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, slugify, and normalizeTextColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3312: return {.

Important local values include label at line 3309; id at line 3310; accent at line 3311. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3308: function normalizeCategory(category = {}) {
3309:   const label = String(category.label || category.title || "New category").trim() || "New category";
3310:   const id = slugify(category.id || label) || "category";
3311:   const accent = normalizeTextColor(category.accent || "") || "#1677a8";
3312:   return {
3313:     accent,
3314:     description: String(category.description || "").trim(),
3315:     id,
3316:     label
3317:   };
3318: }
```

refreshCategoryControls (template-preview.js, lines 3320-3327)

refreshCategoryControls part of the private builder frontend and editing experience. In this file, it appears around lines 3320-3327 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, clone, escapeHtml, join, and renderCategoryDropdown. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3320: function refreshCategoryControls() {
3321:   catalog.categories = (catalog.categories || []).map(normalizeCategory);
3322:   savedPortfolioCatalog.categories = clone(catalog.categories || []);
3323:   if (newCategorySelect) {
3324:     newCategorySelect.innerHTML = catalog.categories.map((category) => `<option
value="${escapeHtml(category.id)}">${escapeHtml(category.label)}</option>`).join("");
3325:   }
3326:   renderCategoryDropdown();
3327: }
```

linkAttributes (template-preview.js, lines 3329-3332)

linkAttributes part of the private builder frontend and editing experience. In this file, it appears around lines 3329-3332 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, and isWebsiteLinkItem. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3331: return /^https?:\/\/i.test(target) ? ' target="_blank" rel="noreferrer"' : "";.

Important local values include target at line 3330. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3329: function linkAttributes(url, item = {}) {
3330:   const target = normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(item, url) });
```

```
3331:   return /^https?:\/\//i.test(target) ? ' target="_blank" rel="noreferrer"' : "";
3332: }
```

isLocalDownloadTarget (template-preview.js, lines 3334-3337)

isLocalDownloadTarget part of the private builder frontend and editing experience. In this file, it appears around lines 3334-3337 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3336: return Boolean(value) && !/^(https?:)?\/\/i.test(value) && !/^(mailto:|tel:|#)/i.test(value);.

Important local values include value at line 3335. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3334: function isLocalDownloadTarget(target = "") {
3335:   const value = normalizeLinkTarget(target, { assumeWeb: true });
3336:   return Boolean(value) && !/^(https?:)?\/\//i.test(value) && !/^(mailto:|tel:|#)/i.test(value);
3337: }
```

downloadAttribute (template-preview.js, lines 3339-3342)

downloadAttribute part of the private builder frontend and editing experience. In this file, it appears around lines 3339-3342 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, isWebsiteLinkItem, and isLocalDownloadTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3341: return isLocalDownloadTarget(value) ? " download" : "";.

Important local values include value at line 3340. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3339: function downloadAttribute(target = "", item = {}) {
3340:   const value = normalizeLinkTarget(target, { assumeWeb: isWebsiteLinkItem(item, target) });
3341:   return isLocalDownloadTarget(value) ? " download" : "";
3342: }
```

resourceLink (template-preview.js, lines 3344-3353)

resourceLink part of the private builder frontend and editing experience. In this file, it appears around lines 3344-3353 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, isWebsiteLinkItem, escapeHtml, linkAttributes, and downloadAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3349: return `<span class="resource-link muted">${label}</span>`;; line 3352: return `<a class="resource-link" href="${escapeHtml(target)}"${linkAttributes(target, item)}${downloadAttribute(target, item)}>${escapeHtml(label)}</a>`;.

Important local values include rawTarget at line 3345; target at line 3346. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3344: function resourceLink(item, label = item.label || item.title || item.name) {
3345:   const rawTarget = item.url || item.artifact || item.href || item.file || item.path || item.src || "";
3346:   const target = normalizeLinkTarget(rawTarget, { assumeWeb: isWebsiteLinkItem(item, rawTarget) });
3347:
3348:   if (!target || item.status === "planned") {
3349:     return `<span class="resource-link muted">${label}</span>`;
3350:   }
```

```
3351:
3352:   return `<a class="resource-link" href="${escapeHtml(target)}"${linkAttributes(target,
item)}${downloadAttribute(target, item)}>${escapeHtml(label)}</a>`;
3353: }
```

pillList (template-preview.js, lines 3355-3360)

pillList part of the private builder frontend and editing experience. In this file, it appears around lines 3355-3360 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3356: return (items || []).map((item) => {; line 3358: return `<span class="tag ${className}">${label}</span>`;.

Important local values include label at line 3357. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3355: function pillList(items, className = "") {
3356:   return (items || []).map((item) => {
3357:     const label = typeof item === "string" ? item : item.name || item.title || item.label || "";
3358:     return `<span class="tag ${className}">${label}</span>`;
3359:   }).join("");
3360: }
```

evidenceList (template-preview.js, lines 3362-3365)

evidenceList part of the private builder frontend and editing experience. In this file, it appears around lines 3362-3365 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3363: if (!items || !items.length) return `<p class="evidence-empty">${emptyMessage}</p>`;; line 3364: return `<ul>${items.map(renderItem).join("")}</ul>`;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3362: function evidenceList(items, renderItem, emptyMessage) {
3363:   if (!items || !items.length) return `<p class="evidence-empty">${emptyMessage}</p>`;
3364:   return `<ul>${items.map(renderItem).join("")}</ul>`;
3365: }
```

detailBlock (template-preview.js, lines 3367-3374)

detailBlock part of the private builder frontend and editing experience. In this file, it appears around lines 3367-3374 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3368: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3367: function detailBlock(title, className, content) {
3368:   return `
3369:     <details class="${className}">
3370:       <summary>${title}</summary>
3371:       ${content}
3372:     </details>
3373:   `;
3374: }
```

mediaGrid (template-preview.js, lines 3376-3398)

mediaGrid part of the private builder frontend and editing experience. In this file, it appears around lines 3376-3398 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, escapeHtml, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3377: if (!items || !items.length) return `<p class="evidence-empty">No project images have been added yet.</p>`;; line 3379: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3376: function mediaGrid(items) {
 3377:   if (!items || !items.length) return `<p class="evidence-empty">No project images have been added
yet.</p>`;
 3378:
 3379:   return `
 3380:     <div class="media-grid">
 3381:       ${items.map((item) => `
 3382:         <figure>
 3383:           ${item.status === "planned" ? `
 3384:             <div class="media-placeholder">${item.title}</div>
 3385:           ` : `
 3386:             <a href="${escapeHtml(normalizeLinkTarget(item.url, { assumeWeb: isWebsiteLinkItem(item,
item.url) }))}"${linkAttributes(item.url, item)}>
 3387:               <img src="${escapeHtml(normalizeLinkTarget(item.url, { assumeWeb: isWebsiteLinkItem(item,
item.url) }))}" alt="${escapeHtml(item.title)}">
 3388:             </a>
 3389:           `}
 3390:           <figcaption>
 3391:             <strong>${escapeHtml(item.title)}</strong>
 3392:             <span>${escapeHtml(item.caption || "")}</span>
 3393:           </figcaption>
 3394:         </figure>
 3395:       `).join("")}
 3396:     </div>
 3397:   `;
 3398: }
```

itemTitle (template-preview.js, lines 3400-3403)

itemTitle part of the private builder frontend and editing experience. In this file, it appears around lines 3400-3403 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3401: if (typeof item === "string") return item;; line 3402: return item?.title || item?.name || item?.label || "Untitled item";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3400: function itemTitle(item) {
3401:   if (typeof item === "string") return item;
3402:   return item?.title || item?.name || item?.label || "Untitled item";
3403: }
```

itemUrl (template-preview.js, lines 3405-3407)

itemUrl part of the private builder frontend and editing experience. In this file, it appears around lines 3405-3407 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3406: return item?.url || item?.artifact || "";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3405: function itemUrl(item) {
3406:   return item?.url || item?.artifact || "";
3407: }
```

itemDescription (template-preview.js, lines 3409-3411)

itemDescription part of the private builder frontend and editing experience. In this file, it appears around lines 3409-3411 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3410: return item?.description || item?.caption || item?.result || item?.method || "";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3409: function itemDescription(item) {
3410:   return item?.description || item?.caption || item?.result || item?.method || "";
3411: }
```

isImageUrl (template-preview.js, lines 3413-3415)

isImageUrl part of the private builder frontend and editing experience. In this file, it appears around lines 3413-3415 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3414: return /\.(png|jpe?g|webp|gif|svg)(\?.*)?$/i.test(url || "");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3413: function isImageUrl(url) {
3414:   return /\.(png|jpe?g|webp|gif|svg)(\?.*)?$/i.test(url || "");
3415: }
```

portfolioLink (template-preview.js, lines 3417-3421)

portfolioLink part of the private builder frontend and editing experience. In this file, it appears around lines 3417-3421 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, escapeHtml, linkAttributes, and downloadAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3418: if (!url) return `<span>${label}</span>`;; line 3420: return `<a href="${escapeHtml(target)}"${linkAttributes(target)}${downloadAttribute(target)}>${escapeHtml(label)}</a>`;.

Important local values include target at line 3419. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3417: function portfolioLink(url, label) {
3418:   if (!url) return `<span>${label}</span>`;
3419:   const target = normalizeLinkTarget(url, { assumeWeb: true });
3420:   return `<a
href="${escapeHtml(target)}"${linkAttributes(target)}${downloadAttribute(target)}>${escapeHtml(label)}</a>`;
3421: }
```

renderPortfolioResourceList (template-preview.js, lines 3423-3444)

renderPortfolioResourceList renders ui, markup, or display output. In this file, it appears around lines 3423-3444 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, itemTitle, itemUrl, itemDescription, portfolioLink, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3424: return `; line 3433: return `.

Important local values include titleText at line 3430; url at line 3431; description at line 3432. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3423: function renderPortfolioResourceList(title, items, emptyMessage) {
3424:   return `
3425:     <section class="portfolio-preview-section">
3426:       <h3>${title}</h3>
3427:       ${(items || []).length ? `
3428:         <div class="portfolio-resource-list">
3429:           ${items.map((item) => {
3430:             const titleText = itemTitle(item);
3431:             const url = itemUrl(item);
3432:             const description = itemDescription(item);
3433:             return `
3434:               <article class="portfolio-resource">
3435:                 <strong>${portfolioLink(url, titleText)}</strong>
3436:                 ${description ? `<p>${description}</p>` : ""}
3437:               </article>
3438:             `;
3439:           }).join("")}
3440:         </div>
3441:       ` : `<p class="evidence-empty">${emptyMessage}</p>`}
3442:     </section>
3443:   `;
3444: }
```

renderPortfolioMedia (template-preview.js, lines 3446-3468)

renderPortfolioMedia renders ui, markup, or display output. In this file, it appears around lines 3446-3468 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, itemUrl, isImageUrl, itemTitle, itemDescription, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3447: return `; line 3454: return `.

Important local values include url at line 3453. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3446: function renderPortfolioMedia(items) {
3447:   return `
3448:     <section class="portfolio-preview-section">
3449:       <h3>Images</h3>
3450:       ${(items || []).length ? `
3451:         <div class="portfolio-media-grid">
3452:           ${items.map((item) => {
3453:             const url = itemUrl(item);
3454:             return `
3455:               <figure>
3456:                 ${isImageUrl(url) ? `<img src="${url}" alt="${itemTitle(item)}">` : `<div class="media-
placeholder">${itemTitle(item)}</div>`}
3457:                 <figcaption>
3458:                   <strong>${itemTitle(item)}</strong>
3459:                   ${itemDescription(item) ? `<span>${itemDescription(item)}</span>` : ""}
3460:                 </figcaption>
3461:               </figure>
3462:             `;
3463:           }).join("")}
3464:         </div>
3465:       ` : `<p class="evidence-empty">No images have been added yet.</p>`}
3466:     </section>
3467:   `;
3468: }
```

renderPortfolioTools (template-preview.js, lines 3470-3486)

renderPortfolioTools renders ui, markup, or display output. In this file, it appears around lines 3470-3486 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, toolName, toolDescription, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3471: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3470: function renderPortfolioTools(project) {
3471:   return `
3472:     <section class="portfolio-preview-section">
3473:       <h3>Tools</h3>
3474:       ${(project.tools || []).length ? `
3475:         <div class="portfolio-tool-list">
3476:           ${project.tools.map((tool) => `
3477:             <article>
3478:               <strong>${toolName(tool)}</strong>
3479:               ${toolDescription(tool) ? `<p>${toolDescription(tool)}</p>` : ""}
3480:             </article>
3481:           `).join("")}
3482:         </div>
3483:       ` : `<p class="evidence-empty">No tools have been added yet.</p>`}
3484:     </section>
3485:   `;
3486: }
```

renderPortfolioCustomSections (template-preview.js, lines 3488-3505)

renderPortfolioCustomSections renders ui, markup, or display output. In this file, it appears around lines 3488-3505 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, portfolioLink, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3489: return (project.sections || []).map((section) => `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3488: function renderPortfolioCustomSections(project) {
3489:   return (project.sections || []).map((section) => `
3490:     <section class="portfolio-preview-section">
3491:       <h3>${section.title}</h3>
3492:       ${section.description ? `<p>${section.description}</p>` : ""}
3493:       ${(section.items || []).length ? `
3494:         <div class="portfolio-resource-list">
3495:           ${section.items.map((item) => `
3496:             <article class="portfolio-resource">
3497:               <strong>${portfolioLink(item.url, item.title || "Untitled subsection")}</strong>
3498:               ${item.description ? `<p>${item.description}</p>` : ""}
3499:             </article>
3500:           `).join("")}
3501:         </div>
3502:       ` : `<p class="evidence-empty">No content has been added yet.</p>`}
3503:     </section>
3504:   `).join("");
3505: }
```

publicCustomSectionBlocks (template-preview.js, lines 3507-3518)

publicCustomSectionBlocks part of the private builder frontend and editing experience. In this file, it appears around lines 3507-3518 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, detailBlock, renderMultilineInlineText, evidenceList, resourceLink, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3508: return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-wide", `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 3507: function publicCustomSectionBlocks(project) {
 3508:   return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-
wide", `
 3509:     ${section.description ? `<p class="evidence-
empty">${renderMultilineInlineText(section.description)}</p>` : ""}
 3510:     ${evidenceList(section.items || [], (item) => `
 3511:       <li>
 3512:         ${item.url ? resourceLink(item, item.title) : `<strong>${item.title}</strong>`}
 3513:         <span>${item.type || "Section item"} &middot; ${item.status || "tracked"}</span>
 3514:         ${item.description ? `<p>${renderMultilineInlineText(item.description)}</p>` : ""}
 3515:       </li>
 3516:     `, "No content has been added yet.")}
 3517:   `)).join("");
 3518: }
```

renderProjectPortfolioPreview (template-preview.js, lines 3520-3556)

renderProjectPortfolioPreview renders ui, markup, or display output. In this file, it appears around lines 3520-3556 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references categoryById, find, filter, previewSectionHasRenderableContent, projectTemplateClass, projectResponsiveClass, projectTemplateStyle, renderParsedBriefBlock, map, parsedPublicSection, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3521: if (!project) return `<p class="evidence-empty">Select a project first.</p>`;; line 3527: return `; line 3538: return `.

Important local values include category at line 3522; accent at line 3523; briefSection at line 3525; otherSections at line 3526. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 3520: function renderProjectPortfolioPreview(project) {
 3521:   if (!project) return `<p class="evidence-empty">Select a project first.</p>`;
 3522:   const category = categoryById(project.category);
 3523:   const accent = category.accent || "#117c7a";
 3524:   if (project.portfolioView) {
 3525:     const briefSection = (project.portfolioView.sections || []).find((section) => section.id ===
"brief");
 3526:     const otherSections = (project.portfolioView.sections || []).filter((section) => section.id !==
"brief" && previewSectionHasRenderableContent(section));
 3527:     return `
 3528:       <article class="portfolio-preview-article ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" style="${projectTemplateStyle(project, accent)}">
 3529:         <header>
 3530:           ${renderParsedBriefBlock(briefSection, project.summary)}
 3531:         </header>
 3532:         <div class="evidence-grid">
 3533:           ${otherSections.map((section, index) => parsedPublicSection(section, index)).join("")}
 3534:         </div>
 3535:       </article>
 3536:     `;
 3537:   }
 3538:   return `
 3539:     <article class="portfolio-preview-article ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" style="${projectTemplateStyle(project, accent)}">
 3540:       <header>
 3541:         <div class="project-brief-default">
 3542:           <h4>Overview</h4>
 3543:           ${project.summary ? `<p class="rich-
paragraph">${renderMultilineInlineText(project.summary)}</p>` : `<p class="evidence-empty">No project overview
has been added yet.</p>`}
 3544:         </div>
 3545:       </header>
 3546:       ${renderPortfolioResourceList("Documents", project.documents, "No documents have been added yet.")}
 3547:       ${renderPortfolioResourceList("Tests", project.tests, "No tests have been added yet.")}
 3548:       ${renderPortfolioResourceList("PCBs Built", project.pcbs, "No PCB files have been added yet.")}
 3549:       ${renderPortfolioMedia(project.media)}
 3550:       ${renderPortfolioTools(project)}
 3551:       ${renderPortfolioResourceList("Languages", (project.languages || []).map((item) => ({ title: typeof
 item === "string" ? item : itemTitle(item) })), "No languages have been added yet.")}
 3552:       ${renderPortfolioResourceList("Links", project.links, "No links have been added yet.")}
 3553:       ${renderPortfolioCustomSections(project)}
 3554:     </article>
 3555:   `;
 3556: }
```

briefSection (template-preview.js, lines 3525-3528)

briefSection part of the private builder frontend and editing experience. In this file, it appears around lines 3525-3528 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, filter, previewSectionHasRenderableContent, projectTemplateClass, projectResponsiveClass, and projectTemplateStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3527: return `.

Important local values include briefSection at line 3525; otherSections at line 3526. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 3525:     const briefSection = (project.portfolioView.sections || []).find((section) => section.id ===
"brief");
 3526:     const otherSections = (project.portfolioView.sections || []).filter((section) => section.id !==
"brief" && previewSectionHasRenderableContent(section));
 3527:     return `
 3528:       <article class="portfolio-preview-article ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" style="${projectTemplateStyle(project, accent)}">
```

otherSections (template-preview.js, lines 3526-3528)

otherSections part of the private builder frontend and editing experience. In this file, it appears around lines 3526-3528 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, previewSectionHasRenderableContent, projectTemplateClass, projectResponsiveClass, and projectTemplateStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3527: return `.

Important local values include otherSections at line 3526. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 3526:     const otherSections = (project.portfolioView.sections || []).filter((section) => section.id !==
"brief" && previewSectionHasRenderableContent(section));
 3527:     return `
 3528:       <article class="portfolio-preview-article ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" style="${projectTemplateStyle(project, accent)}">
```

fullProjectPreviewHtmlExact (template-preview.js, lines 3558-3685)

fullProjectPreviewHtmlExact part of the private builder frontend and editing experience. In this file, it appears around lines 3558-3685 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parseProjectForPortfolio, categoryById, normalizeCategory, displayTitle, parsedPortfolioGlobals, stringify, replaceAll, escapeHtml, var, min, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3561: return `<!DOCTYPE html>; line 3601: return `<!DOCTYPE html>.

Important local values include baseHref at line 3559; parsedProject at line 3580; categorySource at line 3581; previewCategory at line 3582; parsedGlobals at line 3586; previewDataObject at line 3587; previewData at line 3599; title at line 3600. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3558: function fullProjectPreviewHtmlExact(project) {
3559:   const baseHref = `${window.location.origin}/`;
3560:   if (!project) {
3561:     return `<!DOCTYPE html>
```

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <base href="${baseHref}" />
    <link rel="stylesheet" href="styles.css" />
    <title>Project preview</title>
  </head>
  <body class="project-isolated-preview">
    <main class="project-isolated-shell">
      <div class="empty-state">
        <h3>No saved project preview</h3>
        <p>Click Save project before viewing the parsed project preview.</p>
      </div>
    </main>
  </body>
</html>`;
  }
  const parsedProject = project.portfolioView ? project : parseProjectForPortfolio(project);
  const categorySource = categoryById(parsedProject.category);
  const previewCategory = normalizeCategory(categorySource.id ? categorySource : {
    id: parsedProject.category || "project",
    label: displayTitle(parsedProject.category || "Project", "Project")
  });
  const parsedGlobals = parsedPortfolioGlobals();
  const previewDataObject = {
    categories: [previewCategory],
    fieldStyles: parsedGlobals.fieldStyles,
    funFacts: [],
    funFactsRich: null,
    profile: parsedGlobals.profile,
    profileRich: parsedGlobals.profileRich,
    projects: [parsedProject],
    siteContent: parsedGlobals.siteContent,
    siteContentRich: parsedGlobals.siteContentRich,
    siteSections: []
  };
  const previewData = JSON.stringify(previewDataObject).replaceAll("</", "<\\/");
  const title = parsedProject.portfolioView?.title || parsedProject.title || "Project preview";
  return `<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <base href="${baseHref}" />
    <title>${escapeHtml(title)} | Project preview</title>
    <link rel="stylesheet" href="styles.css" />
    <style>
      body.project-isolated-preview {
        min-height: 100vh;
        margin: 0;
        background: var(--paper, #f8fbff);
        color: var(--ink, #172636);
      }

      .project-isolated-shell {
        width: min(1180px, calc(100vw - 32px));
        margin: 0 auto;
        padding: 28px 0 36px;
      }

      .project-isolated-support {
        display: none !important;
      }

      body.project-isolated-preview .project-grid {
        display: block;
      }

      body.project-isolated-preview .category-section {
        padding: 0;
        border: 0;
        background: transparent;
      }

      body.project-isolated-preview .category-heading,
      body.project-isolated-preview .category-description {
        display: none;
      }

      body.project-isolated-preview .category-projects {
        display: block;
      }

      body.project-isolated-preview .project-card {
        margin: 0;
      }

      @media (max-width: 720px) {
        .project-isolated-shell {
          width: min(100vw - 20px, 100%);
          padding: 14px 0 24px;
```

```
3654:         }
3655:       }
3656:     </style>
3657:   </head>
3658:   <body class="project-isolated-preview">
3659:     <main class="project-isolated-shell" aria-label="Isolated project preview">
3660:       <div class="project-isolated-support" aria-hidden="true">
3661:         <label class="search-wrap" for="project-search">
3662:           <span>Search</span>
3663:           <input id="project-search" type="search" tabindex="-1" />
3664:         </label>
3665:         <div id="project-filters">
3666:           <button class="filter-button active" type="button" data-filter="all" tabindex="-1">All</button>
3667:         </div>
3668:         <span id="project-count">0</span>
3669:         <span id="project-track-count">0 Tracks</span>
3670:         <span id="project-track-labels">project categories</span>
3671:         <span id="year">${new Date().getFullYear()}</span>
3672:       </div>
3673:
3674:       <section class="section project-isolated-section" id="projects" aria-labelledby="project-isolated-
title">
3675:         <h1 class="sr-only" id="project-isolated-title">${escapeHtml(title)}</h1>
3676:         <div class="project-grid" id="project-grid" aria-live="polite"></div>
3677:       </section>
     ... excerpt truncated after 120 lines. Open the source file for the rest of this function.
```

renderProjectWebsitePreview (template-preview.js, lines 3687-3705)

renderProjectWebsitePreview renders ui, markup, or display output. In this file, it appears around lines 3687-3705 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references categoryById, applyProjectTemplateToElement, updateProjectPreviewNavigation, add, escapeHtml, querySelector, and fullProjectPreviewHtmlExact. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include category at line 3688; frame at line 3703. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3687: function renderProjectWebsitePreview(project) {
3688:   const category = categoryById(project?.category);
3689:   applyProjectTemplateToElement(projectPreviewDialog, project, category.accent || "#117c7a");
3690:   projectPreviewTitle.textContent = project?.portfolioView?.title || project?.title || "Project preview";
3691:   activeProjectPreviewProject = project || null;
3692:   activeProjectPreviewSectionIndex = -1;
3693:   activeProjectPreviewPath = [];
3694:   activeProjectPreviewForwardStack = [];
3695:   updateProjectPreviewNavigation();
3696:   projectPreviewContent.classList.add("is-website-preview");
3697:   projectPreviewContent.innerHTML = `
3698:     <iframe
3699:       class="portfolio-preview-frame project-preview-frame"
3700:       title="${escapeHtml(project?.title || "Project website preview")}"
3701:     ></iframe>
3702:   `;
3703:   const frame = projectPreviewContent.querySelector("iframe");
3704:   if (frame) frame.srcdoc = fullProjectPreviewHtmlExact(project);
3705: }
```

openProjectPortfolioPreview (template-preview.js, lines 3707-3715)

openProjectPortfolioPreview controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3707-3715 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, find, renderProjectWebsitePreview, add, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include project at line 3708; savedProject at line 3709; previewProject at line 3710. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3707: function openProjectPortfolioPreview() {
3708:   const project = selectedProject();
3709:   const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project?.id);
3710:   const previewProject = savedProject || project;
3711:   renderProjectWebsitePreview(previewProject);
3712:   document.body.classList.add("full-window-open");
3713:   projectPreviewDialog.showModal();
3714:   projectPreviewDialog.scrollTop = 0;
3715: }
```

savedProject (template-preview.js, lines 3709-3714)

savedProject persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 3709-3714 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, renderProjectWebsitePreview, add, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include savedProject at line 3709; previewProject at line 3710. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3709:   const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project?.id);
3710:   const previewProject = savedProject || project;
3711:   renderProjectWebsitePreview(previewProject);
3712:   document.body.classList.add("full-window-open");
3713:   projectPreviewDialog.showModal();
3714:   projectPreviewDialog.scrollTop = 0;
```

updateProjectPreviewNavigation (template-preview.js, lines 3717-3721)

updateProjectPreviewNavigation updates ui, state, cache, or stored data. In this file, it appears around lines 3717-3721 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3717: function updateProjectPreviewNavigation() {
3718:   projectPreviewBack.hidden = activeProjectPreviewSectionIndex < 0;
3719:   projectPreviewForward.hidden = !activeProjectPreviewForwardStack.length;
3720:   updateDialogWindowButtons(projectPreviewDialog);
3721: }
```

openProjectPreviewNode (template-preview.js, lines 3723-3745)

openProjectPreviewNode controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3723-3745 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, previewSectionHasRenderableContent, previewNodeAtPath, parsedItemHasContent, updateProjectPreviewNavigation, displayTitle, and parsedPublicSectionContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 3724: if (!activeProjectPreviewProject?.portfolioView) return;; line 3727: if (!section) return;; line 3729: if (!node) return;; line 3730: if (path.length ? !parsedItemHasContent(node) : !previewSectionHasRenderableContent(section)) return;.

Important local values include sections at line 3725; section at line 3726; node at line 3728. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 3723: function openProjectPreviewNode(sectionIndex, path = [], options = {}) {
 3724:   if (!activeProjectPreviewProject?.portfolioView) return;
 3725:   const sections = (activeProjectPreviewProject.portfolioView.sections || []).filter((section) =>
section.id !== "brief" && previewSectionHasRenderableContent(section));
 3726:   const section = sections[Number(sectionIndex)];
 3727:   if (!section) return;
 3728:   const node = path.length ? previewNodeAtPath(section, path) : section;
 3729:   if (!node) return;
 3730:   if (path.length ? !parsedItemHasContent(node) : !previewSectionHasRenderableContent(section)) return;
 3731:
 3732:   activeProjectPreviewSectionIndex = Number(sectionIndex);
 3733:   activeProjectPreviewPath = path;
 3734:   if (!options.preserveForward) activeProjectPreviewForwardStack = [];
 3735:   updateProjectPreviewNavigation();
 3736:   projectPreviewTitle.textContent = displayTitle(node.title || section.title, "Section");
 3737:   projectPreviewContent.innerHTML = `
 3738:     <article class="portfolio-preview-article">
 3739:       <section class="portfolio-preview-section">
 3740:         ${parsedPublicSectionContent(section, Number(sectionIndex), path)}
 3741:       </section>
 3742:     </article>
 3743:   `;
 3744:   projectPreviewDialog.scrollTop = 0;
 3745: }
```

sections (template-preview.js, lines 3725-3730)

sections part of the private builder frontend and editing experience. In this file, it appears around lines 3725-3730 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, previewSectionHasRenderableContent, previewNodeAtPath, and parsedItemHasContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3727: if (!section) return;; line 3729: if (!node) return;; line 3730: if (path.length ? !parsedItemHasContent(node) : !previewSectionHasRenderableContent(section)) return;.

Important local values include sections at line 3725; section at line 3726; node at line 3728. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 3725:   const sections = (activeProjectPreviewProject.portfolioView.sections || []).filter((section) =>
section.id !== "brief" && previewSectionHasRenderableContent(section));
 3726:   const section = sections[Number(sectionIndex)];
 3727:   if (!section) return;
 3728:   const node = path.length ? previewNodeAtPath(section, path) : section;
 3729:   if (!node) return;
 3730:   if (path.length ? !parsedItemHasContent(node) : !previewSectionHasRenderableContent(section)) return;
```

projectPreviewBackStep (template-preview.js, lines 3747-3769)

projectPreviewBackStep part of the private builder frontend and editing experience. In this file, it appears around lines 3747-3769 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references push, slice, renderProjectPortfolioPreview, updateProjectPreviewNavigation, and openProjectPreviewNode. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3748: if (!activeProjectPreviewProject) return;; line 3761: return;.

Important local values include nextPath at line 3767. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 3747: function projectPreviewBackStep() {
 3748:   if (!activeProjectPreviewProject) return;
 3749:   if (!activeProjectPreviewPath.length) {
 3750:     if (activeProjectPreviewSectionIndex >= 0) {
 3751:       activeProjectPreviewForwardStack.push({
 3752:         path: activeProjectPreviewPath.slice(),
 3753:         sectionIndex: activeProjectPreviewSectionIndex
 3754:       });
 3755:     }
```

```
    projectPreviewTitle.textContent = activeProjectPreviewProject.title || "Project preview";
    projectPreviewContent.innerHTML = renderProjectPortfolioPreview(activeProjectPreviewProject);
    activeProjectPreviewSectionIndex = -1;
    activeProjectPreviewPath = [];
    updateProjectPreviewNavigation();
    return;
  }
  activeProjectPreviewForwardStack.push({
    path: activeProjectPreviewPath.slice(),
    sectionIndex: activeProjectPreviewSectionIndex
  });
  const nextPath = activeProjectPreviewPath.slice(0, -1);
  openProjectPreviewNode(activeProjectPreviewSectionIndex, nextPath, { preserveForward: true });
}
```

projectPreviewForwardStep (template-preview.js, lines 3771-3775)

projectPreviewForwardStep part of the private builder frontend and editing experience. In this file, it appears around lines 3771-3775 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pop, and openProjectPreviewNode. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3772: if (!activeProjectPreviewProject || !activeProjectPreviewForwardStack.length) return;.

Important local values include next at line 3773. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function projectPreviewForwardStep() {
  if (!activeProjectPreviewProject || !activeProjectPreviewForwardStack.length) return;
  const next = activeProjectPreviewForwardStack.pop();
  openProjectPreviewNode(next.sectionIndex, next.path, { preserveForward: true });
}
```

closeOrStepBackProjectPreview (template-preview.js, lines 3777-3783)

closeOrStepBackProjectPreview controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3777-3783 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectPreviewBackStep, and closeProjectPreviewWindow. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3780: return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
function closeOrStepBackProjectPreview() {
  if (activeProjectPreviewProject && (activeProjectPreviewSectionIndex >= 0 ||
activeProjectPreviewPath.length)) {
    projectPreviewBackStep();
    return;
  }
  closeProjectPreviewWindow();
}
```

closeProjectPreviewWindow (template-preview.js, lines 3785-3791)

closeProjectPreviewWindow controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3785-3791 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closeDialogElement. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3785: function closeProjectPreviewWindow() {
3786:   activeProjectPreviewProject = null;
3787:   activeProjectPreviewSectionIndex = -1;
3788:   activeProjectPreviewPath = [];
3789:   activeProjectPreviewForwardStack = [];
3790:   closeDialogElement(projectPreviewDialog);
3791: }
```

publicProjectCard (template-preview.js, lines 3793-3856)

publicProjectCard part of the private builder frontend and editing experience. In this file, it appears around lines 3793-3856 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parsedPublicProjectCard, categoryById, projectTemplateClass, projectResponsiveClass, projectTemplateStyle, detailBlock, map, itemTitle, join, evidenceList, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3794: if (project.portfolioView) return parsedPublicProjectCard(project);; line 3798: return `.

Important local values include category at line 3795; accent at line 3796. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3793: function publicProjectCard(project) {
3794:   if (project.portfolioView) return parsedPublicProjectCard(project);
3795:   const category = categoryById(project.category);
3796:   const accent = category.accent || "#117c7a";
3797:
3798:   return `
3799:     <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" id="${project.id}" style="${projectTemplateStyle(project, accent)}">
3800:       <div class="project-body">
3801:         <h3>${project.title}</h3>
3802:         <p>${project.summary || ""}</p>
3803:
3804:         ${project.highlights && project.highlights.length ? detailBlock("Project highlights", "project-
drawer", `
3805:           <ul class="highlight-list">
3806:             ${project.highlights.map((item) => `<li>${typeof item === "string" ? item :
itemTitle(item)}</li>`).join("")}
3807:           </ul>
3808:         `) : ""}
3809:
3810:         <div class="evidence-grid" aria-label="${project.title} evidence blocks">
3811:           ${detailBlock("Documents", "evidence-block", evidenceList(project.documents, (item) => `
3812:             <li>
3813:               ${resourceLink(item, item.title)}
3814:               <span>${item.type || "Document"} &middot; ${item.status || "tracked"}</span>
3815:             </li>
3816:           `, "No document artifact has been added yet."))}
3817:
3818:           ${detailBlock("Tests and results", "evidence-block", evidenceList(project.tests, (item) => `
3819:             <li>
3820:               ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
3821:               <span>${item.method || "Validation"} &middot; ${item.status || "tracked"}</span>
3822:               ${item.result ? `<p>${item.result}</p>` : ""}
3823:             </li>
3824:           `, "No test artifact has been added yet."))}
3825:
3826:           ${detailBlock("PCBs built", "evidence-block", evidenceList(project.pcbs, (item) => `
3827:             <li>
3828:               ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
3829:               <span>${item.revision || "Revision"} &middot; ${item.status || "tracked"}</span>
3830:             </li>
3831:           `, "No PCB build has been added yet."))}
3832:
3833:           ${detailBlock("Images", "evidence-block evidence-wide", mediaGrid(project.media))}
3834:           ${publicCustomSectionBlocks(project)}
3835:         </div>
3836:
3837:         ${detailBlock("Tools and implementation files", "project-drawer", `
3838:           <div class="project-tooling">
3839:             <div>
3840:               <h4>Tools Used</h4>
3841:               <div class="project-meta">${pillList(project.tools, "tool-tag")}</div>
3842:             </div>
3843:             <div>
```

```
              <h4>Languages</h4>
              <div class="project-meta">${pillList(project.languages, "language-tag")}</div>
            </div>
          </div>
        `)}

        <div class="resource-list">
          ${(project.links || []).map((item) => resourceLink(item)).join("")}
        </div>
      </div>
    </article>
  `;
}
```

renderParsedBriefBlock (template-preview.js, lines 3858-3868)

renderParsedBriefBlock renders ui, markup, or display output. In this file, it appears around lines 3858-3868 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderRichContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3861: if (!briefItem.rich?.blocks?.length && !briefText) return "";; line 3862: return `.

Important local values include briefItem at line 3859; briefText at line 3860. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function renderParsedBriefBlock(section, fallbackSummary = "") {
  const briefItem = section?.items?.[0] || {};
  const briefText = briefItem.description || fallbackSummary || "";
  if (!briefItem.rich?.blocks?.length && !briefText) return "";
  return `
    <details class="project-brief-default parsed-summary" open>
      <summary>Overview</summary>
      ${renderRichContent(briefItem.rich, briefText)}
    </details>
  `;
}
```

previewPathToString (template-preview.js, lines 3870-3872)

previewPathToString part of the private builder frontend and editing experience. In this file, it appears around lines 3870-3872 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3871: return path.join(".");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
function previewPathToString(path = []) {
  return path.join(".");
}
```

previewPathFromString (template-preview.js, lines 3874-3877)

previewPathFromString part of the private builder frontend and editing experience. In this file, it appears around lines 3874-3877 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, filter, and isInteger. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3875: if (!value) return [];; line 3876: return value.split(".").map((item) => Number(item)).filter((item) => Number.isInteger(item));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3874: function previewPathFromString(value = "") {
3875:   if (!value) return [];
3876:   return value.split(".").map((item) => Number(item)).filter((item) => Number.isInteger(item));
3877: }
```

previewNodeAtPath (template-preview.js, lines 3879-3888)

previewNodeAtPath part of the private builder frontend and editing experience. In this file, it appears around lines 3879-3888 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3884: if (!node) return null;; line 3887: return node;.

Important local values include node at line 3880; children at line 3881. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3879: function previewNodeAtPath(section, path = []) {
3880:   let node = section;
3881:   let children = section?.items || [];
3882:   for (const index of path) {
3883:     node = children[index];
3884:     if (!node) return null;
3885:     children = node.children || [];
3886:   }
3887:   return node;
3888: }
```

previewNodeChildren (template-preview.js, lines 3890-3892)

previewNodeChildren part of the private builder frontend and editing experience. In this file, it appears around lines 3890-3892 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3891: return node?.items || node?.children || [];.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3890: function previewNodeChildren(node) {
3891:   return node?.items || node?.children || [];
3892: }
```

previewSectionHasRenderableContent (template-preview.js, lines 3894-3896)

previewSectionHasRenderableContent part of the private builder frontend and editing experience. In this file, it appears around lines 3894-3896 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richHasContent, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3895: return Boolean(section?.description || richHasContent(section?.rich) || (section?.items || []).some(parsedItemHasContent));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3894: function previewSectionHasRenderableContent(section) {
3895:   return Boolean(section?.description || richHasContent(section?.rich) || (section?.items ||
[]).some(parsedItemHasContent));
3896: }
```

previewNodeSummary (template-preview.js, lines 3898-3906)

previewNodeSummary part of the private builder frontend and editing experience. In this file, it appears around lines 3898-3906 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, and renderRichContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3899: if (!rich?.blocks?.length && !text) return "";; line 3900: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3898: function previewNodeSummary(title, rich, text, emptyMessage = "No summary has been added yet.") {
3899:   if (!rich?.blocks?.length && !text) return "";
3900:   return `
3901:     <details class="parsed-summary" open>
3902:       <summary>${escapeHtml(title)}</summary>
3903:       ${renderRichContent(rich, text || "")}
3904:     </details>
3905:   `;
3906: }
```

previewNodeIsOverview (template-preview.js, lines 3908-3911)

previewNodeIsOverview part of the private builder frontend and editing experience. In this file, it appears around lines 3908-3911 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, displayTitle, and endsWith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3910: return item.kind === "summary" || title === "overview" || title.endsWith(" overview");.

Important local values include title at line 3909. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3908: function previewNodeIsOverview(item = {}) {
3909:   const title = normalize(displayTitle(item.title || item.label || ""));
3910:   return item.kind === "summary" || title === "overview" || title.endsWith(" overview");
3911: }
```

previewNodeOverviewDetails (template-preview.js, lines 3913-3931)

previewNodeOverviewDetails part of the private builder frontend and editing experience. In this file, it appears around lines 3913-3931 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, push, renderRichContent, forEach, previewFileList, previewNodeChildren, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3924: if (!blocks.length) return "";; line 3925: return `.

Important local values include overviewItems at line 3914; blocks at line 3915; itemFiles at line 3921. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3913: function previewNodeOverviewDetails(node, children = []) {
3914:   const overviewItems = children.filter(previewNodeIsOverview);
3915:   const blocks = [];
```

```
  if (node?.rich?.blocks?.length || node?.description) {
    blocks.push(renderRichContent(node.rich, node.description || ""));
  }
  overviewItems.forEach((item) => {
    if (item.rich?.blocks?.length || item.description) blocks.push(renderRichContent(item.rich,
item.description || ""));
    const itemFiles = previewFileList(previewNodeChildren(item));
    if (itemFiles) blocks.push(itemFiles);
  });
  if (!blocks.length) return "";
  return `
    <details class="parsed-summary section-overview-default" open>
      <summary>Overview</summary>
      ${blocks.join("")}
    </details>
  `;
}
```

previewNodeCard (template-preview.js, lines 3933-3940)

previewNodeCard part of the private builder frontend and editing experience. In this file, it appears around lines 3933-3940 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references displayTitle, previewPathToString, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3935: return `.

Important local values include title at line 3934. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function previewNodeCard(node, sectionIndex, path) {
  const title = displayTitle(node.title);
  return `
    <button class="section-open-card subsection-open-card" type="button" data-preview-section-
index="${sectionIndex}" data-preview-resource-path="${previewPathToString(path)}">
      <span>${escapeHtml(title)}</span>
    </button>
  `;
}
```

previewChildCards (template-preview.js, lines 3942-3950)

previewChildCards part of the private builder frontend and editing experience. In this file, it appears around lines 3942-3950 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, parsedItemHasContent, previewNodeIsOverview, map, previewNodeCard, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3944: if (!visibleChildren.length) return "";; line 3945: return `.

Important local values include visibleChildren at line 3943. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function previewChildCards(children, sectionIndex, basePath = []) {
  const visibleChildren = children.filter((child) => parsedItemHasContent(child) && !child.url &&
!previewNodeIsOverview(child));
  if (!visibleChildren.length) return "";
  return `
    <div class="subsection-grid section-content-grid">
      ${children.map((child, index) => parsedItemHasContent(child) && !child.url &&
!previewNodeIsOverview(child) ? previewNodeCard(child, sectionIndex, [...basePath, index]) : "").join("")}
    </div>
  `;
}
```

previewFileList (template-preview.js, lines 3952-3969)

previewFileList part of the private builder frontend and editing experience. In this file, it appears around lines 3952-3969 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, map, escapeHtml, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, downloadAttribute, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3954: if (!files.length) return "";; line 3955: return `.

Important local values include files at line 3953. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3952: function previewFileList(children = []) {
3953:   const files = children.filter((child) => child?.url);
3954:   if (!files.length) return "";
3955:   return `
3956:     <div class="section-file-list">
3957:       ${files.map((file) => `
3958:         <a
3959:           class="section-file-link"
3960:           href="${escapeHtml(normalizeLinkTarget(file.url, { assumeWeb: isWebsiteLinkItem(file, file.url)
}))}"
3961:           ${linkAttributes(file.url, file)}
3962:           ${downloadAttribute(file.url, file)}
3963:         >
3964:           ${escapeHtml(file.title || "Open link")}
3965:         </a>
3966:       `).join("")}
3967:     </div>
3968:   `;
3969: }
```

previewNodeContent (template-preview.js, lines 3971-3983)

previewNodeContent part of the private builder frontend and editing experience. In this file, it appears around lines 3971-3983 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references previewFileList, previewNodeChildren, filter, previewNodeIsOverview, previewNodeOverviewDetails, and previewChildCards. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3973: if (!isContainer && node.url) return previewFileList([node]);; line 3976: return `.

Important local values include isContainer at line 3972; children at line 3974; contentChildren at line 3975. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3971: function previewNodeContent(node, sectionIndex, path = []) {
3972:   const isContainer = !node.kind || node.kind === "subsection";
3973:   if (!isContainer && node.url) return previewFileList([node]);
3974:   const children = previewNodeChildren(node);
3975:   const contentChildren = children.filter((child) => !previewNodeIsOverview(child));
3976:   return `
3977:     <article class="parsed-window-panel section-directory">
3978:     ${previewNodeOverviewDetails(node, children)}
3979:     ${previewChildCards(children, sectionIndex, path)}
3980:     ${previewFileList(contentChildren)}
3981:     </article>
3982:   `;
3983: }
```

parsedPublicSectionContent (template-preview.js, lines 3985-3989)

parsedPublicSectionContent part of the private builder frontend and editing experience. In this file, it appears around lines 3985-3989 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references previewNodeAtPath, and previewNodeContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3987: if (!node) return `<p class="evidence-empty">This section could not be found.</p>`;; line 3988: return previewNodeContent(node, sectionIndex, path);.

Important local values include node at line 3986. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3985: function parsedPublicSectionContent(section, sectionIndex = 0, path = []) {
3986:   const node = path.length ? previewNodeAtPath(section, path) : section;
3987:   if (!node) return `<p class="evidence-empty">This section could not be found.</p>`;
3988:   return previewNodeContent(node, sectionIndex, path);
3989: }
```

parsedPublicSection (template-preview.js, lines 3991-3999)

parsedPublicSection part of the private builder frontend and editing experience. In this file, it appears around lines 3991-3999 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, some, escapeHtml, and displayTitle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3994: return `.

Important local values include visibleItems at line 3992; hasImages at line 3993. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3991: function parsedPublicSection(section, index = 0) {
3992:   const visibleItems = (section.items || []).filter(parsedItemHasContent);
3993:   const hasImages = visibleItems.some((item) => item.kind === "image");
3994:   return `
3995:     <button class="section-open-card evidence-block ${hasImages ? "evidence-wide" : ""}" type="button"
data-preview-section-index="${index}">
3996:       <span>${escapeHtml(displayTitle(section.title, "Section"))}</span>
3997:     </button>
3998:   `;
3999: }
```

parsedPublicProjectCard (template-preview.js, lines 4001-4019)

parsedPublicProjectCard part of the private builder frontend and editing experience. In this file, it appears around lines 4001-4019 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references categoryById, find, filter, previewSectionHasRenderableContent, projectTemplateClass, projectResponsiveClass, projectTemplateStyle, renderParsedBriefBlock, map, parsedPublicSection, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4008: return `.

Important local values include category at line 4002; accent at line 4003; view at line 4004; briefSection at line 4005; otherSections at line 4006. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4001: function parsedPublicProjectCard(project) {
4002:   const category = categoryById(project.category);
4003:   const accent = category.accent || "#117c7a";
4004:   const view = project.portfolioView;
4005:   const briefSection = (view.sections || []).find((section) => section.id === "brief");
4006:   const otherSections = (view.sections || []).filter((section) => section.id !== "brief" &&
previewSectionHasRenderableContent(section));
4007:
4008:   return `
4009:     <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" id="${project.id}" style="${projectTemplateStyle(project, accent)}">
4010:       <div class="project-body">
4011:         <h3>${view.title || project.title}</h3>
4012:         ${renderParsedBriefBlock(briefSection, project.summary)}
4013:         <div class="evidence-grid" aria-label="${project.title} parsed project content">
4014:           ${otherSections.map((section, index) => parsedPublicSection(section, index)).join("")}
4015:         </div>
4016:       </div>
4017:     </article>
```

```
4018:   `;
4019: }
```

briefSection (template-preview.js, lines 4005-4006)

briefSection part of the private builder frontend and editing experience. In this file, it appears around lines 4005-4006 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, filter, and previewSectionHasRenderableContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include briefSection at line 4005; otherSections at line 4006. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4005:   const briefSection = (view.sections || []).find((section) => section.id === "brief");
4006:   const otherSections = (view.sections || []).filter((section) => section.id !== "brief" &&
previewSectionHasRenderableContent(section));
```

otherSections (template-preview.js, lines 4006-4006)

otherSections part of the private builder frontend and editing experience. In this file, it appears around lines 4006-4006 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, and previewSectionHasRenderableContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include otherSections at line 4006. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4006:   const otherSections = (view.sections || []).filter((section) => section.id !== "brief" &&
previewSectionHasRenderableContent(section));
```

publicCategorySection (template-preview.js, lines 4021-4036)

publicCategorySection part of the private builder frontend and editing experience. In this file, it appears around lines 4021-4036 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4022: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4021: function publicCategorySection(category, visibleProjects) {
4022:   return `
4023:     <section class="category-section ${visibleProjects.length ? "" : "empty-category-section"}" aria-
labelledby="${category.id}-preview-title">
4024:       <div class="category-heading">
4025:         <div>
4026:           <h3 id="${category.id}-preview-title">${category.label}</h3>
4027:         </div>
4028:         ${visibleProjects.length ? `<span>${visibleProjects.length} project${visibleProjects.length === 1
? "" : "s"}</span>` : ""}
4029:       </div>
4030:       ${visibleProjects.length ? `<p class="category-description">${category.description}</p>` : ""}
4031:       <div class="category-projects">
4032:         ${visibleProjects.map(publicProjectCard).join("")}
4033:       </div>
4034:     </section>
```

```
4035:   `;
4036: }
```

renderedPublicProjects (template-preview.js, lines 4038-4051)

renderedPublicProjects renders ui, markup, or display output. In this file, it appears around lines 4038-4051 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, filter, publicCategorySection, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4040: return `; line 4047: return savedPortfolioCatalog.categories.map((category) => {; line 4049: return publicCategorySection(category, categoryProjects);.

Important local values include categoryProjects at line 4048. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4038: function renderedPublicProjects() {
4039:   if (!savedPortfolioCatalog.projects.length) {
4040:     return `
4041:       <div class="empty-state">
4042:         <h3>No parsed projects saved yet.</h3>
4043:         <p>Open a project, edit it, then click Save to build it into this portfolio preview.</p>
4044:       </div>
4045:     `;
4046:   }
4047:   return savedPortfolioCatalog.categories.map((category) => {
4048:     const categoryProjects = savedPortfolioCatalog.projects.filter((project) => project.category ===
category.id);
4049:     return publicCategorySection(category, categoryProjects);
4050:   }).join("");
4051: }
```

sectionControls (template-preview.js, lines 4053-4060)

sectionControls part of the private builder frontend and editing experience. In this file, it appears around lines 4053-4060 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4054: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4053: function sectionControls(sectionId, uploadKey = "", folder = "") {
4054:   return `
4055:     <div class="local-section-controls">
4056:       <button type="button" title="Edit this section" data-preview-section="${sectionId}">Edit</button>
4057:       ${uploadKey ? `<button type="button" title="Add file or image" data-preview-upload="${uploadKey}"
data-folder="${folder}">+</button>` : ""}
4058:     </div>
4059:   `;
4060: }
```

customSectionBlocks (template-preview.js, lines 4062-4074)

customSectionBlocks part of the private builder frontend and editing experience. In this file, it appears around lines 4062-4074 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, detailBlock, sectionControls, evidenceList, resourceLink, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4063: return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-wide", `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4062: function customSectionBlocks(project) {
4063:   return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-
wide", `
4064:     ${sectionControls(`custom:${section.id}`, "sections", section.id)}
4065:     ${section.description ? `<p class="evidence-empty">${section.description}</p>` : ""}
4066:     ${evidenceList(section.items || [], (item) => `
4067:       <li>
4068:         ${item.url ? resourceLink(item, item.title) : `<strong>${item.title}</strong>`}
4069:         <span>${item.type || "Section item"} &middot; ${item.status || "tracked"}</span>
4070:         ${item.description ? `<p>${item.description}</p>` : ""}
4071:       </li>
4072:     `, "No content has been added yet.")}
4073:   `)).join("");
4074: }
```

projectCard (template-preview.js, lines 4076-4154)

projectCard part of the private builder frontend and editing experience. In this file, it appears around lines 4076-4154 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references categoryById, projectTemplateClass, projectResponsiveClass, projectTemplateStyle, detailBlock, map, join, sectionControls, evidenceList, resourceLink, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4080: return `.

Important local values include category at line 4077; accent at line 4078. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4076: function projectCard(project, isSelected = false) {
4077:   const category = categoryById(project.category);
4078:   const accent = category.accent || "#117c7a";
4079:
4080:   return `
4081:     <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)} ${isSelected ? "preview-project-card" : ""}" id="${project.id}"
style="${projectTemplateStyle(project, accent)}">
4082:       <div class="project-body">
4083:         <div class="local-card-controls">
4084:           <button type="button" title="Open project" data-preview-project="${project.id}">Open</button>
4085:           <button type="button" title="Delete project" data-preview-
delete="${project.id}">Delete</button>
4086:         </div>
4087:         <h3>${project.title}</h3>
4088:         <p>${project.summary}</p>
4089:
4090:         ${project.highlights && project.highlights.length ? detailBlock("Project highlights", "project-
drawer", `
4091:           <ul class="highlight-list">
4092:             ${project.highlights.map((item) => `<li>${item}</li>`).join("")}
4093:           </ul>
4094:         `) : ""}
4095:
4096:         <div class="evidence-grid" aria-label="${project.title} evidence blocks">
4097:           ${detailBlock("Documents", "evidence-block", `
4098:             ${sectionControls("documents", "documents", "documents")}
4099:             ${evidenceList(project.documents, (item) => `
4100:             <li>
4101:               ${resourceLink(item, item.title)}
4102:               <span>${item.type || "Document"} &middot; ${item.status || "tracked"}</span>
4103:             </li>
4104:           `, "No document artifact has been added yet.")}
4105:           `)}
4106:
4107:           ${detailBlock("Tests and results", "evidence-block", `
4108:             ${sectionControls("tests", "tests", "tests")}
4109:             ${evidenceList(project.tests, (item) => `
4110:             <li>
4111:               ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
4112:               <span>${item.method || "Validation"} &middot; ${item.status || "tracked"}</span>
4113:               ${item.result ? `<p>${item.result}</p>` : ""}
4114:             </li>
4115:           `, "No test artifact has been added yet.")}
4116:           `)}
4117:
4118:           ${detailBlock("PCBs built", "evidence-block", `
4119:             ${sectionControls("pcbs", "pcbs", "pcbs")}
4120:             ${evidenceList(project.pcbs, (item) => `
```

```
            <li>
              ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
              <span>${item.revision || "Revision"} &middot; ${item.status || "tracked"}</span>
            </li>
          `, "No PCB build has been added yet.")}
          `)}

          ${detailBlock("Images", "evidence-block evidence-wide", `
            ${sectionControls("media", "media", "images")}
            ${mediaGrid(project.media)}
          `)}
          ${customSectionBlocks(project)}
        </div>

        ${detailBlock("Tools and implementation files", "project-drawer", `
          <div class="project-tooling">
            <div>
              <h4>Tools Used</h4>
              <div class="project-meta">${pillList(project.tools, "tool-tag")}</div>
            </div>
            <div>
              <h4>Languages</h4>
              <div class="project-meta">${pillList(project.languages, "language-tag")}</div>
            </div>
          </div>
        `)}

        <div class="resource-list">
          ${(project.links || []).map((item) => resourceLink(item)).join("")}
        </div>
      </div>
    </article>
  `;
}
```

buildTemplateProject (template-preview.js, lines 4156-4186)

buildTemplateProject creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4156-4186 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, trim, slugify, clone, and defaultDesignModel. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4162: return {.

Important local values include template at line 4157; title at line 4158; id at line 4159; category at line 4160. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function buildTemplateProject() {
  const template = templates.find((item) => item.id === templateSelect.value) || null;
  const title = newProjectTitle.value.trim() || "New Hardware Project";
  const id = slugify(title);
  const category = newCategorySelect.value || pendingCreateCategoryId || "analog-mixed-signal";

  return {
    id,
    title,
    templateId: template?.id || "",
    templateLabel: template?.label || "",
    templateGroup: "Appearance",
    templateVisual: template?.visual ? clone(template.visual) : null,
    category,
    status: "Draft",
    summary: "",
    summaryRich: null,
    focus: [],
    highlights: [],
    documents: [],
    tests: [],
    pcbs: [],
    media: [],
    tools: [],
    languages: [],
    links: [],
    sections: [],
    compileCode: { files: [], activeFileId: "", terminal: "" },
    design: category === "analog-mixed-signal" ? defaultDesignModel() : undefined
  };
}
```

insertProject (template-preview.js, lines 4188-4204)

insertProject part of the private builder frontend and editing experience. In this file, it appears around lines 4188-4204 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, map, and splice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 4192: if (placement === "site-start") return [project, ...nextProjects];; line 4193: if (placement === "site-end") return [...nextProjects, project];; line 4200: if (!indexes.length) return [...nextProjects, project];; line 4203: return nextProjects;.

Important local values include nextProjects at line 4189; placement at line 4190; indexes at line 4195; targetIndex at line 4201. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4188: function insertProject(project) {
4189:   const nextProjects = catalog.projects.filter((item) => item.id !== project.id);
4190:   const placement = placementSelect.value;
4191:
4192:   if (placement === "site-start") return [project, ...nextProjects];
4193:   if (placement === "site-end") return [...nextProjects, project];
4194:
4195:   const indexes = nextProjects
4196:     .map((item, index) => ({ item, index }))
4197:     .filter(({ item }) => item.category === project.category)
4198:     .map(({ index }) => index);
4199:
4200:   if (!indexes.length) return [...nextProjects, project];
4201:   const targetIndex = placement === "category-start" ? indexes[0] : indexes[indexes.length - 1] + 1;
4202:   nextProjects.splice(targetIndex, 0, project);
4203:   return nextProjects;
4204: }
```

renderCategoryDropdown (template-preview.js, lines 4206-4213)

renderCategoryDropdown renders ui, markup, or display output. In this file, it appears around lines 4206-4213 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4206: function renderCategoryDropdown() {
4207:   categoryDropdown.innerHTML = (catalog.categories || []).map((category) => `
4208:     <button type="button" data-create-category="${escapeHtml(category.id)}" style="--category-accent:
${escapeHtml(category.accent || "#117c7a")}">
4209:       <span>${escapeHtml(category.label)}</span>
4210:       <small>${escapeHtml(category.description || "Project category")}</small>
4211:     </button>
4212:   `).join("");
4213: }
```

toggleCategoryDropdown (template-preview.js, lines 4215-4219)

toggleCategoryDropdown part of the private builder frontend and editing experience. In this file, it appears around lines 4215-4219 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include open at line 4216. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4215: function toggleCategoryDropdown(forceOpen = null) {
4216:   const open = forceOpen === null ? categoryDropdown.hidden : forceOpen;
4217:   categoryDropdown.hidden = !open;
4218:   addProjectButton.setAttribute("aria-expanded", String(open));
4219: }
```

createProjectInCategory (template-preview.js, lines 4221-4233)

createProjectInCategory creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4221-4233 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references buildTemplateProject, insertProject, add, toggleCategoryDropdown, setStatus, categoryById, scheduleAutosave, renderAll, and openProjectWindow. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include project at line 4223. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4221: function createProjectInCategory(categoryId) {
4222:   newCategorySelect.value = categoryId;
4223:   const project = buildTemplateProject();
4224:   catalog.projects = insertProject(project);
4225:   selectedProjectId = project.id;
4226:   activeSectionId = "brief";
4227:   expandedCategories.add(categoryId);
4228:   toggleCategoryDropdown(false);
4229:   setStatus(`Project added locally in ${categoryById(categoryId).label || categoryId}. Click Save project
to include it in the portfolio preview.`);
4230:   scheduleAutosave();
4231:   renderAll();
4232:   openProjectWindow(project.id, "brief");
4233: }
```

openProjectCreateDialog (template-preview.js, lines 4235-4245)

openProjectCreateDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 4235-4245 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references categoryById, groupedTemplateOptions, toggleCategoryDropdown, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include category at line 4238. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4235: function openProjectCreateDialog(categoryId) {
4236:   pendingCreateCategoryId = categoryId;
4237:   newCategorySelect.value = categoryId;
4238:   const category = categoryById(categoryId);
4239:   projectCreateCategory.textContent = category.label || "Project category";
4240:   templateSelect.innerHTML = groupedTemplateOptions();
4241:   templateSelect.value = "";
4242:   newProjectTitle.value = "";
4243:   toggleCategoryDropdown(false);
4244:   projectCreateDialog.showModal();
4245: }
```

groupedTemplateOptions (template-preview.js, lines 4247-4253)

groupedTemplateOptions part of the private builder frontend and editing experience. In this file, it appears around lines 4247-4253 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4252: return `${emptyOption}${options}`;.

Important local values include emptyOption at line 4248; options at line 4249. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4247: function groupedTemplateOptions() {
4248:   const emptyOption = `<option value="">No appearance - white project layout</option>`;
4249:   const options = templates
4250:     .map((template) => `<option value="${template.id}">${template.label}</option>`)
4251:     .join("");
4252:   return `${emptyOption}${options}`;
4253: }
```

showTemplatePreview (template-preview.js, lines 4255-4276)

showTemplatePreview controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 4255-4276 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setProperty, map, join, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4256: if (!template) return;.

Important local values include visual at line 4257; palette at line 4258. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4255: function showTemplatePreview(template) {
4256:   if (!template) return;
4257:   const visual = template.visual || {};
4258:   const palette = visual.palette || ["#0f4c5c", "#7dd3c7", "#ffffff"];
4259:
4260:   templatePreviewGroup.textContent = visual.style ? `Appearance / ${visual.style}` : "Appearance";
4261:   templatePreviewTitle.textContent = template.label;
4262:   templatePreviewDescription.textContent = template.description || "";
4263:   templatePreviewCardTitle.textContent = template.label;
4264:   templatePreviewInteraction.textContent = visual.interaction || "Hover and click styles are shown in the
local builder.";
4265:   templatePreviewCard.style.setProperty("--template-primary", palette[0]);
4266:   templatePreviewCard.style.setProperty("--template-hover", palette[1]);
4267:   templatePreviewCard.style.setProperty("--template-paper", palette[2] || "#ffffff");
4268:   templatePreviewCard.style.setProperty("--template-bg", visual.background || palette[0]);
4269:   templatePreviewCard.style.setProperty("--template-panel", visual.panel || palette[2] || "#ffffff");
4270:   templatePreviewCard.style.setProperty("--template-accent", visual.accent || palette[1]);
4271:   templatePreviewCard.style.setProperty("--template-click", visual.click || palette[2] || "#ffffff");
4272:   templatePreviewCard.style.setProperty("--template-click-text", visual.clickText || visual.text ||
"#172636");
4273:   templateVisual.className = `template-visual template-visual-${visual.style || "schematic"}`;
4274:   templatePalette.innerHTML = palette.map((color) => `<span style="background:
${color}">${color}</span>`).join("");
4275:   templateDialog.showModal();
4276: }
```

renderTree (template-preview.js, lines 4278-4319)

renderTree renders ui, markup, or display output. In this file, it appears around lines 4278-4319 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectSearchText, map, filter, includes, projectMatchesSearch, has, escapeHtml, highlightSearchText, join, and updateProjectSearchStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4287: if (cleanQuery && !categoryProjects.length && !categoryHit) return "";; line 4290: return `.

Important local values include cleanQuery at line 4279; matchCount at line 4280; treeMarkup at line 4281; allCategoryProjects at line 4282; categoryHit at line 4283; categoryProjects at line 4284; isExpanded at line 4289. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 4278: function renderTree() {
 4279:   const cleanQuery = projectSearchText(projectSearchQuery);
 4280:   let matchCount = 0;
 4281:   const treeMarkup = catalog.categories.map((category) => {
 4282:     const allCategoryProjects = catalog.projects.filter((project) => project.category === category.id);
 4283:     const categoryHit = cleanQuery && projectSearchText(`${category.label}
${category.description}`).includes(cleanQuery);
 4284:     const categoryProjects = cleanQuery
 4285:       ? allCategoryProjects.filter((project) => categoryHit || projectMatchesSearch(project, category,
cleanQuery))
 4286:       : allCategoryProjects;
 4287:     if (cleanQuery && !categoryProjects.length && !categoryHit) return "";
 4288:     matchCount += categoryProjects.length;
 4289:     const isExpanded = cleanQuery ? true : expandedCategories.has(category.id);
 4290:     return `
 4291:       <section class="tree-category">
 4292:         <div class="tree-category-heading">
 4293:           <button class="tree-category-toggle" type="button" data-toggle-
category="${escapeHtml(category.id)}" aria-expanded="${isExpanded}">
 4294:             <span>${highlightSearchText(category.label, projectSearchQuery)}</span>
 4295:             <small>${allCategoryProjects.length} project${allCategoryProjects.length === 1 ? "" :
"s"}</small>
 4296:           </button>
 4297:         </div>
 4298:         <div class="tree-projects" ${isExpanded ? "" : "hidden"}>
 4299:           ${categoryProjects.length ? categoryProjects.map((project) => `
 4300:             <div class="tree-project-row ${project.id === selectedProjectId ? "active" : ""}">
 4301:               <button type="button" data-project-id="${escapeHtml(project.id)}">
 4302:                 <span>${highlightSearchText(project.title, projectSearchQuery)}</span>
 4303:               </button>
 4304:               <button class="tree-project-delete" type="button" data-delete-
project="${escapeHtml(project.id)}" aria-label="Delete ${escapeHtml(project.title)}" title="Delete project">
 4305:                 &times;
 4306:               </button>
 4307:             </div>
 4308:           `).join("") : cleanQuery ? `<p class="tree-empty">Category matches, but no projects are in it
yet.</p>` : ""}
 4309:         </div>
 4310:       </section>
 4311:     `;
 4312:   }).join("");
 4313:   projectTree.innerHTML = treeMarkup || `
 4314:     <section class="tree-category tree-search-empty">
 4315:       <p class="tree-empty">No project, file, tool, or section matched this search.</p>
 4316:     </section>
 4317:   `;
 4318:   updateProjectSearchStatus(cleanQuery ? matchCount : catalog.projects.length);
 4319: }
```

summaryWordCount (template-preview.js, lines 4321-4323)

summaryWordCount part of the private builder frontend and editing experience. In this file, it appears around lines 4321-4323 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references wordCount, and plainTextFromRich. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4322: return wordCount(project?.summaryRich?.blocks?.length ? plainTextFromRich(project.summaryRich) : project?.summary || "");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 4321: function summaryWordCount(project) {
 4322:   return wordCount(project?.summaryRich?.blocks?.length ? plainTextFromRich(project.summaryRich) :
project?.summary || "");
 4323: }
```

renderSummaryPreview (template-preview.js, lines 4325-4329)

renderSummaryPreview renders ui, markup, or display output. In this file, it appears around lines 4325-4329 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderRichContent, and renderMultilineInlineText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4326: if (project.summaryRich?.blocks?.length) return renderRichContent(project.summaryRich, project.summary || "");; line 4327: if (project.summary) return `<p class="rich-paragraph">${renderMultilineInlineText(project.summary)}</p>`;; line 4328: return `<p class="evidence-empty">No project overview has been added yet.</p>`;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 4325: function renderSummaryPreview(project) {
 4326:   if (project.summaryRich?.blocks?.length) return renderRichContent(project.summaryRich, project.summary
|| "");
 4327:   if (project.summary) return `<p class="rich-
paragraph">${renderMultilineInlineText(project.summary)}</p>`;
 4328:   return `<p class="evidence-empty">No project overview has been added yet.</p>`;
 4329: }
```

renderSummaryBuilder (template-preview.js, lines 4331-4359)

renderSummaryBuilder renders ui, markup, or display output. In this file, it appears around lines 4331-4359 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references summaryWordCount, and renderSummaryPreview. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4335: return `.

Important local values include isEditing at line 4332; summaryWords at line 4333; hasSummary at line 4334. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 4331: function renderSummaryBuilder(project) {
 4332:   const isEditing = summaryEditorProjectId === project.id;
 4333:   const summaryWords = summaryWordCount(project);
 4334:   const hasSummary = summaryWords > 0 || Boolean(project.summaryRich?.blocks?.length);
 4335:   return `
 4336:     <div class="summary-builder wide-field">
 4337:       <div class="summary-builder-heading">
 4338:         <div>
 4339:           <span>Project overview <small>${summaryWords}/1000 words</small></span>
 4340:         </div>
 4341:         <div class="summary-builder-actions">
 4342:           ${isEditing
 4343:             ? `<button class="button secondary" type="button" data-summary-cancel="true">Cancel
overview</button>`
 4344:             : `<button class="button primary" type="button" data-summary-create="true">${hasSummary ?
"Edit overview" : "Create overview"}</button>`}
 4345:           ${hasSummary && !isEditing ? `<button class="button danger-control" type="button" data-summary-
delete="true">Delete overview</button>` : ""}
 4346:         </div>
 4347:       </div>
 4348:       ${isEditing ? `
 4349:         <div class="rich-summary-panel">
 4350:           <p class="rich-editor-hint">Right-click text or an inserted block for formatting, images,
formulas, code blocks, alignment, and delete controls.</p>
 4351:           <div class="rich-summary-editor" data-rich-editor="summary" data-placeholder="Click anywhere
and start writing the project overview." contenteditable="true" spellcheck="true" aria-label="Rich overview
editor"></div>
 4352:           <div class="summary-save-actions">
 4353:             <button class="button primary" type="button" data-summary-save="true">Save overview</button>
 4354:           </div>
 4355:         </div>
 4356:       ` : `<div class="summary-rendered-preview">${renderSummaryPreview(project)}</div>`}
 4357:     </div>
 4358:   `;
 4359: }
```

applyRichTextBlockStyle (template-preview.js, lines 4361-4372)

applyRichTextBlockStyle part of the private builder frontend and editing experience. In this file, it appears around lines 4361-4372 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cleanFontFamily, normalizeFontPx, and normalizeTextColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include fontFamily at line 4362; fontPx at line 4363; color at line 4364. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4361: function applyRichTextBlockStyle(block) {
4362:     const fontFamily = cleanFontFamily(block.dataset.fontFamily || "Arial") || "Arial";
4363:     const fontPx = normalizeFontPx(block.dataset.fontPx || "");
4364:     const color = normalizeTextColor(block.dataset.color || "");
4365:
4366:     block.style.fontFamily = fontFamily || "";
4367:     block.style.fontSize = fontPx ? `${fontPx}px` : "";
4368:     block.style.color = color || "";
4369:     block.style.fontWeight = block.dataset.bold === "true" ? "700" : "";
4370:     block.style.fontStyle = block.dataset.italic === "true" ? "italic" : "";
4371:     block.style.textDecoration = block.dataset.underline === "true" ? "underline" : "";
4372: }
```

createRichTextBlock (template-preview.js, lines 4374-4391)

createRichTextBlock creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4374-4391 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, cleanFontFamily, normalizeFontPx, normalizeTextColor, sanitizeRichInlineHtml, and applyRichTextBlockStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4390: return block;.

Important local values include block at line 4375. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4374: function createRichTextBlock(text = "", fontSize = "normal", align = "left", fontFamily = "Arial", fontPx
= "", color = "", bold = false, html = "", italic = false, underline = false) {
4375:     const block = document.createElement("p");
4376:     block.className = `rich-block rich-text-block rich-text-${fontSize} text-${align}`;
4377:     block.dataset.type = "paragraph";
4378:     block.dataset.fontSize = fontSize;
4379:     block.dataset.align = align;
4380:     block.dataset.fontFamily = cleanFontFamily(fontFamily || "Arial") || "Arial";
4381:     block.dataset.fontPx = normalizeFontPx(fontPx);
4382:     block.dataset.color = normalizeTextColor(color);
4383:     block.dataset.bold = bold ? "true" : "false";
4384:     block.dataset.italic = italic ? "true" : "false";
4385:     block.dataset.underline = underline ? "true" : "false";
4386:     block.spellcheck = true;
4387:     if (html) block.innerHTML = sanitizeRichInlineHtml(html);
4388:     else block.textContent = text;
4389:     applyRichTextBlockStyle(block);
4390:     return block;
4391: }
```

richBlockActions (template-preview.js, lines 4393-4401)

richBlockActions part of the private builder frontend and editing experience. In this file, it appears around lines 4393-4401 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4394: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4393: function richBlockActions(label = "block", options = {}) {
4394:   return `
4395:     <div class="rich-block-actions">
4396:       ${options.movable ? `<button class="rich-drag-handle" type="button" draggable="true" data-rich-
drag-handle aria-label="Move ${escapeHtml(label)}">Move</button>` : ""}
4397:       <button type="button" data-rich-block-action="edit" aria-label="Edit
${escapeHtml(label)}">Edit</button>
4398:       <button class="danger-icon" type="button" data-rich-block-action="delete" aria-label="Delete
${escapeHtml(label)}">Delete</button>
4399:     </div>
4400:   `;
4401: }
```

createRichImageBlock (template-preview.js, lines 4403-4431)

createRichImageBlock creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4403-4431 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, cleanRichImageTitle, normalizeCropAspect, normalizeCropZoom, normalizeCropPosition, richBlockActions, richImageCropStyle, escapeHtml, and normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4430: return figure;.

Important local values include figure at line 4404; align at line 4405; title at line 4406. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4403: function createRichImageBlock(blockData) {
4404:   const figure = document.createElement("figure");
4405:   const align = blockData.align || "center";
4406:   const title = cleanRichImageTitle(blockData);
4407:   figure.className = `rich-block rich-image-block justify-${align}`;
4408:   figure.dataset.type = "image";
4409:   figure.dataset.url = blockData.url || "";
4410:   figure.dataset.title = blockData.title || "";
4411:   figure.dataset.caption = blockData.caption || "";
4412:   figure.dataset.align = align;
4413:   figure.dataset.display = blockData.display === "download" ? "download" : "show";
4414:   figure.dataset.source = blockData.source || "local";
4415:   figure.dataset.cropAspect = normalizeCropAspect(blockData.cropAspect);
4416:   figure.dataset.cropZoom = String(normalizeCropZoom(blockData.cropZoom));
4417:   figure.dataset.cropX = String(normalizeCropPosition(blockData.cropX));
4418:   figure.dataset.cropY = String(normalizeCropPosition(blockData.cropY));
4419:   figure.contentEditable = "false";
4420:   figure.draggable = true;
4421:   figure.title = "Drag image to move it between text.";
4422:   figure.innerHTML = `
4423:     ${richBlockActions("overview image", { movable: true })}
4424:     ${figure.dataset.display === "download" ? `<span class="rich-download-badge">Download only</span>` :
""}
4425:     <span class="rich-image-viewport crop-${figure.dataset.cropAspect === "original" ? "original" :
"active"}"${richImageCropStyle(blockData)}>
4426:       <img src="${escapeHtml(normalizeLinkTarget(blockData.url, { assumeWeb: true }))}"
alt="${escapeHtml(title || "Overview image")}" draggable="true" data-rich-image-drag="true">
4427:     </span>
4428:     ${(title || blockData.caption) ? `<figcaption>${title ? `<strong>${escapeHtml(title)}</strong>` :
""}${blockData.caption ? `<span>${escapeHtml(blockData.caption)}</span>` : ""}</figcaption>` : ""}
4429:   `;
4430:   return figure;
4431: }
```

createRichFormulaBlock (template-preview.js, lines 4433-4446)

createRichFormulaBlock creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4433-4446 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, unwrapFormula, richBlockActions, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4445: return block;.

Important local values include block at line 4434; align at line 4435. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4433: function createRichFormulaBlock(blockData) {
4434:   const block = document.createElement("div");
4435:   const align = blockData.align || "center";
4436:   block.className = `rich-block rich-formula-block justify-${align}`;
4437:   block.dataset.type = "formula";
4438:   block.dataset.formula = unwrapFormula(blockData.formula || "");
4439:   block.dataset.align = align;
4440:   block.contentEditable = "false";
4441:   block.innerHTML = `
4442:     ${richBlockActions("formula")}
4443:     <span class="formula-text">${escapeHtml(unwrapFormula(blockData.formula || ""))}</span>
4444:   `;
4445:   return block;
4446: }
```

createRichCodeBlock (template-preview.js, lines 4448-4465)

createRichCodeBlock creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4448-4465 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, normalizeCodePasteMode, normalizeCodeText, normalizeCodeLanguage, detectCodeLanguage, richBlockActions, escapeHtml, codeLanguageLabel, and tokenizedCodeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4464: return block;.

Important local values include block at line 4449; pasteMode at line 4450; code at line 4451; language at line 4452. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4448: function createRichCodeBlock(blockData = {}) {
4449:   const block = document.createElement("figure");
4450:   const pasteMode = normalizeCodePasteMode(blockData.pasteMode || "source");
4451:   const code = normalizeCodeText(blockData.code || "", pasteMode);
4452:   const language = normalizeCodeLanguage(blockData.language || detectCodeLanguage(code));
4453:   block.className = `rich-block rich-code-block rich-code-language-${language}`;
4454:   block.dataset.type = "code";
4455:   block.dataset.language = language;
4456:   block.dataset.pasteMode = pasteMode;
4457:   block.dataset.code = code;
4458:   block.contentEditable = "false";
4459:   block.innerHTML = `
4460:     ${richBlockActions("code block")}
4461:     <figcaption><span>&lt;/&gt;</span> ${escapeHtml(codeLanguageLabel(language))}<button type="button"
data-rich-block-action="copy-code">Copy code</button></figcaption>
4462:     <pre><code>${tokenizedCodeHtml(code, language)}</code></pre>
4463:   `;
4464:   return block;
4465: }
```

refreshRichImageBlock (template-preview.js, lines 4467-4497)

refreshRichImageBlock part of the private builder frontend and editing experience. In this file, it appears around lines 4467-4497 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCropAspect, normalizeCropZoom, normalizeCropPosition, remove, add, cleanRichImageTitle, richBlockActions, richImageCropStyle, escapeHtml, and normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include align at line 4468; title at line 4483. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4467: function refreshRichImageBlock(block, blockData) {
4468:   const align = blockData.align || block.dataset.align || "center";
4469:   block.dataset.url = blockData.url || block.dataset.url || "";
```

```
  block.dataset.title = blockData.title || "";
  block.dataset.caption = blockData.caption || "";
  block.dataset.align = align;
  block.dataset.display = blockData.display === "download" ? "download" : "show";
  block.dataset.source = blockData.source || block.dataset.source || "local";
  block.dataset.cropAspect = normalizeCropAspect(blockData.cropAspect || block.dataset.cropAspect);
  block.dataset.cropZoom = String(normalizeCropZoom(blockData.cropZoom || block.dataset.cropZoom));
  block.dataset.cropX = String(normalizeCropPosition(blockData.cropX ?? block.dataset.cropX));
  block.dataset.cropY = String(normalizeCropPosition(blockData.cropY ?? block.dataset.cropY));
  block.draggable = true;
  block.title = "Drag image to move it between text.";
  block.classList.remove("justify-left", "justify-center", "justify-right");
  block.classList.add(`justify-${align}`);
  const title = cleanRichImageTitle({ title: block.dataset.title });
  block.innerHTML = `
    ${richBlockActions("overview image", { movable: true })}
    ${block.dataset.display === "download" ? `<span class="rich-download-badge">Download only</span>` :
""}
    <span class="rich-image-viewport crop-${block.dataset.cropAspect === "original" ? "original" :
"active"}"${richImageCropStyle({
      cropAspect: block.dataset.cropAspect,
      cropZoom: block.dataset.cropZoom,
      cropX: block.dataset.cropX,
      cropY: block.dataset.cropY
    })}>
      <img src="${escapeHtml(normalizeLinkTarget(block.dataset.url, { assumeWeb: true }))}"
alt="${escapeHtml(title || "Overview image")}" draggable="true" data-rich-image-drag="true">
    </span>
    ${(title || block.dataset.caption) ? `<figcaption>${title ? `<strong>${escapeHtml(title)}</strong>` :
""}${block.dataset.caption ? `<span>${escapeHtml(block.dataset.caption)}</span>` : ""}</figcaption>` : ""}
  `;
}
```

refreshRichFormulaBlock (template-preview.js, lines 4499-4510)

refreshRichFormulaBlock part of the private builder frontend and editing experience. In this file, it appears around lines 4499-4510 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references unwrapFormula, remove, add, richBlockActions, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include align at line 4500; formula at line 4501. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function refreshRichFormulaBlock(block, blockData) {
  const align = blockData.align || block.dataset.align || "center";
  const formula = unwrapFormula(blockData.formula || block.dataset.formula || "");
  block.dataset.formula = formula;
  block.dataset.align = align;
  block.classList.remove("justify-left", "justify-center", "justify-right");
  block.classList.add(`justify-${align}`);
  block.innerHTML = `
    ${richBlockActions("formula")}
    <span class="formula-text">${escapeHtml(formula)}</span>
  `;
}
```

refreshRichCodeBlock (template-preview.js, lines 4512-4525)

refreshRichCodeBlock part of the private builder frontend and editing experience. In this file, it appears around lines 4512-4525 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCodePasteMode, normalizeCodeText, normalizeCodeLanguage, detectCodeLanguage, richBlockActions, escapeHtml, codeLanguageLabel, and tokenizedCodeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include pasteMode at line 4513; code at line 4514; language at line 4515. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 4512: function refreshRichCodeBlock(block, blockData = {}) {
 4513:   const pasteMode = normalizeCodePasteMode(blockData.pasteMode || block.dataset.pasteMode || "source");
 4514:   const code = normalizeCodeText(blockData.code || block.dataset.code || "", pasteMode);
 4515:   const language = normalizeCodeLanguage(blockData.language || block.dataset.language ||
detectCodeLanguage(code));
 4516:   block.dataset.language = language;
 4517:   block.dataset.pasteMode = pasteMode;
 4518:   block.dataset.code = code;
 4519:   block.className = `rich-block rich-code-block rich-code-language-${language}`;
 4520:   block.innerHTML = `
 4521:     ${richBlockActions("code block")}
 4522:     <figcaption><span>&lt;/&gt;</span> ${escapeHtml(codeLanguageLabel(language))}<button type="button"
data-rich-block-action="copy-code">Copy code</button></figcaption>
 4523:     <pre><code>${tokenizedCodeHtml(code, language)}</code></pre>
 4524:   `;
 4525: }
```

createRichBlockElement (template-preview.js, lines 4527-4544)

createRichBlockElement creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4527-4544 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createRichImageBlock, createRichFormulaBlock, createRichCodeBlock, and createRichTextBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 4528: if (block.type === "image") return createRichImageBlock(block);; line 4529: if (block.type === "formula") return createRichFormulaBlock(block);; line 4530: if (block.type === "code") return createRichCodeBlock(block);; line 4532: return createRichTextBlock(.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 4527: function createRichBlockElement(block) {
 4528:     if (block.type === "image") return createRichImageBlock(block);
 4529:     if (block.type === "formula") return createRichFormulaBlock(block);
 4530:     if (block.type === "code") return createRichCodeBlock(block);
 4531:
 4532:     return createRichTextBlock(
 4533:         block.text || "",
 4534:         block.fontSize || "normal",
 4535:         block.align || "left",
 4536:         block.fontFamily || "Arial",
 4537:         block.fontPx || "",
 4538:         block.color || "",
 4539:         Boolean(block.bold),
 4540:         block.html || "",
 4541:         Boolean(block.italic),
 4542:         Boolean(block.underline)
 4543:     );
 4544: }
```

populateSummaryEditor (template-preview.js, lines 4546-4550)

populateSummaryEditor part of the private builder frontend and editing experience. In this file, it appears around lines 4546-4550 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, and populateRichEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4548: if (!editor) return;.

Important local values include editor at line 4547. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 4546: function populateSummaryEditor(project) {
 4547:     const editor = projectFields.querySelector("[data-rich-editor='summary']");
 4548:     if (!editor) return;
 4549:     populateRichEditor(editor, project);
 4550: }
```

populateStandaloneRichEditor (template-preview.js, lines 4552-4560)

populateStandaloneRichEditor part of the private builder frontend and editing experience. In this file, it appears around lines 4552-4560 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richBlocksFromValue, forEach, append, createRichBlockElement, and createRichTextBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4553: if (!editor) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4552: function populateStandaloneRichEditor(editor, rich, fallbackText = "") {
4553:     if (!editor) return;
4554:     editor.contentEditable = "true";
4555:     editor.tabIndex = 0;
4556:     editor.spellcheck = true;
4557:     editor.innerHTML = "";
4558:     richBlocksFromValue(rich, fallbackText).forEach((block) =>
editor.append(createRichBlockElement(block)));
4559:     if (!editor.children.length) editor.append(createRichTextBlock(""));
4560: }
```

populateRichEditor (template-preview.js, lines 4562-4592)

This function reconstructs rich text editor blocks from saved data. It turns stored text, images, formulas, and code blocks back into editable DOM elements.

A saved project must reopen in an editable form without losing formatting. This function is the load side of the rich editor.

It calls or references selectedProject, richBlocksFromValue, forEach, append, createRichBlockElement, createRichTextBlock, richBlocksForProject, and getByPath. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4563: if (!editor) return;; line 4573: return;; line 4576: if (!project) return;.

Important local values include blocks at line 4580; textPath at line 4585; richPath at line 4586. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4562: function populateRichEditor(editor, project = selectedProject()) {
4563:     if (!editor) return;
4564:     editor.contentEditable = "true";
4565:     editor.tabIndex = 0;
4566:     editor.spellcheck = true;
4567:
4568:     if (editor.dataset.richEditor === "pending-description") {
4569:         editor.innerHTML = "";
4570:         richBlocksFromValue(pendingEditor?.richDescription, pendingEditor?.description || "")
4571:           .forEach((block) => editor.append(createRichBlockElement(block)));
4572:         if (!editor.children.length) editor.append(createRichTextBlock(""));
4573:         return;
4574:     }
4575:
4576:     if (!project) return;
4577:
4578:     editor.innerHTML = "";
4579:
4580:     let blocks = [];
4581:
4582:     if (editor.dataset.richEditor === "summary" && !editor.dataset.richPath) {
4583:         blocks = richBlocksForProject(project);
4584:     } else {
4585:         const textPath = editor.dataset.richTextPath || "";
4586:         const richPath = editor.dataset.richPath || "";
4587:         blocks = richBlocksFromValue(getByPath(project, richPath), getByPath(project, textPath) || "");
4588:     }
4589:
4590:     blocks.forEach((block) => editor.append(createRichBlockElement(block)));
4591:     if (!editor.children.length) editor.append(createRichTextBlock(""));
4592: }
```

populateRichEditors (template-preview.js, lines 4594-4596)

populateRichEditors part of the private builder frontend and editing experience. In this file, it appears around lines 4594-4596 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, and populateRichEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4594: function populateRichEditors(root = document) {
4595:     root.querySelectorAll("[data-rich-editor]").forEach((editor) => populateRichEditor(editor));
4596: }
```

saveRichEditorToProject (template-preview.js, lines 4598-4658)

This function serializes rich editor DOM blocks back into structured project data. It extracts text formatting, image data, captions, formulas, code blocks, and ordering.

The parser and previews cannot rely on raw contenteditable HTML. This function turns the user's editing surface into predictable data.

It calls or references syncFunFactsFromInput, setStatus, syncSiteRichField, syncProfileRichField, selectedProject, extractRichSummary, plainTextFromRich, wordCount, setByPath, scheduleAutosave, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 9 return statement(s). The most important visible returns are: line 4599: if (!editor) return true;; line 4604: return true;; line 4610: return true;; line 4616: return true;. There are 5 additional return statements in the function body.

Important local values include project at line 4624; rich at line 4627; rich at line 4636; plain at line 4637; textPath at line 4648; richPath at line 4649. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4598: function saveRichEditorToProject(editor) {
4599:     if (!editor) return true;
4600:
4601:     if (editor.dataset.richEditor === "fun-facts") {
4602:         syncFunFactsFromInput();
4603:         setStatus("Unsaved local changes.");
4604:         return true;
4605:     }
4606:
4607:     if (editor.dataset.richEditor === "site-field") {
4608:         syncSiteRichField(editor);
4609:         setStatus("Front page text formatting updated locally. Click Save draft before applying to
site.");
4610:         return true;
4611:     }
4612:
4613:     if (editor.dataset.richEditor === "profile-field") {
4614:         syncProfileRichField(editor);
4615:         setStatus("Profile and contact formatting updated locally. Click Save draft before applying to
site.");
4616:         return true;
4617:     }
4618:
4619:     if (editor.dataset.richEditor === "standalone") {
4620:         setStatus("Unsaved dialog changes. Use the dialog save button to keep them.");
4621:         return true;
4622:     }
4623:
4624:     const project = selectedProject();
4625:
4626:     if (editor.dataset.richEditor === "pending-description" && pendingEditor) {
4627:         const rich = extractRichSummary(editor);
4628:         pendingEditor.richDescription = rich;
4629:         pendingEditor.description = plainTextFromRich(rich);
4630:         setStatus("Unsaved local changes.");
4631:         return true;
4632:     }
4633:
4634:     if (!project) return true;
4635:
4636:     const rich = extractRichSummary(editor);
4637:     const plain = plainTextFromRich(rich);
4638:
4639:     if (wordCount(plain) > 1000) {
4640:         setStatus("Overview is limited to 1000 words. Shorten the text before saving.");
4641:         return false;
```

```
4642:     }
4643:
4644:     if (editor.dataset.richEditor === "summary" && !editor.dataset.richPath) {
4645:         project.summaryRich = rich;
4646:         project.summary = plain;
4647:     } else {
4648:         const textPath = editor.dataset.richTextPath;
4649:         const richPath = editor.dataset.richPath;
4650:         if (textPath) setByPath(project, textPath, plain);
4651:         if (richPath) setByPath(project, richPath, rich);
4652:     }
4653:
4654:     setStatus("Unsaved local changes.");
4655:     scheduleAutosave();
4656:     schedulePreviewRender();
4657:     return true;
4658: }
```

saveVisibleRichEditors (template-preview.js, lines 4660-4664)

saveVisibleRichEditors persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 4660-4664 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, and saveRichEditorToProject. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4660: function saveVisibleRichEditors() {
4661:     document.querySelectorAll("[data-rich-editor]").forEach((editor) => {
4662:         saveRichEditorToProject(editor);
4663:     });
4664: }
```

currentRichBlock (template-preview.js, lines 4666-4672)

currentRichBlock part of the private builder frontend and editing experience. In this file, it appears around lines 4666-4672 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getSelection, contains, querySelector, and closest. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4669: if (!node || !editor.contains(node)) return editor.querySelector(".rich-block:last-child");; line 4671: return node.closest(".rich-block") || editor.querySelector(".rich-block:last-child");.

Important local values include selection at line 4667; node at line 4668. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4666: function currentRichBlock(editor) {
4667:   const selection = window.getSelection();
4668:   let node = selection?.anchorNode;
4669:   if (!node || !editor.contains(node)) return editor.querySelector(".rich-block:last-child");
4670:   if (node.nodeType === Node.TEXT_NODE) node = node.parentElement;
4671:   return node.closest(".rich-block") || editor.querySelector(".rich-block:last-child");
4672: }
```

selectionRangeInsideEditor (template-preview.js, lines 4674-4682)

selectionRangeInsideEditor part of the private builder frontend and editing experience. In this file, it appears around lines 4674-4682 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getSelection, getRangeAt, contains, and cloneRange. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4676: if (!selection?.rangeCount) return null;; line 4681: return container && editor.contains(container) ? range.cloneRange() : null;.

Important local values include selection at line 4675; range at line 4677; container at line 4678. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4674: function selectionRangeInsideEditor(editor) {
4675:   const selection = window.getSelection();
4676:   if (!selection?.rangeCount) return null;
4677:   const range = selection.getRangeAt(0);
4678:   const container = range.commonAncestorContainer.nodeType === Node.TEXT_NODE
4679:     ? range.commonAncestorContainer.parentElement
4680:     : range.commonAncestorContainer;
4681:   return container && editor.contains(container) ? range.cloneRange() : null;
4682: }
```

captureRichSelection (template-preview.js, lines 4684-4688)

captureRichSelection part of the private builder frontend and editing experience. In this file, it appears around lines 4684-4688 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectionRangeInsideEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4687: return range;.

Important local values include range at line 4685. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4684: function captureRichSelection(editor) {
4685:   const range = selectionRangeInsideEditor(editor);
4686:   if (range) activeRichSelectionRange = range;
4687:   return range;
4688: }
```

rememberRichFormattingSelection (template-preview.js, lines 4690-4700)

rememberRichFormattingSelection part of the private builder frontend and editing experience. In this file, it appears around lines 4690-4700 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectionRangeInsideEditor, rangeBelongsToEditor, and cloneRange. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4691: if (!editor) return null;; line 4695: if (!range) return null;; line 4699: return range;.

Important local values include range at line 4692. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4690: function rememberRichFormattingSelection(editor = activeSummaryEditor) {
4691:   if (!editor) return null;
4692:   const range = selectionRangeInsideEditor(editor)
4693:     || (rangeBelongsToEditor(activeTextSelectionRange, editor) ? activeTextSelectionRange.cloneRange() :
null)
4694:     || (rangeBelongsToEditor(activeRichSelectionRange, editor) ? activeRichSelectionRange.cloneRange() :
null);
4695:   if (!range) return null;
4696:   activeSummaryEditor = editor;
4697:   activeRichSelectionRange = range.cloneRange();
4698:   if (!range.collapsed) activeTextSelectionRange = range.cloneRange();
4699:   return range;
4700: }
```

restoreRichSelection (template-preview.js, lines 4702-4713)

restoreRichSelection part of the private builder frontend and editing experience. In this file, it appears around lines 4702-4713 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, getSelection, removeAllRanges, addRange, cloneRange, and showPersistentTextSelection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4703: if (!activeRichSelectionRange) return false;; line 4707: if (!container || !editor.contains(container)) return false;; line 4712: return true;.

Important local values include container at line 4704; selection at line 4708. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4702: function restoreRichSelection(editor) {
4703:   if (!activeRichSelectionRange) return false;
4704:   const container = activeRichSelectionRange.commonAncestorContainer.nodeType === Node.TEXT_NODE
4705:     ? activeRichSelectionRange.commonAncestorContainer.parentElement
4706:     : activeRichSelectionRange.commonAncestorContainer;
4707:   if (!container || !editor.contains(container)) return false;
4708:   const selection = window.getSelection();
4709:   selection.removeAllRanges();
4710:   selection.addRange(activeRichSelectionRange.cloneRange());
4711:   showPersistentTextSelection(activeRichSelectionRange);
4712:   return true;
4713: }
```

showPersistentTextSelection (template-preview.js, lines 4715-4745)

showPersistentTextSelection controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 4715-4745 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references set, Highlight, cloneRange, closest, createElement, setAttribute, append, replaceChildren, getClientRects, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4716: if (!range || range.collapsed) return;; line 4720: return;; line 4736: if (rect.width < 1 || rect.height < 1) return;.

Important local values include container at line 4723; editor at line 4726; host at line 4727; marker at line 4737. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4715: function showPersistentTextSelection(range) {
4716:   if (!range || range.collapsed) return;
4717:   if (window.CSS?.highlights && typeof window.Highlight === "function") {
4718:     window.CSS.highlights.set(persistentSelectionHighlightName, new
window.Highlight(range.cloneRange()));
4719:     if (persistentSelectionOverlay) persistentSelectionOverlay.hidden = true;
4720:     return;
4721:   }
4722:
4723:   const container = range.commonAncestorContainer.nodeType === Node.TEXT_NODE
4724:     ? range.commonAncestorContainer.parentElement
4725:     : range.commonAncestorContainer;
4726:   const editor = container?.closest?.("[data-rich-editor]") || activeSummaryEditor;
4727:   const host = editor?.closest("dialog") || document.body;
4728:   if (!persistentSelectionOverlay) {
4729:     persistentSelectionOverlay = document.createElement("div");
4730:     persistentSelectionOverlay.className = "persistent-selection-overlay";
4731:     persistentSelectionOverlay.setAttribute("aria-hidden", "true");
4732:   }
4733:   if (persistentSelectionOverlay.parentElement !== host) host.append(persistentSelectionOverlay);
4734:   persistentSelectionOverlay.replaceChildren();
4735:   [...range.getClientRects()].forEach((rect) => {
4736:     if (rect.width < 1 || rect.height < 1) return;
4737:     const marker = document.createElement("span");
4738:     marker.style.left = `${rect.left}px`;
4739:     marker.style.top = `${rect.top}px`;
4740:     marker.style.width = `${rect.width}px`;
4741:     marker.style.height = `${rect.height}px`;
4742:     persistentSelectionOverlay.append(marker);
4743:   });
4744:   persistentSelectionOverlay.hidden = !persistentSelectionOverlay.childElementCount;
4745: }
```

clearPersistentTextSelection (template-preview.js, lines 4747-4756)

clearPersistentTextSelection part of the private builder frontend and editing experience. In this file, it appears around lines 4747-4756 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replaceChildren. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4753: if (!clearRanges) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4747: function clearPersistentTextSelection(clearRanges = false) {
4748:   window.CSS?.highlights?.delete?.(persistentSelectionHighlightName);
4749:   if (persistentSelectionOverlay) {
4750:     persistentSelectionOverlay.replaceChildren();
4751:     persistentSelectionOverlay.hidden = true;
4752:   }
4753:   if (!clearRanges) return;
4754:   activeRichSelectionRange = null;
4755:   activeTextSelectionRange = null;
4756: }
```

refreshPersistentSelectionHighlight (template-preview.js, lines 4758-4765)

refreshPersistentSelectionHighlight part of the private builder frontend and editing experience. In this file, it appears around lines 4758-4765 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references rangeBelongsToEditor, and showPersistentTextSelection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include range at line 4759. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4758: function refreshPersistentSelectionHighlight() {
4759:   const range = rangeBelongsToEditor(activeTextSelectionRange, activeSummaryEditor)
4760:     ? activeTextSelectionRange
4761:     : rangeBelongsToEditor(activeRichSelectionRange, activeSummaryEditor)
4762:       ? activeRichSelectionRange
4763:       : null;
4764:   if (range) showPersistentTextSelection(range);
4765: }
```

rangeBelongsToEditor (template-preview.js, lines 4767-4773)

rangeBelongsToEditor part of the private builder frontend and editing experience. In this file, it appears around lines 4767-4773 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4768: if (!range || range.collapsed || !editor) return false;; line 4772: return Boolean(container && editor.contains(container));.

Important local values include container at line 4769. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4767: function rangeBelongsToEditor(range, editor) {
4768:   if (!range || range.collapsed || !editor) return false;
4769:   const container = range.commonAncestorContainer.nodeType === Node.TEXT_NODE
4770:     ? range.commonAncestorContainer.parentElement
```

```
4771:     : range.commonAncestorContainer;
4772:   return Boolean(container && editor.contains(container));
4773: }
```

editorFromRange (template-preview.js, lines 4775-4781)

editorFromRange part of the private builder frontend and editing experience. In this file, it appears around lines 4775-4781 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4776: if (!range || range.collapsed) return null;; line 4780: return container?.closest?.("[data-rich-editor]") || null;.

Important local values include container at line 4777. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4775: function editorFromRange(range) {
4776:   if (!range || range.collapsed) return null;
4777:   const container = range.commonAncestorContainer.nodeType === Node.TEXT_NODE
4778:     ? range.commonAncestorContainer.parentElement
4779:     : range.commonAncestorContainer;
4780:   return container?.closest?.("[data-rich-editor]") || null;
4781: }
```

pointInsideRange (template-preview.js, lines 4783-4793)

pointInsideRange part of the private builder frontend and editing experience. In this file, it appears around lines 4783-4793 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getClientRects, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4784: if (!range || range.collapsed) return false;; line 4785: return [...range.getClientRects()].some((rect) =>.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4783: function pointInsideRange(range, x, y) {
4784:   if (!range || range.collapsed) return false;
4785:   return [...range.getClientRects()].some((rect) =>
4786:     rect.width > 0 &&
4787:     rect.height > 0 &&
4788:     x >= rect.left &&
4789:     x <= rect.right &&
4790:     y >= rect.top &&
4791:     y <= rect.bottom
4792:   );
4793: }
```

richEditorFromContextEvent (template-preview.js, lines 4795-4805)

richEditorFromContextEvent part of the private builder frontend and editing experience. In this file, it appears around lines 4795-4805 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references rangeBelongsToEditor, pointInsideRange, and editorFromRange. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4797: if (directEditor) return directEditor;; line 4802: return editorFromRange(savedRange);; line 4804: return null;.

Important local values include directEditor at line 4796; savedRange at line 4798. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4795: function richEditorFromContextEvent(event) {
4796:   const directEditor = event.target.closest?.("[data-rich-editor]");
4797:   if (directEditor) return directEditor;
4798:   const savedRange = rangeBelongsToEditor(activeTextSelectionRange, activeSummaryEditor)
4799:     ? activeTextSelectionRange
4800:     : activeRichSelectionRange;
4801:   if (pointInsideRange(savedRange, event.clientX, event.clientY)) {
4802:     return editorFromRange(savedRange);
4803:   }
4804:   return null;
4805: }
```

displayRichFontName (template-preview.js, lines 4807-4812)

displayRichFontName part of the private builder frontend and editing experience. In this file, it appears around lines 4807-4812 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, replace, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4808: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4807: function displayRichFontName(value = "") {
4808:   return String(value || "")
4809:     .split(",")[0]
4810:     .replace(/^['\"]|['\"]$/g, "")
4811:     .trim() || "Arial";
4812: }
```

rgbColorToHex (template-preview.js, lines 4814-4823)

rgbColorToHex part of the private builder frontend and editing experience. In this file, it appears around lines 4814-4823 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, slice, split, map, join, match, min, toString, and padStart. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 4816: if (/^#[0-9a-f]{6}$/i.test(color)) return color.toLowerCase();; line 4818: return `#${color.slice(1).split("").map((part) => part + part).join("")}`.toLowerCase();; line 4821: if (!match) return color;; line 4822: return `#${match.slice(1, 4).map((part) => Math.min(255, Number(part)).toString(16).padStart(2, "0")).join("")}`;.

Important local values include color at line 4815; match at line 4820. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4814: function rgbColorToHex(value = "") {
4815:   const color = String(value || "").trim();
4816:   if (/^#[0-9a-f]{6}$/i.test(color)) return color.toLowerCase();
4817:   if (/^#[0-9a-f]{3}$/i.test(color)) {
4818:     return `#${color.slice(1).split("").map((part) => part + part).join("")}`.toLowerCase();
4819:   }
4820:   const match = color.match(/^rgba?\(\s*(\d+)\D+(\d+)\D+(\d+)/i);
4821:   if (!match) return color;
4822:   return `#${match.slice(1, 4).map((part) => Math.min(255, Number(part)).toString(16).padStart(2,
"0")).join("")}`;
4823: }
```

textNodesInRichSelection (template-preview.js, lines 4825-4844)

textNodesInRichSelection part of the private builder frontend and editing experience. In this file, it appears around lines 4825-4844 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createTreeWalker, acceptNode, trim, closest, intersectsNode, nextNode, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 4828: if (!node.textContent || (!includeWhitespace && !node.textContent.trim())) return NodeFilter.FILTER_REJECT;; line 4829: if (node.parentElement?.closest("[contenteditable='false']")) return NodeFilter.FILTER_REJECT;; line 4831: return range.intersectsNode(node) ? NodeFilter.FILTER_ACCEPT : NodeFilter.FILTER_REJECT;; line 4833: return NodeFilter.FILTER_REJECT;. There are 1 additional return statements in the function body.

Important local values include walker at line 4826; nodes at line 4837; node at line 4838. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4825: function textNodesInRichSelection(range, editor, includeWhitespace = false) {
4826:   const walker = document.createTreeWalker(editor, NodeFilter.SHOW_TEXT, {
4827:     acceptNode(node) {
4828:       if (!node.textContent || (!includeWhitespace && !node.textContent.trim())) return
NodeFilter.FILTER_REJECT;
4829:       if (node.parentElement?.closest("[contenteditable='false']")) return NodeFilter.FILTER_REJECT;
4830:       try {
4831:         return range.intersectsNode(node) ? NodeFilter.FILTER_ACCEPT : NodeFilter.FILTER_REJECT;
4832:       } catch {
4833:         return NodeFilter.FILTER_REJECT;
4834:       }
4835:     }
4836:   });
4837:   const nodes = [];
4838:   let node = walker.nextNode();
4839:   while (node) {
4840:     nodes.push(node);
4841:     node = walker.nextNode();
4842:   }
4843:   return nodes;
4844: }
```

richBlockFromRange (template-preview.js, lines 4846-4853)

richBlockFromRange part of the private builder frontend and editing experience. In this file, it appears around lines 4846-4853 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4847: if (!range || !editor) return null;; line 4852: return block && editor.contains(block) ? block : null;.

Important local values include container at line 4848; block at line 4851. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4846: function richBlockFromRange(range, editor) {
4847:   if (!range || !editor) return null;
4848:   const container = range.startContainer?.nodeType === Node.TEXT_NODE
4849:     ? range.startContainer.parentElement
4850:     : range.startContainer;
4851:   const block = container?.closest?.(".rich-block");
4852:   return block && editor.contains(block) ? block : null;
4853: }
```

uniformSelectionValue (template-preview.js, lines 4855-4861)

uniformSelectionValue part of the private builder frontend and editing experience. In this file, it appears around lines 4855-4861 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, trim, filter, every, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4857: if (!cleanValues.length) return "";; line 4858: return cleanValues.every((value) => value.toLowerCase() === cleanValues[0].toLowerCase()).

Important local values include cleanValues at line 4856. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4855: function uniformSelectionValue(values) {
4856:   const cleanValues = values.map((value) => String(value || "").trim()).filter(Boolean);
4857:   if (!cleanValues.length) return "";
4858:   return cleanValues.every((value) => value.toLowerCase() === cleanValues[0].toLowerCase())
4859:     ? cleanValues[0]
4860:     : "";
4861: }
```

richSelectionStyleState (template-preview.js, lines 4863-4871)

richSelectionStyleState part of the private builder frontend and editing experience. In this file, it appears around lines 4863-4871 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textNodesInRichSelection, map, getComputedStyle, uniformSelectionValue, rgbColorToHex, displayRichFontName, and round. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4866: return {.

Important local values include textNodes at line 4864; styles at line 4865. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4863: function richSelectionStyleState(range, editor) {
4864:   const textNodes = textNodesInRichSelection(range, editor);
4865:   const styles = textNodes.map((node) => getComputedStyle(node.parentElement || editor));
4866:   return {
4867:     color: uniformSelectionValue(styles.map((style) => rgbColorToHex(style.color))),
4868:     font: uniformSelectionValue(styles.map((style) => displayRichFontName(style.fontFamily))),
4869:     size: uniformSelectionValue(styles.map((style) => String(Math.round(parseFloat(style.fontSize) ||
16))))
4870:   };
4871: }
```

closeSelectionInspectorOptions (template-preview.js, lines 4873-4877)

closeSelectionInspectorOptions controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 4873-4877 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4873: function closeSelectionInspectorOptions(except = "") {
4874:   textSelectionInspector?.querySelectorAll("[data-selection-options]").forEach((options) => {
4875:     if (options.dataset.selectionOptions !== except) options.hidden = true;
4876:   });
4877: }
```

positionSelectionInspector (template-preview.js, lines 4879-4900)

positionSelectionInspector part of the private builder frontend and editing experience. In this file, it appears around lines 4879-4900 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getBoundingClientRect, closest, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4880: if (!textSelectionInspector || selectionInspectorWasMoved) return;.

Important local values include rect at line 4881; hostDialog at line 4882; hostRect at line 4883; bounds at line 4884; inspectorRect at line 4890; inspectorWidth at line 4891; inspectorHeight at line 4892; left at line 4893; plus 2 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4879: function positionSelectionInspector(range) {
4880:   if (!textSelectionInspector || selectionInspectorWasMoved) return;
4881:   const rect = range.getBoundingClientRect();
4882:   const hostDialog = textSelectionInspector.closest("dialog");
4883:   const hostRect = hostDialog?.getBoundingClientRect();
4884:   const bounds = {
4885:     bottom: Math.min(window.innerHeight - 8, (hostRect?.bottom ?? window.innerHeight) - 8),
4886:     left: Math.max(8, (hostRect?.left ?? 0) + 8),
4887:     right: Math.min(window.innerWidth - 8, (hostRect?.right ?? window.innerWidth) - 8),
4888:     top: Math.max(8, (hostRect?.top ?? 0) + 8)
4889:   };
4890:   const inspectorRect = textSelectionInspector.getBoundingClientRect();
4891:   const inspectorWidth = Math.min(inspectorRect.width || 310, bounds.right - bounds.left);
4892:   const inspectorHeight = Math.min(inspectorRect.height || 260, bounds.bottom - bounds.top);
4893:   const left = Math.max(bounds.left, Math.min(rect.left, bounds.right - inspectorWidth));
4894:   const preferredTop = rect.bottom + 10;
4895:   const top = preferredTop + inspectorHeight <= bounds.bottom
4896:     ? preferredTop
4897:     : Math.max(bounds.top, rect.top - inspectorHeight - 10);
4898:   textSelectionInspector.style.left = `${left}px`;
4899:   textSelectionInspector.style.top = `${top}px`;
4900: }
```

showTextSelectionInspector (template-preview.js, lines 4902-4924)

showTextSelectionInspector controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 4902-4924 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toString, trim, closest, append, cloneRange, showPersistentTextSelection, richSelectionStyleState, slice, and positionSelectionInspector. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4903: if (!textSelectionInspector || !editor || !range || range.collapsed) return;; line 4905: if (!selectedText) return;.

Important local values include selectedText at line 4904; inspectorHost at line 4906; state at line 4912. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4902: function showTextSelectionInspector(editor, range) {
4903:   if (!textSelectionInspector || !editor || !range || range.collapsed) return;
4904:   const selectedText = range.toString().trim();
4905:   if (!selectedText) return;
4906:   const inspectorHost = editor.closest("dialog") || document.body;
4907:   if (textSelectionInspector.parentElement !== inspectorHost)
inspectorHost.append(textSelectionInspector);
4908:   activeSummaryEditor = editor;
4909:   activeRichSelectionRange = range.cloneRange();
4910:   activeTextSelectionRange = range.cloneRange();
4911:   showPersistentTextSelection(range);
4912:   const state = richSelectionStyleState(range, editor);
4913:   selectionTextPreview.textContent = selectedText.length > 120 ? `${selectedText.slice(0, 117)}...` :
selectedText;
4914:   selectionTextPreview.title = selectedText;
4915:   selectionFontInput.value = state.font;
4916:   selectionColorInput.value = state.color;
4917:   selectionSizeInput.value = state.size;
4918:   selectionFontInput.placeholder = state.font ? "" : "Mixed";
4919:   selectionColorInput.placeholder = state.color ? "" : "Mixed";
4920:   selectionSizeInput.placeholder = state.size ? "" : "Mixed";
4921:   if (state.color && /^#[0-9a-f]{6}$/i.test(state.color)) selectionColorPicker.value = state.color;
4922:   textSelectionInspector.hidden = false;
4923:   positionSelectionInspector(range);
4924: }
```

hideTextSelectionInspector (template-preview.js, lines 4926-4931)

hideTextSelectionInspector controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 4926-4931 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closeSelectionInspectorOptions. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4927: if (!textSelectionInspector) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4926: function hideTextSelectionInspector() {
4927:   if (!textSelectionInspector) return;
4928:   textSelectionInspector.hidden = true;
4929:   closeSelectionInspectorOptions();
4930:   selectionInspectorWasMoved = false;
4931: }
```

refreshTextSelectionInspector (template-preview.js, lines 4933-4953)

refreshTextSelectionInspector part of the private builder frontend and editing experience. In this file, it appears around lines 4933-4953 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references matches, getSelection, showTextSelectionInspector, hideTextSelectionInspector, and getRangeAt. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 4934: if (!textSelectionInspector || selectionInspectorPointerInside || textSelectionInspector.matches(":focus-within")) return;; line 4939: return;; line 4942: return;; line 4950: return;.

Important local values include selection at line 4935; range at line 4944; node at line 4945; editor at line 4947. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4933: function refreshTextSelectionInspector() {
4934:   if (!textSelectionInspector || selectionInspectorPointerInside ||
textSelectionInspector.matches(":focus-within")) return;
4935:   const selection = window.getSelection();
4936:   if (!selection?.rangeCount || selection.isCollapsed) {
4937:     if (activeTextSelectionRange && !activeTextSelectionRange.collapsed && activeSummaryEditor) {
4938:       showTextSelectionInspector(activeSummaryEditor, activeTextSelectionRange);
4939:       return;
4940:     }
4941:     hideTextSelectionInspector();
4942:     return;
4943:   }
4944:   const range = selection.getRangeAt(0);
4945:   let node = range.commonAncestorContainer;
4946:   if (node.nodeType === Node.TEXT_NODE) node = node.parentElement;
4947:   const editor = node?.closest?.("[data-rich-editor]");
4948:   if (!editor) {
4949:     hideTextSelectionInspector();
4950:     return;
4951:   }
4952:   showTextSelectionInspector(editor, range);
4953: }
```

scheduleTextSelectionInspectorRefresh (template-preview.js, lines 4955-4958)

scheduleTextSelectionInspectorRefresh part of the private builder frontend and editing experience. In this file, it appears around lines 4955-4958 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4955: function scheduleTextSelectionInspectorRefresh(delay = 0) {
4956:   clearTimeout(selectionInspectorRefreshTimer);
4957:   selectionInspectorRefreshTimer = setTimeout(refreshTextSelectionInspector, delay);
4958: }
```

applySelectionInspectorFormat (template-preview.js, lines 4960-4990)

This function applies formatting chosen in the floating selection inspector to the currently highlighted text.

It is what makes font, size, and color controls actually affect only the selected text instead of changing a whole block.

It calls or references cloneRange, cleanFontFamily, applyRichInlineCommand, normalizeTextColor, rgbColorToHex, normalizeFontPx, closeSelectionInspectorOptions, requestAnimationFrame, and showTextSelectionInspector. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 4961: if (!activeSummaryEditor || !activeTextSelectionRange) return;; line 4965: if (!font) return;; line 4971: if (!color) return;; line 4979: if (!size) return;.

Important local values include font at line 4964; color at line 4970; normalized at line 4972; size at line 4978. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4960: function applySelectionInspectorFormat(kind, rawValue) {
4961:   if (!activeSummaryEditor || !activeTextSelectionRange) return;
4962:   activeRichSelectionRange = activeTextSelectionRange.cloneRange();
4963:   if (kind === "font") {
4964:     const font = cleanFontFamily(rawValue);
4965:     if (!font) return;
4966:     selectionFontInput.value = font;
4967:     applyRichInlineCommand(activeSummaryEditor, "fontName", font);
4968:   }
4969:   if (kind === "color") {
4970:     const color = normalizeTextColor(rawValue);
4971:     if (!color) return;
4972:     const normalized = rgbColorToHex(color);
4973:     selectionColorInput.value = normalized;
4974:     if (/^#[0-9a-f]{6}$/i.test(normalized)) selectionColorPicker.value = normalized;
4975:     applyRichInlineCommand(activeSummaryEditor, "foreColor", normalized);
4976:   }
4977:   if (kind === "size") {
4978:     const size = normalizeFontPx(rawValue);
4979:     if (!size) return;
4980:     selectionSizeInput.value = size;
4981:     applyRichInlineCommand(activeSummaryEditor, "fontSize", size);
4982:   }
4983:   closeSelectionInspectorOptions();
4984:   requestAnimationFrame(() => {
4985:     if (activeRichSelectionRange && activeSummaryEditor) {
4986:       activeTextSelectionRange = activeRichSelectionRange.cloneRange();
4987:       showTextSelectionInspector(activeSummaryEditor, activeRichSelectionRange);
4988:     }
4989:   });
4990: }
```

renderSelectionInspectorOptions (template-preview.js, lines 4992-5007)

renderSelectionInspectorOptions renders ui, markup, or display output. In this file, it appears around lines 4992-5007 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, map, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include fontOptions at line 4993; sizeOptions at line 4994. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4992: function renderSelectionInspectorOptions() {
4993:   const fontOptions = textSelectionInspector?.querySelector("[data-selection-options='font']");
4994:   const sizeOptions = textSelectionInspector?.querySelector("[data-selection-options='size']");
4995:   if (fontOptions) {
4996:     fontOptions.innerHTML = commonRichFonts.map((font) => `<button type="button" data-selection-
value="font" data-value="${escapeHtml(font)}" style="font-family:
${escapeHtml(font)}">${escapeHtml(font)}</button>`).join("");
4997:   }
4998:   if (sizeOptions) {
4999:     sizeOptions.innerHTML = commonRichSizes.map((size) => `<button type="button" data-selection-
```

```
value="size" data-value="${size}">${size} px</button>`).join("");
 5000:   }
 5001:   if (selectionColorSwatches) {
 5002:     selectionColorSwatches.innerHTML = commonRichColors.map((color) => `<button type="button" data-
selection-value="color" data-value="${color}" style="--selection-swatch: ${color}" aria-label="Use
${color}"></button>`).join("");
 5003:   }
 5004:   if (richContextColorSwatches) {
 5005:     richContextColorSwatches.innerHTML = commonRichColors.map((color) => `<button type="button" data-
rich-color-value="${color}" style="--selection-swatch: ${color}" aria-label="Use ${color}"></button>`).join("");
 5006:   }
 5007: }
```

applyStyleToRichSelection (template-preview.js, lines 5009-5055)

applyStyleToRichSelection part of the private builder frontend and editing experience. In this file, it appears around lines 5009-5055 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textNodesInRichSelection, every, getComputedStyle, includes, forEach, max, min, splitText, createElement, cleanFontFamily, and 9 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5011: if (!nodes.length) return null;; line 5029: if (start === end) return;; line 5047: if (!formattedNodes.length) return null;; line 5054: return nextRange;.

Important local values include nodes at line 5010; makeBold at line 5012; makeItalic at line 5015; makeUnderline at line 5018; formattedNodes at line 5021; length at line 5024; start at line 5025; end at line 5026; plus 5 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 5009: function applyStyleToRichSelection(editor, range, command, value = "") {
 5010:   const nodes = textNodesInRichSelection(range, editor, true);
 5011:   if (!nodes.length) return null;
 5012:   const makeBold = command === "bold"
 5013:     ? !nodes.every((node) => Number.parseInt(getComputedStyle(node.parentElement || editor).fontWeight,
10) >= 600)
 5014:     : false;
 5015:   const makeItalic = command === "italic"
 5016:     ? !nodes.every((node) => getComputedStyle(node.parentElement || editor).fontStyle === "italic")
 5017:     : false;
 5018:   const makeUnderline = command === "underline"
 5019:     ? !nodes.every((node) => getComputedStyle(node.parentElement ||
editor).textDecorationLine.includes("underline"))
 5020:     : false;
 5021:   const formattedNodes = [];
 5022:
 5023:   nodes.forEach((node) => {
 5024:     const length = node.textContent?.length || 0;
 5025:     let start = node === range.startContainer ? range.startOffset : 0;
 5026:     let end = node === range.endContainer ? range.endOffset : length;
 5027:     start = Math.max(0, Math.min(start, length));
 5028:     end = Math.max(start, Math.min(end, length));
 5029:     if (start === end) return;
 5030:
 5031:     if (end < length) node.splitText(end);
 5032:     const selectedNode = start > 0 ? node.splitText(start) : node;
 5033:     const wrapper = document.createElement("span");
 5034:     wrapper.className = "rich-inline-style";
 5035:     wrapper.style.display = "inline";
 5036:     if (command === "fontName") wrapper.style.fontFamily = cleanFontFamily(value);
 5037:     if (command === "foreColor") wrapper.style.color = normalizeTextColor(value);
 5038:     if (command === "fontSize") wrapper.style.fontSize = `${normalizeFontPx(value)}px`;
 5039:     if (command === "bold") wrapper.style.fontWeight = makeBold ? "700" : "400";
 5040:     if (command === "italic") wrapper.style.fontStyle = makeItalic ? "italic" : "normal";
 5041:     if (command === "underline") wrapper.style.textDecoration = makeUnderline ? "underline" : "none";
 5042:     selectedNode.replaceWith(wrapper);
 5043:     wrapper.append(selectedNode);
 5044:     formattedNodes.push(selectedNode);
 5045:   });
 5046:
 5047:   if (!formattedNodes.length) return null;
 5048:   normalizeInlineStyleSpans(editor);
 5049:   const nextRange = document.createRange();
 5050:   const firstNode = formattedNodes[0];
 5051:   const lastNode = formattedNodes[formattedNodes.length - 1];
 5052:   nextRange.setStart(firstNode, 0);
 5053:   nextRange.setEnd(lastNode, lastNode.textContent?.length || 0);
 5054:   return nextRange;
 5055: }
```

applyRichInlineCommand (template-preview.js, lines 5057-5103)

applyRichInlineCommand part of the private builder frontend and editing experience. In this file, it appears around lines 5057-5103 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectionRangeInsideEditor, cloneRange, rangeBelongsToEditor, applyStyleToRichSelection, showPersistentTextSelection, richBlockFromRange, selectedRichBlock, currentRichBlock, setStatus, selectRichBlock, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 5058: if (!editor) return;; line 5083: return;; line 5098: return;.

Important local values include liveRange at line 5059; savedRange at line 5060; appliedToSelection at line 5068; nextRange at line 5070; block at line 5080. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 5057: function applyRichInlineCommand(editor, command, value = "") {
 5058:   if (!editor) return;
 5059:   const liveRange = selectionRangeInsideEditor(editor);
 5060:   const savedRange = liveRange && !liveRange.collapsed
 5061:     ? liveRange.cloneRange()
 5062:     : rangeBelongsToEditor(activeTextSelectionRange, editor)
 5063:     ? activeTextSelectionRange.cloneRange()
 5064:     : rangeBelongsToEditor(activeRichSelectionRange, editor)
 5065:       ? activeRichSelectionRange.cloneRange()
 5066:       : null;
 5067:
 5068:   let appliedToSelection = false;
 5069:   if (savedRange) {
 5070:     const nextRange = applyStyleToRichSelection(editor, savedRange, command, value);
 5071:     if (nextRange) {
 5072:       activeRichSelectionRange = nextRange.cloneRange();
 5073:       activeTextSelectionRange = nextRange.cloneRange();
 5074:       showPersistentTextSelection(nextRange);
 5075:       appliedToSelection = true;
 5076:     }
 5077:   }
 5078:
 5079:   if (!appliedToSelection) {
 5080:     const block = richBlockFromRange(savedRange, editor) || selectedRichBlock(editor) ||
currentRichBlock(editor);
 5081:     if (!block || block.dataset.type !== "paragraph") {
 5082:       setStatus("Highlight text or click inside a text line before formatting.");
 5083:       return;
 5084:     }
 5085:     selectRichBlock(block);
 5086:     if (command === "fontName") updateCurrentTextBlock(editor, { fontFamily: value });
 5087:     if (command === "foreColor") updateCurrentTextBlock(editor, { color: value });
 5088:     if (command === "fontSize") updateCurrentTextBlock(editor, { fontPx: value });
 5089:     if (command === "bold") updateCurrentTextBlock(editor, { bold: block.dataset.bold !== "true" });
 5090:     if (command === "italic") {
 5091:       updateCurrentTextBlock(editor, { italic: block.dataset.italic !== "true" });
 5092:     }
 5093:     if (command === "underline") {
 5094:       updateCurrentTextBlock(editor, { underline: block.dataset.underline !== "true" });
 5095:     }
 5096:     saveRichEditorToProject(editor);
 5097:     setStatus("Formatting applied and saved locally for the current text line.");
 5098:     return;
 5099:   }
 5100:
 5101:   saveRichEditorToProject(editor);
 5102:   setStatus("Unsaved local changes. Click the section save button to keep this formatting.");
 5103: }
```

selectRichBlock (template-preview.js, lines 5105-5113)

selectRichBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5105-5113 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, remove, add, closest, and indexOf. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include editor at line 5110. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5105: function selectRichBlock(block) {
5106:   document.querySelectorAll(".rich-block.selected").forEach((item) => item.classList.remove("selected"));
5107:   activeSummaryBlock = block || null;
5108:   if (block) {
5109:     block.classList.add("selected");
5110:     const editor = block.closest("[data-rich-editor]");
5111:     if (editor) editor.dataset.activeRichBlockIndex = String([...editor.children].indexOf(block));
5112:   }
5113: }
```

selectedRichBlock (template-preview.js, lines 5115-5123)

selectedRichBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5115-5123 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isInteger, matches, contains, and currentRichBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5116: if (!editor) return null;; line 5119: return editor.children[index];; line 5121: if (activeSummaryBlock && editor.contains(activeSummaryBlock)) return activeSummaryBlock;; line 5122: return currentRichBlock(editor);.

Important local values include index at line 5117. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5115: function selectedRichBlock(editor) {
5116:   if (!editor) return null;
5117:   const index = Number(editor.dataset.activeRichBlockIndex);
5118:   if (Number.isInteger(index) && index >= 0 && editor.children[index]?.matches(".rich-block")) {
5119:     return editor.children[index];
5120:   }
5121:   if (activeSummaryBlock && editor.contains(activeSummaryBlock)) return activeSummaryBlock;
5122:   return currentRichBlock(editor);
5123: }
```

syncRichContextMenuControls (template-preview.js, lines 5125-5144)

syncRichContextMenuControls keeps two places aligned, such as local data and target files. In this file, it appears around lines 5125-5144 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getComputedStyle, displayRichFontName, round, normalizeTextColor, restoreRichSelection, closest, queryCommandState, some, and setAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5126: if (!block || block.dataset.type !== "paragraph") return;.

Important local values include computed at line 5128; family at line 5129; size at line 5130; color at line 5131; bold at line 5133. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5125: function syncRichContextMenuControls(block) {
5126:     if (!block || block.dataset.type !== "paragraph") return;
5127:
5128:     const computed = getComputedStyle(block);
5129:     const family = displayRichFontName(computed.fontFamily) || "Arial";
5130:     const size = String(Math.round(parseFloat(computed.fontSize) || 16));
5131:     const color = normalizeTextColor(block.dataset.color || "") || "#17202a";
5132:     restoreRichSelection(block.closest("[data-rich-editor]"));
5133:     const bold = document.queryCommandState("bold") || block.dataset.bold === "true";
5134:
5135:     if (richFontSelect) {
5136:         richFontSelect.value = [...richFontSelect.options].some((option) => option.value === family ||
option.textContent === family)
5137:             ? family
5138:             : "Arial";
5139:     }
5140:
5141:     if (richFontSize) richFontSize.value = size;
5142:     if (richColorInput) richColorInput.value = color;
5143:     if (richBoldButton) richBoldButton.setAttribute("aria-pressed", bold ? "true" : "false");
5144: }
```

focusTextBlock (template-preview.js, lines 5146-5158)

focusTextBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5146-5158 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectRichBlock, closest, focus, createRange, selectNodeContents, collapse, getSelection, removeAllRanges, addRange, and captureRichSelection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5147: if (!block) return;.

Important local values include editor at line 5149; range at line 5151; selection at line 5154. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5146: function focusTextBlock(block) {
5147:   if (!block) return;
5148:   selectRichBlock(block);
5149:   const editor = block.closest("[data-rich-editor]");
5150:   editor?.focus();
5151:   const range = document.createRange();
5152:   range.selectNodeContents(block);
5153:   range.collapse(false);
5154:   const selection = window.getSelection();
5155:   selection.removeAllRanges();
5156:   selection.addRange(range);
5157:   captureRichSelection(editor);
5158: }
```

setTextBlockCaretOffset (template-preview.js, lines 5160-5174)

setTextBlockCaretOffset part of the private builder frontend and editing experience. In this file, it appears around lines 5160-5174 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectRichBlock, closest, focus, max, min, createRange, setStart, collapse, getSelection, removeAllRanges, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5161: if (!block) return;.

Important local values include editor at line 5163; textNode at line 5165; safeOffset at line 5166; range at line 5167; selection at line 5170. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5160: function setTextBlockCaretOffset(block, offset) {
5161:   if (!block) return;
5162:   selectRichBlock(block);
5163:   const editor = block.closest("[data-rich-editor]");
5164:   editor?.focus();
5165:   const textNode = block.firstChild || block;
5166:   const safeOffset = Math.max(0, Math.min(offset, textNode.textContent?.length || 0));
5167:   const range = document.createRange();
5168:   range.setStart(textNode, safeOffset);
5169:   range.collapse(true);
5170:   const selection = window.getSelection();
5171:   selection.removeAllRanges();
5172:   selection.addRange(range);
5173:   captureRichSelection(editor);
5174: }
```

textBlockCaretOffset (template-preview.js, lines 5176-5183)

textBlockCaretOffset part of the private builder frontend and editing experience. In this file, it appears around lines 5176-5183 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getSelection, contains, getRangeAt, cloneRange, selectNodeContents, setEnd, and toString. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 5178: if (!selection?.rangeCount || !block.contains(selection.anchorNode)) return block.textContent.length;; line 5182: return range.toString().length;.

Important local values include selection at line 5177; range at line 5179. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5176: function textBlockCaretOffset(block) {
5177:   const selection = window.getSelection();
5178:   if (!selection?.rangeCount || !block.contains(selection.anchorNode)) return block.textContent.length;
5179:   const range = selection.getRangeAt(0).cloneRange();
5180:   range.selectNodeContents(block);
5181:   range.setEnd(selection.anchorNode, selection.anchorOffset);
5182:   return range.toString().length;
5183: }
```

matchingTextBlock (template-preview.js, lines 5185-5198)

matchingTextBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5185-5198 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createRichTextBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5186: return createRichTextBlock(.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
5185: function matchingTextBlock(block, text = "") {
5186:     return createRichTextBlock(
5187:         text,
5188:         block?.dataset.fontSize || "normal",
5189:         block?.dataset.align || "left",
5190:         block?.dataset.fontFamily || "Arial",
5191:         block?.dataset.fontPx || "",
5192:         block?.dataset.color || "",
5193:         block?.dataset.bold === "true",
5194:         "",
5195:         block?.dataset.italic === "true",
5196:         block?.dataset.underline === "true"
5197:     );
5198: }
```

richInsertionRange (template-preview.js, lines 5200-5221)

richInsertionRange part of the private builder frontend and editing experience. In this file, it appears around lines 5200-5221 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectionRangeInsideEditor, collapse, restoreRichSelection, querySelectorAll, at, createRange, and selectNodeContents. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5204: return currentRange;; line 5210: return restored;; line 5218: return range;; line 5220: return null;.

Important local values include currentRange at line 5201; restored at line 5207; lastText at line 5213; range at line 5215. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5200: function richInsertionRange(editor) {
5201:   const currentRange = selectionRangeInsideEditor(editor);
5202:   if (currentRange) {
5203:     currentRange.collapse(true);
5204:     return currentRange;
5205:   }
5206:   if (restoreRichSelection(editor)) {
5207:     const restored = selectionRangeInsideEditor(editor);
5208:     if (restored) {
5209:       restored.collapse(true);
5210:       return restored;
5211:     }
```

```
5212:   }
5213:   const lastText = [...editor.querySelectorAll(".rich-text-block")].at(-1);
5214:   if (lastText) {
5215:     const range = document.createRange();
5216:     range.selectNodeContents(lastText);
5217:     range.collapse(false);
5218:     return range;
5219:   }
5220:   return null;
5221: }
```

insertRichBlockAtCursor (template-preview.js, lines 5223-5257)

insertRichBlockAtCursor part of the private builder frontend and editing experience. In this file, it appears around lines 5223-5257 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cloneRange, richInsertionRange, selectedRichBlock, contains, collapse, createRange, setStart, setEnd, extractContents, after, and 5 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include range at line 5224; rangeNode at line 5225; current at line 5228; nextText at line 5229; tail at line 5233; trailingContent at line 5236; reference at line 5242. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5223: function insertRichBlockAtCursor(editor, blockElement, insertionRange = null) {
5224:   const range = insertionRange?.cloneRange() || richInsertionRange(editor);
5225:   const rangeNode = range?.startContainer?.nodeType === Node.TEXT_NODE
5226:     ? range.startContainer.parentElement
5227:     : range?.startContainer;
5228:   const current = rangeNode?.closest?.(".rich-block") || selectedRichBlock(editor);
5229:   let nextText = null;
5230:
5231:   if (range && current?.dataset.type === "paragraph" && current.contains(range.startContainer)) {
5232:     range.collapse(true);
5233:     const tail = document.createRange();
5234:     tail.setStart(range.startContainer, range.startOffset);
5235:     tail.setEnd(current, current.childNodes.length);
5236:     const trailingContent = tail.extractContents();
5237:     current.after(blockElement);
5238:     nextText = matchingTextBlock(current, "");
5239:     nextText.append(trailingContent);
5240:     blockElement.after(nextText);
5241:   } else if (range?.startContainer === editor) {
5242:     const reference = editor.children[range.startOffset] || null;
5243:     editor.insertBefore(blockElement, reference);
5244:     nextText = createRichTextBlock("");
5245:     blockElement.after(nextText);
5246:   } else if (current) {
5247:     current.after(blockElement);
5248:     nextText = matchingTextBlock(current, "");
5249:     blockElement.after(nextText);
5250:   } else {
5251:     editor.append(blockElement);
5252:     nextText = createRichTextBlock("");
5253:     editor.append(nextText);
5254:   }
5255:
5256:   focusTextBlock(nextText);
5257: }
```

insertRichBlockAfterCursor (template-preview.js, lines 5259-5261)

insertRichBlockAfterCursor part of the private builder frontend and editing experience. In this file, it appears around lines 5259-5261 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references insertRichBlockAtCursor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
5259: function insertRichBlockAfterCursor(editor, blockElement) {
5260:   insertRichBlockAtCursor(editor, blockElement);
5261: }
```

cleanupEmptyRichTextBlocks (template-preview.js, lines 5263-5275)

cleanupEmptyRichTextBlocks part of the private builder frontend and editing experience. In this file, it appears around lines 5263-5275 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, richElementPlainText, remove, querySelector, append, and createRichTextBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 5264: if (!editor) return;; line 5267: if (hasVisibleText) return;.

Important local values include hasVisibleText at line 5266; previous at line 5268; next at line 5269; keepTypingBlockAfterImage at line 5270; isOnlyBlock at line 5271. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5263: function cleanupEmptyRichTextBlocks(editor) {
5264:   if (!editor) return;
5265:   [...editor.querySelectorAll(":scope > .rich-text-block")].forEach((block) => {
5266:     const hasVisibleText = richElementPlainText(block);
5267:     if (hasVisibleText) return;
5268:     const previous = block.previousElementSibling;
5269:     const next = block.nextElementSibling;
5270:     const keepTypingBlockAfterImage = previous?.dataset.type === "image" && !next;
5271:     const isOnlyBlock = editor.children.length <= 1;
5272:     if (!isOnlyBlock && !keepTypingBlockAfterImage) block.remove();
5273:   });
5274:   if (!editor.querySelector(":scope > .rich-block")) editor.append(createRichTextBlock(""));
5275: }
```

normalizeRichEditorStructure (template-preview.js, lines 5277-5307)

normalizeRichEditorStructure validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 5277-5307 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references forEach, trim, remove, createRichTextBlock, replaceWith, matches, sanitizeRichInlineHtml, querySelectorAll, add, removeAttribute, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5278: if (!editor) return;; line 5283: return;; line 5287: return;; line 5290: if (node.nodeType !== Node.ELEMENT_NODE || node.matches(".rich-block")) return;.

Important local values include paragraph at line 5285; paragraph at line 5291. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5277: function normalizeRichEditorStructure(editor) {
5278:   if (!editor) return;
5279:   [...editor.childNodes].forEach((node) => {
5280:     if (node.nodeType === Node.TEXT_NODE) {
5281:       if (!node.textContent.trim()) {
5282:         node.remove();
5283:         return;
5284:       }
5285:       const paragraph = createRichTextBlock(node.textContent);
5286:       node.replaceWith(paragraph);
5287:       return;
5288:     }
5289:
5290:     if (node.nodeType !== Node.ELEMENT_NODE || node.matches(".rich-block")) return;
5291:     const paragraph = createRichTextBlock("");
5292:     paragraph.innerHTML = sanitizeRichInlineHtml(node.innerHTML || node.textContent || "");
5293:     node.replaceWith(paragraph);
5294:   });
5295:
5296:   editor.querySelectorAll(".rich-text-block").forEach((block) => {
5297:     block.dataset.type = "paragraph";
```

```
5298:     block.dataset.fontSize ||= "normal";
5299:     block.dataset.align ||= "left";
5300:     block.dataset.fontFamily ||= "Arial";
5301:     block.classList.add("rich-block");
5302:     block.removeAttribute("contenteditable");
5303:     applyRichTextBlockStyle(block);
5304:   });
5305:
5306:   if (!editor.querySelector(":scope > .rich-block")) editor.append(createRichTextBlock(""));
5307: }
```

richElementPlainText (template-preview.js, lines 5309-5313)

richElementPlainText part of the private builder frontend and editing experience. In this file, it appears around lines 5309-5313 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cloneNode, querySelectorAll, forEach, replaceWith, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5312: return String(cloneElement.textContent || "").trim();.

Important local values include cloneElement at line 5310. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5309: function richElementPlainText(element) {
5310:   const cloneElement = element.cloneNode(true);
5311:   cloneElement.querySelectorAll("br").forEach((lineBreak) => lineBreak.replaceWith("\n"));
5312:   return String(cloneElement.textContent || "").trim();
5313: }
```

updateCurrentTextBlock (template-preview.js, lines 5315-5357)

updateCurrentTextBlock updates ui, state, cache, or stored data. In this file, it appears around lines 5315-5357 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedRichBlock, remove, add, call, cleanFontFamily, normalizeFontPx, normalizeTextColor, applyRichTextBlockStyle, and saveRichEditorToProject. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5317: if (!block || block.dataset.type !== "paragraph") return;.

Important local values include block at line 5316. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5315: function updateCurrentTextBlock(editor, updates) {
5316:     const block = selectedRichBlock(editor);
5317:     if (!block || block.dataset.type !== "paragraph") return;
5318:
5319:     if (updates.fontSize) {
5320:         block.dataset.fontSize = updates.fontSize;
5321:         block.classList.remove("rich-text-small", "rich-text-normal", "rich-text-large");
5322:         block.classList.add(`rich-text-${updates.fontSize}`);
5323:     }
5324:
5325:     if (updates.align) {
5326:         block.dataset.align = updates.align;
5327:         block.classList.remove("text-left", "text-center", "text-right");
5328:         block.classList.add(`text-${updates.align}`);
5329:     }
5330:
5331:     if (Object.prototype.hasOwnProperty.call(updates, "fontFamily")) {
5332:         block.dataset.fontFamily = cleanFontFamily(updates.fontFamily);
5333:     }
5334:
5335:     if (Object.prototype.hasOwnProperty.call(updates, "fontPx")) {
5336:         block.dataset.fontPx = normalizeFontPx(updates.fontPx);
5337:     }
5338:
5339:     if (Object.prototype.hasOwnProperty.call(updates, "color")) {
5340:         block.dataset.color = normalizeTextColor(updates.color);
5341:     }
5342:
5343:     if (Object.prototype.hasOwnProperty.call(updates, "bold")) {
```

```
5344:         block.dataset.bold = updates.bold ? "true" : "false";
5345:     }
5346:
5347:     if (Object.prototype.hasOwnProperty.call(updates, "italic")) {
5348:         block.dataset.italic = updates.italic ? "true" : "false";
5349:     }
5350:
5351:     if (Object.prototype.hasOwnProperty.call(updates, "underline")) {
5352:         block.dataset.underline = updates.underline ? "true" : "false";
5353:     }
5354:
5355:     applyRichTextBlockStyle(block);
5356:     saveRichEditorToProject(editor);
5357: }
```

extractRichSummary (template-preview.js, lines 5359-5416)

extractRichSummary part of the private builder frontend and editing experience. In this file, it appears around lines 5359-5416 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeRichEditorStructure, normalizeInlineStyleSpans, querySelectorAll, map, normalizeCropAspect, normalizeCropPosition, normalizeCropZoom, unwrapFormula, normalizeCodeText, querySelector, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 8 return statement(s). The most important visible returns are: line 5365: return {; line 5380: return {; line 5388: if (!code) return null;; line 5389: return {. There are 4 additional return statements in the function body.

Important local values include blocks at line 5362; align at line 5363; code at line 5387; text at line 5396. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5359: function extractRichSummary(editor) {
5360:   normalizeRichEditorStructure(editor);
5361:   normalizeInlineStyleSpans(editor);
5362:   const blocks = [...editor.querySelectorAll(".rich-block")].map((element) => {
5363:     const align = element.dataset.align || "left";
5364:     if (element.dataset.type === "image") {
5365:       return {
5366:         align,
5367:         caption: element.dataset.caption || "",
5368:         cropAspect: normalizeCropAspect(element.dataset.cropAspect),
5369:         cropX: normalizeCropPosition(element.dataset.cropX),
5370:         cropY: normalizeCropPosition(element.dataset.cropY),
5371:         cropZoom: normalizeCropZoom(element.dataset.cropZoom),
5372:         display: element.dataset.display === "download" ? "download" : "show",
5373:         source: element.dataset.source || "local",
5374:         title: element.dataset.title || "",
5375:         type: "image",
5376:         url: element.dataset.url || ""
5377:       };
5378:     }
5379:     if (element.dataset.type === "formula") {
5380:       return {
5381:         align,
5382:         formula: unwrapFormula(element.dataset.formula || element.textContent || ""),
5383:         type: "formula"
5384:       };
5385:     }
5386:     if (element.dataset.type === "code") {
5387:       const code = normalizeCodeText(element.dataset.code || element.querySelector("code")?.textContent
|| "", element.dataset.pasteMode || "source");
5388:       if (!code) return null;
5389:       return {
5390:         code,
5391:         language: normalizeCodeLanguage(element.dataset.language || detectCodeLanguage(code)),
5392:         pasteMode: normalizeCodePasteMode(element.dataset.pasteMode),
5393:         type: "code"
5394:       };
5395:     }
5396:     const text = richElementPlainText(element);
5397:     if (!text) return null;
5398:     if (isFormulaOnly(text)) {
5399:       return { align: element.dataset.align || "center", formula: unwrapFormula(text), type: "formula" };
5400:     }
5401:     return {
5402:       align,
5403:       bold: element.dataset.bold === "true",
5404:       color: element.dataset.color || "",
5405:       fontFamily: element.dataset.fontFamily || "",
5406:       fontPx: element.dataset.fontPx || "",
5407:       fontSize: element.dataset.fontSize || "normal",
5408:       html: sanitizeRichInlineHtml(element.innerHTML),
5409:       italic: element.dataset.italic === "true",
```

```
      text,
      underline: element.dataset.underline === "true",
      type: "paragraph"
    };
  }).filter(Boolean);
  return { blocks };
}
```

uploadSummaryImageFile (template-preview.js, lines 5418-5458)

uploadSummaryImageFile part of the private builder frontend and editing experience. In this file, it appears around lines 5418-5458 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, isAllowedUpload, setStatus, call, withExtension, readFileAsDataUrl, fetch, stringify, json, normalizeCropAspect, and 2 more. Endpoint references found inside it are /api/upload. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5421: if (!projectId || !file) return null;; line 5425: return null;; line 5443: return null;; line 5445: return {.

Important local values include project at line 5419; projectId at line 5420; validation at line 5422; displayTitle at line 5427; fileName at line 5428; data at line 5429; response at line 5430; result at line 5440. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
async function uploadSummaryImageFile(file, options = {}) {
  const project = selectedProject();
  const projectId = options.projectId || project?.id;
  if (!projectId || !file) return null;
  const validation = isAllowedUpload("media", file.name, file.type);
  if (!validation.ok) {
    setStatus(validation.message);
    return null;
  }
  const displayTitle = Object.prototype.hasOwnProperty.call(options, "title") ? options.title :
file.name;
  const fileName = withExtension(displayTitle || file.name, file.name);
  const data = await readFileAsDataUrl(file);
  const response = await fetch("/api/upload", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      projectId,
        section: options.folder || "summary",
      fileName,
      data
    })
  });
  const result = await response.json();
  if (!response.ok) {
    setStatus(result.error || "Image upload failed.");
    return null;
  }
  return {
    align: options.align || "center",
    caption: options.caption || "",
    cropAspect: normalizeCropAspect(options.cropAspect),
    cropX: normalizeCropPosition(options.cropX),
    cropY: normalizeCropPosition(options.cropY),
    cropZoom: normalizeCropZoom(options.cropZoom),
    display: options.display === "download" ? "download" : "show",
    source: "local",
    title: displayTitle || "",
    type: "image",
    url: result.url
  };
}
```

updateSummaryImageDialogVisibility (template-preview.js, lines 5460-5465)

updateSummaryImageDialogVisibility updates ui, state, cache, or stored data. In this file, it appears around lines 5460-5465 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include source at line 5461; isLocal at line 5462. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5460: function updateSummaryImageDialogVisibility() {
5461:   const source = summaryImageSource?.value || "local";
5462:   const isLocal = source === "local";
5463:   document.querySelector(".summary-image-local-field").hidden = !isLocal;
5464:   document.querySelector(".summary-image-url-field").hidden = isLocal;
5465: }
```

openSummaryImageDialog (template-preview.js, lines 5467-5546)

openSummaryImageDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 5467-5546 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, normalizeCropAspect, normalizeCropZoom, updateSummaryImageDialogVisibility, dialogValue, normalizeLinkTarget, trim, setStatus, urlLooksLikeDirectImage, normalizeCropPosition, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 5483: if (!saved) return null;; line 5493: return null;; line 5498: return null;; line 5500: return {. There are 3 additional return statements in the function body.

Important local values include heading at line 5468; submitButton at line 5469; saved at line 5482; file at line 5484; source at line 5485; url at line 5486; existingUrl at line 5489; nextUrl at line 5490; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5467: async function openSummaryImageDialog(existingBlock = null, options = {}) {
5468:   const heading = summaryImageDialog.querySelector("h2");
5469:   const submitButton = summaryImageDialog.querySelector("button[type='submit']");
5470:   if (heading) heading.textContent = existingBlock ? "Edit image" : "Add image to overview";
5471:   if (submitButton) submitButton.textContent = existingBlock ? "Save image" : "Add image";
5472:   summaryImageSource.value = existingBlock?.dataset.source || "local";
5473:   summaryImageFile.value = "";
5474:   summaryImageUrl.value = existingBlock?.dataset.source && existingBlock.dataset.source !== "local" ?
existingBlock.dataset.url || "" : "";
5475:   summaryImageTitle.value = existingBlock?.dataset.title || "";
5476:   summaryImageCaption.value = existingBlock?.dataset.caption || "";
5477:   summaryImageAlign.value = existingBlock?.dataset.align || "center";
5478:   summaryImageDisplay.value = existingBlock?.dataset.display || "show";
5479:   summaryImageCrop.value = normalizeCropAspect(existingBlock?.dataset.cropAspect);
5480:   summaryImageZoom.value = String(normalizeCropZoom(existingBlock?.dataset.cropZoom));
5481:   updateSummaryImageDialogVisibility();
5482:   const saved = await dialogValue(summaryImageDialog);
5483:   if (!saved) return null;
5484:   const file = summaryImageFile.files[0];
5485:   const source = summaryImageSource.value || "local";
5486:   const url = normalizeLinkTarget(summaryImageUrl.value.trim(), { assumeWeb: true });
5487:
5488:   if (source !== "local") {
5489:     const existingUrl = existingBlock?.dataset.url || "";
5490:     const nextUrl = url || existingUrl;
5491:     if (!nextUrl) {
5492:       setStatus("Paste an image URL or share link first.");
5493:       return null;
5494:     }
5495:     const display = summaryImageDisplay.value === "download" ? "download" : "show";
5496:     if (display === "show" && !urlLooksLikeDirectImage(nextUrl)) {
5497:       setStatus("Use a direct image URL for visible images, or choose Downloadable file only for share
links and non-image URLs.");
5498:       return null;
5499:     }
5500:     return {
5501:       align: summaryImageAlign.value,
5502:       caption: summaryImageCaption.value.trim(),
5503:       cropAspect: summaryImageCrop.value,
5504:       cropX: normalizeCropPosition(existingBlock?.dataset.cropX),
5505:       cropY: normalizeCropPosition(existingBlock?.dataset.cropY),
5506:       cropZoom: normalizeCropZoom(summaryImageZoom.value),
5507:       display,
5508:       source,
5509:       title: summaryImageTitle.value.trim() || displayNameFromUrl(nextUrl, source === "drive" ? "Google
Drive image" : "Web image"),
5510:       type: "image",
5511:       url: nextUrl
5512:     };
5513:   }
```

```
5514:
5515:   if (!file && !existingBlock) {
5516:     setStatus("Choose an image first.");
5517:     return null;
5518:   }
5519:   if (!file && existingBlock) {
5520:     return {
5521:       align: summaryImageAlign.value,
5522:       caption: summaryImageCaption.value.trim(),
5523:       cropAspect: summaryImageCrop.value,
5524:       cropX: normalizeCropPosition(existingBlock.dataset.cropX),
5525:       cropY: normalizeCropPosition(existingBlock.dataset.cropY),
5526:       cropZoom: normalizeCropZoom(summaryImageZoom.value),
5527:       display: summaryImageDisplay.value === "download" ? "download" : "show",
5528:       source: existingBlock.dataset.source || "local",
5529:       title: summaryImageTitle.value.trim() || existingBlock.dataset.title || "",
5530:       type: "image",
5531:       url: existingBlock.dataset.url || ""
5532:     };
5533:   }
5534:   return uploadSummaryImageFile(file, {
5535:     align: summaryImageAlign.value,
5536:     caption: summaryImageCaption.value.trim(),
5537:       cropAspect: summaryImageCrop.value,
5538:       cropX: 50,
5539:       cropY: 50,
5540:       cropZoom: normalizeCropZoom(summaryImageZoom.value),
5541:       display: summaryImageDisplay.value === "download" ? "download" : "show",
5542:       folder: options.folder || "summary",
5543:       projectId: options.projectId || "",
5544:     title: summaryImageTitle.value.trim() || file.name
5545:   });
5546: }
```

openSummaryFormulaDialog (template-preview.js, lines 5548-5563)

openSummaryFormulaDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 5548-5563 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, dialogValue, unwrapFormula, trim, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 5556: if (!saved) return null;; line 5560: return null;; line 5562: return { align: summaryFormulaAlign.value, formula, type: "formula" };.

Important local values include heading at line 5549; submitButton at line 5550; saved at line 5555; formula at line 5557. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5548: async function openSummaryFormulaDialog(existingBlock = null) {
5549:   const heading = summaryFormulaDialog.querySelector("h2");
5550:   const submitButton = summaryFormulaDialog.querySelector("button[type='submit']");
5551:   if (heading) heading.textContent = existingBlock ? "Edit formula" : "Add formula";
5552:   if (submitButton) submitButton.textContent = existingBlock ? "Save formula" : "Add formula";
5553:   summaryFormulaInput.value = existingBlock?.dataset.formula || "";
5554:   summaryFormulaAlign.value = existingBlock?.dataset.align || "center";
5555:   const saved = await dialogValue(summaryFormulaDialog);
5556:   if (!saved) return null;
5557:   const formula = unwrapFormula(summaryFormulaInput.value.trim());
5558:   if (!formula) {
5559:     setStatus("Paste or type a formula first.");
5560:     return null;
5561:   }
5562:   return { align: summaryFormulaAlign.value, formula, type: "formula" };
5563: }
```

refreshSummaryCodeDialogPreview (template-preview.js, lines 5565-5575)

refreshSummaryCodeDialogPreview part of the private builder frontend and editing experience. In this file, it appears around lines 5565-5575 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references detectCodeLanguage, normalizeCodeLanguage, normalizeCodeText, and renderRichCodeBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5566: if (!summaryCodePreview || !summaryCodeInput) return;.

Important local values include rawCode at line 5567; language at line 5568; code at line 5571. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5565: function refreshSummaryCodeDialogPreview() {
5566:   if (!summaryCodePreview || !summaryCodeInput) return;
5567:   const rawCode = summaryCodeInput.value || "";
5568:   const language = summaryCodeLanguage?.value === "auto"
5569:     ? detectCodeLanguage(rawCode)
5570:     : normalizeCodeLanguage(summaryCodeLanguage?.value || "javascript");
5571:   const code = normalizeCodeText(rawCode, summaryCodePasteMode?.value || "source");
5572:   summaryCodePreview.innerHTML = code
5573:     ? renderRichCodeBlock({ code, language, pasteMode: summaryCodePasteMode?.value || "source" })
5574:     : `<p class="evidence-empty">Code preview appears here.</p>`;
5575: }
```

beautifyCodeClient (template-preview.js, lines 5577-5592)

beautifyCodeClient part of the private builder frontend and editing experience. In this file, it appears around lines 5577-5592 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, normalizeCodeLanguage, includes, split, map, trim, max, repeat, match, join, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 5582: return normalized.split("\n").map((rawLine) => {; line 5584: if (!line) return "";; line 5590: return nextLine;.

Important local values include normalized at line 5578; lang at line 5579; depth at line 5581; line at line 5583; nextLine at line 5586; opens at line 5587; closes at line 5588. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5577: function beautifyCodeClient(code = "", language = "javascript") {
5578:   const normalized = String(code || "").replace(/\r\n?/g, "\n").replace(/\t/g, "  ");
5579:   const lang = normalizeCodeLanguage(language);
5580:   if (["c", "cpp", "java", "javascript", "verilog", "systemverilog"].includes(lang)) {
5581:     let depth = 0;
5582:     return normalized.split("\n").map((rawLine) => {
5583:       const line = rawLine.trim();
5584:       if (!line) return "";
5585:       if (/^[}\])]/.test(line)) depth = Math.max(0, depth - 1);
5586:       const nextLine = `${"  ".repeat(depth)}${line}`;
5587:       const opens = (line.match(/[{\[(]/g) || []).length;
5588:       const closes = (line.match(/[}\])]/g) || []).length;
5589:       depth = Math.max(0, depth + opens - closes);
5590:       return nextLine;
5591:     }).join("\n").trimEnd();
5592:   }
```

beautifySummaryCodeDialog (template-preview.js, lines 5599-5618)

beautifySummaryCodeDialog part of the private builder frontend and editing experience. In this file, it appears around lines 5599-5618 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references detectCodeLanguage, normalizeCodeLanguage, fetch, now, stringify, json, Error, beautifyCodeClient, and refreshSummaryCodeDialogPreview. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5600: if (!summaryCodeInput) return;.

Important local values include language at line 5601; response at line 5605; result at line 5611. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5599: async function beautifySummaryCodeDialog() {
5600:   if (!summaryCodeInput) return;
5601:   const language = summaryCodeLanguage?.value === "auto"
5602:     ? detectCodeLanguage(summaryCodeInput.value || "")
5603:     : normalizeCodeLanguage(summaryCodeLanguage?.value || "javascript");
5604:   try {
```

```
    const response = await fetch(`/api/code/beautify?t=${Date.now()}`, {
      method: "POST",
      cache: "no-store",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ language, code: summaryCodeInput.value || "" })
    });
    const result = await response.json();
    if (!response.ok || !result.ok) throw new Error(result.error || "Beautifier failed.");
    summaryCodeInput.value = result.code || summaryCodeInput.value;
  } catch {
    summaryCodeInput.value = beautifyCodeClient(summaryCodeInput.value || "", language);
  }
  refreshSummaryCodeDialogPreview();
}
```

openSummaryCodeDialog (template-preview.js, lines 5620-5640)

openSummaryCodeDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 5620-5640 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, detectCodeLanguage, refreshSummaryCodeDialogPreview, dialogValue, normalizeCodePasteMode, normalizeCodeText, setStatus, and normalizeCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 5631: if (!saved) return null;; line 5636: return null;; line 5639: return { code, language, pasteMode, type: "code" };.

Important local values include heading at line 5621; submitButton at line 5622; defaultCode at line 5625; saved at line 5630; pasteMode at line 5632; code at line 5633; language at line 5638. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
async function openSummaryCodeDialog(existingBlock = null, options = {}) {
  const heading = summaryCodeDialog.querySelector("h2");
  const submitButton = summaryCodeDialog.querySelector("button[type='submit']");
  if (heading) heading.textContent = existingBlock ? "Edit code block" : "Add code block";
  if (submitButton) submitButton.textContent = existingBlock ? "Save code" : "Add code";
  const defaultCode = options.code || "";
  summaryCodeLanguage.value = existingBlock?.dataset.language || (defaultCode ?
detectCodeLanguage(defaultCode) : "auto");
  summaryCodePasteMode.value = existingBlock?.dataset.pasteMode || options.pasteMode || "source";
  summaryCodeInput.value = existingBlock?.dataset.code || defaultCode;
  refreshSummaryCodeDialogPreview();
  const saved = await dialogValue(summaryCodeDialog);
  if (!saved) return null;
  const pasteMode = normalizeCodePasteMode(summaryCodePasteMode.value);
  const code = normalizeCodeText(summaryCodeInput.value, pasteMode);
  if (!code) {
    setStatus("Paste or type code before saving the block.");
    return null;
  }
  const language = summaryCodeLanguage.value === "auto" ? detectCodeLanguage(code) :
normalizeCodeLanguage(summaryCodeLanguage.value);
  return { code, language, pasteMode, type: "code" };
}
```

configureSummaryContextMenu (template-preview.js, lines 5642-5665)

configureSummaryContextMenu part of the private builder frontend and editing experience. In this file, it appears around lines 5642-5665 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, querySelectorAll, forEach, and has. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include plainActions at line 5643; action at line 5653; textOnly at line 5660. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function configureSummaryContextMenu(mode = "rich", options = {}) {
  const plainActions = new Set([
    "copy-text",
    "paste-text",
```

```
    "cut-text",
    "select-all-text",
    "toggle-bold",
    "toggle-italic",
    "toggle-underline"
  ]);
  summaryContextMenu.querySelectorAll("[data-rich-action]").forEach((button) => {
    const action = button.dataset.richAction;
    button.hidden = mode === "plain" ? !plainActions.has(action) : false;
  });
  summaryContextMenu.querySelectorAll(".rich-menu-field").forEach((field) => {
    field.hidden = false;
  });
  if (mode === "rich") {
    const textOnly = Boolean(options.textOnly);
    summaryContextMenu.querySelectorAll("[data-rich-action='add-image'], [data-rich-action='add-
formula'], [data-rich-action='add-code'], [data-rich-action='paste-code']").forEach((button) => {
      button.hidden = textOnly;
    });
  }
}
```

applyPlainTextControlFormat (template-preview.js, lines 5667-5701)

applyPlainTextControlFormat part of the private builder frontend and editing experience. In this file, it appears around lines 5667-5701 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references call, cleanFontFamily, normalizeFontPx, normalizeTextColor, rgbColorToHex, storePlainControlStyle, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5668: if (!control) return;.

Important local values include font at line 5670; size at line 5675; color at line 5680. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function applyPlainTextControlFormat(control, updates = {}) {
  if (!control) return;
  if (Object.prototype.hasOwnProperty.call(updates, "fontFamily")) {
    const font = cleanFontFamily(updates.fontFamily);
    control.dataset.builderFontFamily = font;
    control.style.fontFamily = font || "";
  }
  if (Object.prototype.hasOwnProperty.call(updates, "fontPx")) {
    const size = normalizeFontPx(updates.fontPx);
    control.dataset.builderFontPx = size;
    control.style.fontSize = size ? `${size}px` : "";
  }
  if (Object.prototype.hasOwnProperty.call(updates, "color")) {
    const color = normalizeTextColor(updates.color);
    control.dataset.builderColor = color;
    control.style.color = color || "";
    if (richColorInput && color && /^#[0-9a-f]{6}$/i.test(rgbColorToHex(color))) {
      richColorInput.value = rgbColorToHex(color);
    }
  }
  if (Object.prototype.hasOwnProperty.call(updates, "bold")) {
    control.dataset.builderBold = updates.bold ? "true" : "false";
    control.style.fontWeight = updates.bold ? "700" : "";
  }
  if (Object.prototype.hasOwnProperty.call(updates, "italic")) {
    control.dataset.builderItalic = updates.italic ? "true" : "false";
    control.style.fontStyle = updates.italic ? "italic" : "";
  }
  if (Object.prototype.hasOwnProperty.call(updates, "underline")) {
    control.dataset.builderUnderline = updates.underline ? "true" : "false";
    control.style.textDecoration = updates.underline ? "underline" : "";
  }
  storePlainControlStyle(control);
  setStatus("Formatting applied and saved locally for this field.");
}
```

syncPlainContextMenuControls (template-preview.js, lines 5703-5719)

syncPlainContextMenuControls keeps two places aligned, such as local data and target files. In this file, it appears around lines 5703-5719 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getComputedStyle, displayRichFontName, round, rgbColorToHex, some, and setAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5704: if (!control) return;.

Important local values include computed at line 5705; family at line 5706; size at line 5707; color at line 5708. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 5703: function syncPlainContextMenuControls(control) {
 5704:   if (!control) return;
 5705:   const computed = getComputedStyle(control);
 5706:   const family = displayRichFontName(control.dataset.builderFontFamily || computed.fontFamily) ||
"Arial";
 5707:   const size = String(Math.round(parseFloat(control.dataset.builderFontPx || computed.fontSize) || 16));
 5708:   const color = rgbColorToHex(control.dataset.builderColor || computed.color || "#17202a");
 5709:   if (richFontSelect) {
 5710:     richFontSelect.value = [...richFontSelect.options].some((option) => option.value === family ||
option.textContent === family)
 5711:       ? family
 5712:       : "Arial";
 5713:   }
 5714:   if (richFontSize) richFontSize.value = [...richFontSize.options].some((option) => option.value === size
 || option.textContent === size)
 5715:     ? size
 5716:     : "16";
 5717:   if (richColorInput && /^#[0-9a-f]{6}$/i.test(color)) richColorInput.value = color;
 5718:   if (richBoldButton) richBoldButton.setAttribute("aria-pressed", control.dataset.builderBold === "true"
? "true" : "false");
 5719: }
```

plainTextControlFromContextEvent (template-preview.js, lines 5721-5728)

plainTextControlFromContextEvent part of the private builder frontend and editing experience. In this file, it appears around lines 5721-5728 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, toLowerCase, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5723: if (!control || control.closest("#summary-context-menu")) return null;; line 5725: if (control.disabled || control.readOnly) return null;; line 5726: if (["button", "checkbox", "color", "file", "hidden", "image", "radio", "range", "reset", "submit"].includes(type)) return null;; line 5727: return control;.

Important local values include control at line 5722; type at line 5724. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 5721: function plainTextControlFromContextEvent(event) {
 5722:   const control = event.target.closest?.("textarea, input");
 5723:   if (!control || control.closest("#summary-context-menu")) return null;
 5724:   const type = String(control.type || "text").toLowerCase();
 5725:   if (control.disabled || control.readOnly) return null;
 5726:   if (["button", "checkbox", "color", "file", "hidden", "image", "radio", "range", "reset",
"submit"].includes(type)) return null;
 5727:   return control;
 5728: }
```

handlePlainTextAction (template-preview.js, lines 5730-5778)

handlePlainTextAction handles an event, request, command, or user action. In this file, it appears around lines 5730-5778 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references focus, max, min, slice, writeText, setRangeText, dispatchEvent, Event, readText, select, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 5732: if (!control) return;; line 5748: return;; line 5757: return;; line 5763: return;. There are 3 additional return statements in the function body.

Important local values include control at line 5731; selection at line 5734; start at line 5738; end at line 5739; text at line 5742; text at line 5752. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5730: async function handlePlainTextAction(action) {
5731:   const control = activePlainTextControl;
5732:   if (!control) return;
5733:   control.focus({ preventScroll: true });
5734:   const selection = activePlainTextSelection || {
5735:     start: control.selectionStart ?? 0,
5736:     end: control.selectionEnd ?? control.value.length
5737:   };
5738:   const start = Math.max(0, Math.min(selection.start, control.value.length));
5739:   const end = Math.max(start, Math.min(selection.end, control.value.length));
5740:
5741:   if (action === "copy-text" || action === "cut-text") {
5742:     const text = control.value.slice(start, end) || control.value;
5743:     if (text) await navigator.clipboard.writeText(text);
5744:     if (action === "cut-text" && end > start) {
5745:       control.setRangeText("", start, end, "start");
5746:       control.dispatchEvent(new Event("input", { bubbles: true }));
5747:     }
5748:     return;
5749:   }
5750:
5751:   if (action === "paste-text") {
5752:     const text = await navigator.clipboard.readText();
5753:     if (text) {
5754:       control.setRangeText(text, start, end, "end");
5755:       control.dispatchEvent(new Event("input", { bubbles: true }));
5756:     }
5757:     return;
5758:   }
5759:
5760:   if (action === "select-all-text") {
5761:     control.select();
5762:     activePlainTextSelection = { start: 0, end: control.value.length };
5763:     return;
5764:   }
5765:
5766:   if (action === "toggle-bold") {
5767:     applyPlainTextControlFormat(control, { bold: control.dataset.builderBold !== "true" });
5768:     return;
5769:   }
5770:   if (action === "toggle-italic") {
5771:     applyPlainTextControlFormat(control, { italic: control.dataset.builderItalic !== "true" });
5772:     return;
5773:   }
5774:   if (action === "toggle-underline") {
5775:     applyPlainTextControlFormat(control, { underline: control.dataset.builderUnderline !== "true" });
5776:     return;
5777:   }
5778: }
```

handleRichAction (template-preview.js, lines 5780-5878)

handleRichAction handles an event, request, command, or user action. In this file, it appears around lines 5780-5878 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references applyRichInlineCommand, updateCurrentTextBlock, restoreRichSelection, getSelection, toString, selectedRichBlock, writeText, readText, insertPlainTextIntoRichEditor, saveRichEditorToProject, and 20 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 5781: if (!editor) return;; line 5801: return;; line 5810: return;; line 5824: return;. There are 1 additional return statements in the function body.

Important local values include selectedText at line 5797; block at line 5798; text at line 5799; text at line 5805; selection at line 5814; selectedText at line 5815; range at line 5818; block at line 5827; plus 9 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5780: async function handleRichAction(action, editor) {
5781:   if (!editor) return;
5782:   activeSummaryEditor = editor;
5783:   if (action === "toggle-bold") {
5784:     applyRichInlineCommand(editor, "bold");
5785:   }
5786:   if (action === "toggle-italic") {
5787:     applyRichInlineCommand(editor, "italic");
5788:   }
```

```
  if (action === "toggle-underline") {
    applyRichInlineCommand(editor, "underline");
  }
  if (action === "align-left") updateCurrentTextBlock(editor, { align: "left" });
  if (action === "align-center") updateCurrentTextBlock(editor, { align: "center" });
    if (action === "align-right") updateCurrentTextBlock(editor, { align: "right" });
    if (action === "copy-text") {
        restoreRichSelection(editor);
        const selectedText = window.getSelection()?.toString() || "";
        const block = selectedRichBlock(editor);
        const text = selectedText || block?.textContent || "";
        if (text) await navigator.clipboard.writeText(text);
        return;
    }

    if (action === "paste-text") {
        const text = await navigator.clipboard.readText();
        if (text) {
            insertPlainTextIntoRichEditor(editor, text);
            saveRichEditorToProject(editor);
        }
        return;
    }
    if (action === "cut-text") {
        restoreRichSelection(editor);
        const selection = window.getSelection();
        const selectedText = selection?.toString() || "";
        if (selectedText && selection.rangeCount) {
            await navigator.clipboard.writeText(selectedText);
            const range = selection.getRangeAt(0);
            range.deleteContents();
            captureRichSelection(editor);
            cleanupEmptyRichTextBlocks(editor);
            saveRichEditorToProject(editor);
        }
        return;
    }
    if (action === "select-all-text") {
        const block = selectedRichBlock(editor);
        const target = block?.dataset.type === "paragraph" ? block : editor;
        const range = document.createRange();
        range.selectNodeContents(target);
        const selection = window.getSelection();
        selection.removeAllRanges();
        selection.addRange(range);
        activeRichSelectionRange = range.cloneRange();
        activeTextSelectionRange = range.cloneRange();
        showPersistentTextSelection(range);
        showTextSelectionInspector(editor, range);
        return;
    }
  if (action === "add-image") {
      const imageBlock = await openSummaryImageDialog(null, {
          folder: editor.dataset.richFolder || "summary",
          projectId: editor.dataset.richProjectId || ""
      });
    if (imageBlock) {
      insertRichBlockAfterCursor(editor, createRichImageBlock(imageBlock));
      saveRichEditorToProject(editor);
    }
  }
  if (action === "add-formula") {
    const formulaBlock = await openSummaryFormulaDialog();
    if (formulaBlock) {
      insertRichBlockAfterCursor(editor, createRichFormulaBlock(formulaBlock));
      saveRichEditorToProject(editor);
    }
  }
  if (action === "add-code") {
    const selectedText = window.getSelection()?.toString() || "";
    const codeBlock = await openSummaryCodeDialog(null, { code: selectedText, pasteMode: "source" });
    if (codeBlock) {
      insertRichBlockAfterCursor(editor, createRichCodeBlock(codeBlock));
      saveRichEditorToProject(editor);
    }
  }
  if (action === "paste-code") {
    const text = await navigator.clipboard.readText();
    const codeBlock = await openSummaryCodeDialog(null, { code: text, pasteMode: "source" });
    if (codeBlock) {
      insertRichBlockAfterCursor(editor, createRichCodeBlock(codeBlock));
      saveRichEditorToProject(editor);
    }
  }
  if (action === "edit-block") {
    await editSelectedRichBlock(editor);
    saveRichEditorToProject(editor);
  }
  if (action === "delete-block") deleteSelectedRichBlock(editor);
}
```

hideSummaryContextMenu (template-preview.js, lines 5880-5884)

hideSummaryContextMenu controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 5880-5884 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
5880: function hideSummaryContextMenu() {
5881:   summaryContextMenu.hidden = true;
5882:   activePlainTextControl = null;
5883:   activePlainTextSelection = null;
5884: }
```

editSelectedRichBlock (template-preview.js, lines 5886-5909)

editSelectedRichBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5886-5909 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedRichBlock, selectRichBlock, openSummaryImageDialog, refreshRichImageBlock, openSummaryFormulaDialog, refreshRichFormulaBlock, openSummaryCodeDialog, refreshRichCodeBlock, focusTextBlock, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5888: if (!block) return;; line 5893: return;; line 5898: return;; line 5903: return;.

Important local values include block at line 5887; updated at line 5891; updated at line 5896; updated at line 5901. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5886: async function editSelectedRichBlock(editor) {
5887:   const block = selectedRichBlock(editor);
5888:   if (!block) return;
5889:   selectRichBlock(block);
5890:   if (block.dataset.type === "image") {
5891:     const updated = await openSummaryImageDialog(block);
5892:     if (updated) refreshRichImageBlock(block, updated);
5893:     return;
5894:   }
5895:   if (block.dataset.type === "formula") {
5896:     const updated = await openSummaryFormulaDialog(block);
5897:     if (updated) refreshRichFormulaBlock(block, updated);
5898:     return;
5899:   }
5900:   if (block.dataset.type === "code") {
5901:     const updated = await openSummaryCodeDialog(block);
5902:     if (updated) refreshRichCodeBlock(block, updated);
5903:     return;
5904:   }
5905:   if (block.dataset.type === "paragraph") {
5906:     focusTextBlock(block);
5907:     setStatus("Text block selected. Type directly in the block to edit it.");
5908:   }
5909: }
```

copyRichCodeBlock (template-preview.js, lines 5911-5918)

copyRichCodeBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5911-5918 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, writeText, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 5912: if (!block || block.dataset.type !== "code") return false;; line 5914: if (!code) return false;; line 5917: return true;.

Important local values include code at line 5913. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5911: async function copyRichCodeBlock(block) {
5912:   if (!block || block.dataset.type !== "code") return false;
5913:   const code = block.dataset.code || block.querySelector("code")?.textContent || "";
5914:   if (!code) return false;
5915:   await navigator.clipboard.writeText(code);
5916:   setStatus("Code copied to clipboard.");
5917:   return true;
5918: }
```

removeRichBlock (template-preview.js, lines 5920-5931)

removeRichBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5920-5931 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references matches, remove, querySelector, append, createRichTextBlock, and focusTextBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5921: if (!block) return;.

Important local values include nextFocus at line 5922. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5920: function removeRichBlock(block, editor) {
5921:   if (!block) return;
5922:   const nextFocus = block.previousElementSibling?.matches(".rich-text-block")
5923:     ? block.previousElementSibling
5924:     : block.nextElementSibling?.matches(".rich-text-block")
5925:       ? block.nextElementSibling
5926:       : null;
5927:   block.remove();
5928:   activeSummaryBlock = null;
5929:   if (!editor.querySelector(".rich-block")) editor.append(createRichTextBlock(""));
5930:   focusTextBlock(nextFocus || editor.querySelector(".rich-text-block"));
5931: }
```

deleteSelectedRichBlock (template-preview.js, lines 5933-5948)

deleteSelectedRichBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5933-5948 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedRichBlock, openDeleteConfirm, and removeRichBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5935: if (!block) return;.

Important local values include block at line 5934; label at line 5936. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5933: function deleteSelectedRichBlock(editor) {
5934:   const block = selectedRichBlock(editor);
5935:   if (!block) return;
5936:   const label = block.dataset.type === "image"
5937:     ? "this overview image"
5938:     : block.dataset.type === "formula"
5939:       ? "this formula"
5940:       : block.dataset.type === "code"
5941:         ? "this code block"
5942:         : "this text block";
5943:   openDeleteConfirm({
5944:     title: "Delete overview block",
5945:     message: `Are you sure you want to delete ${label}?`,
5946:     onConfirm: () => removeRichBlock(block, editor)
5947:   });
5948: }
```

deleteProjectSummary (template-preview.js, lines 5950-5966)

deleteProjectSummary part of the private builder frontend and editing experience. In this file, it appears around lines 5950-5966 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references openDeleteConfirm, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5951: if (!project) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 5950: function deleteProjectSummary(project) {
 5951:   if (!project) return;
 5952:   openDeleteConfirm({
 5953:     title: "Delete project overview",
 5954:     message: "Are you sure you want to delete the current project overview?",
 5955:     onConfirm: () => {
 5956:       project.summary = "";
 5957:       project.summaryRich = null;
 5958:       summaryEditorProjectId = "";
 5959:       activeSummaryEditor = null;
 5960:       activeSummaryBlock = null;
 5961:       setStatus("Project overview deleted in the editor. Click Save project to update the portfolio
preview.");
 5962:       scheduleAutosave();
 5963:       renderAll();
 5964:     }
 5965:   });
 5966: }
```

saveRichSummary (template-preview.js, lines 5968-5987)

saveRichSummary persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 5968-5987 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, querySelector, extractRichSummary, plainTextFromRich, wordCount, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 5971: if (!project || !editor) return true;; line 5976: return false;; line 5986: return true;.

Important local values include project at line 5969; editor at line 5970; rich at line 5972; plain at line 5973. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 5968: async function saveRichSummary() {
 5969:   const project = selectedProject();
 5970:   const editor = projectFields.querySelector("[data-rich-editor='summary']");
 5971:   if (!project || !editor) return true;
 5972:   const rich = extractRichSummary(editor);
 5973:   const plain = plainTextFromRich(rich);
 5974:   if (wordCount(plain) > 1000) {
 5975:     setStatus("Project overview is limited to 1000 words. Shorten the text before saving.");
 5976:     return false;
 5977:   }
 5978:   project.summaryRich = rich;
 5979:   project.summary = plain;
 5980:   summaryEditorProjectId = "";
 5981:   activeSummaryEditor = null;
 5982:   activeSummaryBlock = null;
 5983:   setStatus("Overview saved in the editor. Click Save project to include this version in the portfolio
preview.");
 5984:   scheduleAutosave();
 5985:   renderAll();
 5986:   return true;
 5987: }
```

renderFields (template-preview.js, lines 5989-6004)

renderFields renders ui, markup, or display output. In this file, it appears around lines 5989-6004 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderSummaryBuilder, requestAnimationFrame, and populateSummaryEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5993: return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
5989: function renderFields(project) {
5990:   if (!project) {
5991:     projectFields.hidden = false;
5992:     projectFields.innerHTML = `<p class="evidence-empty">Add or select a project to begin editing.</p>`;
5993:     return;
5994:   }
5995:
5996:   projectFields.hidden = activeSectionId !== "brief";
5997:   projectWindowTitle.textContent = project.title;
5998:   projectFields.innerHTML = `
5999:     ${renderSummaryBuilder(project)}
6000:   `;
6001:   if (summaryEditorProjectId === project.id) {
6002:     requestAnimationFrame(() => populateSummaryEditor(project));
6003:   }
6004: }
```

sectionOptions (template-preview.js, lines 6006-6016)

sectionOptions part of the private builder frontend and editing experience. In this file, it appears around lines 6006-6016 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, filter, and isAnalogProject. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6012: return [.

Important local values include custom at line 6007. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6006: function sectionOptions(project) {
6007:   const custom = (project.sections || []).map((section) => ({
6008:     id: `custom:${section.id}`,
6009:     label: section.title,
6010:     folder: section.id
6011:   }));
6012:   return [
6013:     ...standardSections.filter((section) => !section.analogOnly || isAnalogProject(project)),
6014:     ...custom
6015:   ];
6016: }
```

custom (template-preview.js, lines 6007-6011)

custom part of the private builder frontend and editing experience. In this file, it appears around lines 6007-6011 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include custom at line 6007. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 6007:   const custom = (project.sections || []).map((section) => ({
 6008:     id: `custom:${section.id}`,
 6009:     label: section.title,
 6010:     folder: section.id
 6011:   }));
```

ensureSiteSections (template-preview.js, lines 6018-6021)

ensureSiteSections part of the private builder frontend and editing experience. In this file, it appears around lines 6018-6021 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clone. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 6018: function ensureSiteSections() {
 6019:   catalog.siteSections = catalog.siteSections || [];
 6020:   savedPortfolioCatalog.siteSections = savedPortfolioCatalog.siteSections || clone(catalog.siteSections
|| []);
 6021: }
```

siteSectionHasContent (template-preview.js, lines 6023-6032)

siteSectionHasContent part of the private builder frontend and editing experience. In this file, it appears around lines 6023-6032 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richHasContent, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6024: return Boolean(.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 6023: function siteSectionHasContent(section) {
 6024:   return Boolean(
 6025:     section?.description ||
 6026:     richHasContent(section?.richDescription) ||
 6027:     section?.backgroundImage ||
 6028:     (section?.links || []).some((link) => link.label || link.url) ||
 6029:     (section?.assets || []).some((asset) => asset.title || asset.url) ||
 6030:     (section?.subsections || []).some((item) => item.title || item.description ||
richHasContent(item.richDescription) || (item.links || []).length)
 6031:   );
 6032: }
```

siteSectionRenderable (template-preview.js, lines 6034-6036)

siteSectionRenderable part of the private builder frontend and editing experience. In this file, it appears around lines 6034-6036 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references siteSectionHasContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6035: return section?.visible !== false && siteSectionHasContent(section);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 6034: function siteSectionRenderable(section) {
 6035:   return section?.visible !== false && siteSectionHasContent(section);
 6036: }
```

siteSectionLinkCount (template-preview.js, lines 6038-6044)

siteSectionLinkCount part of the private builder frontend and editing experience. In this file, it appears around lines 6038-6044 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flatMap, and filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6039: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 6038: function siteSectionLinkCount(section) {
 6039:   return [
 6040:     ...(section.links || []),
 6041:     ...(section.assets || []),
 6042:     ...(section.subsections || []).flatMap((item) => item.links || [])
 6043:   ].filter((item) => item.label || item.url).length;
 6044: }
```

renderSiteSectionList (template-preview.js, lines 6046-6094)

renderSiteSectionList renders ui, markup, or display output. In this file, it appears around lines 6046-6094 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureSiteSections, map, url, siteSectionLinkCount, richHasContent, renderRichContent, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include sections at line 6048. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 6046: function renderSiteSectionList() {
 6047:   ensureSiteSections();
 6048:   const sections = catalog.siteSections || [];
 6049:   siteSectionList.innerHTML = sections.length ? sections.map((section) => `
 6050:     <article class="site-section-item ${section.visible === false ? "is-hidden-section" : ""}"
style="${section.backgroundImage ? `--site-section-bg: url('${section.backgroundImage}')` : ""}">
 6051:       <div>
 6052:         <strong>${section.title || "Untitled section"}</strong>
 6053:         <span>${section.visible === false ? "Hidden from website" : "Visible on website"} ·
${(section.subsections || []).length} subsection${(section.subsections || []).length === 1 ? "" : "s"} ·
${siteSectionLinkCount(section)} link${siteSectionLinkCount(section) === 1 ? "" : "s"}</span>
 6054:         ${section.description || richHasContent(section.richDescription) ?
renderRichContent(section.richDescription, section.description || "") : ""}
 6055:       </div>
 6056:       ${(section.subsections || []).length ? `
 6057:         <div class="site-subsection-list">
 6058:           ${(section.subsections || []).map((item, index) => `
 6059:             <article class="site-subsection-item">
 6060:               <div>
 6061:                 <strong>${item.title || "Untitled subsection"}</strong>
 6062:                 ${item.description || richHasContent(item.richDescription) ?
renderRichContent(item.richDescription, item.description || "") : ""}
 6063:               </div>
 6064:               <div class="site-section-actions">
 6065:                 <button type="button" data-site-subsection-edit="${section.id}" data-
index="${index}">Edit</button>
 6066:                 <button class="danger-icon" type="button" data-site-subsection-delete="${section.id}"
data-index="${index}">Delete</button>
 6067:               </div>
 6068:             </article>
 6069:           `).join("")}
 6070:         </div>
 6071:       ` : ""}
 6072:       ${(section.links || []).length ? `
 6073:         <div class="site-link-list">
 6074:           ${(section.links || []).map((link, index) => `
 6075:             <span>
 6076:               ${link.label || "Untitled link"}
 6077:               <button type="button" data-site-link-edit="${section.id}" data-
index="${index}">Edit</button>
```

```
 6078:               <button type="button" data-site-link-delete="${section.id}" data-
index="${index}">Delete</button>
 6079:             </span>
 6080:           `).join("")}
 6081:         </div>
 6082:       ` : ""}
 6083:       <div class="site-section-actions">
 6084:         <button type="button" data-site-section-toggle="${section.id}">${section.visible === false ?
"Show" : "Hide"}</button>
 6085:         <button type="button" data-site-section-edit="${section.id}">Edit</button>
 6086:         <button type="button" data-site-subsection-add="${section.id}">Add subsection</button>
 6087:         <button type="button" data-site-link-add="${section.id}">Add link</button>
 6088:         <button type="button" data-site-section-bg="${section.id}">Background</button>
 6089:         ${section.backgroundImage ? `<button type="button" data-site-section-bg-
clear="${section.id}">Clear bg</button>` : ""}
 6090:         <button class="danger-icon" type="button" data-site-section-
delete="${section.id}">Delete</button>
 6091:       </div>
 6092:     </article>
 6093:   `).join("") : `<p class="evidence-empty">No extra portfolio areas added yet.</p>`;
 6094: }
```

siteSectionDialogValue (template-preview.js, lines 6096-6118)

siteSectionDialogValue part of the private builder frontend and editing experience. In this file, it appears around lines 6096-6118 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references populateStandaloneRichEditor, dialogValue, trim, setStatus, extractRichSummary, plainTextFromRich, and slugify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 6105: if (!saved) return null;; line 6109: return null;; line 6112: return {.

Important local values include saved at line 6104; title at line 6106; richDescription at line 6111. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 6096: async function siteSectionDialogValue(section = {}) {
 6097:   sectionDialogTitle.textContent = section.id ? "Edit section" : "Create section";
 6098:   sectionTitle.placeholder = "Professional profile, Personal background, Sports...";
 6099:   sectionDescription.dataset.placeholder = "Short section overview shown on the website";
 6100:   sectionDescription.dataset.richFolder = section.id || "new-portfolio-section";
 6101:   sectionDescription.dataset.richProjectId = "_site-sections";
 6102:   sectionTitle.value = section.title || "";
 6103:   populateStandaloneRichEditor(sectionDescription, section.richDescription, section.description || "");
 6104:   const saved = await dialogValue(sectionDialog);
 6105:   if (!saved) return null;
 6106:   const title = sectionTitle.value.trim();
 6107:   if (!title) {
 6108:     setStatus("Type a section title before saving.");
 6109:     return null;
 6110:   }
 6111:   const richDescription = extractRichSummary(sectionDescription);
 6112:   return {
 6113:     description: plainTextFromRich(richDescription, "\n").trim(),
 6114:     id: section.id || slugify(title),
 6115:     richDescription,
 6116:     title
 6117:   };
 6118: }
```

siteSubsectionDialogValue (template-preview.js, lines 6120-6143)

siteSubsectionDialogValue part of the private builder frontend and editing experience. In this file, it appears around lines 6120-6143 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references populateStandaloneRichEditor, dialogValue, trim, setStatus, extractRichSummary, plainTextFromRich, and slugify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 6129: if (!saved) return null;; line 6133: return null;; line 6136: return {.

Important local values include saved at line 6128; title at line 6130; richDescription at line 6135. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6120: async function siteSubsectionDialogValue(subsection = {}) {
6121:   sectionDialogTitle.textContent = subsection.id ? "Edit subsection" : "Create subsection";
6122:   sectionTitle.placeholder = "Sports, Hobbies, Resume links, Origin...";
6123:   sectionDescription.dataset.placeholder = "Short subsection text shown inside the portfolio area";
6124:   sectionDescription.dataset.richFolder = subsection.id || "new-portfolio-subsection";
6125:   sectionDescription.dataset.richProjectId = "_site-sections";
6126:   sectionTitle.value = subsection.title || "";
6127:   populateStandaloneRichEditor(sectionDescription, subsection.richDescription, subsection.description ||
"");
6128:   const saved = await dialogValue(sectionDialog);
6129:   if (!saved) return null;
6130:   const title = sectionTitle.value.trim();
6131:   if (!title) {
6132:     setStatus("Type a subsection title before saving.");
6133:     return null;
6134:   }
6135:   const richDescription = extractRichSummary(sectionDescription);
6136:   return {
6137:     description: plainTextFromRich(richDescription, "\n").trim(),
6138:     id: subsection.id || slugify(title),
6139:     links: subsection.links || [],
6140:     richDescription,
6141:     title
6142:   };
6143: }
```

dialogLinkValue (template-preview.js, lines 6145-6169)

dialogLinkValue part of the private builder frontend and editing experience. In this file, it appears around lines 6145-6169 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references removeEventListener, resolve, trim, setStatus, normalizeLinkTarget, addEventListener, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 6146: return new Promise((resolve) => {; line 6155: return;; line 6162: return;.

Important local values include onClose at line 6151; label at line 6157; url at line 6158. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6145: function dialogLinkValue(link = {}) {
6146:   return new Promise((resolve) => {
6147:     siteLinkEyebrow.textContent = link.url ? "Edit portfolio link" : "Add portfolio link";
6148:     siteLinkTitle.textContent = link.url ? "Edit link" : "Add link";
6149:     siteLinkLabel.value = link.label || "";
6150:     siteLinkUrl.value = link.url || "";
6151:     const onClose = () => {
6152:       siteLinkDialog.removeEventListener("close", onClose);
6153:       if (siteLinkDialog.returnValue !== "save") {
6154:         resolve(null);
6155:         return;
6156:       }
6157:       const label = siteLinkLabel.value.trim();
6158:       const url = siteLinkUrl.value.trim();
6159:       if (!label || !url) {
6160:         setStatus("Type both a link label and URL before saving.");
6161:         resolve(null);
6162:         return;
6163:       }
6164:       resolve({ ...link, label, url: normalizeLinkTarget(url, { assumeWeb: true }) });
6165:     };
6166:     siteLinkDialog.addEventListener("close", onClose);
6167:     siteLinkDialog.showModal();
6168:   });
6169: }
```

onClose (template-preview.js, lines 6151-6165)

onClose part of the private builder frontend and editing experience. In this file, it appears around lines 6151-6165 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references removeEventListener, resolve, trim, setStatus, and normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6155: return;; line 6162: return;.

Important local values include onClose at line 6151; label at line 6157; url at line 6158. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6151:     const onClose = () => {
6152:       siteLinkDialog.removeEventListener("close", onClose);
6153:       if (siteLinkDialog.returnValue !== "save") {
6154:         resolve(null);
6155:         return;
6156:       }
6157:       const label = siteLinkLabel.value.trim();
6158:       const url = siteLinkUrl.value.trim();
6159:       if (!label || !url) {
6160:         setStatus("Type both a link label and URL before saving.");
6161:         resolve(null);
6162:         return;
6163:       }
6164:       resolve({ ...link, label, url: normalizeLinkTarget(url, { assumeWeb: true }) });
6165:     };
```

addSiteSection (template-preview.js, lines 6171-6191)

addSiteSection part of the private builder frontend and editing experience. In this file, it appears around lines 6171-6191 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureSiteSections, siteSectionDialogValue, slugify, push, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6174: if (!input) return;.

Important local values include input at line 6173; id at line 6175. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6171: async function addSiteSection() {
6172:   ensureSiteSections();
6173:   const input = await siteSectionDialogValue();
6174:   if (!input) return;
6175:   const id = input.id || slugify(input.title);
6176:   catalog.siteSections.push({
6177:     id,
6178:     title: input.title,
6179:     description: input.description,
6180:     richDescription: input.richDescription,
6181:     visible: false,
6182:     backgroundImage: "",
6183:     links: [],
6184:     assets: [],
6185:     subsections: []
6186:   });
6187:   markDraftNeedsSave();
6188:   setStatus("Portfolio area added locally.");
6189:   scheduleAutosave();
6190:   renderAll();
6191: }
```

editSiteSection (template-preview.js, lines 6193-6206)

editSiteSection part of the private builder frontend and editing experience. In this file, it appears around lines 6193-6206 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureSiteSections, find, siteSectionDialogValue, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6196: if (!section) return;; line 6198: if (!input) return;.

Important local values include section at line 6195; input at line 6197. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6193: async function editSiteSection(sectionId) {
6194:   ensureSiteSections();
6195:   const section = catalog.siteSections.find((item) => item.id === sectionId);
6196:   if (!section) return;
6197:   const input = await siteSectionDialogValue(section);
6198:   if (!input) return;
6199:   section.title = input.title;
6200:   section.description = input.description;
6201:   section.richDescription = input.richDescription;
6202:   markDraftNeedsSave();
6203:   setStatus("Portfolio area edited locally.");
6204:   scheduleAutosave();
6205:   renderAll();
6206: }
```

addSiteSubsection (template-preview.js, lines 6208-6219)

addSiteSubsection part of the private builder frontend and editing experience. In this file, it appears around lines 6208-6219 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, siteSubsectionDialogValue, push, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6210: if (!section) return;; line 6212: if (!input) return;.

Important local values include section at line 6209; input at line 6211. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6208: async function addSiteSubsection(sectionId) {
6209:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6210:   if (!section) return;
6211:   const input = await siteSubsectionDialogValue();
6212:   if (!input) return;
6213:   section.subsections = section.subsections || [];
6214:   section.subsections.push(input);
6215:   markDraftNeedsSave();
6216:   setStatus("Portfolio subsection added locally.");
6217:   scheduleAutosave();
6218:   renderAll();
6219: }
```

section (template-preview.js, lines 6209-6218)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6209-6218 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, siteSubsectionDialogValue, push, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6210: if (!section) return;; line 6212: if (!input) return;.

Important local values include section at line 6209; input at line 6211. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6209:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6210:   if (!section) return;
6211:   const input = await siteSubsectionDialogValue();
6212:   if (!input) return;
6213:   section.subsections = section.subsections || [];
6214:   section.subsections.push(input);
6215:   markDraftNeedsSave();
6216:   setStatus("Portfolio subsection added locally.");
```

```
6217:   scheduleAutosave();
6218:   renderAll();
```

editSiteSubsection (template-preview.js, lines 6221-6232)

editSiteSubsection part of the private builder frontend and editing experience. In this file, it appears around lines 6221-6232 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, siteSubsectionDialogValue, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6224: if (!section || !subsection) return;; line 6226: if (!input) return;.

Important local values include section at line 6222; subsection at line 6223; input at line 6225. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6221: async function editSiteSubsection(sectionId, index) {
6222:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6223:   const subsection = section?.subsections?.[Number(index)];
6224:   if (!section || !subsection) return;
6225:   const input = await siteSubsectionDialogValue(subsection);
6226:   if (!input) return;
6227:   section.subsections[Number(index)] = input;
6228:   markDraftNeedsSave();
6229:   setStatus("Portfolio subsection edited locally.");
6230:   scheduleAutosave();
6231:   renderAll();
6232: }
```

section (template-preview.js, lines 6222-6231)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6222-6231 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, siteSubsectionDialogValue, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6224: if (!section || !subsection) return;; line 6226: if (!input) return;.

Important local values include section at line 6222; subsection at line 6223; input at line 6225. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6222:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6223:   const subsection = section?.subsections?.[Number(index)];
6224:   if (!section || !subsection) return;
6225:   const input = await siteSubsectionDialogValue(subsection);
6226:   if (!input) return;
6227:   section.subsections[Number(index)] = input;
6228:   markDraftNeedsSave();
6229:   setStatus("Portfolio subsection edited locally.");
6230:   scheduleAutosave();
6231:   renderAll();
```

deleteSiteSubsection (template-preview.js, lines 6234-6248)

deleteSiteSubsection part of the private builder frontend and editing experience. In this file, it appears around lines 6234-6248 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, openDeleteConfirm, splice, markDraftNeedsSave, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6237: if (!section || !subsection) return;.

Important local values include section at line 6235; subsection at line 6236. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6234: function deleteSiteSubsection(sectionId, index) {
6235:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6236:   const subsection = section?.subsections?.[Number(index)];
6237:   if (!section || !subsection) return;
6238:   openDeleteConfirm({
6239:     title: "Delete subsection",
6240:     message: `Are you sure you want to delete "${subsection.title}"?`,
6241:     onConfirm: () => {
6242:       section.subsections.splice(Number(index), 1);
6243:       markDraftNeedsSave();
6244:       scheduleAutosave();
6245:       renderAll();
6246:     }
6247:   });
6248: }
```

section (template-preview.js, lines 6235-6247)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6235-6247 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, openDeleteConfirm, splice, markDraftNeedsSave, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6237: if (!section || !subsection) return;.

Important local values include section at line 6235; subsection at line 6236. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6235:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6236:   const subsection = section?.subsections?.[Number(index)];
6237:   if (!section || !subsection) return;
6238:   openDeleteConfirm({
6239:     title: "Delete subsection",
6240:     message: `Are you sure you want to delete "${subsection.title}"?`,
6241:     onConfirm: () => {
6242:       section.subsections.splice(Number(index), 1);
6243:       markDraftNeedsSave();
6244:       scheduleAutosave();
6245:       renderAll();
6246:     }
6247:   });
```

addSiteSectionLink (template-preview.js, lines 6250-6261)

addSiteSectionLink part of the private builder frontend and editing experience. In this file, it appears around lines 6250-6261 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, dialogLinkValue, push, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6252: if (!section) return;; line 6254: if (!input) return;.

Important local values include section at line 6251; input at line 6253. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6250: async function addSiteSectionLink(sectionId) {
6251:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6252:   if (!section) return;
6253:   const input = await dialogLinkValue();
6254:   if (!input) return;
6255:   section.links = section.links || [];
6256:   section.links.push(input);
6257:   markDraftNeedsSave();
6258:   setStatus("Portfolio link added locally.");
6259:   scheduleAutosave();
6260:   renderAll();
```

```
6261: }
```

section (template-preview.js, lines 6251-6260)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6251-6260 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, dialogLinkValue, push, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6252: if (!section) return;; line 6254: if (!input) return;.

Important local values include section at line 6251; input at line 6253. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6251:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6252:   if (!section) return;
6253:   const input = await dialogLinkValue();
6254:   if (!input) return;
6255:   section.links = section.links || [];
6256:   section.links.push(input);
6257:   markDraftNeedsSave();
6258:   setStatus("Portfolio link added locally.");
6259:   scheduleAutosave();
6260:   renderAll();
```

editSiteSectionLink (template-preview.js, lines 6263-6274)

editSiteSectionLink part of the private builder frontend and editing experience. In this file, it appears around lines 6263-6274 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, dialogLinkValue, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6266: if (!section || !link) return;; line 6268: if (!input) return;.

Important local values include section at line 6264; link at line 6265; input at line 6267. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6263: async function editSiteSectionLink(sectionId, index) {
6264:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6265:   const link = section?.links?.[Number(index)];
6266:   if (!section || !link) return;
6267:   const input = await dialogLinkValue(link);
6268:   if (!input) return;
6269:   section.links[Number(index)] = input;
6270:   markDraftNeedsSave();
6271:   setStatus("Portfolio link edited locally.");
6272:   scheduleAutosave();
6273:   renderAll();
6274: }
```

section (template-preview.js, lines 6264-6273)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6264-6273 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, dialogLinkValue, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6266: if (!section || !link) return;; line 6268: if (!input) return;.

Important local values include section at line 6264; link at line 6265; input at line 6267. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6264:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6265:   const link = section?.links?.[Number(index)];
6266:   if (!section || !link) return;
6267:   const input = await dialogLinkValue(link);
6268:   if (!input) return;
6269:   section.links[Number(index)] = input;
6270:   markDraftNeedsSave();
6271:   setStatus("Portfolio link edited locally.");
6272:   scheduleAutosave();
6273:   renderAll();
```

deleteSiteSectionLink (template-preview.js, lines 6276-6284)

deleteSiteSectionLink part of the private builder frontend and editing experience. In this file, it appears around lines 6276-6284 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, splice, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6278: if (!section?.links?.[Number(index)]) return;.

Important local values include section at line 6277. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6276: function deleteSiteSectionLink(sectionId, index) {
6277:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6278:   if (!section?.links?.[Number(index)]) return;
6279:   section.links.splice(Number(index), 1);
6280:   markDraftNeedsSave();
6281:   setStatus("Portfolio link deleted locally.");
6282:   scheduleAutosave();
6283:   renderAll();
6284: }
```

section (template-preview.js, lines 6277-6283)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6277-6283 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, splice, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6278: if (!section?.links?.[Number(index)]) return;.

Important local values include section at line 6277. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6277:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6278:   if (!section?.links?.[Number(index)]) return;
6279:   section.links.splice(Number(index), 1);
6280:   markDraftNeedsSave();
6281:   setStatus("Portfolio link deleted locally.");
6282:   scheduleAutosave();
6283:   renderAll();
```

toggleSiteSectionVisibility (template-preview.js, lines 6286-6294)

toggleSiteSectionVisibility part of the private builder frontend and editing experience. In this file, it appears around lines 6286-6294 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6288: if (!section) return;.

Important local values include section at line 6287. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6286: function toggleSiteSectionVisibility(sectionId) {
6287:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6288:   if (!section) return;
6289:   section.visible = section.visible === false;
6290:   markDraftNeedsSave();
6291:   setStatus(section.visible ? "Portfolio area will show on the website preview." : "Portfolio area hidden
from the website preview.");
6292:   scheduleAutosave();
6293:   renderAll();
6294: }
```

section (template-preview.js, lines 6287-6293)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6287-6293 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6288: if (!section) return;.

Important local values include section at line 6287. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6287:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6288:   if (!section) return;
6289:   section.visible = section.visible === false;
6290:   markDraftNeedsSave();
6291:   setStatus(section.visible ? "Portfolio area will show on the website preview." : "Portfolio area hidden
from the website preview.");
6292:   scheduleAutosave();
6293:   renderAll();
```

deleteSiteSection (template-preview.js, lines 6296-6310)

deleteSiteSection part of the private builder frontend and editing experience. In this file, it appears around lines 6296-6310 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, openDeleteConfirm, filter, markDraftNeedsSave, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6298: if (!section) return;.

Important local values include section at line 6297. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6296: function deleteSiteSection(sectionId) {
6297:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6298:   if (!section) return;
6299:   openDeleteConfirm({
6300:     title: "Delete portfolio area",
6301:     message: `Are you sure you want to delete "${section.title}"?`,
6302:     onConfirm: () => {
6303:       catalog.siteSections = (catalog.siteSections || []).filter((item) => item.id !== sectionId);
6304:       savedPortfolioCatalog.siteSections = (savedPortfolioCatalog.siteSections || []).filter((item) =>
item.id !== sectionId);
6305:       markDraftNeedsSave();
6306:       scheduleAutosave();
6307:       renderAll();
6308:     }
6309:   });
6310: }
```

section (template-preview.js, lines 6297-6309)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6297-6309 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, openDeleteConfirm, filter, markDraftNeedsSave, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6298: if (!section) return;.

Important local values include section at line 6297. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 6297:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
 6298:   if (!section) return;
 6299:   openDeleteConfirm({
 6300:     title: "Delete portfolio area",
 6301:     message: `Are you sure you want to delete "${section.title}"?`,
 6302:     onConfirm: () => {
 6303:       catalog.siteSections = (catalog.siteSections || []).filter((item) => item.id !== sectionId);
 6304:       savedPortfolioCatalog.siteSections = (savedPortfolioCatalog.siteSections || []).filter((item) =>
item.id !== sectionId);
 6305:       markDraftNeedsSave();
 6306:       scheduleAutosave();
 6307:       renderAll();
 6308:     }
 6309:   });
```

setSiteSectionBackground (template-preview.js, lines 6312-6343)

setSiteSectionBackground part of the private builder frontend and editing experience. In this file, it appears around lines 6312-6343 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, chooseFile, startsWith, setStatus, readFileAsDataUrl, fetch, now, stringify, json, markDraftNeedsSave, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 6314: if (!section) return;; line 6316: if (!file) return;; line 6319: return;; line 6336: return;.

Important local values include section at line 6313; file at line 6315; data at line 6321; response at line 6322; result at line 6333. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6312: async function setSiteSectionBackground(sectionId) {
6313:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6314:   if (!section) return;
6315:   const file = await chooseFile();
6316:   if (!file) return;
6317:   if (!file.type.startsWith("image/")) {
6318:     setStatus("Choose an image file for a section background.");
6319:     return;
6320:   }
6321:   const data = await readFileAsDataUrl(file);
6322:   const response = await fetch(`/api/upload?t=${Date.now()}`, {
6323:     method: "POST",
6324:     cache: "no-store",
6325:     headers: { "Content-Type": "application/json" },
6326:     body: JSON.stringify({
6327:       projectId: "_site-sections",
6328:       section: section.id,
6329:       fileName: file.name,
6330:       data
6331:     })
6332:   });
6333:   const result = await response.json();
6334:   if (!response.ok) {
6335:     setStatus(result.error || "Background upload failed.");
6336:     return;
6337:   }
6338:   section.backgroundImage = result.url;
6339:   markDraftNeedsSave();
6340:   setStatus("Portfolio area background saved locally.");
6341:   scheduleAutosave();
6342:   renderAll();
6343: }
```

section (template-preview.js, lines 6313-6320)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6313-6320 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, chooseFile, startsWith, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 6314: if (!section) return;; line 6316: if (!file) return;; line 6319: return;.

Important local values include section at line 6313; file at line 6315. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6313:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6314:   if (!section) return;
6315:   const file = await chooseFile();
6316:   if (!file) return;
6317:   if (!file.type.startsWith("image/")) {
6318:     setStatus("Choose an image file for a section background.");
6319:     return;
6320:   }
```

clearSiteSectionBackground (template-preview.js, lines 6345-6353)

clearSiteSectionBackground part of the private builder frontend and editing experience. In this file, it appears around lines 6345-6353 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6347: if (!section) return;.

Important local values include section at line 6346. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6345: function clearSiteSectionBackground(sectionId) {
6346:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6347:   if (!section) return;
6348:   section.backgroundImage = "";
6349:   markDraftNeedsSave();
6350:   setStatus("Portfolio area background cleared locally.");
6351:   scheduleAutosave();
6352:   renderAll();
6353: }
```

section (template-preview.js, lines 6346-6352)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6346-6352 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6347: if (!section) return;.

Important local values include section at line 6346. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6346:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6347:   if (!section) return;
6348:   section.backgroundImage = "";
6349:   markDraftNeedsSave();
6350:   setStatus("Portfolio area background cleared locally.");
```

```
 6351:   scheduleAutosave();
 6352:   renderAll();
```

renderSectionTabs (template-preview.js, lines 6355-6369)

renderSectionTabs renders ui, markup, or display output. In this file, it appears around lines 6355-6369 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sectionOptions, map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6358: return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 6355: function renderSectionTabs(project) {
 6356:   if (!project) {
 6357:     sectionTabs.innerHTML = "";
 6358:     return;
 6359:   }
 6360:
 6361:   sectionTabs.innerHTML = `
 6362:     ${sectionOptions(project).map((section) => `
 6363:       <button class="${section.id === activeSectionId ? "active" : ""}" type="button" data-section-
id="${section.id}">
 6364:         ${section.label}
 6365:       </button>
 6366:     `).join("")}
 6367:     <button type="button" data-add-section="true">Add section</button>
 6368:   `;
 6369: }
```

renderTextArray (template-preview.js, lines 6371-6389)

renderTextArray renders ui, markup, or display output. In this file, it appears around lines 6371-6389 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, join, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6373: return `.

Important local values include items at line 6372. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 6371: function renderTextArray(project, key, label) {
 6372:   const items = project[key] || [];
 6373:   return `
 6374:     <div class="section-actions">
 6375:       <button class="button primary" type="button" data-open-editor="text-array" data-key="${key}" data-
mode="add">Add ${label}</button>
 6376:     </div>
 6377:     <div class="builder-items">
 6378:       ${items.map((item, index) => `
 6379:         <article class="builder-item">
 6380:           <div>
 6381:             <strong>${typeof item === "string" ? item : item.title || item.name || item.label ||
""}</strong>
 6382:           </div>
 6383:           <button type="button" data-open-editor="text-array" data-key="${key}" data-mode="edit" data-
index="${index}">Edit</button>
 6384:           <button class="danger-icon" type="button" data-delete-array="${key}" data-
index="${index}">Delete</button>
 6385:         </article>
 6386:       `).join("") || `<p class="evidence-empty">No ${label.toLowerCase()} added yet.</p>`}
 6387:     </div>
 6388:   `;
 6389: }
```

toolName (template-preview.js, lines 6391-6393)

toolName part of the private builder frontend and editing experience. In this file, it appears around lines 6391-6393 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6392: return typeof item === "string" ? item : item.name || item.title || item.label || "";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6391: function toolName(item) {
6392:   return typeof item === "string" ? item : item.name || item.title || item.label || "";
6393: }
```

toolDescription (template-preview.js, lines 6395-6397)

toolDescription part of the private builder frontend and editing experience. In this file, it appears around lines 6395-6397 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6396: return typeof item === "string" ? "" : item.description || item.usedFor || "";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6395: function toolDescription(item) {
6396:   return typeof item === "string" ? "" : item.description || item.usedFor || "";
6397: }
```

renderToolsSection (template-preview.js, lines 6399-6420)

renderToolsSection renders ui, markup, or display output. In this file, it appears around lines 6399-6420 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, toolName, richHasContent, renderRichContent, toolDescription, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6401: return `.

Important local values include items at line 6400. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6399: function renderToolsSection(project) {
6400:   const items = project.tools || [];
6401:   return `
6402:     <div class="section-actions">
6403:       <button class="button primary" type="button" data-open-editor="tool" data-mode="add">Add
tool</button>
6404:     </div>
6405:     <div class="builder-items tool-items">
6406:       ${items.map((item, index) => `
6407:         <article class="builder-item tool-item">
6408:           <div>
6409:             <strong>${toolName(item) || "Untitled tool"}</strong>
6410:           </div>
6411:           ${richHasContent(item?.richDescription)
6412:             ? renderRichContent(item.richDescription, toolDescription(item))
6413:             : `<p>${toolDescription(item) || "No usage description added yet."}</p>`}
6414:           <button type="button" data-open-editor="tool" data-mode="edit" data-
index="${index}">Edit</button>
6415:           <button class="danger-icon" type="button" data-delete-array="tools" data-
index="${index}">Delete</button>
```

```
6416:         </article>
6417:       `).join("") || `<p class="evidence-empty">No tools added yet.</p>`}
6418:     </div>
6419:   `;
6420: }
```

renderObjectSection (template-preview.js, lines 6422-6450)

renderObjectSection renders ui, markup, or display output. In this file, it appears around lines 6422-6450 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, richHasContent, renderRichContent, renderMultilineInlineText, join, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6424: return `; line 6434: return `.

Important local values include items at line 6423; title at line 6431; url at line 6432; description at line 6433. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6422: function renderObjectSection(project, key, folder, label) {
6423:   const items = project[key] || [];
6424:   return `
6425:     <div class="section-actions">
6426:       <button class="button primary" type="button" data-open-editor="object" data-key="${key}" data-
mode="add">Add text item</button>
6427:       ${folder ? `<button class="button secondary" type="button" data-upload="${key}" data-
folder="${folder}">Add file or image</button>` : ""}
6428:     </div>
6429:     <div class="builder-items">
6430:       ${items.map((item, index) => {
6431:         const title = item.title || item.name || item.label || "Untitled item";
6432:         const url = item.url || item.artifact || "";
6433:         const description = item.description || item.caption || item.result || item.method || "";
6434:         return `
6435:           <article class="builder-item">
6436:             <div>
6437:               <strong>${title}</strong>
6438:               <span>${url || "No file attached"}</span>
6439:               ${richHasContent(item.richDescription)
6440:                 ? renderRichContent(item.richDescription, description)
6441:                 : description ? `<p>${renderMultilineInlineText(description)}</p>` : ""}
6442:             </div>
6443:             <button type="button" data-open-editor="object" data-key="${key}" data-mode="edit" data-
index="${index}">Edit</button>
6444:             <button class="danger-icon" type="button" data-delete-item="${key}" data-
index="${index}">Delete</button>
6445:           </article>
6446:         `;
6447:       }).join("") || `<p class="evidence-empty">No ${label.toLowerCase()} added yet.</p>`}
6448:     </div>
6449:   `;
6450: }
```

designArray (template-preview.js, lines 6452-6458)

designArray part of the private builder frontend and editing experience. In this file, it appears around lines 6452-6458 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureDesignModel, getByPath, isArray, and setByPath. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6455: if (Array.isArray(items)) return items;; line 6457: return getByPath(project, pathValue);.

Important local values include items at line 6454. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6452: function designArray(project, pathValue) {
6453:   ensureDesignModel(project);
6454:   const items = getByPath(project, pathValue);
6455:   if (Array.isArray(items)) return items;
6456:   setByPath(project, pathValue, []);
6457:   return getByPath(project, pathValue);
```

```
6458: }
```

makeDesignItem (template-preview.js, lines 6460-6466)

makeDesignItem part of the private builder frontend and editing experience. In this file, it appears around lines 6460-6466 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 6461: if (kind === "reference") return { title, description, url, type: "Reference", status: url ? "linked" : "draft" };; line 6462: if (kind === "analysis") return { title, description, type: "Math / Analysis", status: "draft" };; line 6463: if (kind === "simulation-file") return { title, description, url, type: "Simulation file", status: url ? "uploaded" : "draft" };; line 6464: if (kind === "simulation-result") return { title, description, url, type: "Simulation result", status: url ? "uploaded" : "draft" };. There are 1 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6460: function makeDesignItem(kind, title, description = "", url = "") {
6461:   if (kind === "reference") return { title, description, url, type: "Reference", status: url ? "linked" :
"draft" };
6462:   if (kind === "analysis") return { title, description, type: "Math / Analysis", status: "draft" };
6463:   if (kind === "simulation-file") return { title, description, url, type: "Simulation file", status: url
? "uploaded" : "draft" };
6464:   if (kind === "simulation-result") return { title, description, url, type: "Simulation result", status:
url ? "uploaded" : "draft" };
6465:   return { title, description, url, type: "Document", status: url ? "uploaded" : "draft" };
6466: }
```

renderDesignObjectSection (template-preview.js, lines 6468-6496)

renderDesignObjectSection renders ui, markup, or display output. In this file, it appears around lines 6468-6496 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references designArray, map, richHasContent, renderRichContent, renderMultilineInlineText, join, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6470: return `; line 6480: return `.

Important local values include items at line 6469; title at line 6477; url at line 6478; description at line 6479. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6468: function renderDesignObjectSection(project, pathValue, folder, label, kind = "document") {
6469:   const items = designArray(project, pathValue);
6470:   return `
6471:     <div class="section-actions">
6472:       <button class="button primary" type="button" data-open-editor="design-object" data-design-
path="${pathValue}" data-design-kind="${kind}" data-mode="add">Add ${label}</button>
6473:       ${folder ? `<button class="button secondary" type="button" data-upload-design="${pathValue}" data-
folder="${folder}" data-design-kind="${kind}">Add file</button>` : ""}
6474:     </div>
6475:     <div class="builder-items">
6476:       ${items.map((item, index) => {
6477:         const title = item.title || item.name || item.label || "Untitled item";
6478:         const url = item.url || item.artifact || "";
6479:         const description = item.description || item.caption || item.result || item.method || "";
6480:         return `
6481:           <article class="builder-item">
6482:             <div>
6483:               <strong>${title}</strong>
6484:               <span>${url || "No file attached"}</span>
6485:               ${richHasContent(item.richDescription)
6486:                 ? renderRichContent(item.richDescription, description)
6487:                 : description ? `<p>${renderMultilineInlineText(description)}</p>` : ""}
6488:             </div>
6489:             <button type="button" data-open-editor="design-object" data-design-path="${pathValue}" data-
design-kind="${kind}" data-mode="edit" data-index="${index}">Edit</button>
6490:             <button class="danger-icon" type="button" data-delete-design-item="${pathValue}" data-
index="${index}">Delete</button>
6491:           </article>
```

```
6492:         `;
6493:       }).join("") || `<p class="evidence-empty">No ${label.toLowerCase()} added yet.</p>`}
6494:     </div>
6495:   `;
6496: }
```

renderDesignSummaryField (template-preview.js, lines 6498-6530)

renderDesignSummaryField renders ui, markup, or display output. In this file, it appears around lines 6498-6530 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getByPath, summaryRichPathFromTextPath, escapeHtml, and overviewFolderFromPath. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6504: return `.

Important local values include textValue at line 6499; richPath at line 6500; richValue at line 6501; hasValue at line 6502. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6498: function renderDesignSummaryField(project, pathValue, label, placeholder) {
6499:     const textValue = getByPath(project, pathValue) || "";
6500:     const richPath = summaryRichPathFromTextPath(pathValue);
6501:     const richValue = getByPath(project, richPath);
6502:     const hasValue = Boolean(textValue || richValue?.blocks?.length);
6503:
6504:     return `
6505:     <div class="wide-field design-summary-field rich-design-summary-field">
6506:       <div class="summary-builder-heading">
6507:         <span>
6508:           ${label}
6509:           ${hasValue ? `<button class="inline-danger-button" type="button" data-clear-design-
summary="${pathValue}">Clear overview</button>` : ""}
6510:         </span>
6511:       </div>
6512:
6513:       <div class="rich-summary-panel">
6514:         <p class="rich-editor-hint">Right-click to add images, code blocks, apply bold, color, font,
numeric size, alignment, or formulas.</p>
6515:
6516:         <div
6517:           class="rich-summary-editor"
6518:           data-rich-editor="section-overview"
6519:           data-rich-text-path="${escapeHtml(pathValue)}"
6520:           data-rich-path="${escapeHtml(richPath)}"
6521:           data-rich-folder="${escapeHtml(overviewFolderFromPath(pathValue))}"
6522:           data-placeholder="${escapeHtml(placeholder || "")}"
6523:           contenteditable="true"
6524:           spellcheck="true"
6525:           aria-label="${escapeHtml(label)} rich editor"
6526:         ></div>
6527:       </div>
6528:     </div>
6529:   `;
6530: }
```

designPathHasContent (template-preview.js, lines 6532-6536)

designPathHasContent part of the private builder frontend and editing experience. In this file, it appears around lines 6532-6536 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getByPath, isArray, some, itemTitle, richHasContent, and summaryRichPathFromTextPath. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6534: if (Array.isArray(value)) return value.some((item) => itemTitle(item) || item?.description || item?.url || item?.artifact || richHasContent(item?.richDescription));; line 6535: return Boolean(value || richHasContent(getByPath(project, summaryRichPathFromTextPath(pathValue))));.

Important local values include value at line 6533. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6532: function designPathHasContent(project, pathValue) {
6533:   const value = getByPath(project, pathValue);
```

```
6534:   if (Array.isArray(value)) return value.some((item) => itemTitle(item) || item?.description || item?.url
|| item?.artifact || richHasContent(item?.richDescription));
6535:   return Boolean(value || richHasContent(getByPath(project, summaryRichPathFromTextPath(pathValue))));
6536: }
```

renderEmptyDynamicSectionPrompt (template-preview.js, lines 6538-6548)

renderEmptyDynamicSectionPrompt renders ui, markup, or display output. In this file, it appears around lines 6538-6548 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6539: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6538: function renderEmptyDynamicSectionPrompt(label = "section") {
6539:   return `
6540:     <div class="compile-code-empty">
6541:       <h3>No ${escapeHtml(label)} content added yet</h3>
6542:       <p>Create your own sections and subsections with the Add section button. Empty predefined groups
stay hidden so the project structure is controlled by you.</p>
6543:       <div class="section-actions">
6544:         <button class="button primary" type="button" data-add-section="true">Add section</button>
6545:       </div>
6546:     </div>
6547:   `;
6548: }
```

renderAnalogDesignWorkspace (template-preview.js, lines 6550-6593)

renderAnalogDesignWorkspace renders ui, markup, or display output. In this file, it appears around lines 6550-6593 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureDesignModel, designPathHasContent, push, renderDesignSummaryField, renderDesignObjectSection, renderObjectSection, join, renderEmptyDynamicSectionPrompt, and renderPendingEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6587: return `.

Important local values include cards at line 6552. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6550: function renderAnalogDesignWorkspace(project) {
6551:   ensureDesignModel(project);
6552:   const cards = [];
6553:   if (designPathHasContent(project, "design.brief.summary") || designPathHasContent(project,
"design.brief.files")) {
6554:     cards.push(`
6555:       <section class="design-workspace-card">
6556:         <h3>Design Overview</h3>
6557:         ${renderDesignSummaryField(project, "design.brief.summary", "Design overview", "State the analog
design objective, topology, constraints, and design decisions.")}
6558:         ${renderDesignObjectSection(project, "design.brief.files", "design-brief", "brief file",
"document")}
6559:       </section>
6560:     `);
6561:   }
6562:   if (
6563:     designPathHasContent(project, "design.documentation.summary") ||
6564:     designPathHasContent(project, "design.documentation.files") ||
6565:     designPathHasContent(project, "design.documentation.references") ||
6566:     designPathHasContent(project, "design.documentation.mathAnalysis")
6567:   ) {
6568:     cards.push(`
6569:       <section class="design-workspace-card">
6570:         <h3>Documentation</h3>
6571:         ${renderDesignSummaryField(project, "design.documentation.summary", "Documentation overview",
"Summarize schematics, notes, datasheets, calculations, and supporting documents.")}
6572:         ${renderDesignObjectSection(project, "design.documentation.files", "documentation", "document",
"document")}
6573:         ${designPathHasContent(project, "design.documentation.references") ?
```

```
`<h4>References</h4>${renderDesignObjectSection(project, "design.documentation.references", "", "reference",
"reference")}` : ""}
 6574:         ${designPathHasContent(project, "design.documentation.mathAnalysis") ? `<h4>Math /
Analysis</h4>${renderDesignObjectSection(project, "design.documentation.mathAnalysis", "", "analysis item",
"analysis")}` : ""}
 6575:       </section>
 6576:     `);
 6577:   }
 6578:   if ((project.pcbs || []).length || (project.media || []).length) {
 6579:     cards.push(`
 6580:       <section class="design-workspace-card">
 6581:         <h3>PCB and Visual Evidence</h3>
 6582:         ${(project.pcbs || []).length ? renderObjectSection(project, "pcbs", "pcbs", "PCBs") : ""}
 6583:         ${(project.media || []).length ? renderObjectSection(project, "media", "images", "Images") : ""}
 6584:       </section>
 6585:     `);
 6586:   }
 6587:   return `
 6588:     <div class="section-stack analog-design-workspace">
 6589:       ${cards.join("") || renderEmptyDynamicSectionPrompt("design")}
 6590:       ${renderPendingEditor()}
 6591:     </div>
 6592:   `;
 6593: }
```

renderAnalogSimulationWorkspace (template-preview.js, lines 6595-6628)

renderAnalogSimulationWorkspace renders ui, markup, or display output. In this file, it appears around lines 6595-6628 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureDesignModel, designPathHasContent, push, renderDesignSummaryField, renderDesignObjectSection, join, renderEmptyDynamicSectionPrompt, and renderPendingEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6622: return `.

Important local values include cards at line 6597. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 6595: function renderAnalogSimulationWorkspace(project) {
 6596:   ensureDesignModel(project);
 6597:   const cards = [];
 6598:   if (designPathHasContent(project, "design.simulation.summary")) {
 6599:     cards.push(`
 6600:       <section class="design-workspace-card">
 6601:         <h3>Simulation Overview</h3>
 6602:         ${renderDesignSummaryField(project, "design.simulation.summary", "Simulation overview", "Explain
what was simulated, why it mattered, and what the results proved.")}
 6603:       </section>
 6604:     `);
 6605:   }
 6606:   if (designPathHasContent(project, "design.simulation.files")) {
 6607:     cards.push(`
 6608:       <section class="design-workspace-card">
 6609:         <h3>Simulation Files</h3>
 6610:         ${renderDesignObjectSection(project, "design.simulation.files", "simulations", "simulation file",
"simulation-file")}
 6611:       </section>
 6612:     `);
 6613:   }
 6614:   if (designPathHasContent(project, "design.simulation.results")) {
 6615:     cards.push(`
 6616:       <section class="design-workspace-card">
 6617:         <h3>Simulation Results</h3>
 6618:         ${renderDesignObjectSection(project, "design.simulation.results", "simulation-results",
"simulation result", "simulation-result")}
 6619:       </section>
 6620:     `);
 6621:   }
 6622:   return `
 6623:     <div class="section-stack analog-design-workspace">
 6624:       ${cards.join("") || renderEmptyDynamicSectionPrompt("simulation")}
 6625:       ${renderPendingEditor()}
 6626:     </div>
 6627:   `;
 6628: }
```

pendingEditorUsesRichDescription (template-preview.js, lines 6630-6632)

pendingEditorUsesRichDescription part of the private builder frontend and editing experience. In this file, it appears around lines 6630-6632 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6631: return Boolean(editor && editor.type !== "text-array");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6630: function pendingEditorUsesRichDescription(editor = pendingEditor) {
6631:   return Boolean(editor && editor.type !== "text-array");
6632: }
```

pendingEditorFolder (template-preview.js, lines 6634-6641)

pendingEditorFolder part of the private builder frontend and editing experience. In this file, it appears around lines 6634-6641 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slugify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 6635: if (!editor) return "content";; line 6636: if (editor.type === "custom-section") return `custom-${slugify(editor.sectionId || "section")}-overview`;; line 6637: if (editor.type === "custom") return `custom-${slugify(editor.sectionId || "section")}-subsection`;; line 6638: if (editor.type === "design-object") return `design-${slugify(editor.kind || "content")}`;. There are 2 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6634: function pendingEditorFolder(editor = pendingEditor) {
6635:   if (!editor) return "content";
6636:   if (editor.type === "custom-section") return `custom-${slugify(editor.sectionId ||
"section")}-overview`;
6637:   if (editor.type === "custom") return `custom-${slugify(editor.sectionId || "section")}-subsection`;
6638:   if (editor.type === "design-object") return `design-${slugify(editor.kind || "content")}`;
6639:   if (editor.type === "object") return slugify(editor.key || "content");
6640:   return slugify(editor.type || "content");
6641: }
```

renderPendingRichDescriptionEditor (template-preview.js, lines 6643-6666)

renderPendingRichDescriptionEditor renders ui, markup, or display output. In this file, it appears around lines 6643-6666 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, and pendingEditorFolder. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6649: return `.

Important local values include label at line 6644. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6643: function renderPendingRichDescriptionEditor(editor = pendingEditor) {
6644:   const label = editor?.type === "tool"
6645:     ? "What it was used for"
6646:     : editor?.type === "custom-section"
6647:       ? "Section content"
6648:       : "Content";
6649:   return `
6650:     <div class="pending-rich-description rich-design-summary-field">
6651:       <div class="summary-builder-heading"><span>${escapeHtml(label)}</span></div>
6652:       <div class="rich-summary-panel">
6653:         <p class="rich-editor-hint">Right-click to add images, code blocks, apply bold, color, font,
```

```
numeric size, alignment, or formulas.</p>
 6654:         <div
 6655:           class="rich-summary-editor"
 6656:           data-rich-editor="pending-description"
 6657:           data-rich-folder="${escapeHtml(pendingEditorFolder(editor))}"
 6658:           data-placeholder="Type or paste the complete content here."
 6659:           contenteditable="true"
 6660:           spellcheck="true"
 6661:           aria-label="${escapeHtml(label)} rich editor"
 6662:         ></div>
 6663:       </div>
 6664:     </div>
 6665:   `;
 6666: }
```

renderPendingEditor (template-preview.js, lines 6668-6692)

renderPendingEditor renders ui, markup, or display output. In this file, it appears around lines 6668-6692 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderPendingRichDescriptionEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6669: if (!pendingEditor) return "";; line 6674: return `.

Important local values include title at line 6670; isTool at line 6671; isSimpleText at line 6672; isSectionDetails at line 6673. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6668: function renderPendingEditor() {
6669:   if (!pendingEditor) return "";
6670:   const title = pendingEditor.title || "";
6671:   const isTool = pendingEditor.type === "tool";
6672:   const isSimpleText = pendingEditor.type === "text-array";
6673:   const isSectionDetails = pendingEditor.type === "custom-section";
6674:   return `
6675:     <form class="pending-editor" id="pending-editor">
6676:       <div>
6677:         <h3>${pendingEditor.mode === "edit" ? "Edit" : "Add"} ${isTool ? "tool" : isSimpleText ? "item" :
isSectionDetails ? "section" : "subsection"}</h3>
6678:       </div>
6679:       <div class="${isTool ? "tool-editor-grid" : "pending-editor-grid"}">
6680:         <label>
6681:           <span>${isTool ? "Tool name" : "Title"}</span>
6682:           <input name="title" type="text" value="${title}" required>
6683:         </label>
6684:         ${isSimpleText ? "" : renderPendingRichDescriptionEditor(pendingEditor)}
6685:       </div>
6686:       <div class="pending-editor-actions">
6687:         <button class="button primary" type="submit">Save</button>
6688:         <button class="button secondary" type="button" data-cancel-pending="true">Cancel</button>
6689:       </div>
6690:     </form>
6691:   `;
6692: }
```

openPendingEditor (template-preview.js, lines 6694-6701)

openPendingEditor controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 6694-6701 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderSectionContent, selectedProject, requestAnimationFrame, populateRichEditors, querySelector, and scrollIntoView. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6694: function openPendingEditor(config) {
6695:   pendingEditor = config;
6696:   renderSectionContent(selectedProject());
6697:   requestAnimationFrame(() => {
6698:     populateRichEditors(sectionContent);
```

```
6699:     document.querySelector("#pending-editor")?.scrollIntoView({ behavior: "smooth", block: "nearest" });
6700:   });
6701: }
```

savePendingEditor (template-preview.js, lines 6703-6785)

savePendingEditor persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 6703-6785 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, trim, querySelector, extractRichSummary, plainTextFromRich, setStatus, push, makeItemForSection, designArray, makeDesignItem, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 6705: if (!project || !pendingEditor) return;; line 6712: return;; line 6763: if (!section) return;; line 6775: if (!section) return;.

Important local values include project at line 6704; title at line 6706; richEditor at line 6707; richDescription at line 6708; description at line 6709; nextTool at line 6717; key at line 6726; existing at line 6729; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6703: function savePendingEditor(form) {
6704:   const project = selectedProject();
6705:   if (!project || !pendingEditor) return;
6706:   const title = form.elements.title.value.trim();
6707:   const richEditor = form.querySelector("[data-rich-editor='pending-description']");
6708:   const richDescription = richEditor ? extractRichSummary(richEditor) : null;
6709:   const description = richEditor ? plainTextFromRich(richDescription) : "";
6710:   if (!title) {
6711:     setStatus("Type a title before saving.");
6712:     return;
6713:   }
6714:
6715:   if (pendingEditor.type === "tool") {
6716:     project.tools = project.tools || [];
6717:     const nextTool = { name: title, description, richDescription };
6718:     if (pendingEditor.mode === "edit") {
6719:       project.tools[Number(pendingEditor.index)] = nextTool;
6720:     } else {
6721:       project.tools.push(nextTool);
6722:     }
6723:   }
6724:
6725:   if (pendingEditor.type === "object") {
6726:     const key = pendingEditor.key;
6727:     project[key] = project[key] || [];
6728:     if (pendingEditor.mode === "edit") {
6729:       const existing = project[key][Number(pendingEditor.index)] || {};
6730:       const existingUrl = existing.url || existing.artifact || "";
6731:       const nextItem = makeItemForSection(key, title, description, existingUrl);
6732:       project[key][Number(pendingEditor.index)] = { ...existing, ...nextItem, richDescription };
6733:     } else {
6734:       const nextItem = makeItemForSection(key, title, description);
6735:       project[key].push({ ...nextItem, richDescription });
6736:     }
6737:   }
6738:
6739:   if (pendingEditor.type === "design-object") {
6740:     const items = designArray(project, pendingEditor.path);
6741:     const kind = pendingEditor.kind || "document";
6742:     if (pendingEditor.mode === "edit") {
6743:       const existing = items[Number(pendingEditor.index)] || {};
6744:       const existingUrl = existing.url || existing.artifact || "";
6745:       items[Number(pendingEditor.index)] = { ...existing, ...makeDesignItem(kind, title, description,
existingUrl), richDescription };
6746:     } else {
6747:       items.push({ ...makeDesignItem(kind, title, description), richDescription });
6748:     }
6749:   }
6750:
6751:   if (pendingEditor.type === "text-array") {
6752:     const key = pendingEditor.key;
6753:     project[key] = project[key] || [];
6754:     if (pendingEditor.mode === "edit") {
6755:       project[key][Number(pendingEditor.index)] = title;
6756:     } else {
6757:       project[key].push(title);
6758:     }
6759:   }
6760:
6761:   if (pendingEditor.type === "custom") {
6762:     const section = (project.sections || []).find((item) => item.id === pendingEditor.sectionId);
```

```
 6763:     if (!section) return;
 6764:     section.items = section.items || [];
 6765:     const nextItem = { title, description, richDescription, type: "Text", status: "draft" };
 6766:     if (pendingEditor.mode === "edit") {
 6767:       section.items[Number(pendingEditor.index)] = { ...section.items[Number(pendingEditor.index)],
...nextItem };
 6768:     } else {
 6769:       section.items.push(nextItem);
 6770:     }
 6771:   }
 6772:
 6773:   if (pendingEditor.type === "custom-section") {
 6774:     const section = (project.sections || []).find((item) => item.id === pendingEditor.sectionId);
 6775:     if (!section) return;
 6776:     section.title = title;
 6777:     section.description = description;
 6778:     section.richDescription = richDescription;
 6779:   }
 6780:
 6781:   pendingEditor = null;
 6782:   setStatus("Saved in the project editor. Click Save project to include this version in the portfolio
preview.");
 6783:   scheduleAutosave();
 6784:   renderAll();
 6785: }
```

section (template-preview.js, lines 6762-6765)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6762-6765 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6763: if (!section) return;.

Important local values include section at line 6762; nextItem at line 6765. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 6762:     const section = (project.sections || []).find((item) => item.id === pendingEditor.sectionId);
 6763:     if (!section) return;
 6764:     section.items = section.items || [];
 6765:     const nextItem = { title, description, richDescription, type: "Text", status: "draft" };
```

section (template-preview.js, lines 6774-6779)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6774-6779 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6775: if (!section) return;.

Important local values include section at line 6774. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 6774:     const section = (project.sections || []).find((item) => item.id === pendingEditor.sectionId);
 6775:     if (!section) return;
 6776:     section.title = title;
 6777:     section.description = description;
 6778:     section.richDescription = richDescription;
 6779:   }
```

openEditorFromButton (template-preview.js, lines 6787-6879)

openEditorFromButton controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 6787-6879 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, openPendingEditor, toolName, toolDescription, designArray, and find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 6789: if (!project) return;; line 6804: return;; line 6819: return;; line 6836: return;. There are 2 additional return statements in the function body.

Important local values include project at line 6788; type at line 6790; mode at line 6791; index at line 6792; item at line 6795; key at line 6808; item at line 6809; pathValue at line 6823; plus 7 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6787: function openEditorFromButton(button) {
6788:   const project = selectedProject();
6789:   if (!project) return;
6790:   const type = button.dataset.openEditor;
6791:   const mode = button.dataset.mode || "add";
6792:   const index = button.dataset.index;
6793:
6794:   if (type === "tool") {
6795:     const item = mode === "edit" ? project.tools?.[Number(index)] : {};
6796:     openPendingEditor({
6797:       type,
6798:       mode,
6799:       index,
6800:       title: toolName(item),
6801:       description: toolDescription(item),
6802:       richDescription: item?.richDescription || null
6803:     });
6804:     return;
6805:   }
6806:
6807:   if (type === "object") {
6808:     const key = button.dataset.key;
6809:     const item = mode === "edit" ? project[key]?.[Number(index)] : {};
6810:     openPendingEditor({
6811:       type,
6812:       mode,
6813:       key,
6814:       index,
6815:       title: item?.title || item?.name || item?.label || "",
6816:       description: item?.description || item?.caption || item?.result || item?.method || "",
6817:       richDescription: item?.richDescription || null
6818:     });
6819:     return;
6820:   }
6821:
6822:   if (type === "design-object") {
6823:     const pathValue = button.dataset.designPath;
6824:     const items = designArray(project, pathValue);
6825:     const item = mode === "edit" ? items?.[Number(index)] : {};
6826:     openPendingEditor({
6827:       type,
6828:       mode,
6829:       path: pathValue,
6830:       kind: button.dataset.designKind || "document",
6831:       index,
6832:       title: item?.title || item?.name || item?.label || "",
6833:       description: item?.description || item?.caption || item?.result || item?.method || "",
6834:       richDescription: item?.richDescription || null
6835:     });
6836:     return;
6837:   }
6838:
6839:
6840:   if (type === "custom") {
6841:     const section = (project.sections || []).find((item) => item.id === button.dataset.sectionId);
6842:     const item = mode === "edit" ? section?.items?.[Number(index)] : {};
6843:     openPendingEditor({
6844:       type,
6845:       mode,
6846:       sectionId: button.dataset.sectionId,
6847:       index,
6848:       title: item?.title || "",
6849:       description: item?.description || "",
6850:       richDescription: item?.richDescription || null
6851:     });
6852:     return;
6853:   }
6854:
6855:   if (type === "text-array") {
6856:     const key = button.dataset.key;
6857:     const item = mode === "edit" ? project[key]?.[Number(index)] : "";
6858:     openPendingEditor({
6859:       type,
```

```
6860:       mode,
6861:       key,
6862:       index,
6863:       title: typeof item === "string" ? item : item?.title || item?.name || item?.label || ""
6864:     });
6865:     return;
6866:   }
6867:
6868:   if (type === "custom-section") {
6869:     const section = (project.sections || []).find((item) => item.id === button.dataset.sectionId);
6870:     openPendingEditor({
6871:       type,
6872:       mode: "edit",
6873:       sectionId: button.dataset.sectionId,
6874:       title: section?.title || "",
6875:       description: section?.description || "",
6876:       richDescription: section?.richDescription || null
6877:     });
6878:   }
6879: }
```

section (template-preview.js, lines 6841-6842)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6841-6842 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include section at line 6841; item at line 6842. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6841:     const section = (project.sections || []).find((item) => item.id === button.dataset.sectionId);
6842:     const item = mode === "edit" ? section?.items?.[Number(index)] : {};
```

section (template-preview.js, lines 6869-6877)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6869-6877 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and openPendingEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include section at line 6869. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6869:     const section = (project.sections || []).find((item) => item.id === button.dataset.sectionId);
6870:     openPendingEditor({
6871:       type,
6872:       mode: "edit",
6873:       sectionId: button.dataset.sectionId,
6874:       title: section?.title || "",
6875:       description: section?.description || "",
6876:       richDescription: section?.richDescription || null
6877:     });
```

builderPreviewContent (template-preview.js, lines 6881-6888)

builderPreviewContent creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 6881-6888 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richHasContent, renderRichContent, renderMultilineInlineText, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6887: return `<div class="builder-item-preview">${content}</div>`;.

Important local values include content at line 6882. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6881: function builderPreviewContent(rich, text = "", emptyText = "No content has been added yet.") {
6882:   const content = richHasContent(rich)
6883:     ? renderRichContent(rich, text || "")
6884:     : text
6885:       ? `<p class="rich-paragraph">${renderMultilineInlineText(text)}</p>`
6886:       : `<p class="evidence-empty">${escapeHtml(emptyText)}</p>`;
6887:   return `<div class="builder-item-preview">${content}</div>`;
6888: }
```

renderCustomSection (template-preview.js, lines 6890-6933)

renderCustomSection renders ui, markup, or display output. In this file, it appears around lines 6890-6933 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, builderPreviewContent, map, escapeHtml, richHasContent, join, and renderPendingEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6892: if (!section) return `<p class="evidence-empty">Section not found.</p>`;; line 6894: return `.

Important local values include section at line 6891. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6890: function renderCustomSection(project, sectionId) {
6891:   const section = (project.sections || []).find((item) => item.id === sectionId);
6892:   if (!section) return `<p class="evidence-empty">Section not found.</p>`;
6893:
6894:   return `
6895:     <div class="section-window-heading">
6896:       <h3>${section.title || "Custom section"}</h3>
6897:       <div>
6898:         <button type="button" data-open-editor="custom-section" data-section-id="${section.id}">Edit
section</button>
6899:         <button class="danger-icon" type="button" data-delete-section="${section.id}">Delete
section</button>
6900:       </div>
6901:     </div>
6902:     ${builderPreviewContent(section.richDescription, section.description || "", "No section content has
been added yet.")}
6903:     <div class="section-actions">
6904:       <button class="button primary" type="button" data-add-custom-item="${section.id}">Add
subsection</button>
6905:       <button class="button secondary" type="button" data-upload-custom="${section.id}">Add file or
image</button>
6906:     </div>
6907:     <div class="builder-items">
6908:       ${(section.items || []).map((item, index) => `
6909:         <article class="builder-item">
6910:           <div>
6911:             <strong>${escapeHtml(item.title || (item.url ? "Untitled file" : "Untitled
subsection"))}</strong>
6912:             ${item.url ? `<span>${escapeHtml(item.url)}</span>` : ""}
6913:             ${(richHasContent(item.richDescription) || item.description) ?
builderPreviewContent(item.richDescription, item.description || "", "") : ""}
6914:             ${(item.children || []).length ? `
6915:               <div class="builder-child-files">
6916:                 ${(item.children || []).map((child, childIndex) => `
6917:                   <div>
6918:                     <span>${escapeHtml(child.title || "Untitled file")}</span>
6919:                     <button class="danger-icon" type="button" data-delete-custom-
child="${escapeHtml(section.id)}" data-index="${index}" data-child-index="${childIndex}">Delete</button>
6920:                   </div>
6921:                 `).join("")}
6922:               </div>
6923:             ` : ""}
6924:           </div>
6925:           ${item.url ? "" : `<button type="button" data-upload-custom-
subsection="${escapeHtml(section.id)}" data-index="${index}">Add file</button>`}
6926:           ${item.url ? "" : `<button type="button" data-open-editor="custom" data-mode="edit" data-
section-id="${escapeHtml(section.id)}" data-index="${index}">Edit</button>`}
```

```
 6927:           <button class="danger-icon" type="button" data-delete-custom-item="${section.id}" data-
index="${index}">Delete</button>
 6928:         </article>
 6929:       `).join("") || `<p class="evidence-empty">No subsections added yet.</p>`}
 6930:     </div>
 6931:     ${renderPendingEditor()}
 6932:   `;
 6933: }
```

section (template-preview.js, lines 6891-6892)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6891-6892 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6892: if (!section) return `<p class="evidence-empty">Section not found.</p>`;.

Important local values include section at line 6891. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 6891:   const section = (project.sections || []).find((item) => item.id === sectionId);
 6892:   if (!section) return `<p class="evidence-empty">Section not found.</p>`;
```

compileLanguageOptions (template-preview.js, lines 6935-6940)

compileLanguageOptions part of the compile code language/tool/build/run flow. In this file, it appears around lines 6935-6940 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCodeLanguage, map, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6937: return supportedCodeLanguages.map((language) => `.

Important local values include normalized at line 6936. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 6935: function compileLanguageOptions(selected = "javascript") {
 6936:   const normalized = normalizeCodeLanguage(selected);
 6937:   return supportedCodeLanguages.map((language) => `
 6938:     <option value="${escapeHtml(language.id)}"${language.id === normalized ? " selected" :
""}>${escapeHtml(language.label)}</option>
 6939:   `).join("");
 6940: }
```

compileRoleOptions (template-preview.js, lines 6942-6947)

compileRoleOptions part of the compile code language/tool/build/run flow. In this file, it appears around lines 6942-6947 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCompileFileRole, map, escapeHtml, compileFileRoleLabel, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6944: return ["design", "testbench"].map((role) => `.

Important local values include normalized at line 6943. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 6942: function compileRoleOptions(selected = "design") {
 6943:   const normalized = normalizeCompileFileRole(selected, "verilog");
 6944:   return ["design", "testbench"].map((role) => `
```

```
 6945:     <option value="${escapeHtml(role)}"${role === normalized ? " selected" :
""}>${escapeHtml(compileFileRoleLabel(role))}</option>
 6946:   `).join("");
 6947: }
```

hdlTestbenchTemplate (template-preview.js, lines 6949-6973)

hdlTestbenchTemplate part of the private builder frontend and editing experience. In this file, it appears around lines 6949-6973 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slugify, replace, dut, clk, reset_n, done, dumpfile, dumpvars, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6953: return [.

Important local values include tbName at line 6950; dutName at line 6951; commentPrefix at line 6952. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6949: function hdlTestbenchTemplate(language = "systemverilog", designName = "design") {
6950:   const tbName = `tb_${slugify(designName || "design").replace(/-/g, "_") || "design"}`;
6951:   const dutName = slugify(designName || "design").replace(/-/g, "_") || "design";
6952:   const commentPrefix = language === "verilog" ? "//" : "//";
6953:   return [
6954:     "`timescale 1ns/1ps",
6955:     "",
6956:     `module ${tbName};`,
6957:     `  ${commentPrefix} Declare testbench signals here.`,
6958:     `  ${commentPrefix} Example: reg clk = 0; reg reset_n = 0; wire done;`,
6959:     "",
6960:     `  ${commentPrefix} Instantiate the design under test and connect the signals.`,
6961:     `  ${commentPrefix} Example: ${dutName} dut (.clk(clk), .reset_n(reset_n), .done(done));`,
6962:     "",
6963:     "  initial begin",
6964:     '    $dumpfile("waveform.vcd");',
6965:     `    $dumpvars(0, ${tbName});`,
6966:     "",
6967:     `    ${commentPrefix} Drive stimulus over time here.`,
6968:     "    #100;",
6969:     "    $finish;",
6970:     "  end",
6971:     "endmodule"
6972:   ].join("\n");
6973: }
```

firstHdlModuleName (template-preview.js, lines 6975-6978)

firstHdlModuleName part of the private builder frontend and editing experience. In this file, it appears around lines 6975-6978 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references match. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6977: return match?.[1] || "design";.

Important local values include match at line 6976. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6975: function firstHdlModuleName(code = "") {
6976:   const match = String(code || "").match(/\bmodule\s+([A-Za-z_$][\w$]*)/);
6977:   return match?.[1] || "design";
6978: }
```

compileAppendDestinations (template-preview.js, lines 6980-7026)

compileAppendDestinations part of the compile code language/tool/build/run flow. In this file, it appears around lines 6980-7026 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isAnalogProject, ensureDesignModel, forEach, push, summaryRichPathFromTextPath, and walkItems. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6981: if (!project) return [];; line 7025: return destinations;.

Important local values include destinations at line 6982; walkItems at line 7007; itemPath at line 7009; itemTitle at line 7010. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6980: function compileAppendDestinations(project) {
6981:   if (!project) return [];
6982:   const destinations = [{
6983:     id: "project-overview",
6984:     label: "Project overview",
6985:     richPath: "summaryRich",
6986:     textPath: "summary"
6987:   }];
6988:
6989:   if (isAnalogProject(project)) {
6990:     ensureDesignModel(project);
6991:     [
6992:       ["design-overview", "Design overview", "design.brief.summary"],
6993:       ["documentation-overview", "Documentation overview", "design.documentation.summary"],
6994:       ["simulation-overview", "Simulation overview", "design.simulation.summary"]
6995:     ].forEach(([id, label, textPath]) => {
6996:       destinations.push({ id, label, richPath: summaryRichPathFromTextPath(textPath), textPath });
6997:     });
6998:   }
6999:
7000:   (project.sections || []).forEach((section, index) => {
7001:     destinations.push({
7002:       id: `custom-${section.id || index}`,
7003:       label: `${section.title || `Custom section ${index + 1}`} overview`,
7004:       richPath: `sections.${index}.richDescription`,
7005:       textPath: `sections.${index}.description`
7006:     });
7007:     const walkItems = (items = [], prefix = `sections.${index}.items`, labelPrefix = section.title ||
`Custom section ${index + 1}`) => {
7008:       (items || []).forEach((item, itemIndex) => {
7009:         const itemPath = `${prefix}.${itemIndex}`;
7010:         const itemTitle = item.title || `Subsection ${itemIndex + 1}`;
7011:         if (!item.url) {
7012:           destinations.push({
7013:             id: `custom-${section.id || index}-${itemPath}`,
7014:             label: `${labelPrefix} / ${itemTitle} overview`,
7015:             richPath: `${itemPath}.richDescription`,
7016:             textPath: `${itemPath}.description`
7017:           });
7018:         }
7019:         walkItems(item.children || item.items || [], `${itemPath}.children`, `${labelPrefix} /
${itemTitle}`);
7020:       });
7021:     };
7022:     walkItems(section.items || []);
7023:   });
7024:
7025:   return destinations;
7026: }
```

compileAppendDestinationOptions (template-preview.js, lines 7028-7036)

compileAppendDestinationOptions part of the compile code language/tool/build/run flow. In this file, it appears around lines 7028-7036 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references compileAppendDestinations, some, map, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7033: return destinations.map((destination) => `.

Important local values include destinations at line 7029; selected at line 7030. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7028: function compileAppendDestinationOptions(project, selectedId = "") {
7029:   const destinations = compileAppendDestinations(project);
7030:   const selected = destinations.some((destination) => destination.id === selectedId)
7031:     ? selectedId
7032:     : destinations[0]?.id || "";
7033:   return destinations.map((destination) => `
```

```
 7034:     <option value="${escapeHtml(destination.id)}"${destination.id === selected ? " selected" :
""}>${escapeHtml(destination.label)}</option>
 7035:   `).join("");
 7036: }
```

activeCompileFile (template-preview.js, lines 7038-7041)

activeCompileFile part of the compile code language/tool/build/run flow. In this file, it appears around lines 7038-7041 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureCompileCode, and find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7040: return workspace.files.find((file) => file.id === workspace.activeFileId) || workspace.files[0] || null;.

Important local values include workspace at line 7039. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7038: function activeCompileFile(project) {
 7039:   const workspace = ensureCompileCode(project);
 7040:   return workspace.files.find((file) => file.id === workspace.activeFileId) || workspace.files[0] ||
null;
 7041: }
```

compileLogTimestamp (template-preview.js, lines 7043-7047)

compileLogTimestamp part of the compile code language/tool/build/run flow. In this file, it appears around lines 7043-7047 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toISOString, isNaN, getTime, and toLocaleTimeString. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7045: if (Number.isNaN(date.getTime())) return "";; line 7046: return date.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit", second: "2-digit" });.

Important local values include date at line 7044. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7043: function compileLogTimestamp(value = new Date().toISOString()) {
 7044:   const date = new Date(value);
 7045:   if (Number.isNaN(date.getTime())) return "";
 7046:   return date.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit", second: "2-digit" });
 7047: }
```

addCompileMessage (template-preview.js, lines 7049-7062)

addCompileMessage part of the compile code language/tool/build/run flow. In this file, it appears around lines 7049-7062 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureCompileCode, slugify, now, random, toString, slice, toISOString, includes, and updateCompileMessagesPanel. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7051: if (!workspace || !text) return;.

Important local values include workspace at line 7050. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7049: function addCompileMessage(project, text, level = "info") {
 7050:   const workspace = ensureCompileCode(project);
 7051:   if (!workspace || !text) return;
 7052:   workspace.messages = [
```

```
7053:     {
7054:       id: slugify(`message-${Date.now()}-${Math.random().toString(16).slice(2)}`),
7055:       at: new Date().toISOString(),
7056:       level: ["info", "success", "warning", "error"].includes(level) ? level : "info",
7057:       text: String(text)
7058:     },
7059:     ...(workspace.messages || [])
7060:   ].slice(0, 80);
7061:   updateCompileMessagesPanel(project);
7062: }
```

renderCompileMessages (template-preview.js, lines 7064-7073)

renderCompileMessages renders ui, markup, or display output. In this file, it appears around lines 7064-7073 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, map, escapeHtml, compileLogTimestamp, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7066: if (!messages.length) return `<p class="compile-message-empty">No messages yet.</p>`;; line 7067: return messages.map((message) => `.

Important local values include messages at line 7065. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7064: function renderCompileMessages(workspace) {
7065:   const messages = Array.isArray(workspace?.messages) ? workspace.messages : [];
7066:   if (!messages.length) return `<p class="compile-message-empty">No messages yet.</p>`;
7067:   return messages.map((message) => `
7068:     <div class="compile-message compile-message-${escapeHtml(message.level || "info")}">
7069:       <time>${escapeHtml(compileLogTimestamp(message.at))}</time>
7070:       <span>${escapeHtml(message.text)}</span>
7071:     </div>
7072:   `).join("");
7073: }
```

updateCompileTerminalPanel (template-preview.js, lines 7075-7084)

updateCompileTerminalPanel updates ui, state, cache, or stored data. In this file, it appears around lines 7075-7084 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references activeCompileFile, ensureCompileCode, querySelector, and requestAnimationFrame. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7079: if (!terminalElement) return;.

Important local values include workspace at line 7076; terminal at line 7077; terminalElement at line 7078. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7075: function updateCompileTerminalPanel(project, file = activeCompileFile(project)) {
7076:   const workspace = ensureCompileCode(project);
7077:   const terminal = file?.lastResult?.terminal || workspace?.terminal || compileTerminalStatus ||
"Compiler output will appear here after you save or run a source file.";
7078:   const terminalElement = sectionContent.querySelector("[data-compile-terminal]");
7079:   if (!terminalElement) return;
7080:   terminalElement.textContent = terminal;
7081:   requestAnimationFrame(() => {
7082:     terminalElement.scrollTop = terminalElement.scrollHeight;
7083:   });
7084: }
```

updateCompileMessagesPanel (template-preview.js, lines 7086-7091)

updateCompileMessagesPanel updates ui, state, cache, or stored data. In this file, it appears around lines 7086-7091 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureCompileCode, querySelector, and renderCompileMessages. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7089: if (!log) return;.

Important local values include workspace at line 7087; log at line 7088. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7086: function updateCompileMessagesPanel(project) {
7087:   const workspace = ensureCompileCode(project);
7088:   const log = sectionContent.querySelector("[data-compile-messages]");
7089:   if (!log) return;
7090:   log.innerHTML = renderCompileMessages(workspace);
7091: }
```

showCompileOutput (template-preview.js, lines 7093-7105)

showCompileOutput part of the compile code language/tool/build/run flow. In this file, it appears around lines 7093-7105 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references activeCompileFile, updateCompileTerminalPanel, querySelector, add, scrollIntoView, requestAnimationFrame, and remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7097: if (!panel || !terminal) return;.

Important local values include panel at line 7095; terminal at line 7096. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7093: function showCompileOutput(project, file = activeCompileFile(project)) {
7094:   updateCompileTerminalPanel(project, file);
7095:   const panel = sectionContent.querySelector("[data-compile-terminal-panel]");
7096:   const terminal = sectionContent.querySelector("[data-compile-terminal]");
7097:   if (!panel || !terminal) return;
7098:   panel.classList.add("is-focused");
7099:   panel.scrollIntoView({ behavior: "smooth", block: "center" });
7100:   requestAnimationFrame(() => {
7101:     terminal.focus?.({ preventScroll: true });
7102:     terminal.scrollTop = terminal.scrollHeight;
7103:   });
7104:   window.setTimeout(() => panel.classList.remove("is-focused"), 1400);
7105: }
```

activeCompileWaveform (template-preview.js, lines 7107-7110)

activeCompileWaveform part of the compile code language/tool/build/run flow. In this file, it appears around lines 7107-7110 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references activeCompileFile, and ensureCompileCode. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7109: return file?.lastResult?.waveform || file?.waveform || workspace?.waveform || null;.

Important local values include workspace at line 7108. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7107: function activeCompileWaveform(project, file = activeCompileFile(project)) {
7108:   const workspace = ensureCompileCode(project);
7109:   return file?.lastResult?.waveform || file?.waveform || workspace?.waveform || null;
7110: }
```

scopeSignalValueAt (template-preview.js, lines 7112-7117)

scopeSignalValueAt part of the private builder frontend and editing experience. In this file, it appears around lines 7112-7117 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 7114: if (value === "1") return "1";; line 7115: if (value === "0") return "0";; line 7116: return "x";.

Important local values include value at line 7113. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7112: function scopeSignalValueAt(changes = [], index = 0) {
7113:   const value = String(changes[index]?.value ?? "x").toLowerCase();
7114:   if (value === "1") return "1";
7115:   if (value === "0") return "0";
7116:   return "x";
7117: }
```

renderScalarWavePath (template-preview.js, lines 7119-7140)

renderScalarWavePath renders ui, markup, or display output. In this file, it appears around lines 7119-7140 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, map, sort, max, min, forEach, xFor, yFor, scopeSignalValueAt, and toFixed. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7139: return path;.

Important local values include normalized at line 7120; top at line 7123; bottom at line 7124; middle at line 7125; yFor at line 7126; xFor at line 7127; path at line 7128; x at line 7130; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7119: function renderScalarWavePath(changes = [], maxTime = 1, rowY = 0, left = 150, width = 680) {
7120:   const normalized = (Array.isArray(changes) && changes.length ? changes : [{ time: 0, value: "x" }])
7121:     .map((change) => ({ time: Number(change.time) || 0, value: String(change.value ?? "x") }))
7122:     .sort((a, b) => a.time - b.time);
7123:   const top = rowY + 5;
7124:   const bottom = rowY + 23;
7125:   const middle = rowY + 14;
7126:   const yFor = (value) => value === "1" ? top : value === "0" ? bottom : middle;
7127:   const xFor = (time) => left + Math.max(0, Math.min(1, (Number(time) || 0) / Math.max(1, maxTime))) *
width;
7128:   let path = "";
7129:   normalized.forEach((change, index) => {
7130:     const x = xFor(change.time);
7131:     const y = yFor(scopeSignalValueAt(normalized, index));
7132:     if (index === 0) {
7133:       path += `M ${x.toFixed(1)} ${y.toFixed(1)}`;
7134:     } else {
7135:       path += ` H ${x.toFixed(1)} V ${y.toFixed(1)}`;
7136:     }
7137:   });
7138:   path += ` H ${(left + width).toFixed(1)}`;
7139:   return path;
7140: }
```

yFor (template-preview.js, lines 7126-7137)

yFor part of the private builder frontend and editing experience. In this file, it appears around lines 7126-7137 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references max, min, forEach, xFor, scopeSignalValueAt, and toFixed. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include yFor at line 7126; xFor at line 7127; path at line 7128; x at line 7130; y at line 7131. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7126:   const yFor = (value) => value === "1" ? top : value === "0" ? bottom : middle;
 7127:   const xFor = (time) => left + Math.max(0, Math.min(1, (Number(time) || 0) / Math.max(1, maxTime))) *
width;
 7128:   let path = "";
 7129:   normalized.forEach((change, index) => {
 7130:     const x = xFor(change.time);
 7131:     const y = yFor(scopeSignalValueAt(normalized, index));
 7132:     if (index === 0) {
 7133:       path += `M ${x.toFixed(1)} ${y.toFixed(1)}`;
 7134:     } else {
 7135:       path += ` H ${x.toFixed(1)} V ${y.toFixed(1)}`;
 7136:     }
 7137:   });
```

xFor (template-preview.js, lines 7127-7137)

xFor part of the private builder frontend and editing experience. In this file, it appears around lines 7127-7137 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references max, min, forEach, yFor, scopeSignalValueAt, and toFixed. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include xFor at line 7127; path at line 7128; x at line 7130; y at line 7131. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7127:   const xFor = (time) => left + Math.max(0, Math.min(1, (Number(time) || 0) / Math.max(1, maxTime))) *
width;
 7128:   let path = "";
 7129:   normalized.forEach((change, index) => {
 7130:     const x = xFor(change.time);
 7131:     const y = yFor(scopeSignalValueAt(normalized, index));
 7132:     if (index === 0) {
 7133:       path += `M ${x.toFixed(1)} ${y.toFixed(1)}`;
 7134:     } else {
 7135:       path += ` H ${x.toFixed(1)} V ${y.toFixed(1)}`;
 7136:     }
 7137:   });
```

renderBusWaveLabels (template-preview.js, lines 7142-7152)

renderBusWaveLabels renders ui, markup, or display output. In this file, it appears around lines 7142-7152 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, map, sort, slice, max, min, toFixed, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7147: return normalized.map((change, index) => {; line 7150: return `<text x="${Math.min(left + width - 70, x + 4).toFixed(1)}" y="${rowY + 19}" class="scope-value-label">${escapeHtml(value)}</text>${index ? `<line x1="${x.toFixed(1)}" x2="${x.toFixed(1)}" y1="${rowY + 5}" y2="${r.

Important local values include normalized at line 7143; x at line 7148; value at line 7149. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7142: function renderBusWaveLabels(changes = [], maxTime = 1, rowY = 0, left = 150, width = 680) {
 7143:   const normalized = (Array.isArray(changes) && changes.length ? changes : [{ time: 0, value: "x" }])
 7144:     .map((change) => ({ time: Number(change.time) || 0, value: String(change.value ?? "x") }))
 7145:     .sort((a, b) => a.time - b.time)
 7146:     .slice(0, 12);
 7147:   return normalized.map((change, index) => {
 7148:     const x = left + Math.max(0, Math.min(1, change.time / Math.max(1, maxTime))) * width;
 7149:     const value = change.value.length > 12 ? `${change.value.slice(0, 12)}...` : change.value;
 7150:     return `<text x="${Math.min(left + width - 70, x + 4).toFixed(1)}" y="${rowY + 19}" class="scope-
value-label">${escapeHtml(value)}</text>${index ? `<line x1="${x.toFixed(1)}" x2="${x.toFixed(1)}" y1="${rowY +
5}" y2="${rowY + 25}" class="scope-transition" />` : ""}`;
 7151:   }).join("");
 7152: }
```

renderHdlScope (template-preview.js, lines 7154-7203)

renderHdlScope renders ui, markup, or display output. In this file, it appears around lines 7154-7203 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, slice, max, flatMap, map, round, toFixed, join, escapeHtml, renderScalarWavePath, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 7157: return `; line 7173: return `<line x1="${x.toFixed(1)}" x2="${x.toFixed(1)}" y1="18" y2="${height - 18}" class="scope-grid" /><text x="${x.toFixed(1)}" y="14" class="scope-time-label">${label}</text>`;; line 7180: return `; line 7190: return `.

Important local values include signals at line 7155; left at line 7164; width at line 7165; rowHeight at line 7166; top at line 7167; height at line 7168; maxTime at line 7169; grid at line 7170; plus 7 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7154: function renderHdlScope(waveform) {
 7155:   const signals = Array.isArray(waveform?.signals) ? waveform.signals.slice(0, 16) : [];
 7156:   if (!signals.length) {
 7157:     return `
 7158:       <div class="compile-scope-empty">
 7159:         <strong>No waveform scope yet.</strong>
 7160:         <span>Run a Verilog/SystemVerilog design with a testbench that calls <code>$dumpfile</code> and
<code>$dumpvars</code>.</span>
 7161:       </div>
 7162:     `;
 7163:   }
 7164:   const left = 165;
 7165:   const width = 760;
 7166:   const rowHeight = 34;
 7167:   const top = 28;
 7168:   const height = top + signals.length * rowHeight + 28;
 7169:   const maxTime = Math.max(1, Number(waveform.maxTime) || Math.max(...signals.flatMap((signal) =>
(signal.changes || []).map((change) => Number(change.time) || 0)), 1));
 7170:   const grid = [0, 0.25, 0.5, 0.75, 1].map((ratio) => {
 7171:     const x = left + ratio * width;
 7172:     const label = Math.round(maxTime * ratio);
 7173:     return `<line x1="${x.toFixed(1)}" x2="${x.toFixed(1)}" y1="18" y2="${height - 18}" class="scope-
grid" /><text x="${x.toFixed(1)}" y="14" class="scope-time-label">${label}</text>`;
 7174:   }).join("");
 7175:   const rows = signals.map((signal, index) => {
 7176:     const y = top + index * rowHeight;
 7177:     const name = String(signal.name || signal.reference || `signal_${index + 1}`);
 7178:     const changes = Array.isArray(signal.changes) ? signal.changes : [];
 7179:     const scalar = Number(signal.width || 1) <= 1;
 7180:     return `
 7181:       <g class="scope-row">
 7182:         <text x="12" y="${y + 20}" class="scope-signal-label">${escapeHtml(name.length > 24 ?
`${name.slice(0, 24)}...` : name)}</text>
 7183:         <line x1="${left}" x2="${left + width}" y1="${y + 27}" y2="${y + 27}" class="scope-row-line" />
 7184:         ${scalar
 7185:           ? `<path d="${renderScalarWavePath(changes, maxTime, y, left, width)}" class="scope-wave" />`
 7186:           : `<line x1="${left}" x2="${left + width}" y1="${y + 15}" y2="${y + 15}" class="scope-bus"
/>${renderBusWaveLabels(changes, maxTime, y, left, width)}`}
 7187:       </g>
 7188:     `;
 7189:   }).join("");
 7190:   return `
 7191:     <div class="compile-scope-meta">
 7192:       <span>${escapeHtml(waveform.source || "waveform.vcd")}</span>
 7193:       <span>${signals.length} signal${signals.length === 1 ? "" : "s"}</span>
 7194:       <span>0 to ${escapeHtml(maxTime)} ${escapeHtml(waveform.timeScale || "ticks")}</span>
 7195:     </div>
 7196:     <div class="compile-scope-scroll">
 7197:       <svg class="compile-scope-svg" viewBox="0 0 ${left + width + 20} ${height}" role="img" aria-
label="HDL waveform scope">
 7198:         ${grid}
 7199:         ${rows}
 7200:       </svg>
 7201:     </div>
 7202:   `;
 7203: }
```

renderCompileScopePanel (template-preview.js, lines 7205-7217)

renderCompileScopePanel renders ui, markup, or display output. In this file, it appears around lines 7205-7217 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references activeCompileFile, activeCompileWaveform, isHdlLanguage, and renderHdlScope. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7207: if (!isHdlLanguage(file?.language) && !waveform) return "";; line 7208: return `.

Important local values include waveform at line 7206. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7205: function renderCompileScopePanel(project, file = activeCompileFile(project)) {
7206:   const waveform = activeCompileWaveform(project, file);
7207:   if (!isHdlLanguage(file?.language) && !waveform) return "";
7208:   return `
7209:     <section class="compile-scope-panel" data-compile-scope-panel aria-label="HDL signal scope">
7210:       <div class="compile-terminal-heading">
7211:         <span class="compile-terminal-title"><span aria-hidden="true" class="scope-dot"></span> Signal
scope</span>
7212:         <button type="button" data-compile-show-scope>Show scope</button>
7213:       </div>
7214:       ${renderHdlScope(waveform)}
7215:     </section>
7216:   `;
7217: }
```

showCompileScope (template-preview.js, lines 7219-7228)

showCompileScope part of the compile code language/tool/build/run flow. In this file, it appears around lines 7219-7228 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references activeCompileFile, querySelector, addCompileMessage, add, scrollIntoView, and remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7223: return;.

Important local values include panel at line 7220. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7219: function showCompileScope(project, file = activeCompileFile(project)) {
7220:   const panel = sectionContent.querySelector("[data-compile-scope-panel]");
7221:   if (!panel) {
7222:     addCompileMessage(project, "No HDL scope is available for the active source yet.", "warning");
7223:     return;
7224:   }
7225:   panel.classList.add("is-focused");
7226:   panel.scrollIntoView({ behavior: "smooth", block: "center" });
7227:   window.setTimeout(() => panel.classList.remove("is-focused"), 1400);
7228: }
```

copyCompileOutput (template-preview.js, lines 7230-7245)

copyCompileOutput part of the compile code language/tool/build/run flow. In this file, it appears around lines 7230-7245 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references activeCompileFile, ensureCompileCode, trim, addCompileMessage, writeText, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7235: return;.

Important local values include workspace at line 7231; text at line 7232. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7230: async function copyCompileOutput(project, file = activeCompileFile(project)) {
7231:   const workspace = ensureCompileCode(project);
7232:   const text = file?.lastResult?.terminal || workspace?.terminal || compileTerminalStatus || "";
7233:   if (!text.trim()) {
```

```
7234:     addCompileMessage(project, "There is no terminal output to copy yet.", "warning");
7235:     return;
7236:   }
7237:   try {
7238:     await navigator.clipboard.writeText(text);
7239:     addCompileMessage(project, "Terminal output copied to the clipboard.", "success");
7240:     setStatus("Terminal output copied.");
7241:   } catch {
7242:     addCompileMessage(project, "Terminal output could not be copied by this browser.", "warning");
7243:     setStatus("Terminal output could not be copied.");
7244:   }
7245: }
```

clearCompileOutput (template-preview.js, lines 7247-7256)

clearCompileOutput part of the compile code language/tool/build/run flow. In this file, it appears around lines 7247-7256 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references activeCompileFile, ensureCompileCode, updateCompileTerminalPanel, addCompileMessage, setStatus, and scheduleAutosave. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include workspace at line 7248. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7247: function clearCompileOutput(project, file = activeCompileFile(project)) {
7248:   const workspace = ensureCompileCode(project);
7249:   if (file?.lastResult) file.lastResult.terminal = "";
7250:   if (workspace) workspace.terminal = "";
7251:   compileTerminalStatus = "";
7252:   updateCompileTerminalPanel(project, file);
7253:   addCompileMessage(project, "Terminal output cleared.", "info");
7254:   setStatus("Terminal output cleared.");
7255:   scheduleAutosave();
7256: }
```

richBlocksForCompileAppend (template-preview.js, lines 7258-7265)

richBlocksForCompileAppend part of the compile code language/tool/build/run flow. In this file, it appears around lines 7258-7265 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getByPath, richBlocksFromValue, filter, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 7261: return richBlocksFromValue(existingRich, existingText).filter((block) => {; line 7262: if (block.type !== "paragraph") return true;; line 7263: return Boolean(String(block.text || "").trim() || block.html);.

Important local values include existingRich at line 7259; existingText at line 7260. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7258: function richBlocksForCompileAppend(project, destination) {
7259:   const existingRich = getByPath(project, destination.richPath);
7260:   const existingText = getByPath(project, destination.textPath) || "";
7261:   return richBlocksFromValue(existingRich, existingText).filter((block) => {
7262:     if (block.type !== "paragraph") return true;
7263:     return Boolean(String(block.text || "").trim() || block.html);
7264:   });
7265: }
```

codeBlockForCompileFile (template-preview.js, lines 7267-7274)

codeBlockForCompileFile part of the compile code language/tool/build/run flow. In this file, it appears around lines 7267-7274 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCodeText, normalizeCodeLanguage, and detectCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7268: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
7267: function codeBlockForCompileFile(file) {
7268:   return {
7269:     code: normalizeCodeText(file?.code || "", "source"),
7270:     language: normalizeCodeLanguage(file?.language || detectCodeLanguage(file?.code || "", file?.fileName
|| "")),
7271:     pasteMode: "source",
7272:     type: "code"
7273:   };
7274: }
```

compileFileDirtyLabel (template-preview.js, lines 7276-7283)

compileFileDirtyLabel part of the compile code language/tool/build/run flow. In this file, it appears around lines 7276-7283 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 7277: if (!file) return "";; line 7278: if (file.dirty) return "Unsaved";; line 7279: if (file.lastResult?.ok === true) return "Compiled";; line 7280: if (file.lastResult?.ok === false) return "Needs fix";. There are 2 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
7276: function compileFileDirtyLabel(file) {
7277:   if (!file) return "";
7278:   if (file.dirty) return "Unsaved";
7279:   if (file.lastResult?.ok === true) return "Compiled";
7280:   if (file.lastResult?.ok === false) return "Needs fix";
7281:   if (file.savedAt) return "Saved";
7282:   return "Draft";
7283: }
```

renderCompileCodeSection (template-preview.js, lines 7285-7408)

This function draws the Compile Code workspace inside a project. It builds the file explorer, editor, language controls, compile/build/run/simulate/scope buttons, terminal panel, message log, and append-code controls.

The builder needs an IDE-like surface inside each project. This renderer turns saved compile workspace state into the screen the user edits and runs.

It calls or references ensureCompileCode, activeCompileFile, escapeHtml, map, match, replace, compileFileDirtyLabel, join, defaultCodeFileName, compileLanguageOptions, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7289: return `.

Important local values include workspace at line 7286; activeFile at line 7287; terminal at line 7288. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7285: function renderCompileCodeSection(project) {
7286:   const workspace = ensureCompileCode(project);
7287:   const activeFile = activeCompileFile(project);
7288:   const terminal = activeFile?.lastResult?.terminal || workspace.terminal || compileTerminalStatus ||
"Compiler output will appear here after you save or run a source file.";
7289:   return `
7290:     <div class="compile-code-workspace">
7291:       <aside class="compile-code-sidebar" aria-label="Compile code files">
7292:         <div class="compile-code-sidebar-heading">
7293:           <h3>Compile Code</h3>
7294:           <small>${workspace.files.length} source file${workspace.files.length === 1 ? "" : "s"}</small>
7295:         </div>
7296:         <div class="compile-code-workspace-tree" aria-label="Project workspace tree">
7297:           <details open>
7298:             <summary>${escapeHtml(project.title || "Project workspace")}</summary>
```

```
 7299:             <div class="compile-code-tree-files">
 7300:               ${workspace.files.map((file) => `
 7301:                 <button class="compile-code-tree-file${file.id === workspace.activeFileId ? " is-active"
: ""}" type="button" data-compile-select="${escapeHtml(file.id)}">
 7302:                   <span class="compile-file-extension">${escapeHtml((file.fileName.match(/\.[^.]+$/)?.[0]
 || "src").replace(".", ""))}</span>
 7303:                   <span>${escapeHtml(file.fileName)}</span>
 7304:                   <small>${escapeHtml(compileFileDirtyLabel(file))}</small>
 7305:                 </button>
 7306:               `).join("") || `<p>No source files yet.</p>`}
 7307:             </div>
 7308:           </details>
 7309:         </div>
 7310:         <div class="compile-code-actions">
 7311:           <button type="button" data-compile-add>Add code file</button>
 7312:           <button type="button" data-compile-add-testbench>Add testbench</button>
 7313:           <button type="button" data-compile-import>Import file</button>
 7314:           <button type="button" data-compile-tools>Check compilers</button>
 7315:         </div>
 7316:       </aside>
 7317:
 7318:       <section class="compile-code-main" aria-label="Compile code editor">
 7319:         ${activeFile ? `
 7320:           <div class="compile-code-meta-grid">
 7321:             <label>
 7322:               <span>Title</span>
 7323:               <input type="text" data-compile-field="title" value="${escapeHtml(activeFile.title || "")}"
 placeholder="Source title" />
 7324:             </label>
 7325:             <label>
 7326:               <span>File name</span>
 7327:               <input type="text" data-compile-field="fileName" value="${escapeHtml(activeFile.fileName ||
 "")}" placeholder="${escapeHtml(defaultCodeFileName(activeFile.language))}" />
 7328:             </label>
 7329:             <label>
 7330:               <span>Language</span>
 7331:               <select data-compile-
field="language">${compileLanguageOptions(activeFile.language)}</select>
 7332:             </label>
 7333:             <label>
 7334:               <span>${isHdlLanguage(activeFile.language) ? "HDL role" : "Source role"}</span>
 7335:               ${isHdlLanguage(activeFile.language)
 7336:                 ? `<select data-compile-field="role">${compileRoleOptions(activeFile.role)}</select>`
 7337:                 : `<input type="text" value="Program source" disabled />`}
 7338:             </label>
 7339:           </div>
 7340:           <div class="compile-code-editor-grid compile-code-editor-grid-single">
 7341:             <section class="compile-code-preview-panel compile-code-active-panel" aria-label="Active
syntax highlighted code editor">
 7342:               <div class="compile-code-preview-heading">
 7343:                 <span>${escapeHtml(codeLanguageLabel(activeFile.language))} active editor</span>
 7344:               </div>
 7345:               <div class="compile-code-active-editor">
 7346:                 <pre class="compile-code-preview" aria-hidden="true"><code data-compile-
preview>${tokenizedCodeHtml(activeFile.code || "", activeFile.language)}</code></pre>
 7347:                 <textarea data-compile-field="code" data-compile-active-editor spellcheck="false"
wrap="off" rows="18" placeholder="Type or paste code here">${escapeHtml(activeFile.code || "")}</textarea>
 7348:               </div>
 7349:             </section>
 7350:           </div>
 7351:           <label class="compile-stdin-field">
 7352:             <span>Program input, optional</span>
 7353:             <textarea data-compile-field="stdin" rows="4" placeholder="Input passed to stdin when the
code runs">${escapeHtml(activeFile.stdin || "")}</textarea>
 7354:           </label>
 7355:           <div class="compile-code-command-bar">
 7356:             <button type="button" data-compile-save>Save source</button>
 7357:             <button type="button" data-compile-beautify>Beautify</button>
 7358:             <button type="button" data-compile-compile>Compile</button>
 7359:             <button type="button" data-compile-build>Build project</button>
 7360:             ${isHdlLanguage(activeFile.language)
 7361:               ? `<button type="button" data-compile-simulate>Simulate</button>`
 7362:               : `<button type="button" data-compile-run>Run</button>`}
 7363:             <button class="compile-output-button" type="button" data-compile-show-output title="Open the
terminal output for this source"><span class="compile-command-icon" aria-hidden="true">&amp;gt;_</span><span>Show
output</span></button>
 7364:             ${isHdlLanguage(activeFile.language) ? `<button class="compile-output-button" type="button"
data-compile-show-scope title="Open the HDL waveform scope"><span class="compile-command-icon" aria-
hidden="true">~</span><span>Show scope</span></button>` : ""}
 7365:             <button type="button" data-compile-install>Install tools</button>
 7366:             <button class="danger-icon" type="button" data-compile-delete>Delete source</button>
 7367:           </div>
 7368:           <section class="compile-append-panel" aria-label="Append code to project section">
 7369:             <div>
 7370:               <strong>Append code to project</strong>
 7371:               <span>Insert the active source as a formatted code block into this project.</span>
 7372:             </div>
 7373:             <label>
 7374:               <span>Destination</span>
 7375:               <select data-compile-append-target>${compileAppendDestinationOptions(project,
workspace.appendTargetId)}</select>
 7376:             </label>
```

```
 7377:             <div class="compile-append-actions">
 7378:               <button type="button" data-compile-append-code>Append code to project</button>
 7379:             </div>
 7380:           </section>
 7381:         ` : `
 7382:           <div class="compile-code-empty">
 7383:             <h3>No compile file selected</h3>
 7384:             <p>Add a source file for C, C++, SystemVerilog, LTspice, Java, JavaScript, Python, or
HTML.</p>
 7385:           </div>
 7386:         `}
 7387:         <section class="compile-terminal-panel" data-compile-terminal-panel aria-label="Compiler terminal
 output">
 7388:           <div class="compile-terminal-heading">
 7389:             <span class="compile-terminal-title"><span aria-hidden="true" class="terminal-dot"></span>
OMB Local Terminal</span>
 7390:             <span class="compile-terminal-controls">
 7391:               <button type="button" data-compile-copy-output>Copy output</button>
 7392:               <button type="button" data-compile-clear-output>Clear output</button>
 7393:             </span>
 7394:           </div>
 7395:           <pre class="compile-terminal" data-compile-terminal tabindex="0" role="log" aria-
live="polite">${escapeHtml(terminal)}</pre>
 7396:         </section>
 7397:         ${renderCompileScopePanel(project, activeFile)}
 7398:         <section class="compile-messages-panel" aria-label="Compile messages log">
 7399:           <div class="compile-terminal-heading">
 7400:             <span>Messages log</span>
 7401:             <button type="button" data-compile-clear-messages>Clear log</button>
 7402:           </div>
 7403:           <div class="compile-messages-log" data-compile-
messages>${renderCompileMessages(workspace)}</div>
 7404:         </section>
      ... excerpt truncated after 120 lines. Open the source file for the rest of this function.
```

renderSectionContent (template-preview.js, lines 7410-7472)

renderSectionContent renders ui, markup, or display output. In this file, it appears around lines 7410-7472 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sectionOptions, find, startsWith, renderCustomSection, replace, requestAnimationFrame, populateRichEditors, isAnalogProject, renderAnalogDesignWorkspace, renderObjectSection, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 7414: return;; line 7423: return;; line 7429: return;; line 7435: if (isAnalogProject(project)) return renderAnalogDesignWorkspace(project);. There are 1 additional return statements in the function body.

Important local values include section at line 7417; isOverview at line 7418; renderers at line 7432; cards at line 7436. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7410: function renderSectionContent(project) {
 7411:   if (!project) {
 7412:     sectionContent.hidden = true;
 7413:     sectionContent.innerHTML = "";
 7414:     return;
 7415:   }
 7416:
 7417:   const section = sectionOptions(project).find((item) => item.id === activeSectionId) ||
standardSections[0];
 7418:   const isOverview = section.id === "brief";
 7419:   projectFields.hidden = !isOverview;
 7420:   sectionContent.hidden = isOverview;
 7421:   if (isOverview) {
 7422:     sectionContent.innerHTML = "";
 7423:     return;
 7424:   }
 7425:
 7426:   if (section.id.startsWith("custom:")) {
 7427:     sectionContent.innerHTML = renderCustomSection(project, section.id.replace("custom:", ""));
 7428:     requestAnimationFrame(() => populateRichEditors(sectionContent));
 7429:     return;
 7430:   }
 7431:
 7432:   const renderers = {
 7433:     brief: () => "",
 7434:     design: () => {
 7435:       if (isAnalogProject(project)) return renderAnalogDesignWorkspace(project);
 7436:       const cards = [
 7437:         (project.documents || []).length ? `<section><h3>Documents</h3>${renderObjectSection(project,
"documents", "documents", "Documents")}</section>` : "",
 7438:         (project.pcbs || []).length ? `<section><h3>PCBs Built</h3>${renderObjectSection(project, "pcbs",
 "pcbs", "PCBs")}</section>` : "",
```

```
        (project.media || []).length ? `<section><h3>Images</h3>${renderObjectSection(project, "media",
"images", "Images")}</section>` : ""
      ].filter(Boolean);
      return `
        <div class="section-stack">
          ${cards.join("") || renderEmptyDynamicSectionPrompt("design")}
          ${renderPendingEditor()}
        </div>
      `;
    },
    simulation: () => isAnalogProject(project) ? renderAnalogSimulationWorkspace(project) : "",
    "compile-code": () => renderCompileCodeSection(project),
    tests: () => `${renderObjectSection(project, "tests", "tests", "Tests")}${renderPendingEditor()}`,
    tools: () => `
      <div class="section-stack">
        <section>
          <h3>Tools Used</h3>
          ${renderToolsSection(project)}
        </section>
        <section>
          <h3>Languages</h3>
          ${renderTextArray(project, "languages", "Language")}
        </section>
        <section>
          <h3>Links</h3>
          ${renderObjectSection(project, "links", "", "Links")}
        </section>
        ${renderPendingEditor()}
      </div>
    `
  };

    sectionContent.innerHTML = renderers[section.id]();
    requestAnimationFrame(() => populateRichEditors(sectionContent));
}
```

fullPortfolioPreviewHtml (template-preview.js, lines 7474-7476)

fullPortfolioPreviewHtml part of the private builder frontend and editing experience. In this file, it appears around lines 7474-7476 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fullPortfolioPreviewHtmlExact. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7475: return fullPortfolioPreviewHtmlExact();.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
function fullPortfolioPreviewHtml() {
  return fullPortfolioPreviewHtmlExact();
}
```

previewValue (template-preview.js, lines 7478-7480)

previewValue part of the private builder frontend and editing experience. In this file, it appears around lines 7478-7480 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references call. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7479: return source && Object.prototype.hasOwnProperty.call(source, key) ? source[key] : fallback;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
function previewValue(source, key, fallback) {
  return source && Object.prototype.hasOwnProperty.call(source, key) ? source[key] : fallback;
}
```

fullPortfolioPreviewHtmlExact (template-preview.js, lines 7482-7702)

This function builds the local preview HTML that should match the public website layout. It combines profile data, fun facts, sections, projects, contacts, AI placement, and generated project markup into one preview frame.

The user needs to see exactly what will be pushed before publishing. This function is the truth test between builder state and website output.

It calls or references parsedPortfolioGlobals, normalizeSiteContent, previewValue, normalizeProfile, normalizeFieldStyles, stringify, replaceAll, escapeHtml, plainFieldStyleAttribute, renderRichFieldContent, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7506: return `<!DOCTYPE html>.

Important local values include baseHref at line 7483; parsedGlobals at line 7484; siteContent at line 7485; profile at line 7486; fieldStyles at line 7487; profileRich at line 7488; siteContentRich at line 7489; ownerName at line 7490; plus 3 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7482: function fullPortfolioPreviewHtmlExact(dataOverride = null) {
 7483:   const baseHref = `${window.location.origin}/`;
 7484:   const parsedGlobals = dataOverride ? null : parsedPortfolioGlobals();
 7485:   const siteContent = normalizeSiteContent(previewValue(dataOverride, "siteContent",
parsedGlobals?.siteContent || {}));
 7486:   const profile = normalizeProfile(previewValue(dataOverride, "profile", parsedGlobals?.profile || {}));
 7487:   const fieldStyles = normalizeFieldStyles(previewValue(dataOverride, "fieldStyles",
parsedGlobals?.fieldStyles || {}));
 7488:   const profileRich = previewValue(dataOverride, "profileRich", parsedGlobals?.profileRich || {});
 7489:   const siteContentRich = previewValue(dataOverride, "siteContentRich", parsedGlobals?.siteContentRich ||
 {});
 7490:   const ownerName = profile.displayName || "Portfolio";
 7491:   const portfolioLabel = profile.portfolioLabel || "Portfolio";
 7492:   const previewDataObject = {
 7493:     categories: previewValue(dataOverride, "categories", parsedGlobals?.categories || []),
 7494:     fieldStyles,
 7495:     funFacts: previewValue(dataOverride, "funFacts", parsedGlobals?.funFacts || []),
 7496:     funFactsRich: previewValue(dataOverride, "funFactsRich", parsedGlobals?.funFactsRich || null),
 7497:     profile,
 7498:     profileRich,
 7499:     projects: previewValue(dataOverride, "projects", savedPortfolioCatalog.projects || []),
 7500:     siteContent,
 7501:     siteContentRich,
 7502:     siteSections: previewValue(dataOverride, "siteSections", parsedGlobals?.siteSections || [])
 7503:   };
 7504:   const previewData = JSON.stringify(previewDataObject).replaceAll("</", "<\\/");
 7505:
 7506:   return `<!DOCTYPE html>
 7507: <html lang="en">
 7508:   <head>
 7509:     <meta charset="UTF-8" />
 7510:     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 7511:     <base href="${baseHref}" />
 7512:     <meta name="description" content="A portfolio website built with OMB Portfolio Builder for projects,
documents, diagrams, source code, tests, links, and profile information." />
 7513:     <meta name="robots" content="index, follow" />
 7514:     <meta name="author" content="${escapeHtml(ownerName)}" />
 7515:     <meta name="portfolio-ai-endpoint" content="https://omb-portfolio-ai.maurice-baraza-
otieno.workers.dev/api/portfolio-ai" />
 7516:     <title>${escapeHtml(ownerName)} | ${escapeHtml(portfolioLabel)}</title>
 7517:     <link rel="stylesheet" href="styles.css" />
 7518:   </head>
 7519:   <body>
 7520:     <header class="site-header" id="top">
 7521:       <a class="brand" href="#top" aria-label="Go to top">
 7522:         <span class="brand-lockup" aria-hidden="true">
 7523:           <img class="brand-icon" alt="" hidden />
 7524:           <span class="omb-engraving">${escapeHtml(profile.brandText || "Portfolio")}</span>
 7525:         </span>
 7526:         <span>
 7527:           <strong${plainFieldStyleAttribute(fieldStyles, "profile-display-
name")}>${renderRichFieldContent(profileRich.displayName, ownerName)}</strong>
 7528:           <small${plainFieldStyleAttribute(fieldStyles, "profile-portfolio-
label")}>${renderRichFieldContent(profileRich.portfolioLabel, portfolioLabel)}</small>
 7529:         </span>
 7530:       </a>
 7531:
 7532:       <div class="header-right">
 7533:         <nav class="site-nav" aria-label="Primary navigation">
 7534:           <a href="#ask-ai">Ask AI</a>
 7535:           <a href="#projects">Projects</a>
 7536:           <a href="#resume">Resume</a>
 7537:           <a href="#process">Process</a>
 7538:           <a href="#contact">Contact</a>
 7539:         </nav>
 7540:         <a class="header-avatar" href="#contact" aria-label="Go to contact information" hidden>
 7541:           <img alt="" />
 7542:         </a>
 7543:       </div>
 7544:     </header>
```

```
 7545:
 7546:     <main>
 7547:       <section class="hero" aria-labelledby="hero-title">
 7548:         <img class="hero-image" alt="" hidden />
 7549:         <div class="hero-shade" aria-hidden="true"></div>
 7550:         <div class="hero-content">
 7551:           <p class="eyebrow"${plainFieldStyleAttribute(fieldStyles, "site-hero-
eyebrow")}>${renderRichFieldContent(siteContentRich.heroEyebrow, siteContent.heroEyebrow)}</p>
 7552:           <h1 id="hero-title"${plainFieldStyleAttribute(fieldStyles, "site-hero-
title")}>${renderRichFieldContent(siteContentRich.heroTitle, siteContent.heroTitle)}</h1>
 7553:           <p class="hero-copy"${plainFieldStyleAttribute(fieldStyles, "site-hero-
copy")}>${renderRichFieldContent(siteContentRich.heroCopy, siteContent.heroCopy)}</p>
 7554:           <div class="hero-actions">
 7555:             <a class="button primary" href="#projects">View projects</a>
 7556:             <a class="button secondary" href="#resume" hidden>View resume</a>
 7557:             <a class="button secondary" href="#" target="_blank" rel="noreferrer" hidden>GitHub
profile</a>
 7558:             <a class="ai-jump-link" href="#ask-ai" aria-label="Jump to the portfolio AI assistant">
 7559:               <svg viewBox="0 0 24 24" aria-hidden="true">
 7560:                 <path d="M12 3v3m-5 7H4m16 0h-3M7.5 7.5 5.4 5.4m11.1 2.1 2.1-2.1M8 10h8v7a2 2 0 0 1-2
2h-4a2 2 0 0 1-2-2z" />
 7561:                 <path d="M10 13h.01M14 13h.01M10 16h4" />
 7562:               </svg>
 7563:               <span>Ask AI</span>
 7564:             </a>
 7565:           </div>
 7566:         </div>
 7567:       </section>
 7568:
 7569:       <section class="fun-facts-section" id="fun-facts-callout" aria-label="Fun facts" hidden></section>
 7570:
 7571:       <section class="builder-download-section" aria-label="Download portfolio builder">
 7572:         <a class="builder-download-link" href="https://github.com/otienomaurice/omb-portfolio-
builder/releases/latest/download/OMB-Portfolio-Builder-Setup-latest-x64.exe" download>
 7573:           <svg viewBox="0 0 24 24" aria-hidden="true">
 7574:             <path d="M12 3v11m0 0 4-4m-4 4-4-4" />
 7575:             <path d="M5 17v2h14v-2" />
 7576:           </svg>
 7577:           <span>Download builder application</span>
 7578:         </a>
 7579:       </section>
 7580:
 7581:       <section class="summary-band" aria-label="Portfolio highlights">
 7582:         <div class="metric">
 7583:           <strong id="project-count">0</strong>
 7584:           <span>presented projects</span>
 7585:         </div>
 7586:         <div class="metric">
 7587:           <strong id="project-track-count">0 Tracks</strong>
 7588:           <span id="project-track-labels">project categories</span>
 7589:         </div>
 7590:         <div class="metric">
 7591:           <strong>Artifacts</strong>
 7592:           <span>source, diagrams, and reports</span>
 7593:         </div>
 7594:       </section>
 7595:
 7596:       <section class="section" id="projects" aria-labelledby="projects-title">
 7597:         <div class="section-heading">
 7598:           <div>
 7599:             <p class="eyebrow">Selected work</p>
 7600:             <h2 id="projects-title">Engineering Projects</h2>
 7601:           </div>
     ... excerpt truncated after 120 lines. Open the source file for the rest of this function.
```

renderPreview (template-preview.js, lines 7704-7706)

renderPreview renders ui, markup, or display output. In this file, it appears around lines 7704-7706 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fullPortfolioPreviewHtmlExact. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
7704: function renderPreview() {
7705:   portfolioPreviewFrame.srcdoc = fullPortfolioPreviewHtmlExact();
7706: }
```

openPortfolioPreview (template-preview.js, lines 7708-7713)

openPortfolioPreview controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 7708-7713 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references saveAllSections, renderPreview, add, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7709: if (!(await saveAllSections({ refreshOpenPreviews: false }))) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
7708: async function openPortfolioPreview() {
7709:   if (!(await saveAllSections({ refreshOpenPreviews: false }))) return;
7710:   renderPreview();
7711:   document.body.classList.add("full-window-open");
7712:   portfolioPreviewDialog.showModal();
7713: }
```

renderAll (template-preview.js, lines 7715-7732)

renderAll renders ui, markup, or display output. In this file, it appears around lines 7715-7732 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureSiteSections, selectedProject, sectionOptions, some, renderFunFactsEditor, renderSiteContentEditor, renderProfileEditor, renderSiteSectionList, renderTree, renderFields, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include project at line 7718. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7715: function renderAll() {
7716:   ensureSiteSections();
7717:   if (!selectedProjectId && catalog.projects.length) selectedProjectId = catalog.projects[0].id;
7718:   const project = selectedProject();
7719:   if (project && !sectionOptions(project).some((section) => section.id === activeSectionId)) {
7720:     activeSectionId = "brief";
7721:   }
7722:   renderFunFactsEditor();
7723:   renderSiteContentEditor();
7724:   renderProfileEditor();
7725:   renderSiteSectionList();
7726:   renderTree();
7727:   renderFields(project);
7728:   renderSectionTabs(project);
7729:   renderSectionContent(project);
7730:   if (portfolioPreviewDialog?.open) renderPreview();
7731:   updateBuilderWorkflow();
7732: }
```

updateProjectField (template-preview.js, lines 7734-7768)

updateProjectField updates ui, state, cache, or stored data. In this file, it appears around lines 7734-7768 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, split, map, trim, filter, limitWords, slugify, includes, setStatus, scheduleAutosave, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7736: if (!project) return;; line 7751: if (!nextId) return;.

Important local values include project at line 7735; needsChromeRender at line 7738; statusMessage at line 7739; limited at line 7743; nextId at line 7750. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7734: function updateProjectField(field, value) {
7735:   const project = selectedProject();
7736:   if (!project) return;
7737:
7738:   let needsChromeRender = false;
7739:   let statusMessage = "Unsaved local changes.";
7740:   if (field === "focus") {
7741:     project.focus = value.split(",").map((item) => item.trim()).filter(Boolean);
7742:   } else if (field === "summary") {
7743:     const limited = limitWords(value, 1000);
7744:     project.summary = limited;
7745:     if (limited !== value && document.activeElement?.dataset?.field === "summary") {
7746:       document.activeElement.value = limited;
7747:       statusMessage = "Project overview is limited to 1000 words.";
7748:     }
7749:   } else if (field === "id") {
7750:     const nextId = slugify(value);
7751:     if (!nextId) return;
7752:     project.id = nextId;
7753:     selectedProjectId = nextId;
7754:     needsChromeRender = true;
7755:   } else {
7756:     project[field] = value;
7757:     needsChromeRender = ["title", "category", "status"].includes(field);
7758:   }
7759:
7760:   if (field === "title") projectWindowTitle.textContent = value || "Untitled project";
7761:   setStatus(statusMessage);
7762:   scheduleAutosave();
7763:   if (needsChromeRender) {
7764:     scheduleChromeRender();
7765:   } else {
7766:     schedulePreviewRender();
7767:   }
7768: }
```

renameSelectedProject (template-preview.js, lines 7770-7775)

renameSelectedProject part of the private builder frontend and editing experience. In this file, it appears around lines 7770-7775 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, and beginTitleRename. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7772: if (!project) return;.

Important local values include project at line 7771. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7770: function renameSelectedProject() {
7771:   const project = selectedProject();
7772:   if (!project) return;
7773:   projectTitleMenu.hidden = true;
7774:   beginTitleRename();
7775: }
```

projectMetaRows (template-preview.js, lines 7777-7813)

projectMetaRows part of the private builder frontend and editing experience. In this file, it appears around lines 7777-7813 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references categoryById, projectTemplateFor, find, flatMap, filter, and toLocaleString. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7778: if (!project) return [];; line 7799: return [.

Important local values include category at line 7779; template at line 7780; savedProject at line 7781; design at line 7782; designFileCount at line 7783; uploadedFileCount at line 7791. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7777: function projectMetaRows(project) {
7778:   if (!project) return [];
7779:   const category = categoryById(project.category);
7780:   const template = projectTemplateFor(project);
7781:   const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project.id);
7782:   const design = project.design || {};
7783:   const designFileCount = [
7784:     ...(design.brief?.files || []),
7785:     ...(design.documentation?.files || []),
7786:     ...(design.documentation?.references || []),
7787:     ...(design.documentation?.mathAnalysis || []),
7788:     ...(design.simulation?.files || []),
7789:     ...(design.simulation?.results || [])
7790:   ].length;
7791:   const uploadedFileCount = [
7792:     ...(project.documents || []),
7793:     ...(project.tests || []),
7794:     ...(project.pcbs || []),
7795:     ...(project.media || []),
7796:     ...(project.links || []),
7797:     ...(project.sections || []).flatMap((section) => section.items || [])
7798:   ].filter((item) => item.url || item.artifact).length + designFileCount;
7799:   return [
7800:     ["Title", project.title || "Untitled project"],
7801:     ["Project ID / folder", project.id || "Not assigned"],
7802:     ["Category", category.label || project.category || "Not assigned"],
7803:     ["Status", project.status || "Draft"],
7804:     ["Appearance", template.label || project.templateLabel || "No appearance - white project layout"],
7805:     ["Appearance ID", project.templateId || "None"],
7806:     ["Portfolio preview", savedProject ? "Saved into parsed portfolio preview" : "Not yet saved into
portfolio preview"],
7807:     ["Custom sections", String((project.sections || []).length)],
7808:     ["Project files / links", String(uploadedFileCount)],
7809:     ["Tools", String((project.tools || []).length)],
7810:     ["Languages", String((project.languages || []).length)],
7811:     ["Last parsed", savedProject?.portfolioView?.builtAt ? new
Date(savedProject.portfolioView.builtAt).toLocaleString() : "Not parsed yet"]
7812:   ];
7813: }
```

savedProject (template-preview.js, lines 7781-7782)

savedProject persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 7781-7782 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include savedProject at line 7781; design at line 7782. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7781:   const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project.id);
7782:   const design = project.design || {};
```

openProjectMetaDetails (template-preview.js, lines 7815-7827)

openProjectMetaDetails controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 7815-7827 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, projectMetaRows, map, join, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7817: if (!project) return;.

Important local values include project at line 7816. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7815: function openProjectMetaDetails() {
7816:   const project = selectedProject();
```

```
  if (!project) return;
  projectTitleMenu.hidden = true;
  projectMetaTitle.textContent = project.title || "Project details";
  projectMetaGrid.innerHTML = projectMetaRows(project).map(([label, value]) => `
    <article>
      <span>${label}</span>
      <strong>${value}</strong>
    </article>
  `).join("");
  if (!projectMetaDialog.open) projectMetaDialog.showModal();
}
```

selectTitleText (template-preview.js, lines 7829-7835)

selectTitleText part of the private builder frontend and editing experience. In this file, it appears around lines 7829-7835 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getSelection, createRange, selectNodeContents, removeAllRanges, and addRange. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include selection at line 7830; range at line 7831. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function selectTitleText() {
  const selection = window.getSelection();
  const range = document.createRange();
  range.selectNodeContents(projectWindowTitle);
  selection.removeAllRanges();
  selection.addRange(range);
}
```

beginTitleRename (template-preview.js, lines 7837-7846)

beginTitleRename part of the private builder frontend and editing experience. In this file, it appears around lines 7837-7846 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, focus, selectTitleText, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7839: if (!project) return;.

Important local values include project at line 7838. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function beginTitleRename() {
  const project = selectedProject();
  if (!project) return;
  originalTitleDraft = project.title || "";
  projectWindowTitle.contentEditable = "true";
  projectWindowTitle.focus();
  selectTitleText();
  titleRenameActions.hidden = false;
  setStatus("Edit the title, then click Save title.");
}
```

endTitleRename (template-preview.js, lines 7848-7865)

endTitleRename part of the private builder frontend and editing experience. In this file, it appears around lines 7848-7865 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, trim, getSelection, removeAllRanges, setStatus, scheduleAutosave, and scheduleChromeRender. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7850: if (!project) return;.

Important local values include project at line 7849; nextTitle at line 7851. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7848: function endTitleRename(saveChanges) {
 7849:   const project = selectedProject();
 7850:   if (!project) return;
 7851:   const nextTitle = projectWindowTitle.textContent.trim();
 7852:   projectWindowTitle.contentEditable = "false";
 7853:   titleRenameActions.hidden = true;
 7854:   window.getSelection()?.removeAllRanges();
 7855:   if (saveChanges && nextTitle) {
 7856:     project.title = nextTitle;
 7857:     projectWindowTitle.textContent = nextTitle;
 7858:     setStatus("Project title saved in the editor. Click Save project to include this version in the
portfolio preview.");
 7859:     scheduleAutosave();
 7860:     scheduleChromeRender();
 7861:   } else {
 7862:     projectWindowTitle.textContent = originalTitleDraft || project.title || "Untitled project";
 7863:     setStatus("Title edit canceled.");
 7864:   }
 7865: }
```

makeItemForSection (template-preview.js, lines 7867-7874)

makeItemForSection part of the private builder frontend and editing experience. In this file, it appears around lines 7867-7874 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 7868: if (key === "documents") return { title, type: "Document", url, status: url ? "uploaded" : "draft" };; line 7869: if (key === "tests") return { name: title, method: description || "Validation notes", status: url ? "uploaded" : "draft", artifact: url, result: description };; line 7870: if (key === "pcbs") return { name: title, revision: "Rev A", status: url ? "uploaded" : "draft", artifact: url };; line 7871: if (key === "media") return { title, url, status: url ? "uploaded" : "draft", caption: description };. There are 2 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 7867: function makeItemForSection(key, title, description = "", url = "") {
 7868:   if (key === "documents") return { title, type: "Document", url, status: url ? "uploaded" : "draft" };
 7869:   if (key === "tests") return { name: title, method: description || "Validation notes", status: url ?
"uploaded" : "draft", artifact: url, result: description };
 7870:   if (key === "pcbs") return { name: title, revision: "Rev A", status: url ? "uploaded" : "draft",
artifact: url };
 7871:   if (key === "media") return { title, url, status: url ? "uploaded" : "draft", caption: description };
 7872:   if (key === "links") return { label: title, url };
 7873:   return { title, description, url, status: url ? "uploaded" : "draft" };
 7874: }
```

addTextItem (template-preview.js, lines 7876-7891)

addTextItem part of the private builder frontend and editing experience. In this file, it appears around lines 7876-7891 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, prompt, includes, push, makeItemForSection, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7879: if (!project || !title) return;.

Important local values include project at line 7877; title at line 7878; description at line 7885. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7876: function addTextItem(key) {
 7877:   const project = selectedProject();
```

```
 7878:   const title = prompt("Type a title for this item.");
 7879:   if (!project || !title) return;
 7880:
 7881:   project[key] = project[key] || [];
 7882:   if (["highlights", "tools", "languages"].includes(key)) {
 7883:     project[key].push(title);
 7884:   } else {
 7885:     const description = prompt("Add a short description if you want.") || "";
 7886:     project[key].push(makeItemForSection(key, title, description));
 7887:   }
 7888:   setStatus("Item added in the editor. Click Save project to include this version in the portfolio
preview.");
 7889:   scheduleAutosave();
 7890:   renderAll();
 7891: }
```

editObjectItem (template-preview.js, lines 7893-7914)

editObjectItem part of the private builder frontend and editing experience. In this file, it appears around lines 7893-7914 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, prompt, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7896: if (!item) return;; line 7900: if (!nextTitle) return;.

Important local values include project at line 7894; item at line 7895; currentTitle at line 7898; nextTitle at line 7899; nextDescription at line 7901. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7893: function editObjectItem(key, index) {
 7894:   const project = selectedProject();
 7895:   const item = project?.[key]?.[index];
 7896:   if (!item) return;
 7897:
 7898:   const currentTitle = item.title || item.name || item.label || "";
 7899:   const nextTitle = prompt("Edit title.", currentTitle);
 7900:   if (!nextTitle) return;
 7901:   const nextDescription = prompt("Edit description.", item.description || item.caption || item.result ||
item.method || "") || "";
 7902:
 7903:   if ("title" in item) item.title = nextTitle;
 7904:   if ("name" in item) item.name = nextTitle;
 7905:   if ("label" in item) item.label = nextTitle;
 7906:   if ("caption" in item) item.caption = nextDescription;
 7907:   if ("result" in item) item.result = nextDescription;
 7908:   if ("description" in item) item.description = nextDescription;
 7909:   if (!("caption" in item) && !("result" in item) && !("description" in item)) item.description =
nextDescription;
 7910:
 7911:   setStatus("Item edited in the editor. Click Save project to include this version in the portfolio
preview.");
 7912:   scheduleAutosave();
 7913:   renderAll();
 7914: }
```

chooseFile (template-preview.js, lines 7916-7922)

chooseFile part of the private builder frontend and editing experience. In this file, it appears around lines 7916-7922 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references resolve, and click. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7917: return new Promise((resolve) => {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 7916: function chooseFile() {
 7917:   return new Promise((resolve) => {
 7918:     filePicker.value = "";
 7919:     filePicker.onchange = () => resolve(filePicker.files[0] || null);
 7920:     filePicker.click();
 7921:   });
 7922: }
```

newCompileFile (template-preview.js, lines 7924-7942)

newCompileFile part of the compile code language/tool/build/run flow. In this file, it appears around lines 7924-7942 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCodeLanguage, safeClientCodeFileName, defaultCodeFileName, normalizeCompileFileRole, inferCompileFileRole, slugify, now, random, toString, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7928: return {.

Important local values include normalized at line 7925; fileName at line 7926; role at line 7927. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7924: function newCompileFile(language = "javascript", seed = {}) {
 7925:   const normalized = normalizeCodeLanguage(language);
 7926:   const fileName = safeClientCodeFileName(seed.fileName || defaultCodeFileName(normalized), normalized);
 7927:   const role = normalizeCompileFileRole(seed.role || inferCompileFileRole(fileName, seed.code || "",
normalized), normalized);
 7928:   return {
 7929:     id: slugify(`${fileName}-${Date.now()}-${Math.random().toString(16).slice(2)}`),
 7930:     title: seed.title || fileName,
 7931:     fileName,
 7932:     language: normalized,
 7933:     role,
 7934:     code: String(seed.code || ""),
 7935:     stdin: "",
 7936:     savedPath: "",
 7937:     savedAt: "",
 7938:     waveform: null,
 7939:     lastResult: null,
 7940:     dirty: true
 7941:   };
 7942: }
```

activeCompileWorkspaceAndFile (template-preview.js, lines 7944-7947)

activeCompileWorkspaceAndFile part of the compile code language/tool/build/run flow. In this file, it appears around lines 7944-7947 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, ensureCompileCode, and activeCompileFile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7946: return { workspace, file: activeCompileFile(project) };.

Important local values include workspace at line 7945. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7944: function activeCompileWorkspaceAndFile(project = selectedProject()) {
7945:   const workspace = ensureCompileCode(project);
7946:   return { workspace, file: activeCompileFile(project) };
7947: }
```

updateCompilePreview (template-preview.js, lines 7949-7952)

updateCompilePreview updates ui, state, cache, or stored data. In this file, it appears around lines 7949-7952 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, and tokenizedCodeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include preview at line 7950. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7949: function updateCompilePreview(file) {
7950:   const preview = sectionContent.querySelector("[data-compile-preview]");
7951:   if (preview && file) preview.innerHTML = tokenizedCodeHtml(file.code || "", file.language);
7952: }
```

syncCompileEditorScroll (template-preview.js, lines 7954-7960)

syncCompileEditorScroll part of the compile code language/tool/build/run flow. In this file, it appears around lines 7954-7960 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, and querySelector. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7957: if (!preview || !textarea) return;.

Important local values include editor at line 7955; preview at line 7956. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7954: function syncCompileEditorScroll(textarea) {
7955:   const editor = textarea?.closest(".compile-code-active-editor");
7956:   const preview = editor?.querySelector(".compile-code-preview");
7957:   if (!preview || !textarea) return;
7958:   preview.scrollTop = textarea.scrollTop;
7959:   preview.scrollLeft = textarea.scrollLeft;
7960: }
```

compilePayload (template-preview.js, lines 7962-7984)

compilePayload part of the compile code language/tool/build/run flow. In this file, it appears around lines 7962-7984 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureCompileCode, inferCompileFileRole, and map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7964: return {.

Important local values include workspace at line 7963. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7962: function compilePayload(project, file, options = {}) {
7963:   const workspace = ensureCompileCode(project);
7964:   return {
7965:     projectId: project.id,
7966:     fileId: file.id,
7967:     title: file.title,
7968:     fileName: file.fileName,
7969:     language: file.language,
7970:     role: file.role || inferCompileFileRole(file.fileName, file.code, file.language),
7971:     code: file.code,
7972:     stdin: file.stdin || "",
7973:     workspaceFiles: (workspace.files || []).map((workspaceFile) => ({
7974:       id: workspaceFile.id,
7975:       title: workspaceFile.title,
7976:       fileName: workspaceFile.fileName,
7977:       language: workspaceFile.language,
7978:       role: workspaceFile.role || inferCompileFileRole(workspaceFile.fileName, workspaceFile.code,
workspaceFile.language),
7979:       code: workspaceFile.code || ""
7980:     })),
7981:     action: options.action || "",
7982:     forceRebuild: options.forceRebuild === true
7983:   };
7984: }
```

selectedCompileAppendDestination (template-preview.js, lines 7986-7990)

selectedCompileAppendDestination part of the compile code language/tool/build/run flow. In this file, it appears around lines 7986-7990 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureCompileCode, compileAppendDestinations, and find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7989: return destinations.find((destination) => destination.id === workspace.appendTargetId) || destinations[0] || null;.

Important local values include workspace at line 7987; destinations at line 7988. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7986: function selectedCompileAppendDestination(project) {
 7987:   const workspace = ensureCompileCode(project);
 7988:   const destinations = compileAppendDestinations(project);
 7989:   return destinations.find((destination) => destination.id === workspace.appendTargetId) ||
destinations[0] || null;
 7990: }
```

appendCompileCodeToProject (template-preview.js, lines 7992-8019)

This function inserts a compile workspace source file into a chosen project section or subsection as a formatted code block.

Successful code needs a clean path from experimentation into portfolio evidence. This function moves code from the IDE workspace into the public-facing project narrative.

It calls or references codeBlockForCompileFile, trim, setStatus, addCompileMessage, selectedCompileAppendDestination, richBlocksForCompileAppend, push, setByPath, plainTextFromRich, toISOString, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 7997: return false;; line 8004: return false;; line 8018: return true;.

Important local values include block at line 7993; destination at line 8000; blocks at line 8007. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 7992: function appendCompileCodeToProject(project, file) {
 7993:   const block = codeBlockForCompileFile(file);
 7994:   if (!block.code.trim()) {
 7995:     setStatus("Type or import code before appending it to the project.");
 7996:     addCompileMessage(project, "Append stopped because the active source is empty.", "warning");
 7997:     return false;
 7998:   }
 7999:
 8000:   const destination = selectedCompileAppendDestination(project);
 8001:   if (!destination) {
 8002:     setStatus("No project destination is available for this code block.");
 8003:     addCompileMessage(project, "Append stopped because no destination section is available.", "warning");
 8004:     return false;
 8005:   }
 8006:
 8007:   const blocks = richBlocksForCompileAppend(project, destination);
 8008:   blocks.push(block);
 8009:   setByPath(project, destination.richPath, { blocks });
 8010:   setByPath(project, destination.textPath, plainTextFromRich({ blocks }));
 8011:
 8012:   file.lastAppendedAt = new Date().toISOString();
 8013:   addCompileMessage(project, `${file.fileName || file.title || "Source code"} appended to
${destination.label}.`, "success");
 8014:   setStatus(`Code appended to ${destination.label}. Save project to include it in the portfolio
preview.`);
 8015:   scheduleAutosave();
 8016:   schedulePreviewRender();
 8017:   updateCompileMessagesPanel(project);
 8018:   return true;
 8019: }
```

saveCompileFile (template-preview.js, lines 8021-8038)

saveCompileFile persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 8021-8038 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, stringify, compilePayload, json, Error, toISOString, and addCompileMessage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8022: if (!project || !file) return null;; line 8037: return result.saved;.

Important local values include response at line 8023; result at line 8029. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8021: async function saveCompileFile(project, file) {
8022:   if (!project || !file) return null;
8023:   const response = await fetch(`/api/code/save?t=${Date.now()}`, {
8024:     method: "POST",
8025:     cache: "no-store",
8026:     headers: { "Content-Type": "application/json" },
8027:     body: JSON.stringify(compilePayload(project, file))
8028:   });
8029:   const result = await response.json();
8030:   if (!response.ok || !result.ok) throw new Error(result.error || "Code could not be saved.");
8031:   file.savedPath = result.saved?.sourcePath || "";
8032:   file.savedAt = result.saved?.savedAt || new Date().toISOString();
8033:   file.fileName = result.saved?.fileName || file.fileName;
8034:   file.language = result.saved?.language || file.language;
8035:   file.dirty = false;
8036:   addCompileMessage(project, `Saved ${file.fileName || "source file"}.`, "info");
8037:   return result.saved;
8038: }
```

importCompileFile (template-preview.js, lines 8040-8063)

importCompileFile part of the compile code language/tool/build/run flow. In this file, it appears around lines 8040-8063 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureCompileCode, chooseFile, flatMap, split, pop, toLowerCase, includes, setStatus, text, detectCodeLanguage, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8043: if (!file) return;; line 8048: return;.

Important local values include workspace at line 8041; file at line 8042; allowedExtensions at line 8044; extension at line 8045; code at line 8050; language at line 8051; profile at line 8052; sourceFile at line 8053. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8040: async function importCompileFile(project) {
8041:   const workspace = ensureCompileCode(project);
8042:   const file = await chooseFile();
8043:   if (!file) return;
8044:   const allowedExtensions = supportedCodeLanguages.flatMap((item) => item.extensions || []);
8045:   const extension = `.${String(file.name || "").split(".").pop().toLowerCase()}`;
8046:   if (!allowedExtensions.includes(extension)) {
8047:     setStatus("Choose a supported source file: C, C++, Verilog, SystemVerilog, LTspice, Java, JavaScript,
Python, or HTML.");
8048:     return;
8049:   }
8050:   const code = await file.text();
8051:   const language = detectCodeLanguage(code, file.name);
8052:   const profile = codeLanguageProfile(language);
8053:   const sourceFile = newCompileFile(profile.id, {
8054:     fileName: file.name,
8055:     title: file.name.replace(/\.[^.]+$/, ""),
8056:     code
8057:   });
8058:   workspace.files.push(sourceFile);
8059:   workspace.activeFileId = sourceFile.id;
8060:   setStatus(`Imported ${file.name}. Click Save source before compiling.`);
8061:   scheduleAutosave();
8062:   renderSectionContent(project);
8063: }
```

beautifyCompileFile (template-preview.js, lines 8065-8085)

beautifyCompileFile part of the compile code language/tool/build/run flow. In this file, it appears around lines 8065-8085 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, stringify, compilePayload, json, Error, setStatus, scheduleAutosave, and renderSectionContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage

keys detected.

It has 1 return statement(s). The most important visible returns are: line 8066: if (!file) return;.

Important local values include response at line 8068; result at line 8074. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8065: async function beautifyCompileFile(project, file) {
8066:   if (!file) return;
8067:   try {
8068:     const response = await fetch(`/api/code/beautify?t=${Date.now()}`, {
8069:       method: "POST",
8070:       cache: "no-store",
8071:       headers: { "Content-Type": "application/json" },
8072:       body: JSON.stringify(compilePayload(project, file))
8073:     });
8074:     const result = await response.json();
8075:     if (!response.ok || !result.ok) throw new Error(result.error || "Beautifier failed.");
8076:     file.code = result.code || file.code;
8077:     file.language = result.language || file.language;
8078:     file.dirty = true;
8079:     setStatus("Code beautified. Review it, then save source.");
8080:     scheduleAutosave();
8081:     renderSectionContent(project);
8082:   } catch (error) {
8083:     setStatus(error.message || "Code beautifier failed.");
8084:   }
8085: }
```

compileActiveFile (template-preview.js, lines 8087-8145)

This function packages the current compile workspace, saves the active file state, calls the backend compile endpoint, then updates terminal output, messages, waveform data, dirty flags, and UI state.

It is the frontend half of the Compile button. Without it, the editor would not connect to server-side compilers or immediately show output.

It calls or references ensureCompileCode, isHdlLanguage, addCompileMessage, updateCompileTerminalPanel, updateCompileMessagesPanel, saveCompileFile, fetch, now, stringify, compilePayload, and 5 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8088: if (!file) return;.

Important local values include workspace at line 8089; action at line 8090; actionLabel at line 8091; response at line 8102; result at line 8111; compileResult at line 8112. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8087: async function compileActiveFile(project, file, options = {}) {
8088:   if (!file) return;
8089:   const workspace = ensureCompileCode(project);
8090:   const action = options.action || (isHdlLanguage(file.language) ? "simulate" : "run");
8091:   const actionLabel = action === "compile" ? "Compile" : action === "build" ? "Build project" : action
=== "simulate" ? "Simulate" : "Run";
8092:   file.lastResult = {
8093:     ok: null,
8094:     terminal: `${actionLabel} started. Please wait...`
8095:   };
8096:   workspace.terminal = file.lastResult.terminal;
8097:   addCompileMessage(project, `${actionLabel} started for ${file.fileName || file.title || "source
file"}.`, "info");
8098:   updateCompileTerminalPanel(project, file);
8099:   updateCompileMessagesPanel(project);
8100:   try {
8101:     await saveCompileFile(project, file);
8102:     const response = await fetch(`/api/code/compile?t=${Date.now()}`, {
8103:       method: "POST",
8104:       cache: "no-store",
8105:       headers: { "Content-Type": "application/json" },
8106:       body: JSON.stringify(compilePayload(project, file, {
8107:         action,
8108:         forceRebuild: options.forceRebuild === true || action === "build"
8109:       }))
8110:     });
8111:     const result = await response.json();
8112:     const compileResult = result.result || {};
8113:     file.lastResult = {
8114:       ok: Boolean(result.ok),
8115:       language: compileResult.language || file.language,
8116:       terminal: compileResult.terminal || result.error || "No compiler output was returned.",
8117:       waveform: compileResult.waveform || null,
8118:       finishedAt: new Date().toISOString()
8119:     };
8120:     file.waveform = compileResult.waveform || null;
8121:     ensureCompileCode(project).waveform = compileResult.waveform || null;
```

```
    if (compileResult.saved) {
      file.savedPath = compileResult.saved.sourcePath || file.savedPath;
      file.savedAt = compileResult.saved.savedAt || file.savedAt;
    }
    file.dirty = false;
    ensureCompileCode(project).terminal = file.lastResult.terminal;
    updateCompileTerminalPanel(project, file);
    addCompileMessage(project, result.ok ? `${actionLabel} succeeded for ${file.fileName}.` :
`${actionLabel} completed with errors for ${file.fileName}.`, result.ok ? "success" : "error");
    setStatus(result.ok ? `${actionLabel} succeeded.` : `${actionLabel} completed with errors.`);
    scheduleAutosave();
    renderSectionContent(project);
  } catch (error) {
    file.lastResult = {
      ok: false,
      terminal: error.message || "Compile/run failed.",
      finishedAt: new Date().toISOString()
    };
    ensureCompileCode(project).terminal = file.lastResult.terminal;
    updateCompileTerminalPanel(project, file);
    addCompileMessage(project, `${actionLabel} failed for ${file.fileName || "source file"}.`, "error");
    setStatus(error.message || `${actionLabel} failed.`);
    renderSectionContent(project);
  }
}
```

checkCompileTools (template-preview.js, lines 8147-8162)

checkCompileTools part of the compile code language/tool/build/run flow. In this file, it appears around lines 8147-8162 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, json, Error, entries, map, join, ensureCompileCode, renderSectionContent, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8154: return `${status.ready ? "READY" : "MISSING"} ${status.label} (${id}) - ${toolText}`;.

Important local values include response at line 8149; result at line 8150; lines at line 8152; toolText at line 8153. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
async function checkCompileTools(project) {
  try {
    const response = await fetch(`/api/code/tools?t=${Date.now()}`, { cache: "no-store" });
    const result = await response.json();
    if (!response.ok || !result.ok) throw new Error(result.error || "Compiler status failed.");
    const lines = Object.entries(result.tools?.languages || {}).map(([id, status]) => {
      const toolText = Object.entries(status.tools || {}).map(([tool, value]) => `${tool}:
${value}`).join(", ") || "no compiler required";
      return `${status.ready ? "READY" : "MISSING"} ${status.label} (${id}) - ${toolText}`;
    });
    compileTerminalStatus = [`Compile workspace: ${result.tools?.compileRoot || ""}`,
...lines].join("\n");
    ensureCompileCode(project).terminal = compileTerminalStatus;
    renderSectionContent(project);
  } catch (error) {
    setStatus(error.message || "Compiler status failed.");
  }
}
```

installCompileTools (template-preview.js, lines 8164-8187)

installCompileTools part of the compile code language/tool/build/run flow. In this file, it appears around lines 8164-8187 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references codeLanguageLabel, renderSectionContent, fetch, now, stringify, json, toISOString, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8165: if (!file) return;.

Important local values include response at line 8169; result at line 8175. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8164: async function installCompileTools(project, file) {
8165:   if (!file) return;
8166:   file.lastResult = { ok: null, terminal: `Installing tools for ${codeLanguageLabel(file.language)}. This
can take several minutes...` };
8167:   renderSectionContent(project);
8168:   try {
8169:     const response = await fetch(`/api/code/install-tools?t=${Date.now()}`, {
8170:       method: "POST",
8171:       cache: "no-store",
8172:       headers: { "Content-Type": "application/json" },
8173:       body: JSON.stringify({ language: file.language })
8174:     });
8175:     const result = await response.json();
8176:     file.lastResult = {
8177:       ok: Boolean(result.ok),
8178:       terminal: result.terminal || result.error || "Tool installation finished.",
8179:       finishedAt: new Date().toISOString()
8180:     };
8181:     setStatus(result.ok ? "Compiler tool installation finished." : "Compiler tool installation could not
complete.");
8182:     renderSectionContent(project);
8183:   } catch (error) {
8184:     file.lastResult = { ok: false, terminal: error.message || "Tool installation failed." };
8185:     renderSectionContent(project);
8186:   }
8187: }
```

readFileAsDataUrl (template-preview.js, lines 8189-8196)

readFileAsDataUrl loads data from disk, network, browser storage, or a service. In this file, it appears around lines 8189-8196 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references FileReader, resolve, and readAsDataURL. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8190: return new Promise((resolve, reject) => {.

Important local values include reader at line 8191. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8189: function readFileAsDataUrl(file) {
8190:   return new Promise((resolve, reject) => {
8191:     const reader = new FileReader();
8192:     reader.onload = () => resolve(reader.result);
8193:     reader.onerror = reject;
8194:     reader.readAsDataURL(file);
8195:   });
8196: }
```

uploadProfileAsset (template-preview.js, lines 8198-8242)

uploadProfileAsset part of the private builder frontend and editing experience. In this file, it appears around lines 8198-8242 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references syncProfileFromInputs, chooseFile, Set, has, startsWith, setStatus, includes, extensionFor, toLowerCase, readFileAsDataUrl, and 9 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 8201: if (!file) return;; line 8206: return;; line 8213: return;; line 8232: return;.

Important local values include profile at line 8199; file at line 8200; imageKinds at line 8203; allowedResume at line 8210; data at line 8217; response at line 8218; result at line 8229. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8198: async function uploadProfileAsset(kind) {
8199:   const profile = syncProfileFromInputs();
8200:   const file = await chooseFile();
8201:   if (!file) return;
8202:
8203:   const imageKinds = new Set(["profileImage", "heroImage", "brandImage"]);
```

```
  if (imageKinds.has(kind) && !file.type.startsWith("image/")) {
    setStatus("Choose an image file for this profile asset.");
    return;
  }

  if (kind === "resumeUrl") {
    const allowedResume = [".pdf", ".doc", ".docx"].includes(extensionFor(file.name).toLowerCase());
    if (!allowedResume) {
      setStatus("Choose a PDF, DOC, or DOCX resume file.");
      return;
    }
  }

  const data = await readFileAsDataUrl(file);
  const response = await fetch(`/api/upload?t=${Date.now()}`, {
    method: "POST",
    cache: "no-store",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      projectId: "_site-profile",
      section: kind,
      fileName: file.name,
      data
    })
  });
  const result = await response.json();
  if (!response.ok) {
    setStatus(result.error || "Profile asset upload failed.");
    return;
  }

  profile[kind] = result.url;
  catalog.profile = normalizeProfile(profile);
  markDraftNeedsSave();
  setStatus("Profile asset saved locally. Click Save draft before applying to site.");
  scheduleAutosave();
  schedulePreviewRender();
  renderProfileEditor();
}
```

extensionFor (template-preview.js, lines 8244-8247)

extensionFor part of the private builder frontend and editing experience. In this file, it appears around lines 8244-8247 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references match. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8246: return match ? match[1] : "";.

Important local values include match at line 8245. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function extensionFor(fileName) {
  const match = String(fileName || "").match(/(\.[a-z0-9]+)$/i);
  return match ? match[1] : "";
}
```

withExtension (template-preview.js, lines 8249-8253)

withExtension part of the private builder frontend and editing experience. In this file, it appears around lines 8249-8253 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and extensionFor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8251: if (!trimmed) return fallbackFileName;; line 8252: return /\.[a-z0-9]+$/i.test(trimmed) ? trimmed : `${trimmed}${extensionFor(fallbackFileName)}`;.

Important local values include trimmed at line 8250. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function withExtension(name, fallbackFileName) {
```

```
8250:   const trimmed = String(name || "").trim();
8251:   if (!trimmed) return fallbackFileName;
8252:   return /\.[a-z0-9]+$/i.test(trimmed) ? trimmed : `${trimmed}${extensionFor(fallbackFileName)}`;
8253: }
```

uploadProfile (template-preview.js, lines 8255-8286)

uploadProfile part of the private builder frontend and editing experience. In this file, it appears around lines 8255-8286 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8285: return profiles[key] || profiles.sections;.

Important local values include anyProjectFile at line 8256; profiles at line 8263. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8255: function uploadProfile(key) {
8256:   const anyProjectFile = {
8257:     label: "project file",
8258:     accept: "",
8259:     extensions: [],
8260:     mimeStarts: [],
8261:     allFiles: true
8262:   };
8263:   const profiles = {
8264:     media: {
8265:       ...anyProjectFile,
8266:       label: "image or file"
8267:     },
8268:     pcbs: {
8269:       ...anyProjectFile,
8270:       label: "PCB artifact"
8271:     },
8272:     tests: {
8273:       ...anyProjectFile,
8274:       label: "test artifact"
8275:     },
8276:     documents: {
8277:       ...anyProjectFile,
8278:       label: "document"
8279:     },
8280:     sections: {
8281:       ...anyProjectFile,
8282:       label: "section file"
8283:     }
8284:   };
8285:   return profiles[key] || profiles.sections;
8286: }
```

allowedFileMessage (template-preview.js, lines 8288-8291)

allowedFileMessage part of the private builder frontend and editing experience. In this file, it appears around lines 8288-8291 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8289: if (profile.allFiles) return `Add any local file type for this ${profile.label}.`;; line 8290: return `Add a ${profile.label}: ${profile.extensions.join(", ")}`;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
8288: function allowedFileMessage(profile) {
8289:   if (profile.allFiles) return `Add any local file type for this ${profile.label}.`;
8290:   return `Add a ${profile.label}: ${profile.extensions.join(", ")}`;
8291: }
```

isAllowedUpload (template-preview.js, lines 8293-8306)

isAllowedUpload part of the private builder frontend and editing experience. In this file, it appears around lines 8293-8306 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uploadProfile, extensionFor, toLowerCase, includes, startsWith, some, and allowedFileMessage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 8295: if (profile.allFiles) return { ok: true, message: "" };; line 8298: return { ok: false, message: "Images can only accept image files. Add PDFs under Documents, Tests, or a custom section instead." };; line 8302: return {.

Important local values include profile at line 8294; extension at line 8296; allowedByExtension at line 8300; allowedByMime at line 8301. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8293: function isAllowedUpload(key, fileName, mimeType = "") {
8294:   const profile = uploadProfile(key);
8295:   if (profile.allFiles) return { ok: true, message: "" };
8296:   const extension = extensionFor(fileName).toLowerCase();
8297:   if (key === "media" && !profile.extensions.includes(extension) && !mimeType.startsWith("image/")) {
8298:     return { ok: false, message: "Images can only accept image files. Add PDFs under Documents, Tests, or
a custom section instead." };
8299:   }
8300:   const allowedByExtension = profile.extensions.includes(extension);
8301:   const allowedByMime = profile.mimeStarts.some((prefix) => mimeType.startsWith(prefix));
8302:   return {
8303:     ok: allowedByExtension || allowedByMime,
8304:     message: `${fileName || "This file"} is not a supported ${profile.label}.
${allowedFileMessage(profile)}.`
8305:   };
8306: }
```

validateAssetForSection (template-preview.js, lines 8308-8317)

validateAssetForSection validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 8308-8317 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isAllowedUpload, normalizeLinkTarget, extensionFromUrl, and extensionFor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 8310: return isAllowedUpload(key, asset.fileName || asset.file?.name || "", asset.file?.type || "");; line 8314: return { ok: true, message: "" };; line 8316: return isAllowedUpload(key, url || asset.title || "", "");.

Important local values include url at line 8312. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8308: function validateAssetForSection(key, asset) {
8309:   if (asset.source === "local") {
8310:     return isAllowedUpload(key, asset.fileName || asset.file?.name || "", asset.file?.type || "");
8311:   }
8312:   const url = normalizeLinkTarget(asset.url || "", { assumeWeb: true });
8313:   if (!extensionFromUrl(url) && !extensionFor(asset.title || "")) {
8314:     return { ok: true, message: "" };
8315:   }
8316:   return isAllowedUpload(key, url || asset.title || "", "");
8317: }
```

dialogValue (template-preview.js, lines 8319-8328)

dialogValue part of the private builder frontend and editing experience. In this file, it appears around lines 8319-8328 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references removeEventListener, resolve, addEventListener, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8320: return new Promise((resolve) => {.

Important local values include onClose at line 8321. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8319: function dialogValue(dialog) {
8320:   return new Promise((resolve) => {
8321:     const onClose = () => {
8322:       dialog.removeEventListener("close", onClose);
8323:       resolve(dialog.returnValue === "save");
8324:     };
8325:     dialog.addEventListener("close", onClose);
8326:     dialog.showModal();
8327:   });
8328: }
```

onClose (template-preview.js, lines 8321-8324)

onClose part of the private builder frontend and editing experience. In this file, it appears around lines 8321-8324 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references removeEventListener, and resolve. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include onClose at line 8321. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8321:     const onClose = () => {
8322:       dialog.removeEventListener("close", onClose);
8323:       resolve(dialog.returnValue === "save");
8324:     };
```

openAssetDialog (template-preview.js, lines 8330-8392)

openAssetDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 8330-8392 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uploadProfile, updateAssetDialogVisibility, dialogValue, trim, setStatus, withExtension, isAllowedUpload, normalizeLinkTarget, displayNameFromUrl, validateAssetForSection, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 8344: if (!saved) return null;; line 8355: return null;; line 8362: return null;; line 8367: return null;. There are 2 additional return statements in the function body.

Important local values include profile at line 8331; saved at line 8343; file at line 8346; url at line 8347; fileName at line 8348; title at line 8349; validation at line 8359; validation at line 8371; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8330: async function openAssetDialog(key) {
8331:   const profile = uploadProfile(key);
8332:   assetDialogTitle.textContent = `Add ${profile.label}`;
8333:   assetSource.value = "local";
8334:   assetFile.value = "";
8335:   assetFile.accept = profile.accept;
8336:   assetUrl.value = "";
8337:   assetTitle.value = "";
8338:   assetCaption.value = "";
8339:   captionSource.value = "text";
8340:   captionFile.value = "";
8341:   updateAssetDialogVisibility();
8342:
8343:   const saved = await dialogValue(assetDialog);
8344:   if (!saved) return null;
8345:
8346:   let file = null;
8347:   let url = assetUrl.value.trim();
8348:   let fileName = "";
```

```
  let title = assetTitle.value.trim();

  if (assetSource.value === "local") {
    file = assetFile.files[0];
    if (!file) {
      setStatus("Choose a local file first.");
      return null;
    }
    fileName = withExtension(title, file.name);
    title = title || file.name;
    const validation = isAllowedUpload(key, fileName, file.type);
    if (!validation.ok) {
      setStatus(validation.message);
      return null;
    }
  } else {
    if (!url) {
      setStatus("Paste a web or Google Drive link first.");
      return null;
    }
    url = normalizeLinkTarget(url, { assumeWeb: true });
    title = title || displayNameFromUrl(url, "Linked asset");
    const validation = validateAssetForSection(key, { source: assetSource.value, url, title });
    if (!validation.ok) {
      setStatus(validation.message);
      return null;
    }
  }

  let caption = assetCaption.value.trim();
  if (captionSource.value === "file" && captionFile.files[0]) {
    caption = await captionFile.files[0].text();
  }

  return {
    caption,
    file,
    fileName,
    kind: file ? "file" : inferUrlAssetKind(url, assetSource.value),
    source: assetSource.value,
    title,
    url
  };
}
```

uploadToSection (template-preview.js, lines 8394-8454)

uploadToSection part of the private builder frontend and editing experience. In this file, it appears around lines 8394-8454 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, openAssetDialog, validateAssetForSection, setStatus, labelForUrlAssetKind, readFileAsDataUrl, fetch, stringify, json, find, and 4 more. Endpoint references found inside it are /api/upload. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 8396: if (!project) return;; line 8399: if (!asset) return;; line 8404: return;; line 8426: return;. There are 1 additional return statements in the function body.

Important local values include project at line 8395; asset at line 8398; validation at line 8401; sectionFolder at line 8407; url at line 8408; type at line 8409; data at line 8412; response at line 8413; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
async function uploadToSection(key, folder, customSectionId = "", customItemIndex = null) {
  const project = selectedProject();
  if (!project) return;

  const asset = await openAssetDialog(key);
  if (!asset) return;

  const validation = validateAssetForSection(key, asset);
  if (!validation.ok) {
    setStatus(validation.message);
    return;
  }

  const sectionFolder = customSectionId || folder || key;
  let url = asset.url;
  let type = asset.source === "local" ? "File" : labelForUrlAssetKind(asset.kind);

  if (asset.file) {
    const data = await readFileAsDataUrl(asset.file);
    const response = await fetch("/api/upload", {
```

```
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        projectId: project.id,
        section: sectionFolder,
        fileName: asset.fileName,
        data
      })
    });
    const result = await response.json();
    if (!response.ok) {
      setStatus(result.error || "Upload failed.");
      return;
    }
    url = result.url;
    type = asset.file.type || "File";
  }

  if (customSectionId) {
    const section = (project.sections || []).find((item) => item.id === customSectionId);
    const uploadedItem = { title: asset.title, description: asset.caption, type, url, status:
asset.source === "local" ? "uploaded" : "linked" };
    if (customItemIndex !== null && customItemIndex !== "") {
      const parent = section?.items?.[Number(customItemIndex)];
      if (!parent || parent.url) return;
      parent.children = parent.children || [];
      parent.children.push(uploadedItem);
    } else {
      section.items = section.items || [];
      section.items.push(uploadedItem);
    }
  } else {
    project[key] = project[key] || [];
    project[key].push(makeItemForSection(key, asset.title, asset.caption, url));
  }

  setStatus(asset.source === "local"
    ? `Saved ${asset.fileName} to ${url}. Click Save project to include it in the portfolio preview.`
    : `Linked ${asset.title}. Click Save project to include it in the portfolio preview.`);
  scheduleAutosave();
  renderAll();
}
```

section (template-preview.js, lines 8433-8434)

section part of the private builder frontend and editing experience. In this file, it appears around lines 8433-8434 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include section at line 8433; uploadedItem at line 8434. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
    const section = (project.sections || []).find((item) => item.id === customSectionId);
    const uploadedItem = { title: asset.title, description: asset.caption, type, url, status:
asset.source === "local" ? "uploaded" : "linked" };
```

uploadToDesignSection (template-preview.js, lines 8456-8502)

uploadToDesignSection part of the private builder frontend and editing experience. In this file, it appears around lines 8456-8502 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, openAssetDialog, validateAssetForSection, setStatus, labelForUrlAssetKind, readFileAsDataUrl, fetch, stringify, json, designArray, and 4 more. Endpoint references found inside it are /api/upload. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 8458: if (!project) return;; line 8461: if (!asset) return;; line 8466: return;; line 8486: return;.

Important local values include project at line 8457; asset at line 8460; validation at line 8463; url at line 8469; type at line 8470; data at line 8472; response at line 8473; result at line 8483; plus 1 more local variable(s). These variables show what the function is assembling,

checking, or handing to another layer.

Relevant source excerpt:

```
8456: async function uploadToDesignSection(pathValue, folder, kind = "document") {
8457:   const project = selectedProject();
8458:   if (!project) return;
8459:
8460:   const asset = await openAssetDialog(kind === "simulation-result" ? "tests" : "sections");
8461:   if (!asset) return;
8462:
8463:   const validation = validateAssetForSection(kind === "simulation-result" ? "tests" : "sections", asset);
8464:   if (!validation.ok) {
8465:     setStatus(validation.message);
8466:     return;
8467:   }
8468:
8469:   let url = asset.url;
8470:   let type = asset.source === "local" ? "File" : labelForUrlAssetKind(asset.kind);
8471:   if (asset.file) {
8472:     const data = await readFileAsDataUrl(asset.file);
8473:     const response = await fetch("/api/upload", {
8474:       method: "POST",
8475:       headers: { "Content-Type": "application/json" },
8476:       body: JSON.stringify({
8477:         projectId: project.id,
8478:         section: folder || "design",
8479:         fileName: asset.fileName,
8480:         data
8481:       })
8482:     });
8483:     const result = await response.json();
8484:     if (!response.ok) {
8485:       setStatus(result.error || "Upload failed.");
8486:       return;
8487:     }
8488:     url = result.url;
8489:     type = asset.file.type || "File";
8490:   }
8491:
8492:   const items = designArray(project, pathValue);
8493:   items.push({
8494:     ...makeDesignItem(kind, asset.title, asset.caption, url),
8495:     type
8496:   });
8497:   setStatus(asset.source === "local"
8498:     ? `Saved ${asset.fileName} to ${url}. Click Save project to include it in the portfolio preview.`
8499:     : `Linked ${asset.title}. Click Save project to include it in the portfolio preview.`);
8500:   scheduleAutosave();
8501:   renderAll();
8502: }
```

openSectionDialog (template-preview.js, lines 8504-8523)

openSectionDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 8504-8523 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references dialogValue, trim, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 8511: if (!saved) return null;; line 8516: return null;; line 8519: return {.

Important local values include saved at line 8510; title at line 8513. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8504: async function openSectionDialog() {
8505:   sectionDialogTitle.textContent = "Create a section";
8506:   sectionTitle.placeholder = "Design, Measurements, Firmware...";
8507:   sectionDescription.placeholder = "Optional description shown to recruiters";
8508:   sectionTitle.value = "";
8509:   sectionDescription.value = "";
8510:   const saved = await dialogValue(sectionDialog);
8511:   if (!saved) return null;
8512:
8513:   const title = sectionTitle.value.trim();
8514:   if (!title) {
8515:     setStatus("Type a section title before saving.");
8516:     return null;
8517:   }
8518:
8519:   return {
```

```
8520:     description: sectionDescription.value.trim(),
8521:     title
8522:   };
8523: }
```

openCategoryDialog (template-preview.js, lines 8525-8531)

openCategoryDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 8525-8531 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references showModal, and focus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
8525: function openCategoryDialog() {
8526:   categoryTitle.value = "";
8527:   categoryDescription.value = "";
8528:   categoryAccent.value = "#1677a8";
8529:   categoryDialog.showModal();
8530:   setTimeout(() => categoryTitle.focus(), 0);
8531: }
```

saveCategoryFromDialog (template-preview.js, lines 8533-8559)

saveCategoryFromDialog persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 8533-8559 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, setStatus, slugify, Set, map, has, normalizeCategory, add, refreshCategoryControls, renderAll, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8537: return;.

Important local values include label at line 8534; baseId at line 8539; ids at line 8540; id at line 8541; suffix at line 8542; category at line 8547. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8533: function saveCategoryFromDialog() {
8534:   const label = categoryTitle.value.trim();
8535:   if (!label) {
8536:     setStatus("Type a category name before saving.");
8537:     return;
8538:   }
8539:   const baseId = slugify(label);
8540:   const ids = new Set((catalog.categories || []).map((category) => category.id));
8541:   let id = baseId;
8542:   let suffix = 2;
8543:   while (ids.has(id)) {
8544:     id = `${baseId}-${suffix}`;
8545:     suffix += 1;
8546:   }
8547:   const category = normalizeCategory({
8548:     accent: categoryAccent.value,
8549:     description: categoryDescription.value,
8550:     id,
8551:     label
8552:   });
8553:   catalog.categories = [...(catalog.categories || []), category];
8554:   expandedCategories.add(category.id);
8555:   refreshCategoryControls();
8556:   renderAll();
8557:   scheduleAutosave();
8558:   setStatus(`Category "${category.label}" added locally. It is now available under Add new project.`);
8559: }
```

addCustomSection (template-preview.js, lines 8561-8582)

addCustomSection part of the private builder frontend and editing experience. In this file, it appears around lines 8561-8582 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, openSectionDialog, slugify, Set, map, has, push, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8563: if (!project) return;; line 8566: if (!sectionInput) return;.

Important local values include project at line 8562; sectionInput at line 8565; id at line 8568; existingIds at line 8569; suffix at line 8570. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8561: async function addCustomSection() {
8562:   const project = selectedProject();
8563:   if (!project) return;
8564:
8565:   const sectionInput = await openSectionDialog();
8566:   if (!sectionInput) return;
8567:
8568:   let id = slugify(sectionInput.title);
8569:   const existingIds = new Set((project.sections || []).map((section) => section.id));
8570:   let suffix = 2;
8571:   while (existingIds.has(id)) {
8572:     id = `${slugify(sectionInput.title)}-${suffix}`;
8573:     suffix += 1;
8574:   }
8575:
8576:   project.sections = project.sections || [];
8577:   project.sections.push({ id, title: sectionInput.title, description: sectionInput.description, items: []
});
8578:   activeSectionId = `custom:${id}`;
8579:   setStatus("New section added in the editor. Click Save project to include this version in the portfolio
preview.");
8580:   scheduleAutosave();
8581:   renderAll();
8582: }
```

addCustomItem (template-preview.js, lines 8584-8596)

addCustomItem part of the private builder frontend and editing experience. In this file, it appears around lines 8584-8596 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, find, prompt, push, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8588: if (!section || !title) return;.

Important local values include project at line 8585; section at line 8586; title at line 8587; description at line 8589. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8584: function addCustomItem(sectionId) {
8585:   const project = selectedProject();
8586:   const section = (project.sections || []).find((item) => item.id === sectionId);
8587:   const title = prompt("Type a subsection title.");
8588:   if (!section || !title) return;
8589:   const description = prompt("Paste or type a text description if you want.") || "";
8590:
8591:   section.items = section.items || [];
8592:   section.items.push({ title, description, type: "Text", status: "draft" });
8593:   setStatus("Subsection added locally.");
8594:   scheduleAutosave();
8595:   renderAll();
8596: }
```

section (template-preview.js, lines 8586-8589)

section part of the private builder frontend and editing experience. In this file, it appears around lines 8586-8589 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and prompt. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8588: if (!section || !title) return;.

Important local values include section at line 8586; title at line 8587; description at line 8589. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8586:   const section = (project.sections || []).find((item) => item.id === sectionId);
8587:   const title = prompt("Type a subsection title.");
8588:   if (!section || !title) return;
8589:   const description = prompt("Paste or type a text description if you want.") || "";
```

editCustomItem (template-preview.js, lines 8598-8609)

editCustomItem part of the private builder frontend and editing experience. In this file, it appears around lines 8598-8609 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, find, prompt, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8601: if (!item) return;; line 8603: if (!title) return;.

Important local values include project at line 8599; item at line 8600; title at line 8602. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8598: function editCustomItem(sectionId, index) {
8599:   const project = selectedProject();
8600:   const item = (project.sections || []).find((section) => section.id === sectionId)?.items?.[index];
8601:   if (!item) return;
8602:   const title = prompt("Edit subsection title.", item.title || "");
8603:   if (!title) return;
8604:   item.title = title;
8605:   item.description = prompt("Edit description.", item.description || "") || "";
8606:   setStatus("Subsection edited locally.");
8607:   scheduleAutosave();
8608:   renderAll();
8609: }
```

item (template-preview.js, lines 8600-8608)

item part of the private builder frontend and editing experience. In this file, it appears around lines 8600-8608 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, prompt, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8601: if (!item) return;; line 8603: if (!title) return;.

Important local values include item at line 8600; title at line 8602. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8600:   const item = (project.sections || []).find((section) => section.id === sectionId)?.items?.[index];
8601:   if (!item) return;
8602:   const title = prompt("Edit subsection title.", item.title || "");
8603:   if (!title) return;
8604:   item.title = title;
8605:   item.description = prompt("Edit description.", item.description || "") || "";
8606:   setStatus("Subsection edited locally.");
8607:   scheduleAutosave();
8608:   renderAll();
```

catalogForSave (template-preview.js, lines 8611-8633)

catalogForSave part of the private builder frontend and editing experience. In this file, it appears around lines 8611-8633 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parsedPortfolioGlobals, and clone. Endpoint references found inside it are /api/apply-catalog. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8614: return {; line 8620: return {.

Important local values include parsedGlobals at line 8612. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8611: function catalogForSave(endpoint) {
8612:   const parsedGlobals = parsedPortfolioGlobals();
8613:   if (endpoint === "/api/apply-catalog") {
8614:     return {
8615:       ...savedPortfolioCatalog,
8616:       ...parsedGlobals,
8617:       projects: clone(savedPortfolioCatalog.projects || [])
8618:     };
8619:   }
8620:   return {
8621:     ...catalog,
8622:     categories: parsedGlobals.categories,
8623:     fieldStyles: parsedGlobals.fieldStyles,
8624:     funFacts: parsedGlobals.funFacts,
8625:     funFactsRich: parsedGlobals.funFactsRich,
8626:     profile: parsedGlobals.profile,
8627:     profileRich: parsedGlobals.profileRich,
8628:     siteContent: parsedGlobals.siteContent,
8629:     siteContentRich: parsedGlobals.siteContentRich,
8630:     siteSections: clone(catalog.siteSections || []),
8631:     projects: clone(catalog.projects || [])
8632:   };
8633: }
```

saveCatalog (template-preview.js, lines 8635-8697)

This function saves the builder catalog through the backend, coordinates parser output, and updates local saved state after successful persistence.

Most builder edits mean nothing until the catalog is saved. This function is the local persistence checkpoint before previewing or publishing.

It calls or references catalogForSave, showBuilderError, setStatus, setSaveState, fetch, now, stringify, json, and showPublishResult. Endpoint references found inside it are /api/apply-catalog, and /api/save-draft. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 8645: return;; line 8655: return;; line 8675: return;; line 8683: return;.

Important local values include targetCatalog at line 8636; applying at line 8662; response at line 8665; result at line 8671. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8635: async function saveCatalog(endpoint, message) {
8636:   const targetCatalog = catalogForSave(endpoint);
8637:   if (endpoint === "/api/apply-catalog" && !draftSavedSinceChanges) {
8638:     showBuilderError(
8639:       "Save draft required",
8640:       "Please click Save draft before applying this portfolio to the live site.",
8641:       "Apply to site was stopped before commit and push. Save the draft, then click Apply to site again."
8642:     );
8643:     setStatus("Apply to site stopped. Save draft first.");
8644:     setSaveState("dirty", "Save draft needed");
8645:     return;
8646:   }
8647:   if (endpoint === "/api/apply-catalog" && !(targetCatalog.projects || []).length) {
8648:     showBuilderError(
8649:       "No saved project preview",
8650:       "Save a project or use Save all sections before applying to the live site.",
8651:       "Apply to site was stopped before commit and push."
8652:     );
8653:     setStatus("Apply to site stopped. Save a project or use Save all sections first.");
8654:     setSaveState("dirty", "Preview needs projects");
8655:     return;
8656:   }
8657:
8658:   try {
8659:     clearTimeout(autosaveTimer);
8660:     saveDraftButton.disabled = true;
8661:     applyCatalogButton.disabled = true;
8662:     const applying = endpoint === "/api/apply-catalog";
8663:     setStatus(applying ? "Applying site changes..." : "Saving draft...");
8664:     setSaveState(applying ? "publishing" : "saving", applying ? "Publishing..." : "Saving...");
8665:     const response = await fetch(`${endpoint}?t=${Date.now()}`, {
8666:       method: "POST",
```

```
8667:       cache: "no-store",
8668:       headers: { "Content-Type": "application/json" },
8669:       body: JSON.stringify({ catalog: targetCatalog })
8670:     });
8671:     const result = await response.json();
8672:     if (!response.ok) {
8673:       setStatus(result.error || "Save failed.");
8674:       setSaveState("error", endpoint === "/api/apply-catalog" ? "Publish failed" : "Save failed");
8675:       return;
8676:     }
8677:     if (result.publish) {
8678:       setStatus(result.publish.pushed
8679:         ? `${message}: ${result.file}. Pushed to ${result.publish.branch}.`
8680:         : `${message}: ${result.file}. Git push failed: ${result.publish.error}`);
8681:       setSaveState(result.publish.pushed ? "published" : "error", result.publish.pushed ? "Site applied"
: "Push failed");
8682:       showPublishResult(result);
8683:       return;
8684:     }
8685:     if (endpoint === "/api/save-draft") {
8686:       draftSavedSinceChanges = true;
8687:       setSaveState("saved", "Draft saved");
8688:     }
8689:     setStatus(`${message}: ${result.file}`);
8690:   } catch {
8691:     setStatus("Save failed. Make sure the local server window is still running.");
8692:     setSaveState("error", "Save failed");
8693:   } finally {
8694:     saveDraftButton.disabled = false;
8695:     applyCatalogButton.disabled = false;
8696:   }
8697: }
```

loadData (template-preview.js, lines 8699-8745)

This function loads templates, projects, saved site content, profile data, categories, and current builder state into the frontend.

It is the startup data path for the builder UI. If it fails, panels may appear empty even when files exist on disk.

It calls or references all, fetch, now, json, call, normalizeFunFacts, clone, normalizeFieldStyles, normalizeProfile, normalizeSiteContent, and 13 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include catalogData at line 8704; templateData at line 8705. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8699: async function loadData() {
8700:   const [catalogResponse, templateResponse] = await Promise.all([
8701:     fetch(`/api/catalog?t=${Date.now()}`, { cache: "no-store" }),
8702:     fetch(`/api/templates?t=${Date.now()}`, { cache: "no-store" })
8703:   ]);
8704:   const catalogData = await catalogResponse.json();
8705:   const templateData = await templateResponse.json();
8706:
8707:   catalog = catalogData.catalog;
8708:   catalog.funFacts = Object.prototype.hasOwnProperty.call(catalog, "funFacts")
8709:     ? normalizeFunFacts(catalog.funFacts)
8710:     : clone(defaultFunFacts);
8711:   catalog.funFactsRich = catalog.funFactsRich || null;
8712:   catalog.fieldStyles = normalizeFieldStyles(catalog.fieldStyles || {});
8713:   catalog.siteContentRich = catalog.siteContentRich || {};
8714:   catalog.profileRich = catalog.profileRich || {};
8715:   catalog.profile = normalizeProfile(catalog.profile || {});
8716:   catalog.siteContent = normalizeSiteContent(catalog.siteContent || {});
8717:   catalog.siteSections = catalog.siteSections || [];
8718:   catalog.projects = (catalog.projects || []).filter((project) => !starterProjectIds.has(project.id));
8719:   catalog.projects.forEach((project) => {
8720:     if (isAnalogProject(project)) ensureDesignModel(project);
8721:     ensureCompileCode(project);
8722:   });
8723:   savedPortfolioCatalog = {
8724:     categories: clone(catalog.categories || []),
8725:     fieldStyles: clone(catalog.fieldStyles || {}),
8726:     funFacts: clone(catalog.funFacts || []),
8727:     funFactsRich: clone(catalog.funFactsRich || null),
8728:     profile: clone(normalizeProfile(catalog.profile || {})),
8729:     profileRich: clone(catalog.profileRich || {}),
8730:     siteContent: clone(catalog.siteContent || defaultSiteContent),
8731:     siteContentRich: clone(catalog.siteContentRich || {}),
8732:     projects: [],
8733:     siteSections: clone(catalog.siteSections || [])
8734:   };
8735:   templates = templateData.templates || [];
8736:   setStatus(`Loaded ${catalogData.source} catalog. Builder controls are local-only.`);
```

```
  draftSavedSinceChanges = true;
  setSaveState("loaded", "Loaded");

  templateSelect.innerHTML = groupedTemplateOptions();
  refreshCategoryControls();
  selectedProjectId = catalog.projects[0]?.id || "";
  expandedCategories = new Set(catalog.categories.map((category) => category.id));
  renderAll();
}
```

prepareBuilderGuideTopicLinks (template-preview.js, lines 8758-8768)

prepareBuilderGuideTopicLinks part of the private builder frontend and editing experience. In this file, it appears around lines 8758-8768 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, querySelector, setAttribute, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8761: if (!summary) return;.

Important local values include summary at line 8760. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function prepareBuilderGuideTopicLinks() {
  builderGuideSections?.querySelectorAll("[data-guide-section]").forEach((section) => {
    const summary = section.querySelector("summary");
    if (!summary) return;
    summary.setAttribute("role", "link");
    summary.setAttribute("tabindex", "0");
    summary.setAttribute("aria-label", `Open ${summary.textContent.trim()}`);
    summary.title = "Open this guide topic";
    section.open = false;
  });
}
```

updateBuilderGuideStats (template-preview.js, lines 8770-8849)

updateBuilderGuideStats updates ui, state, cache, or stored data. In this file, it appears around lines 8770-8849 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, entries, forEach, querySelector, guideSectionText, join, toLowerCase, filterBuilderGuide, trim, querySelectorAll, and 10 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 8771: if (!builderGuideDialog) return;; line 8791: return [; line 8799: if (!builderGuideSections) return;.

Important local values include projectCount at line 8772; categoryCount at line 8773; parsedCount at line 8774; targetLabel at line 8775; values at line 8778; node at line 8785; query at line 8800; sections at line 8801; plus 5 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function updateBuilderGuideStats() {
  if (!builderGuideDialog) return;
  const projectCount = catalog.projects?.length || 0;
  const categoryCount = catalog.categories?.length || 0;
  const parsedCount = savedPortfolioCatalog.projects?.length || 0;
  const targetLabel = currentPublishTarget?.remote
    ? `Target: ${currentPublishTarget.remote.replace(/^https:\/\/github\.com\//i, "")}`
    : "Target: not connected";
  const values = {
    projects: `Projects: ${projectCount} (${parsedCount} parsed)`,
    categories: `Categories: ${categoryCount}`,
    draft: draftSavedSinceChanges ? "Draft: saved" : "Draft: unsaved changes",
    target: targetLabel
  };
  Object.entries(values).forEach(([key, value]) => {
    const node = builderGuideDialog.querySelector(`[data-guide-stat='${key}']`);
    if (node) node.textContent = value;
  });
}

function guideSectionText(section) {
```

```
  return [
    section.querySelector("summary")?.textContent || "",
    section.textContent || "",
    section.dataset.guideSection || ""
  ].join(" ").toLowerCase();
}

function filterBuilderGuide() {
  if (!builderGuideSections) return;
  const query = String(builderGuideSearch?.value || "").trim().toLowerCase();
  const sections = [...builderGuideSections.querySelectorAll("[data-guide-section]")];
  let visibleCount = 0;
  sections.forEach((section) => {
    const topicMatch = activeBuilderGuideTopic === "all" || (section.dataset.guideSection ||
"").split(/\s+/).includes(activeBuilderGuideTopic);
    const queryMatch = !query || guideSectionText(section).includes(query);
    const visible = topicMatch && queryMatch;
    section.hidden = !visible;
    if (visible) {
      visibleCount += 1;
      section.open = false;
    }
  });
  if (builderGuideResults) {
    const topicText = activeBuilderGuideTopic === "all" ? "all topics" : activeBuilderGuideTopic;
    builderGuideResults.textContent = visibleCount
      ? `Showing ${visibleCount} guide topic${visibleCount === 1 ? "" : "s"} for ${topicText}${query ? `
matching "${query}"` : ""}.`
      : `No guide topics match "${query}".`;
  }
}

function setBuilderGuideTopic(topic = "all") {
  activeBuilderGuideTopic = topic || "all";
  builderGuideDialog?.querySelectorAll("[data-guide-topic]").forEach((button) => {
    button.setAttribute("aria-pressed", button.dataset.guideTopic === activeBuilderGuideTopic ? "true" :
"false");
  });
  filterBuilderGuide();
}

function closeGuideAndFocus(target) {
  closeDialogElement(builderGuideDialog);
  requestAnimationFrame(() => {
    target?.scrollIntoView?.({ behavior: "smooth", block: "center" });
    if (target?.focus) target.focus({ preventScroll: true });
  });
}

function handleBuilderGuideAction(action = "") {
  if (action === "profile") closeGuideAndFocus(profileDisplayNameInput);
  if (action === "site-content") closeGuideAndFocus(siteHeroTitleInput);
  if (action === "fun-facts") closeGuideAndFocus(funFactsInput);
  if (action === "projects") closeGuideAndFocus(projectTree);
  if (action === "preview") {
    closeDialogElement(builderGuideDialog);
    openPortfolioPreviewButton?.click();
  }
  if (action === "target") {
    closeDialogElement(builderGuideDialog);
    publishTargetOpen?.click();
  }
```

guideSectionText (template-preview.js, lines 8790-8796)

guideSectionText part of the private builder frontend and editing experience. In this file, it appears around lines 8790-8796 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, join, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8791: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
function guideSectionText(section) {
  return [
    section.querySelector("summary")?.textContent || "",
    section.textContent || "",
    section.dataset.guideSection || ""
  ].join(" ").toLowerCase();
}
```

filterBuilderGuide (template-preview.js, lines 8798-8819)

filterBuilderGuide part of the private builder frontend and editing experience. In this file, it appears around lines 8798-8819 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, querySelectorAll, forEach, split, includes, and guideSectionText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8799: if (!builderGuideSections) return;.

Important local values include query at line 8800; sections at line 8801; visibleCount at line 8802; topicMatch at line 8804; queryMatch at line 8805; visible at line 8806; topicText at line 8814. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 8798: function filterBuilderGuide() {
 8799:   if (!builderGuideSections) return;
 8800:   const query = String(builderGuideSearch?.value || "").trim().toLowerCase();
 8801:   const sections = [...builderGuideSections.querySelectorAll("[data-guide-section]")];
 8802:   let visibleCount = 0;
 8803:   sections.forEach((section) => {
 8804:     const topicMatch = activeBuilderGuideTopic === "all" || (section.dataset.guideSection ||
"").split(/\s+/).includes(activeBuilderGuideTopic);
 8805:     const queryMatch = !query || guideSectionText(section).includes(query);
 8806:     const visible = topicMatch && queryMatch;
 8807:     section.hidden = !visible;
 8808:     if (visible) {
 8809:       visibleCount += 1;
 8810:       section.open = false;
 8811:     }
 8812:   });
 8813:   if (builderGuideResults) {
 8814:     const topicText = activeBuilderGuideTopic === "all" ? "all topics" : activeBuilderGuideTopic;
 8815:     builderGuideResults.textContent = visibleCount
 8816:       ? `Showing ${visibleCount} guide topic${visibleCount === 1 ? "" : "s"} for ${topicText}${query ? `
matching "${query}"` : ""}.`
 8817:       : `No guide topics match "${query}".`;
 8818:   }
 8819: }
```

setBuilderGuideTopic (template-preview.js, lines 8821-8827)

setBuilderGuideTopic part of the private builder frontend and editing experience. In this file, it appears around lines 8821-8827 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, setAttribute, and filterBuilderGuide. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 8821: function setBuilderGuideTopic(topic = "all") {
 8822:   activeBuilderGuideTopic = topic || "all";
 8823:   builderGuideDialog?.querySelectorAll("[data-guide-topic]").forEach((button) => {
 8824:     button.setAttribute("aria-pressed", button.dataset.guideTopic === activeBuilderGuideTopic ? "true" :
"false");
 8825:   });
 8826:   filterBuilderGuide();
 8827: }
```

closeGuideAndFocus (template-preview.js, lines 8829-8835)

closeGuideAndFocus controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 8829-8835 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closeDialogElement, requestAnimationFrame, and focus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
8829: function closeGuideAndFocus(target) {
8830:   closeDialogElement(builderGuideDialog);
8831:   requestAnimationFrame(() => {
8832:     target?.scrollIntoView?.({ behavior: "smooth", block: "center" });
8833:     if (target?.focus) target.focus({ preventScroll: true });
8834:   });
8835: }
```

handleBuilderGuideAction (template-preview.js, lines 8837-8854)

handleBuilderGuideAction handles an event, request, command, or user action. In this file, it appears around lines 8837-8854 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closeGuideAndFocus, closeDialogElement, and click. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
8837: function handleBuilderGuideAction(action = "") {
8838:   if (action === "profile") closeGuideAndFocus(profileDisplayNameInput);
8839:   if (action === "site-content") closeGuideAndFocus(siteHeroTitleInput);
8840:   if (action === "fun-facts") closeGuideAndFocus(funFactsInput);
8841:   if (action === "projects") closeGuideAndFocus(projectTree);
8842:   if (action === "preview") {
8843:     closeDialogElement(builderGuideDialog);
8844:     openPortfolioPreviewButton?.click();
8845:   }
8846:   if (action === "target") {
8847:     closeDialogElement(builderGuideDialog);
8848:     publishTargetOpen?.click();
8849:   }
8850:   if (action === "updates") {
8851:     closeDialogElement(builderGuideDialog);
8852:     appUpdateCheckButton?.click();
8853:   }
8854: }
```

openBuilderGuideTopic (template-preview.js, lines 8856-8878)

openBuilderGuideTopic controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 8856-8878 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, trim, replaceChildren, cloneNode, add, append, createElement, showModal, requestAnimationFrame, focus, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8857: if (!section || !builderGuideTopicDialog || !builderGuideTopicTitle || !builderGuideTopicBody) return;.

Important local values include summary at line 8858; source at line 8859; title at line 8860; copy at line 8864; empty at line 8868. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8856: function openBuilderGuideTopic(section) {
8857:   if (!section || !builderGuideTopicDialog || !builderGuideTopicTitle || !builderGuideTopicBody) return;
8858:   const summary = section.querySelector("summary");
8859:   const source = section.querySelector(":scope > div");
8860:   const title = summary?.textContent?.trim() || "Builder guide topic";
8861:   builderGuideTopicTitle.textContent = title;
8862:   builderGuideTopicBody.replaceChildren();
8863:   if (source) {
8864:     const copy = source.cloneNode(true);
8865:     copy.classList.add("builder-guide-topic-copy");
8866:     builderGuideTopicBody.append(copy);
8867:   } else {
```

```
    const empty = document.createElement("p");
    empty.textContent = "This guide topic does not have written instructions yet.";
    builderGuideTopicBody.append(empty);
  }
  if (!builderGuideTopicDialog.open) builderGuideTopicDialog.showModal();
  requestAnimationFrame(() => {
    builderGuideTopicBody.scrollTop = 0;
    builderGuideTopicBody.focus({ preventScroll: true });
    updateDialogWindowButtons(builderGuideTopicDialog);
  });
}
```

handleRichEditorContextMenu (template-preview.js, lines 9312-9372)

handleRichEditorContextMenu handles an event, request, command, or user action. In this file, it appears around lines 9312-9372 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richEditorFromContextEvent, preventDefault, stopPropagation, closest, clearPersistentTextSelection, hideTextSelectionInspector, configureSummaryContextMenu, selectionRangeInsideEditor, cloneRange, contains, and 9 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9316: if (!editor && !textBlock) return;.

Important local values include editor at line 9313; textBlock at line 9314; nextEditor at line 9320; liveRange at line 9333; savedRange at line 9334; savedContainer at line 9337; menuHost at line 9352; hostRect at line 9359; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
function handleRichEditorContextMenu(event) {
    const editor = richEditorFromContextEvent(event);
    const textBlock = event.target.closest?.(".rich-text-block");

    if (!editor && !textBlock) return;

    event.preventDefault();
    event.stopPropagation();
    const nextEditor = editor || textBlock.closest("[data-rich-editor]");
    if (activeSummaryEditor && nextEditor && activeSummaryEditor !== nextEditor) {
      activeRichSelectionRange = null;
      activeTextSelectionRange = null;
      clearPersistentTextSelection(true);
      summaryContextMenu.hidden = true;
    }
    hideTextSelectionInspector();

    activePlainTextControl = null;
    activePlainTextSelection = null;
    activeSummaryEditor = nextEditor;
    configureSummaryContextMenu("rich", { textOnly: activeSummaryEditor.dataset.richTextOnly === "true"
});
    const liveRange = selectionRangeInsideEditor(activeSummaryEditor);
    const savedRange = activeTextSelectionRange && !activeTextSelectionRange.collapsed
      ? activeTextSelectionRange.cloneRange()
      : null;
    const savedContainer = savedRange?.commonAncestorContainer?.nodeType === Node.TEXT_NODE
      ? savedRange.commonAncestorContainer.parentElement
      : savedRange?.commonAncestorContainer;
    if (liveRange && !liveRange.collapsed) {
      activeRichSelectionRange = liveRange.cloneRange();
      activeTextSelectionRange = liveRange.cloneRange();
    } else if (savedRange && savedContainer && activeSummaryEditor.contains(savedContainer)) {
      activeRichSelectionRange = savedRange.cloneRange();
      restoreRichSelection(activeSummaryEditor);
    } else {
      captureRichSelection(activeSummaryEditor);
    }
    selectRichBlock(event.target.closest?.(".rich-block") || currentRichBlock(activeSummaryEditor));
    syncRichContextMenuControls(activeSummaryBlock);

    const menuHost = activeSummaryEditor.closest("dialog") || document.body;
    if (summaryContextMenu.parentElement !== menuHost) menuHost.append(summaryContextMenu);

    summaryContextMenu.style.left = `${event.clientX}px`;
    summaryContextMenu.style.top = `${event.clientY}px`;
    summaryContextMenu.style.maxHeight = "";
    summaryContextMenu.hidden = false;
    const hostRect = menuHost.getBoundingClientRect();
    const bounds = {
      bottom: Math.min(window.innerHeight - 8, hostRect.bottom - 8),
      left: Math.max(8, hostRect.left + 8),
      right: Math.min(window.innerWidth - 8, hostRect.right - 8),
```

```
9364:       top: Math.max(8, hostRect.top + 8)
9365:     };
9366:     summaryContextMenu.style.maxHeight = `${Math.max(180, bounds.bottom - bounds.top)}px`;
9367:     const menuRect = summaryContextMenu.getBoundingClientRect();
9368:     const left = Math.max(bounds.left, Math.min(event.clientX, bounds.right - menuRect.width));
9369:     const top = Math.max(bounds.top, Math.min(event.clientY, bounds.bottom - menuRect.height));
9370:     summaryContextMenu.style.left = `${left}px`;
9371:     summaryContextMenu.style.top = `${top}px`;
9372: }
```

handlePlainTextContextMenu (template-preview.js, lines 9398-9433)

handlePlainTextContextMenu handles an event, request, command, or user action. In this file, it appears around lines 9398-9433 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references plainTextControlFromContextEvent, preventDefault, hideTextSelectionInspector, configureSummaryContextMenu, syncPlainContextMenuControls, closest, append, getBoundingClientRect, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9400: if (!control) return;.

Important local values include control at line 9399; menuHost at line 9414; hostRect at line 9420; bounds at line 9421; menuRect at line 9428; left at line 9429; top at line 9430. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9398: function handlePlainTextContextMenu(event) {
9399:   const control = plainTextControlFromContextEvent(event);
9400:   if (!control) return;
9401:
9402:   event.preventDefault();
9403:   hideTextSelectionInspector();
9404:   activeSummaryEditor = null;
9405:   activeSummaryBlock = null;
9406:   activePlainTextControl = control;
9407:   activePlainTextSelection = {
9408:     start: control.selectionStart ?? 0,
9409:     end: control.selectionEnd ?? control.value.length
9410:   };
9411:   configureSummaryContextMenu("plain");
9412:   syncPlainContextMenuControls(control);
9413:
9414:   const menuHost = control.closest("dialog") || document.body;
9415:   if (summaryContextMenu.parentElement !== menuHost) menuHost.append(summaryContextMenu);
9416:   summaryContextMenu.style.left = `${event.clientX}px`;
9417:   summaryContextMenu.style.top = `${event.clientY}px`;
9418:   summaryContextMenu.style.maxHeight = "";
9419:   summaryContextMenu.hidden = false;
9420:   const hostRect = menuHost.getBoundingClientRect();
9421:   const bounds = {
9422:     bottom: Math.min(window.innerHeight - 8, hostRect.bottom - 8),
9423:     left: Math.max(8, hostRect.left + 8),
9424:     right: Math.min(window.innerWidth - 8, hostRect.right - 8),
9425:     top: Math.max(8, hostRect.top + 8)
9426:   };
9427:   summaryContextMenu.style.maxHeight = `${Math.max(140, bounds.bottom - bounds.top)}px`;
9428:   const menuRect = summaryContextMenu.getBoundingClientRect();
9429:   const left = Math.max(bounds.left, Math.min(event.clientX, bounds.right - menuRect.width));
9430:   const top = Math.max(bounds.top, Math.min(event.clientY, bounds.bottom - menuRect.height));
9431:   summaryContextMenu.style.left = `${left}px`;
9432:   summaryContextMenu.style.top = `${top}px`;
9433: }
```

handleRichEditorPaste (template-preview.js, lines 9437-9480)

This function controls paste behavior inside rich editors. It decides whether clipboard content is an image, plain text, HTML, code, or formula-like text, then inserts it using the builder's block model.

Pasting is where editors often become messy. This function keeps pasted content usable while preventing links from being misread as formulas and images from landing in broken positions.

It calls or references closest, find, startsWith, preventDefault, setStatus, uploadSummaryImageFile, getAsFile, insertRichBlockAtCursor, createRichImageBlock, saveRichEditorToProject, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 9439: if (!editor) return;; line 9447: return;; line 9463: return;; line 9471: return;. There are 1 additional return statements in the function body.

Important local values include editor at line 9438; imageItem at line 9441; imageBlock at line 9450; pastedHtml at line 9466; pastedText at line 9474. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 9437: async function handleRichEditorPaste(event) {
 9438:     const editor = event.target.closest("[data-rich-editor]");
 9439:     if (!editor) return;
 9440:
 9441:     const imageItem = [...(event.clipboardData?.items || [])].find((item) =>
item.type.startsWith("image/"));
 9442:
 9443:     if (imageItem) {
 9444:         event.preventDefault();
 9445:         if (editor.dataset.richTextOnly === "true") {
 9446:           setStatus("Images can only be pasted into overview editors, not title, profile, or contact
fields.");
 9447:           return;
 9448:         }
 9449:
 9450:         const imageBlock = await uploadSummaryImageFile(imageItem.getAsFile(), {
 9451:             align: "center",
 9452:             display: "show",
 9453:             folder: editor.dataset.richFolder || "summary",
 9454:             projectId: editor.dataset.richProjectId || "",
 9455:             title: ""
 9456:         });
 9457:
 9458:         if (imageBlock) {
 9459:             insertRichBlockAtCursor(editor, createRichImageBlock(imageBlock));
 9460:             saveRichEditorToProject(editor);
 9461:         }
 9462:
 9463:         return;
 9464:     }
 9465:
 9466:     const pastedHtml = event.clipboardData?.getData("text/html") || "";
 9467:     if (pastedHtml) {
 9468:       event.preventDefault();
 9469:       insertHtmlIntoRichEditor(editor, pastedHtml);
 9470:       saveRichEditorToProject(editor);
 9471:       return;
 9472:     }
 9473:
 9474:     const pastedText = event.clipboardData?.getData("text/plain") || "";
 9475:     if (!pastedText) return;
 9476:
 9477:     event.preventDefault();
 9478:     insertPlainTextIntoRichEditor(editor, pastedText);
 9479:     saveRichEditorToProject(editor);
 9480: }
```

restoreRichInsertionPoint (template-preview.js, lines 9494-9503)

restoreRichInsertionPoint part of the private builder frontend and editing experience. In this file, it appears around lines 9494-9503 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references restoreRichSelection, focusTextBlock, selectedRichBlock, querySelector, focus, selectionRangeInsideEditor, and deleteContents. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 9499: if (!range) return null;; line 9502: return range;.

Important local values include range at line 9498. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 9494: function restoreRichInsertionPoint(editor) {
 9495:     if (!restoreRichSelection(editor)) focusTextBlock(selectedRichBlock(editor) ||
editor.querySelector(".rich-text-block"));
 9496:     editor.focus({ preventScroll: true });
 9497:
 9498:     const range = selectionRangeInsideEditor(editor);
 9499:     if (!range) return null;
 9500:
 9501:     range.deleteContents();
 9502:     return range;
 9503: }
```

insertFragmentIntoRichEditor (template-preview.js, lines 9505-9521)

insertFragmentIntoRichEditor part of the private builder frontend and editing experience. In this file, it appears around lines 9505-9521 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references restoreRichInsertionPoint, insertNode, createRange, setStartAfter, setStart, collapse, getSelection, removeAllRanges, addRange, and captureRichSelection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9507: if (!range || !fragment) return;.

Important local values include range at line 9506; lastInsertedNode at line 9509; nextRange at line 9512; selection at line 9517. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9505: function insertFragmentIntoRichEditor(editor, fragment) {
9506:     const range = restoreRichInsertionPoint(editor);
9507:     if (!range || !fragment) return;
9508:
9509:     const lastInsertedNode = fragment.lastChild;
9510:     range.insertNode(fragment);
9511:
9512:     const nextRange = document.createRange();
9513:     if (lastInsertedNode) nextRange.setStartAfter(lastInsertedNode);
9514:     else nextRange.setStart(range.startContainer, range.startOffset);
9515:     nextRange.collapse(true);
9516:
9517:     const selection = window.getSelection();
9518:     selection.removeAllRanges();
9519:     selection.addRange(nextRange);
9520:     captureRichSelection(editor);
9521: }
```

insertPlainTextIntoRichEditor (template-preview.js, lines 9523-9533)

insertPlainTextIntoRichEditor part of the private builder frontend and editing experience. In this file, it appears around lines 9523-9533 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, createDocumentFragment, split, forEach, append, createElement, createTextNode, and insertFragmentIntoRichEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9525: if (!textToInsert) return;.

Important local values include textToInsert at line 9524; fragment at line 9527. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9523: function insertPlainTextIntoRichEditor(editor, pastedText) {
9524:     const textToInsert = String(pastedText || "").replace(/\r\n?/g, "\n");
9525:     if (!textToInsert) return;
9526:
9527:     const fragment = document.createDocumentFragment();
9528:     textToInsert.split("\n").forEach((line, index) => {
9529:         if (index > 0) fragment.append(document.createElement("br"));
9530:         if (line) fragment.append(document.createTextNode(line));
9531:     });
9532:     insertFragmentIntoRichEditor(editor, fragment);
9533: }
```

insertHtmlIntoRichEditor (template-preview.js, lines 9535-9543)

insertHtmlIntoRichEditor part of the private builder frontend and editing experience. In this file, it appears around lines 9535-9543 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, createElement, sanitizeRichInlineHtml, insertFragmentIntoRichEditor, and cloneNode. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include normalizedHtml at line 9536; template at line 9540. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9535: function insertHtmlIntoRichEditor(editor, pastedHtml) {
9536:     const normalizedHtml = String(pastedHtml || "")
9537:       .replace(/<\/(?:p|div|li|h[1-6]|tr)>/gi, "<br>")
9538:       .replace(/<(?:p|div|li|h[1-6]|tr|td|th)[^>]*>/gi, "")
9539:       .replace(/<br\s*\/?>\s*(<br\s*\/?>\s*){2,}/gi, "<br><br>");
9540:     const template = document.createElement("template");
9541:     template.innerHTML = sanitizeRichInlineHtml(normalizedHtml);
9542:     insertFragmentIntoRichEditor(editor, template.content.cloneNode(true));
9543: }
```

handleRichEditorKeydown (template-preview.js, lines 9565-9610)

handleRichEditorKeydown handles an event, request, command, or user action. In this file, it appears around lines 9565-9610 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, includes, toLowerCase, preventDefault, applyRichInlineCommand, execCommand, captureRichSelection, selectionRangeInsideEditor, contains, deleteContents, and 13 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 9567: if (!editor) return;; line 9577: return;; line 9580: if (event.key !== "Enter") return;; line 9583: if (!block) return;. There are 2 additional return statements in the function body.

Important local values include editor at line 9566; command at line 9571; block at line 9582; range at line 9592; tail at line 9595; trailingContent at line 9598; nextBlock at line 9599; nextRange at line 9602; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9565: function handleRichEditorKeydown(event) {
9566:     const editor = event.target.closest("[data-rich-editor]");
9567:     if (!editor) return;
9568:
9569:     if ((event.ctrlKey || event.metaKey) && ["b", "i", "u"].includes(event.key.toLowerCase())) {
9570:       event.preventDefault();
9571:       const command = event.key.toLowerCase() === "b"
9572:         ? "bold"
9573:         : event.key.toLowerCase() === "i"
9574:           ? "italic"
9575:           : "underline";
9576:       applyRichInlineCommand(editor, command);
9577:       return;
9578:     }
9579:
9580:     if (event.key !== "Enter") return;
9581:
9582:     const block = event.target.closest(".rich-text-block");
9583:     if (!block) return;
9584:
9585:     event.preventDefault();
9586:     if (event.shiftKey) {
9587:       document.execCommand("insertLineBreak", false);
9588:       captureRichSelection(editor);
9589:       return;
9590:     }
9591:
9592:     const range = selectionRangeInsideEditor(editor);
9593:     if (!range || !block.contains(range.startContainer)) return;
9594:     range.deleteContents();
9595:     const tail = document.createRange();
9596:     tail.setStart(range.startContainer, range.startOffset);
9597:     tail.setEnd(block, block.childNodes.length);
9598:     const trailingContent = tail.extractContents();
9599:     const nextBlock = matchingTextBlock(block, "");
9600:     nextBlock.append(trailingContent);
9601:     block.after(nextBlock);
9602:     const nextRange = document.createRange();
9603:     nextRange.selectNodeContents(nextBlock);
9604:     nextRange.collapse(true);
9605:     const selection = window.getSelection();
9606:     selection.removeAllRanges();
9607:     selection.addRange(nextRange);
9608:     captureRichSelection(editor);
9609:     setStatus("Unsaved local changes.");
9610: }
```

handleRichEditorFocus (template-preview.js, lines 9632-9637)

handleRichEditorFocus handles an event, request, command, or user action. In this file, it appears around lines 9632-9637 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, execCommand, and captureRichSelection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9634: if (!editor) return;.

Important local values include editor at line 9633. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9632: function handleRichEditorFocus(event) {
9633:   const editor = event.target.closest("[data-rich-editor]");
9634:   if (!editor) return;
9635:   document.execCommand("defaultParagraphSeparator", false, "p");
9636:   captureRichSelection(editor);
9637: }
```

handleRichEditorInput (template-preview.js, lines 9639-9644)

handleRichEditorInput handles an event, request, command, or user action. In this file, it appears around lines 9639-9644 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, captureRichSelection, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9641: if (!editor) return;.

Important local values include editor at line 9640. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9639: function handleRichEditorInput(event) {
9640:   const editor = event.target.closest("[data-rich-editor]");
9641:   if (!editor) return;
9642:   captureRichSelection(editor);
9643:   setStatus("Unsaved local changes.");
9644: }
```

handleRichEditorBeforeInput (template-preview.js, lines 9646-9653)

handleRichEditorBeforeInput handles an event, request, command, or user action. In this file, it appears around lines 9646-9653 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, and clearPersistentTextSelection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9648: if (!editor) return;.

Important local values include editor at line 9647; inputType at line 9649. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9646: function handleRichEditorBeforeInput(event) {
9647:   const editor = event.target.closest("[data-rich-editor]");
9648:   if (!editor) return;
9649:   const inputType = event.inputType || "";
9650:   if (!/insertFromPaste|insertFromDrop/i.test(inputType) && /^(insert|delete|history)/i.test(inputType))
{
9651:     clearPersistentTextSelection(true);
9652:   }
9653: }
```

finishTextSelectionGesture (template-preview.js, lines 9728-9753)

finishTextSelectionGesture part of the private builder frontend and editing experience. In this file, it appears around lines 9728-9753 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getSelection, selectionRangeInsideEditor, cloneRange, showPersistentTextSelection, rangeBelongsToEditor, clearPersistentTextSelection, and refreshTextSelectionInspector. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9729: if (event?.button === 2 || event?.target?.closest?.("#text-selection-inspector, #summary-context-menu")) return;.

Important local values include selection at line 9733; node at line 9734; editor at line 9736; range at line 9737. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 9728: function finishTextSelectionGesture(event) {
 9729:   if (event?.button === 2 || event?.target?.closest?.("#text-selection-inspector, #summary-context-
menu")) return;
 9730:   selectionGestureActive = false;
 9731:   clearTimeout(selectionGestureFinishTimer);
 9732:   selectionGestureFinishTimer = setTimeout(() => {
 9733:     const selection = window.getSelection();
 9734:     let node = selection?.anchorNode;
 9735:     if (node?.nodeType === Node.TEXT_NODE) node = node.parentElement;
 9736:     const editor = node?.closest?.("[data-rich-editor]");
 9737:     const range = editor ? selectionRangeInsideEditor(editor) : null;
 9738:   if (editor && range && !range.collapsed) {
 9739:       activeSummaryEditor = editor;
 9740:       activeRichSelectionRange = range.cloneRange();
 9741:       activeTextSelectionRange = range.cloneRange();
 9742:       selectionGestureProducedRange = true;
 9743:       showPersistentTextSelection(range);
 9744:     } else if (!selectionGestureProducedRange) {
 9745:       if (editor && rangeBelongsToEditor(activeTextSelectionRange, editor)) {
 9746:         showPersistentTextSelection(activeTextSelectionRange);
 9747:       } else {
 9748:         clearPersistentTextSelection(true);
 9749:       }
 9750:     }
 9751:     refreshTextSelectionInspector();
 9752:   }, 40);
 9753: }
```

endSelectionInspectorDrag (template-preview.js, lines 9814-9819)

endSelectionInspectorDrag part of the private builder frontend and editing experience. In this file, it appears around lines 9814-9819 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9815: if (!activeSelectionInspectorDrag) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 9814: function endSelectionInspectorDrag() {
 9815:   if (!activeSelectionInspectorDrag) return;
 9816:   activeSelectionInspectorDrag.target?.releasePointerCapture?.(activeSelectionInspectorDrag.pointerId);
 9817:   textSelectionInspector?.classList.remove("is-dragging");
 9818:   activeSelectionInspectorDrag = null;
 9819: }
```

handleSelectionInspectorTextField (template-preview.js, lines 9843-9849)

handleSelectionInspectorTextField handles an event, request, command, or user action. In this file, it appears around lines 9843-9849 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references preventDefault, and applySelectionInspectorFormat. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9844: if (event.type === "keydown" && event.key !== "Enter") return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
9843: function handleSelectionInspectorTextField(event) {
9844:   if (event.type === "keydown" && event.key !== "Enter") return;
9845:   if (event.type === "keydown") event.preventDefault();
9846:   if (event.target === selectionFontInput) applySelectionInspectorFormat("font", event.target.value);
9847:   if (event.target === selectionColorInput) applySelectionInspectorFormat("color", event.target.value);
9848:   if (event.target === selectionSizeInput) applySelectionInspectorFormat("size", event.target.value);
9849: }
```

scheduleSelectionInspectorTextField (template-preview.js, lines 9851-9859)

scheduleSelectionInspectorTextField part of the private builder frontend and editing experience. In this file, it appears around lines 9851-9859 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cleanFontFamily, applySelectionInspectorFormat, normalizeTextColor, and normalizeFontPx. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include target at line 9853. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9851: function scheduleSelectionInspectorTextField(event) {
9852:   clearTimeout(selectionInspectorInputTimer);
9853:   const target = event.target;
9854:   selectionInspectorInputTimer = setTimeout(() => {
9855:     if (target === selectionFontInput && cleanFontFamily(target.value))
applySelectionInspectorFormat("font", target.value);
9856:     if (target === selectionColorInput && normalizeTextColor(target.value))
applySelectionInspectorFormat("color", target.value);
9857:     if (target === selectionSizeInput && normalizeFontPx(target.value))
applySelectionInspectorFormat("size", target.value);
9858:   }, 350);
9859: }
```

focusAdjacentImageText (template-preview.js, lines 9871-9882)

focusAdjacentImageText part of the private builder frontend and editing experience. In this file, it appears around lines 9871-9882 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, matches, matchingTextBlock, before, after, focusTextBlock, and saveRichEditorToProject. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9873: if (!editor) return;.

Important local values include editor at line 9872; sibling at line 9874; textBlock at line 9875. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9871: function focusAdjacentImageText(block, direction) {
9872:   const editor = block?.closest("[data-rich-editor]");
9873:   if (!editor) return;
9874:   const sibling = direction === "before" ? block.previousElementSibling : block.nextElementSibling;
9875:   const textBlock = sibling?.matches(".rich-text-block") ? sibling : matchingTextBlock(block, "");
9876:   if (textBlock !== sibling) {
9877:     if (direction === "before") block.before(textBlock);
9878:     else block.after(textBlock);
9879:   }
9880:   focusTextBlock(textBlock);
9881:   saveRichEditorToProject(editor);
9882: }
```

handleRichCaretClick (template-preview.js, lines 9884-9889)

handleRichCaretClick handles an event, request, command, or user action. In this file, it appears around lines 9884-9889 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, and focusAdjacentImageText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 9886: if (!caretButton) return false;; line 9888: return true;.

Important local values include caretButton at line 9885. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9884: function handleRichCaretClick(event) {
9885:   const caretButton = event.target.closest("[data-rich-caret]");
9886:   if (!caretButton) return false;
9887:   focusAdjacentImageText(caretButton.closest(".rich-image-block"), caretButton.dataset.richCaret);
9888:   return true;
9889: }
```

handleRichDragStart (template-preview.js, lines 9891-9910)

handleRichDragStart handles an event, request, command, or user action. In this file, it appears around lines 9891-9910 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, preventDefault, selectRichBlock, add, clearData, and setData. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9898: return;.

Important local values include handle at line 9892; imageBlock at line 9893; isControl at line 9894; block at line 9895. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9891: function handleRichDragStart(event) {
9892:   const handle = event.target.closest("[data-rich-drag-handle]");
9893:   const imageBlock = event.target.closest(".rich-image-block");
9894:   const isControl = event.target.closest("button, input, select, textarea, a, .rich-block-actions, [data-
rich-caret]");
9895:   const block = handle?.closest(".rich-block") || (!isControl ? imageBlock : null);
9896:   if (!block) {
9897:     if (imageBlock) event.preventDefault();
9898:     return;
9899:   }
9900:   activeRichDragBlock = block;
9901:   selectRichBlock(activeRichDragBlock);
9902:   activeRichDragBlock.classList.add("is-dragging");
9903:   if (event.dataTransfer) {
9904:     event.dataTransfer.clearData();
9905:     event.dataTransfer.effectAllowed = "move";
9906:     event.dataTransfer.dropEffect = "move";
9907:     event.dataTransfer.setData("text/x-rich-block", "move");
9908:     event.dataTransfer.setData("text/plain", activeRichDragBlock.dataset.title ||
activeRichDragBlock.dataset.type || "image");
9909:   }
9910: }
```

handleRichDragOver (template-preview.js, lines 9912-9919)

handleRichDragOver handles an event, request, command, or user action. In this file, it appears around lines 9912-9919 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, preventDefault, and updateRichDropMarker. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 9913: if (!activeRichDragBlock) return;; line 9915: if (!editor || editor !== activeRichDragBlock.closest("[data-rich-editor]")) return;.

Important local values include editor at line 9914. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9912: function handleRichDragOver(event) {
9913:   if (!activeRichDragBlock) return;
9914:   const editor = event.target.closest("[data-rich-editor]");
9915:   if (!editor || editor !== activeRichDragBlock.closest("[data-rich-editor]")) return;
9916:   event.preventDefault();
9917:   if (event.dataTransfer) event.dataTransfer.dropEffect = "move";
9918:   updateRichDropMarker(editor, activeRichDragBlock, event.clientX, event.clientY, event.target);
9919: }
```

richRangeFromPoint (template-preview.js, lines 9921-9933)

richRangeFromPoint part of the private builder frontend and editing experience. In this file, it appears around lines 9921-9933 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references caretRangeFromPoint, caretPositionFromPoint, createRange, setStart, and collapse. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 9922: if (document.caretRangeFromPoint) return document.caretRangeFromPoint(x, y);; line 9929: return range;; line 9932: return null;.

Important local values include position at line 9924; range at line 9926. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9921: function richRangeFromPoint(x, y) {
9922:   if (document.caretRangeFromPoint) return document.caretRangeFromPoint(x, y);
9923:   if (document.caretPositionFromPoint) {
9924:     const position = document.caretPositionFromPoint(x, y);
9925:     if (position) {
9926:       const range = document.createRange();
9927:       range.setStart(position.offsetNode, position.offset);
9928:       range.collapse(true);
9929:       return range;
9930:     }
9931:   }
9932:   return null;
9933: }
```

richDropLocation (template-preview.js, lines 9935-9950)

richDropLocation part of the private builder frontend and editing experience. In this file, it appears around lines 9935-9950 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references validDropRangeForEditor, richRangeFromPoint, contains, and elementFromPoint. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9943: return {.

Important local values include previousPointerEvents at line 9936; dropRange at line 9939; pointTarget at line 9940. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9935: function richDropLocation(editor, movingBlock, x, y, eventTarget = null) {
9936:   const previousPointerEvents = movingBlock?.style.pointerEvents || "";
9937:   if (movingBlock) movingBlock.style.pointerEvents = "none";
9938:   try {
9939:     const dropRange = validDropRangeForEditor(editor, movingBlock, richRangeFromPoint(x, y));
9940:     const pointTarget = eventTarget && !movingBlock?.contains(eventTarget)
9941:       ? eventTarget
9942:       : document.elementFromPoint(x, y);
9943:     return {
9944:       dropRange,
9945:       target: pointTarget?.closest?.(".rich-block") || null
9946:     };
9947:   } finally {
9948:     if (movingBlock) movingBlock.style.pointerEvents = previousPointerEvents;
9949:   }
9950: }
```

validDropRangeForEditor (template-preview.js, lines 9952-9960)

validDropRangeForEditor part of the private builder frontend and editing experience. In this file, it appears around lines 9952-9960 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, and closest. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 9956: if (!range || !rangeNode || !editor.contains(rangeNode)) return null;; line 9957: if (movingBlock?.contains(rangeNode)) return null;; line 9958: if (rangeNode.closest("[contenteditable='false']")) return null;; line 9959: return range;.

Important local values include rangeNode at line 9953. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9952: function validDropRangeForEditor(editor, movingBlock, range) {
9953:   const rangeNode = range?.startContainer?.nodeType === Node.TEXT_NODE
9954:     ? range.startContainer.parentElement
9955:     : range?.startContainer;
9956:   if (!range || !rangeNode || !editor.contains(rangeNode)) return null;
9957:   if (movingBlock?.contains(rangeNode)) return null;
9958:   if (rangeNode.closest("[contenteditable='false']")) return null;
9959:   return range;
9960: }
```

ensureRichDropMarker (template-preview.js, lines 9962-9971)

ensureRichDropMarker part of the private builder frontend and editing experience. In this file, it appears around lines 9962-9971 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, setAttribute, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9970: return activeRichDropMarker;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
9962: function ensureRichDropMarker() {
9963:   if (!activeRichDropMarker) {
9964:     activeRichDropMarker = document.createElement("div");
9965:     activeRichDropMarker.className = "rich-drop-marker";
9966:     activeRichDropMarker.setAttribute("aria-hidden", "true");
9967:     document.body.append(activeRichDropMarker);
9968:   }
9969:   activeRichDropMarker.hidden = false;
9970:   return activeRichDropMarker;
9971: }
```

hideRichDropMarker (template-preview.js, lines 9973-9975)

hideRichDropMarker controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 9973-9975 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
9973: function hideRichDropMarker() {
9974:   if (activeRichDropMarker) activeRichDropMarker.hidden = true;
9975: }
```

updateRichDropMarker (template-preview.js, lines 9977-10006)

updateRichDropMarker updates ui, state, cache, or stored data. In this file, it appears around lines 9977-10006 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureRichDropMarker, richDropLocation, getClientRects, getBoundingClientRect, max, min, and contains. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 9978: if (!editor || !movingBlock) return;; line 9993: return;.

Important local values include marker at line 9979; rangeRect at line 9982; anchor at line 9983; anchorRect at line 9986; rect at line 9987; targetRect at line 9996; imageRect at line 9999; beforeTarget at line 10000. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
 9977: function updateRichDropMarker(editor, movingBlock, x, y, eventTarget = null) {
 9978:   if (!editor || !movingBlock) return;
 9979:   const marker = ensureRichDropMarker();
 9980:   const { dropRange, target } = richDropLocation(editor, movingBlock, x, y, eventTarget);
 9981:   if (dropRange) {
 9982:     const rangeRect = dropRange.getClientRects()[0];
 9983:     const anchor = dropRange.startContainer.nodeType === Node.TEXT_NODE
 9984:       ? dropRange.startContainer.parentElement
 9985:       : dropRange.startContainer;
 9986:     const anchorRect = anchor?.getBoundingClientRect?.();
 9987:     const rect = rangeRect || anchorRect || editor.getBoundingClientRect();
 9988:     marker.className = "rich-drop-marker caret";
 9989:     marker.style.left = `${Math.max(rect.left, Math.min(x, rect.right || rect.left))}px`;
 9990:     marker.style.top = `${rect.top}px`;
 9991:     marker.style.width = "2px";
 9992:     marker.style.height = `${Math.max(22, rect.height || 22)}px`;
 9993:     return;
 9994:   }
 9995:
 9996:   const targetRect = (target && editor.contains(target) && target !== movingBlock)
 9997:     ? target.getBoundingClientRect()
 9998:     : editor.getBoundingClientRect();
 9999:   const imageRect = movingBlock.getBoundingClientRect();
10000:   const beforeTarget = !target || y < targetRect.top + targetRect.height / 2;
10001:   marker.className = "rich-drop-marker block";
10002:   marker.style.left = `${targetRect.left}px`;
10003:   marker.style.top = `${beforeTarget ? targetRect.top : targetRect.bottom}px`;
10004:   marker.style.width = `${Math.min(targetRect.width, Math.max(160, imageRect.width ||
targetRect.width))}px`;
10005:   marker.style.height = "3px";
10006: }
```

moveRichBlockToPoint (template-preview.js, lines 10008-10029)

moveRichBlockToPoint part of the private builder frontend and editing experience. In this file, it appears around lines 10008-10029 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richDropLocation, remove, insertRichBlockAtCursor, contains, append, getBoundingClientRect, before, after, cleanupEmptyRichTextBlocks, focusAdjacentImageText, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 10009: if (!editor || !movingBlock) return;; line 10013: if (target === movingBlock) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
10008: function moveRichBlockToPoint(editor, movingBlock, x, y, eventTarget = null, options = {}) {
10009:   if (!editor || !movingBlock) return;
10010:   const { dropRange, target } = richDropLocation(editor, movingBlock, x, y, eventTarget);
10011:
10012:   movingBlock.classList.remove("is-dragging");
10013:   if (target === movingBlock) return;
10014:
10015:   if (dropRange) {
10016:     insertRichBlockAtCursor(editor, movingBlock, dropRange);
10017:   } else if (!target || !editor.contains(target)) {
10018:     editor.append(movingBlock);
10019:   } else if (y < target.getBoundingClientRect().top + target.getBoundingClientRect().height / 2) {
10020:     target.before(movingBlock);
10021:   } else {
```

```
10022:     target.after(movingBlock);
10023:   }
10024:
10025:   cleanupEmptyRichTextBlocks(editor);
10026:   if (options.focusAfter !== false) focusAdjacentImageText(movingBlock, "after");
10027:   if (options.commit !== false) saveRichEditorToProject(editor);
10028:   if (options.focusAfter === false) selectRichBlock(movingBlock);
10029: }
```

handleRichDrop (template-preview.js, lines 10031-10039)

handleRichDrop handles an event, request, command, or user action. In this file, it appears around lines 10031-10039 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, preventDefault, moveRichBlockToPoint, and hideRichDropMarker. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 10032: if (!activeRichDragBlock) return;; line 10034: if (!editor || editor !== activeRichDragBlock.closest("[data-rich-editor]")) return;.

Important local values include editor at line 10033. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10031: function handleRichDrop(event) {
10032:   if (!activeRichDragBlock) return;
10033:   const editor = event.target.closest("[data-rich-editor]");
10034:   if (!editor || editor !== activeRichDragBlock.closest("[data-rich-editor]")) return;
10035:   event.preventDefault();
10036:   moveRichBlockToPoint(editor, activeRichDragBlock, event.clientX, event.clientY, event.target);
10037:   hideRichDropMarker();
10038:   activeRichDragBlock = null;
10039: }
```

imageMoveDragControl (template-preview.js, lines 10041-10043)

imageMoveDragControl part of the private builder frontend and editing experience. In this file, it appears around lines 10041-10043 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 10042: return target.closest("button, input, select, textarea, a, .rich-block-actions, [data-rich-caret]");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
10041: function imageMoveDragControl(target) {
10042:   return target.closest("button, input, select, textarea, a, .rich-block-actions, [data-rich-caret]");
10043: }
```

beginRichImageMoveDrag (template-preview.js, lines 10045-10061)

beginRichImageMoveDrag part of the private builder frontend and editing experience. In this file, it appears around lines 10045-10061 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, and imageMoveDragControl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 10046: if (event.button !== 0) return;; line 10048: if (!block || imageMoveDragControl(event.target)) return;; line 10049: if (event.target.closest(".rich-image-viewport.crop-active")) return;; line 10051: if (!editor) return;.

Important local values include block at line 10047; editor at line 10050. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10045: function beginRichImageMoveDrag(event) {
10046:   if (event.button !== 0) return;
10047:   const block = event.target.closest(".rich-image-block");
10048:   if (!block || imageMoveDragControl(event.target)) return;
10049:   if (event.target.closest(".rich-image-viewport.crop-active")) return;
10050:   const editor = block.closest("[data-rich-editor]");
10051:   if (!editor) return;
10052:   activeRichImageMoveDrag = {
10053:     block,
10054:     editor,
10055:     moved: false,
10056:     pointerId: event.pointerId,
10057:     startX: event.clientX,
10058:     startY: event.clientY
10059:   };
10060:   block.setPointerCapture?.(event.pointerId);
10061: }
```

moveRichImageMoveDrag (template-preview.js, lines 10063-10077)

moveRichImageMoveDrag part of the private builder frontend and editing experience. In this file, it appears around lines 10063-10077 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references hypot, selectRichBlock, add, updateRichDropMarker, and preventDefault. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 10064: if (!activeRichImageMoveDrag) return;; line 10067: if (!drag.moved && distance < 6) return;.

Important local values include drag at line 10065; distance at line 10066. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10063: function moveRichImageMoveDrag(event) {
10064:   if (!activeRichImageMoveDrag) return;
10065:   const drag = activeRichImageMoveDrag;
10066:   const distance = Math.hypot(event.clientX - drag.startX, event.clientY - drag.startY);
10067:   if (!drag.moved && distance < 6) return;
10068:   if (!drag.moved) {
10069:     drag.moved = true;
10070:     activeRichDragBlock = drag.block;
10071:     selectRichBlock(drag.block);
10072:     drag.block.classList.add("is-dragging");
10073:     document.body.classList.add("rich-image-dragging");
10074:   }
10075:   updateRichDropMarker(drag.editor, drag.block, event.clientX, event.clientY, event.target);
10076:   event.preventDefault();
10077: }
```

endRichImageMoveDrag (template-preview.js, lines 10079-10092)

endRichImageMoveDrag part of the private builder frontend and editing experience. In this file, it appears around lines 10079-10092 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references moveRichBlockToPoint, hideRichDropMarker, preventDefault, and remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 10080: if (!activeRichImageMoveDrag) return;.

Important local values include drag at line 10081. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10079: function endRichImageMoveDrag(event) {
10080:   if (!activeRichImageMoveDrag) return;
10081:   const drag = activeRichImageMoveDrag;
10082:   drag.block.releasePointerCapture?.(drag.pointerId);
10083:   if (drag.moved) {
10084:     moveRichBlockToPoint(drag.editor, drag.block, event.clientX, event.clientY);
10085:     hideRichDropMarker();
10086:     event.preventDefault();
10087:   }
10088:   drag.block.classList.remove("is-dragging");
```

```
10089:   document.body.classList.remove("rich-image-dragging");
10090:   activeRichImageMoveDrag = null;
10091:   activeRichDragBlock = null;
10092: }
```

beginImageCropDrag (template-preview.js, lines 10094-10109)

beginImageCropDrag part of the private builder frontend and editing experience. In this file, it appears around lines 10094-10109 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, normalizeCropPosition, max, getBoundingClientRect, and preventDefault. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 10096: if (!viewport || event.button !== 0) return;.

Important local values include viewport at line 10095; block at line 10097. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10094: function beginImageCropDrag(event) {
10095:   const viewport = event.target.closest(".rich-image-viewport.crop-active");
10096:   if (!viewport || event.button !== 0) return;
10097:   const block = viewport.closest(".rich-image-block");
10098:   activeImageCropDrag = {
10099:     block,
10100:     editor: block.closest("[data-rich-editor]"),
10101:     startX: event.clientX,
10102:     startY: event.clientY,
10103:     cropX: normalizeCropPosition(block.dataset.cropX),
10104:     cropY: normalizeCropPosition(block.dataset.cropY),
10105:     width: Math.max(1, viewport.getBoundingClientRect().width),
10106:     height: Math.max(1, viewport.getBoundingClientRect().height)
10107:   };
10108:   event.preventDefault();
10109: }
```

moveImageCropDrag (template-preview.js, lines 10111-10121)

moveImageCropDrag part of the private builder frontend and editing experience. In this file, it appears around lines 10111-10121 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCropPosition, querySelector, and setProperty. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 10112: if (!activeImageCropDrag) return;.

Important local values include nextX at line 10114; nextY at line 10115; viewport at line 10118. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10111: function moveImageCropDrag(event) {
10112:   if (!activeImageCropDrag) return;
10113:   const { block, startX, startY, cropX, cropY, width, height } = activeImageCropDrag;
10114:   const nextX = normalizeCropPosition(cropX - ((event.clientX - startX) / width) * 100);
10115:   const nextY = normalizeCropPosition(cropY - ((event.clientY - startY) / height) * 100);
10116:   block.dataset.cropX = String(nextX);
10117:   block.dataset.cropY = String(nextY);
10118:   const viewport = block.querySelector(".rich-image-viewport");
10119:   viewport?.style.setProperty("--crop-x", `${nextX}%`);
10120:   viewport?.style.setProperty("--crop-y", `${nextY}%`);
10121: }
```

endImageCropDrag (template-preview.js, lines 10123-10127)

endImageCropDrag part of the private builder frontend and editing experience. In this file, it appears around lines 10123-10127 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references saveRichEditorToProject. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 10124: if (!activeImageCropDrag) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
10123: function endImageCropDrag() {
10124:   if (!activeImageCropDrag) return;
10125:   saveRichEditorToProject(activeImageCropDrag.editor);
10126:   activeImageCropDrag = null;
10127: }
```

handleStandaloneRichClick (template-preview.js, lines 10211-10226)

handleStandaloneRichClick handles an event, request, command, or user action. In this file, it appears around lines 10211-10226 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references handleRichCaretClick, closest, selectRichBlock, copyRichCodeBlock, editSelectedRichBlock, deleteSelectedRichBlock, and saveRichEditorToProject. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 10212: if (handleRichCaretClick(event)) return;; line 10216: if (!blockAction) return;; line 10221: return;.

Important local values include blockAction at line 10213; richBlock at line 10214; editor at line 10217. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10211: async function handleStandaloneRichClick(event) {
10212:   if (handleRichCaretClick(event)) return;
10213:   const blockAction = event.target.closest("[data-rich-block-action]");
10214:   const richBlock = event.target.closest(".rich-block");
10215:   if (richBlock) selectRichBlock(richBlock);
10216:   if (!blockAction) return;
10217:   const editor = blockAction.closest("[data-rich-editor]");
10218:   selectRichBlock(blockAction.closest(".rich-block"));
10219:   if (blockAction.dataset.richBlockAction === "copy-code") {
10220:     await copyRichCodeBlock(blockAction.closest(".rich-block"));
10221:     return;
10222:   }
10223:   if (blockAction.dataset.richBlockAction === "edit") await editSelectedRichBlock(editor);
10224:   if (blockAction.dataset.richBlockAction === "delete") deleteSelectedRichBlock(editor);
10225:   saveRichEditorToProject(editor);
10226: }
```

hasDataset (template-preview.js, lines 10323-10327)

hasDataset part of the private builder frontend and editing experience. In this file, it appears around lines 10323-10327 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references call, and addCustomSection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 10326: return;.

Important local values include hasDataset at line 10323. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10323:   const hasDataset = (key) => Object.prototype.hasOwnProperty.call(button.dataset, key);
10324:   if (button.dataset.addSection) {
10325:     await addCustomSection();
10326:     return;
10327:   }
```

Representative opening snippet

```
1: const builderGuideOpen = document.querySelector("#builder-guide-open");
2: const builderGuideDialog = document.querySelector("#builder-guide-dialog");
3: const builderGuideClose = document.querySelector("#builder-guide-close");
4: const builderGuideSearch = document.querySelector("#builder-guide-search");
5: const builderGuideResults = document.querySelector("#builder-guide-results");
```

```
const builderGuideSections = document.querySelector("#builder-guide-sections");
const builderGuideTopicDialog = document.querySelector("#builder-guide-topic-dialog");
const builderGuideTopicTitle = document.querySelector("#builder-guide-topic-title");
const builderGuideTopicBody = document.querySelector("#builder-guide-topic-body");
const builderGuideTopicClose = document.querySelector("#builder-guide-topic-close");
const templateSelect = document.querySelector("#template-select");
const newCategorySelect = document.querySelector("#new-category-select");
const placementSelect = document.querySelector("#placement-select");
const newProjectTitle = document.querySelector("#new-project-title");
const addProjectButton = document.querySelector("#add-project");
const addCategoryButton = document.querySelector("#add-category");
const addSiteSectionButton = document.querySelector("#add-site-section");
const funFactsInput = document.querySelector("#fun-facts-input");
const funFactsCount = document.querySelector("#fun-facts-count");
const saveFunFactsButton = document.querySelector("#save-fun-facts");
const clearFunFactsButton = document.querySelector("#clear-fun-facts");
const siteHeroEyebrowInput = document.querySelector("#site-hero-eyebrow");
const siteHeroTitleInput = document.querySelector("#site-hero-title");
const siteHeroCopyInput = document.querySelector("#site-hero-copy");
const saveSiteContentButton = document.querySelector("#save-site-content");
const profileDisplayNameInput = document.querySelector("#profile-display-name");
const profilePortfolioLabelInput = document.querySelector("#profile-portfolio-label");
const profileContactIntroInput = document.querySelector("#profile-contact-intro");
const profileEmailInput = document.querySelector("#profile-email");
const profilePhoneInput = document.querySelector("#profile-phone");
const profileGithubInput = document.querySelector("#profile-github");
const profileLinkedinInput = document.querySelector("#profile-linkedin");
const profileWebsiteInput = document.querySelector("#profile-website");
const saveProfileContentButton = document.querySelector("#save-profile-content");
const profileImageUploadButton = document.querySelector("#profile-image-upload");
const profileHeroUploadButton = document.querySelector("#profile-hero-upload");
const profileResumeUploadButton = document.querySelector("#profile-resume-upload");
const profileBrandUploadButton = document.querySelector("#profile-brand-upload");
const profileAssetStatus = document.querySelector("#profile-asset-status");
const categoryDropdown = document.querySelector("#category-dropdown");
const siteSectionList = document.querySelector("#site-section-list");
const projectCreateDialog = document.querySelector("#project-create-dialog");
const projectCreateForm = document.querySelector("#project-create-form");
const projectCreateCategory = document.querySelector("#project-create-category");
const projectCreateCancel = document.querySelector("#project-create-cancel");
const categoryDialog = document.querySelector("#category-dialog");
const categoryForm = document.querySelector("#category-form");
const categoryTitle = document.querySelector("#category-title");
const categoryDescription = document.querySelector("#category-description");
const categoryAccent = document.querySelector("#category-accent");
const categoryCancel = document.querySelector("#category-cancel");
const projectTree = document.querySelector("#project-tree");
const projectSearchInput = document.querySelector("#project-search");
const projectSearchClear = document.querySelector("#project-search-clear");
const projectSearchStatus = document.querySelector("#project-search-status");
const projectDialog = document.querySelector("#project-dialog");
const projectWindowTitle = document.querySelector("#project-window-title");
const projectWindowClose = document.querySelector("#project-window-close");
const projectWindowDelete = document.querySelector("#project-window-delete");
const viewProjectPreviewButton = document.querySelector("#view-project-preview");
const saveProjectButton = document.querySelector("#save-project");
const saveProjectCloseButton = document.querySelector("#save-project-close");
const projectTitleMenu = document.querySelector("#project-title-menu");
const titleRenameActions = document.querySelector("#title-rename-actions");
const titleRenameSave = document.querySelector("#title-rename-save");
const titleRenameCancel = document.querySelector("#title-rename-cancel");
const projectMetaDialog = document.querySelector("#project-meta-dialog");
const projectMetaTitle = document.querySelector("#project-meta-title");
const projectMetaGrid = document.querySelector("#project-meta-grid");
const projectMetaClose = document.querySelector("#project-meta-close");
const projectFields = document.querySelector("#project-fields");
const sectionTabs = document.querySelector("#section-tabs");
```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.

- Change it on a feature branch from development.

- Regenerate docs if the file purpose, API surface, or behavior changes.

- Run the smallest relevant smoke test before merging into development.